Universität Rostock Traditio et Innovatio

Generating Educational Questions using Large Language Models: An Evaluation of Quality and Alignment with Pedagogical Principles

Faculty of Computer Science and Electrical Engineering
of the University of Rostock
Study Programme "Computer Science (2021)"

Bachelor's Thesis

to obtain the Academic Degree
Bachelor of Science

submitted by

Malte Gunnar Grube

E-Mail:	malte.grube@uni-rostock.de
Matriculation Number:	221200075
Date of Submission:	15.08.2025
Processing Period:	29.04.2025 – 15.08.2025
First Examiner:	Prof. Dr. rer.nat. Clemens H. Cap
Second Examiner:	Jun.-Prof. Dr. Charlott Rubach

Acknowledgements

Firstly, I want to thank Professor Clemens Cap, my primary advisor from the Department of Information and Communication Services (IuK) at the University of Rostock's Faculty of Computer Science and Electrical Engineering, for all of his help and financial support during this project. I am equally thankful to Professor Charlott Rubach, my secondary advisor from the University of Rostock's Faculty of Philosophy and the Institute of School Pedagogy and Educational Research (ISB). Her knowledge and guidance made this interdisciplinary research possible. I want to thank all coworkers and staff members who took the time and shared their knowledge and expertise to help with this job. Finally, I wish to thank my family and friends for being supportive and understanding while I worked on this thesis.

Abstract

This Bachelor's thesis investigates the potential of Large Language Models for Automated Question Generation regarding two aspects: the adherence to given instructional content and the models' performance across Bloom's revised Taxonomy, particularly focusing on alignment with desired cognitive levels. The work is structured via two primary research questions, which will be answered by going through two certain experiments. The methodology integrates both quantitative and qualitative evaluations. On the one hand, quantitative assessment utilizes cosine similarity measures and Large Language Model-based adherence scoring to assess content fidelity between generated questions and desired source content. On the other hand, qualitative analysis involves expert- and student-based evaluation using structured rubrics that assess criteria including e.g. relevance, clarity, answerability, and especially correctness to the source content in the first experiment. In contrast to correctness, the second experiment systematically explores the relationship between question formats (Multiple-Choice vs. Open-Ended) and cognitive complexity levels across Bloom's Taxonomy, analyzing how these models behave when generating questions with varying instructional information. Results indicate a significant impact of prompt complexity, since vastly formulated prompts do not imply higher scoring of e.g. content fidelity, focusing on the first experiment. Even state-of-the-art models struggle with this task, and generating questions for educational purposes – including Bloom's revised Taxonomy – remains a challenging task, regarding the second experiment. Overall, this research demonstrates the promise and challenges of using Large Language Models for educational question generation and proposes a comprehensive evaluation that combines automated metrics with expert / student assessments to present future directions for improving Automated Question Generation systems.

Keywords: Large Language Models; LLMs; Automated Question Generation; AQG; Content Fidelity; Content Adherence; Bloom's Taxonomy; Pedagogical Effectiveness; Bloom Alignment

Table of Contents

List of Figures

List of Tables

List of Abbreviations

AQG Automated Question Generation

BLEU BiLingual Evaluation Understudy

BLEURT BiLingual Evaluation Understudy with Representations from Transformers

CNN Convolutional Neural Network

CoT Chain-of-Thought

FFN Feed-Forward Network

LLM Large Language Model

MCQ Multiple-Choice Question

METEOR Metric for Evaluation of Translation with Explicit ORdering

MoE Mixture of Experts

NLP Natural Language Processing

RAG Retrieval-Augmented Generation

RNN Recurrent Neural Network

ROUGE Recall-Oriented Understudy for Gisting Evaluation

RQUGE Reference-free QUestion Generation Evaluation

SLM Small Language Model

Markdown & Source Code Listings

I Introduction

This thesis investigates the capabilities of Large Language Models (LLMs) for the automatic generation and evaluation of educational questions. The focus is on the growing need for effective educational tools that can support both teaching and learning.

1.1 Motivation

Effective learning often relies on engaging actively with educational content, instead of reading it passively [Ste+21]. One effective teaching strategy is to ask questions about the content that the students are dealing with, which can help them to understand and comprehend the material better [Ste+21]. However, creating questions that are both linguistically and pedagogically valuable can be a time-consuming task for educators [Ste+21; Vu+24]. Moreover, many educational materials and textbooks lack sufficient questions, so testing the students' comprehension is limited [Ste+21].

LLMs have shown remarkable skills in understanding and generating human-like text across various tasks [Wan+24]. Additionally, these models have potential to assist educators via Automated Question Generation (AQG) [Vu+24; LA24]. Utilizing LLMs for AQG could enable educators to produce a larger volume and wider variety of questions efficiently [SCS24; HWY24], resulting in more time on direct student interaction [Dou+24; ARA24] and personalized learning [LA24].

1.2 Role of AI in Education

Originality of the Problem. Existing educational AQG systems have certain limitations. Many prior approaches focused on the linguistic quality while disregarding the pedagogical value and appropriateness of questions, as stated in [Hor+20; Ste+21]. Outdated models [CXJ24; Elk+23; Bho+23] were often used, potentially due to power and cost constraints [ZWC25; Vu+24]. Furthermore, novel models with *reasoning* capabilities, known from e.g. OpenAI o1 [Ope+24c] and DeepSeek R1 [Dee+25a], were not leveraged in the literature. A significant research gap exists in generating cognitively challenging questions, particularly focusing the alignment with higher levels of Bloom's Taxonomy. While LLMs are good at generating questions for lower cognitive levels, they have limitations at the higher levels [Elk+23; SDS24], e.g. often generating oversimplified or unrealistic questions [DWK24]. In addition, the missing evaluation of content adherence related to a source text is a critical aspect that is highlighted in limitations of prior work [Dou+24; ARA24].

Relevance of the Problem. The development of effective educational tools is increasingly important to support both teaching and learning. Automating question generation, particularly for educational assessments, can significantly reduce the workload for educators, leading to saving the educators' time [BFL24; HWY24; Vu+24; CXJ24]. LLMs can support the creation of scalable educational content, such as question banks for various learning modules [Vu+24]. Furthermore, these models have potential to enable personalized education for varying needs and learning styles, enhancing student engagement and understanding [HWY24].

Resulting Contributions. The Bachelor's thesis contributes to the field through offering two systematic evaluations for assessing educational questions generated by LLMs. A key focus, on the one hand, is evaluating the adherence of generated questions to certain ISO-OSI model-based source materials, to ensure that the are based on this. On the other hand, the capabilities of LLMs in AQG while considering the cognitive levels of Bloom's Taxonomy are investigated. The evaluation's insights imply that even recent LLMs occasionally struggle with generating questions that are faithful to given source materials, and useful for educational purposes. This whole thesis project is made available on GitHub: `https://github.com/mgrube753/bachelor-thesis`.

1.3 Research Questions

Regarding the identified potential and limitations of LLMs in educational contexts, this thesis aims to provide a comprehensive evaluation of their effectiveness in generating educational questions. To achieve this, the research will be guided by the following questions:

RQ1: *To what extent do LLMs adhere to the content of diverse provided instructional texts when generating questions?* Evaluating this aspect is crucial to generate reliable educational questions. This involves assessing whether the questions can be answered using the source material [MDS25; Moo+22; SDS24] and measuring the content fidelity based on semantic similarity to reference content [LZ24]. To avoid the generation of misleading or irrelevant information, called "hallucination", ensuring that the generated questions are aligned with the source material is realized in research [ARA24].

RQ2: *How does the relationship between diverse question formats and Bloom's Taxonomy levels influence the pedagogical effectiveness of LLM-generated questions?* Previous work has pointed out challenges in generating questions for higher Bloom's levels [SDS24] and the need of pedagogical evaluation beyond linguistic tests [Hor+20; Ste+21]. This research question aims to explore how specific question formats perform across different cognitive levels of Bloom's revised Taxonomy [Kra02] and how this influences the usefulness for educational intentions.

1.4 Thesis Outline

This Bachelor's thesis is organized as follows: Section **II** provides the theoretical foundations necessary to understand the research, including an introduction to Large Language Models, Bloom's Taxonomy, prompting strategies, and evaluation methodologies. Section **III** reviews related work in the field of LLMs and AQG. Section **IV** addresses the experimental design, outlining the experimental setup, and the methodological approach used to evaluate LLMs' capabilities in generating educational questions. Section **V** describes the implementation and execution of the experiments, including the used APIs, the experimental procedure, the promptings, the data generation process and the evaluation methods. Section **VI** presents the evaluation of results, offering both quantitative and qualitative analyses for both experiments of this work. Section **VII** concludes the thesis by summarizing key findings, discussing practical implications for education, and also suggesting directions for future research.

II Theoretical Foundations

This section provides the theoretical foundations for the work presented in this thesis. It covers the necessary background knowledge, including the fundamentals of LLMs, the hierarchical structure of Bloom's Taxonomy, plus various performance enhancement and evaluation methodologies.

2.1 Understanding Large Language Models

Architectural Fundamentals. LLMs – the base of AQG – are commonly based on the novel neural network architecture known as transformers [NNN24], released by Google [Vas+17]. It was originally proposed for machine translation [Min+25; Vas+17]. Shown in Google's paper, what makes the transformer architecture so special is that it is solely based on the attention mechanism, so it does not rely on the former approaches called Recurrent Neural Networks (RNNs) or Convolutional Neural Networks (CNNs). Additionally, this architecture is capable of better parallelization, faster training times and better performance, as to be seen in its intended task [Vas+17]. Attention is essential for models such as the transformer, allowing to model long-range dependencies effectively, but also focus and weight on important information in the input sequence [PG24].

The transformer follows an encoder-decoder structure [PG24; Min+25]. Obtained from [Min+25; Vas+17], both the encoder and decoder consist of a stack of identical layers, and in the original paper, $N = 6$ layers were used, split in two columns in the following figure:

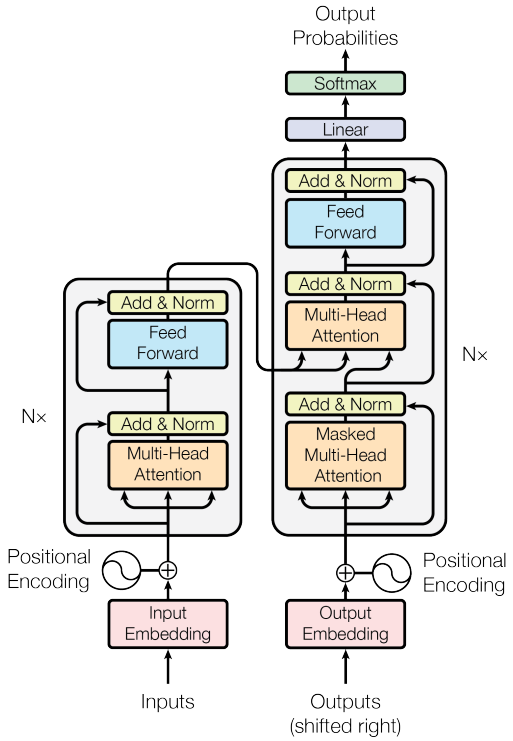

Figure 1: Transformer Architecture [Vas+17].

Preprocessing Steps. To prepare the text for the transformer model, several steps are performed before the actual encoding and decoding process begins:

- **Tokenization**. It is a crucial step for transformers and LLMs, where the text is parsed into several units called tokens [Nav+24; Rai+24; Zha+25; NNN24], which represent characters, a group of these or words. With tokens, the next step can be performed.
- **Creating Embeddings**. Each token is then converted into a numeric vector representation of the same dimension [Vas+17]. By this, the token's meaning is mapped in a mathematical space, where comparable phrases are close together [Rai+24].
- **Positional Encoding**. As stated in [Vas+17], positional encoding is added to the input embeddings to provide information about the position of each token in the sequence. This is necessary because transformers do not have a built-in order, unlike RNNs or CNNs.

Left: Encoder. The encoder structure is primarily designed for processing and interpreting the input text [Wan+24]. It extracts step-by-step features and the result is passed to the decoder [Liu+25]. Each encoder layer consists of two sub-layers:

- **Orange: Multi-Head Self-Attention** [Vas+17]. In this layer, every position in the encoder's sequence can attend to all other positions in the previous layer of the encoder, which is the input of the current layer. By this [Liu+25], the model can capture dependencies between differently positioned tokens in the input sequence.
- **Blue: Position-wise Feed-Forward Network (FFN)** [Vas+17]. This layer is applied independently to each position in the sequence. By using this layer [Liu+25], the encoder can process and extract features from the attention's output.

Right: Decoder. The decoder instead is responsible for generating the output sequence based on the encoded input. This results in a sequence of generated tokens [Rai+24]. Each decoder layer is a bit different from the encoder layer structure. It consists of the following three sub-layers:

- **Orange$_1$: Masked Multi-Head Self-Attention** [Vas+17]. This layer is a modified version of the encoder's self-attention mechanism, but it prevents attending to future sequence positions. By this, the model can only use information from the past tokens and the current one in its decoder layer.
- **Orange$_2$: Multi-Head Attention** [Vas+17]. This layer explicitly allows the decoder to attend over the encoder's output, enabling it to incorporate information from the input sequence.
- **Blue: Position-wise FFN** [Vas+17]. This layer is similar to the one in the encoder, applied independently to each position in the sequence. In the end, the decoder output is passed to a linear layer and a softmax function to finally generate the next-token probabilities.

Resulting Architectures. According to the survey [Sha+24], the transformer architecture has been split into three main categories, as follows:

- **Encoder-only:** Earlier architectures such as Google BERT [Dev+19] and its derivates, e.g. Huawei ERNIE [Zha+19] and Meta RoBERTa [Liu+19] are solely based on the encoder component. This results a niche for these models being suitable for specific Natural Language Processing (NLP) tasks focused on comprehension. According to the survey [Hou+24], these models process and encode input into hidden representations to capture word relationships and contextual information. Here, models like BERT observe both the left and right context of the currently focused word.

- **Decoder-only**: More current models e.g. Meta's LLaMA series [Tou+23b], OpenAI's ChatGPT [Ope22] plus the publication of GPT-4 [Ope+24a] focus on token generation based solely on the preceding tokens [Sha+24]. According to [Hou+24], without the reliance on the encoder, decoder-only transformers can be used for diverse generation tasks, such as code generation and code completion.

- **Encoder-Decoder**: Certain past architectures such as Google's T5 [Raf+23] and their BART model [Lew+19] integrated both the encoder and decoder components. This led to hybrid capabilities, combining the understanding of the input and the proper generation of the output. According to [Wan+24], these models are applicable to e.g. machine translation, text summarization and question answering. However, the complexity of the dual architecture can lead to demanding training processes and slower inference times.

Mixture of Experts (MoE). Another notable variant of the transformer architecture is the MoE [Sha+17] architecture, as shown in the overview by [Nav+24]. In this architecture, multiple experts are integrated. Typically, an MoE system with several experts includes a mixture of an FFN for each expert, positioned right after the attention block, and a router mechanism, which decides which token(s) are processed by which expert(s). These are advantageous in terms of computational efficiency while being as powerful as dense models. That means, the size of the model can be increased without increasing the computational cost proportionally, because only a subset of the experts is activated for each input. Prominent examples of MoE models are xAI Grok-1 [xAI24b], Gemini 1.5 by Google [Tea+24a], the DeepSeek models V3 [Dee+25b] and its *reasoning* variant R1 [Dee+25a], and the recently released LLaMA 4 family by Meta [Met25].

2.2 Framing Bloom's Taxonomy for LLM Applications

Given LLMs' capabilities, these enable the automation of the often time-consuming question generation process for educators [Vu+24]. The integration of such models into education offers an opportunity to enhance the learning experience for students and support teachers in advancing educational content [Nav+24]. With the help of LLMs, customized study materials and practice questions can be generated to develop personalized learning for students [ARA24; Bho+23; HWY24], while freeing up the educators' valuable time for both teaching and student interaction [Nav+24].

Challenge of Alignment. For automated content creation and personalized learning, the ability to differentiate questions based on certain levels of cognitive complexity is crucial [LZ24; ZWC25]. However, aligning the automated questions with specific cognitive levels still remains a key challenge. LLMs per se show great potential in AQG with respect to different cognitive levels [MDS25; Blo+23; DWK24; SDS24; SCS24]. By using different prompting strategies, as illustrated in the **Performance Enhancements** and the according **Question Generation Research** section, the models can be guided to generate questions that are of high quality and better aligned with the cognitive levels of Bloom's Taxonomy.

Overall, certain approaches can generate questions across all levels by fine-tuning a base model [DWK24; ZWC25], but [DWK24] lacks the proper evaluation of the cognitive adherence to Bloom's Taxonomy, as the authors' student-based study only evaluated the preference of the generated questions between GPT-4 and their BloomLLM. On the contrary, the authors of TwinStar [ZWC25] did a thorough but solely automated evaluation of their AQG system. In this case, their LLaMA2-based system was compared to LLMs such as GPT-4, and TwinStar outperformed the other models regarding the cognitive adherence. This insight is interesting since the larger models performed worse than the 13 billion parameter version of LLAMA2. Additionally, outdated models were used here and fundamentally in the literature, so the usage of current LLMs can lead to other results.

However, despite their proficiency at lower levels, LLMs such as ChatGPT [Ope22] occasionally struggle with adherence at the higher levels [DWK24; Elk+23]. This limitation primarily stems from their tendency to generate questions which are oversimplified or even unrealistic [DWK24], or the given cognitive level from the prompt was not properly covered [SDS24]. In the latter paper, the evaluation has shown that several LLMs struggle with the adherence to the given cognitive level, but GPT-3.5 and GPT-4 outperformed the other models, such as LLaMA2 [Tou+23b], by $\approx 20\%$, depending on the prompt specification. In addition to that, generating *Creating*-level questions was the most challenging task for almost every model used in the study.

Furthermore, in the study of [MDS25], it was observed that a generated question was better suited to the *Analyzing* level, instead of the *Applying*. It is possible to minimize this problem by prompting the model in a specific way, as the authors have shown via their eight-shot prompting setting. This method requires example context and questions to be provided in the prompt, which is not feasible for every author, as it requires a lot of effort to create the contexts and questions.

Cognitive Hierarchy. In the literature, as depicted above, Bloom's Taxonomy [KBM64] and primarily its revised version [Kra02] are often discussed in the context of question generation and evaluation with respect to LLMs [SDS24; SCS24; Elk+23]. For crucial understanding of the taxonomy, it follows a hierarchical structure where the six main categories are organized by increasing complexity, with each level building upon a less complex predecessor excepting the lowest level [Kra02]. The following figure[1] shows the cognitive levels of Bloom's Taxonomy in its revised version.

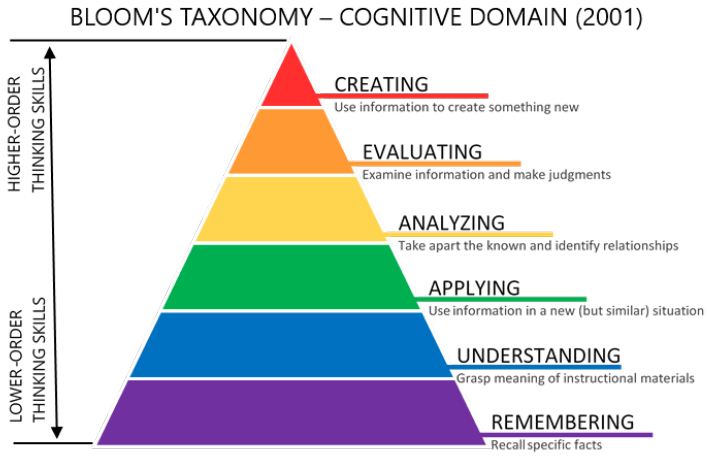

Figure 2: Hierarchy of Bloom's Revised Taxonomy.

The revised taxonomy [Kra02] is divided into six levels. Each level is associated with a specific cognitive skill, as follows with ascending complexity:

- **Remembering:** Formerly known as "Knowledge", this level refers to the retrieval of relevant knowledge from long-term memory by focusing on recognition and recall of facts.
- **Understanding:** This level, previously called "Comprehension", involves identifying the meaning of instructional messages, such as oral, written and graphic communication.
- **Applying:** This level is about carrying out or using a procedure in a given situation.
- **Analyzing:** This involves breaking down material into its components and determining their relationships to one another or to an overall structure or purpose.
- **Evaluating:** This level requires making judgements based on established criteria and standards.
- **Creating:** This is the highest cognitive level and involves assembling or reorganizing elements into a coherent whole.

Each of these cognitive levels is associated with a set of action verbs that can be used to formulate learning objectives and assessment questions. A comprehensive list of such verbs, categorized by these cognitive levels, can be found at the University of Arkansas' website[2]. The following table offers a brief overview of the cognitive levels and their corresponding action verbs:

[1] https://citt.ufl.edu/resources/the-learning-process/designing-the-learning-experience/blooms-taxonomy/blooms-taxonomy-graphic-description/, accessed 04/29/2025

[2] https://tips.uark.edu/blooms-taxonomy-verb-chart/, accessed 05/13/2025

Table 1: Bloom's Taxonomy Action Verbs.

Remembering	Understanding	Applying	Analyzing	Evaluating	Creating
Define	Clarify	Apply	Analyze	Assess	Assemble
Describe	Classify	Calculate	Break down	Conclude	Code
Enumerate	Compare	Demonstrate	Detect	Criticize	Compile
Identify	Contrast	Determine	Differentiate	Defend	Construct
List	Detail	Examine	Distinguish	Evaluate	Create
Match	Explain	Illustrate	Examine	Grade	Design
Name	Paraphrase	Modify	Investigate	Interpret	Develop
Outline	Rewrite	Simulate	Optimize	Judge	Enhance
Quote	Summarize	Solve	Relate	Justify	Improve
Select	Translate	Use	Separate	Verify	Reorganize

2.3 Performance Enhancements

To address these challenges and to refine the capabilities of LLMs for specific tasks, such as AQG, various performance enhancement strategies have been proposed. These techniques aim to improve the quality and relevance of the generated content, as listed in the following:

In-Context Learning. As highlighted by [Zha+25], the advent of GPT-3 [Bro+20] introduced the concept of in-context learning. This method allows LLMs to adapt to new tasks by using a small set of examples directly within the prompt at inference time [Min+25]. OpenAI further delineated this into subcategories such as zero-shot, one-shot, and few-shot learning, depending on how many examples are provided in the prompt [Bro+20]. By this, in-context learning is efficient as it requires minimal task-specific data and no parameter updates in the model itself, while LLMs can adapt without any costly retraining [PG24; Hou+24].

Chain-of-Thought (CoT). This technique, introduced by Google [Wei+22], enhances in-context learning by prompting reasoning steps to elicit the LLMs' step-by-step reasoning [Zha+25]. Moreover, this method can lead to higher quality questions [SDS24], as depicted in [Wei+22]. What makes CoT so powerful nowadays is that most of the state-of-the-art LLMs are performing techniques like this on their own before returning the final answer to the user [Ope+24c], visible in Table **2** as the term of *reasoning*.

Role-based Prompting. This technique involves assigning a specific role or persona to the model while prompting it [Zha+25] to better fulfill the task instruction. The initial phrase of a prompt can look like "Behave as an expert in the field of X". Independently of the survey [Zha+25],

the authors of this method [Kon+24] have shown that this approach highlights the potential to trigger CoT implicitly and effectively, leading to enhanced reasoning outcomes. It is worth noting that reasoning here refers to the model's abilities, not the generative step before providing the final answer to the user.

Self-Refinement. By iteratively refining LLM-generated output based on feedback [Mad+23], the overall process and the corresponding solutions can be improved [Zha+25]. For instance, the authors of MCQGen [HWY24] applied this technique to their AQG system, as the model is critiquing and improving its initial outputs in a recursive loop.

Using Special Marks. Emphasizing important parts of the prompt can be achieved by using special marks like quotation marks or line breaks [Zha+25] to help the LLM visually distinguish between different parts of the prompt. As an example, to emphasize a text block, it can be enclosed in docstring quotation marks, as it is known from Python programming[3].

Providing Scoring Standards. For tasks which involve scoring a text, it is crucial to provide a detailed description of the scoring standards in the prompt [Zha+25]. For instance, using a clear rubric helps the LLM to assess the quality of generated questions, as it was done in [SDS24]. However, in this work – probably because of the usage of an outdated LLM –, the evaluation results differ from the human evaluation, which has to be considered when using this technique.

Retrieval-Augmented Generation (RAG). This method [Lew+20], as used in approach [HWY24], highlights how external knowledge sources can be integrated at AQG prompts. For this technique, querying a comprehensive database is needed to retrieve relevant information that is useful the context of the question. This can also be realized via e.g. knowledge graphs with domain-specific knowledge, which can be based on course content [YZ24].

Model Fine-Tuning. A key strategy to adapt general-purpose LLMs to specific tasks by training them on domain-specific data [Hou+24]. By this process, models can learn knowledge relevant to a specific context, but traditional full fine-tuning requires vast amounts of data and computational resources [Hou+24]. For instance, [DWK24] fine-tuned their AQG model BloomLLM with more than 1000 questions, which is still a relatively small amount of data. In the end, this GPT-3.5-based model was preferred across all Bloom levels over GPT-4 according to AQG.

Integration of Contextual Knowledge. Several studies [BFL24; Blo+23; Bho+23] have injected topic-related contextual information directly into the prompt to avoid hallucinating and give users control over the source of knowledge [BFL24].

[3] https://peps.python.org/pep-0257/, accessed 06/16/2025

2.4 Evaluation Methodologies

Evaluation is always necessary to assess the quality of LLMs for the task at hand. According to [Dou+24], evaluating automatically generated learning resources is important because they often lack the deep qualities necessary for their effectiveness. The methods to evaluate LLMs at e.g. AQG can be divided into several categories:

n-gram-based Metrics. These metrics follow a reference-based approach, where the generated text is compared to a reference text. The evaluation is based on the overlap of sequences of n words between the generated and reference text [Guo+24]. The most common metrics in this category are:

- *BiLingual Evaluation Understudy (BLEU)* [Pap+01] evaluates the average n-gram precision against a reference text, so how many n-grams from the generated text are also present in the reference text [Guo+24].
- *Recall-Oriented Understudy for Gisting Evaluation (ROUGE)*, proposed by [Lin04], calculates recall, which is the ratio of n-grams from a ground truth text that are also present in the generated text [Guo+24].
- *Metric for Evaluation of Translation with Explicit ORdering (METEOR)* [BL05] calculates the harmonic mean of precision and recall, so it provides a balanced evaluation [Guo+24].

Diversity-based Metric. The more recent *Distinct-n* [Li+16] measures text diversity by calculating the proportion of unique n-grams, though it is considered overly simplistic [Guo+24].

Semantic Similarity. Metrics in this category assess sentence-level comparison, which is vital beyond word-level comparisons [Guo+24]. Regarding *BERTScore* [Zha+20], it compares contextual sentence-level embedding vectors returned from a pre-trained BERT model. These generated vectors are compared by calculating cosine similarity between the embeddings of the generated and the reference text [Guo+24], as this scoring metric was adopted in the study of [LZ24].

Other Metrics. Apart from these categories, diverse other methods and metrics can be used to evaluate the generated questions. For instance, [YZ24] utilized – besides *Diversity* – the metrics *Perplexity*, and *Toxicity* to assess the linguistic question's quality.

In addition to that, researchers developing new evaluation metrics compare their approaches against a range of existing ones. For instance, [Ngu+24] benchmarked their proposed metric against modern metrics e.g. *BERTScore*, Google's *BiLingual Evaluation Understudy with Representations from Transformers (BLEURT)* – predicting human judgements of text quality – [SDP20], and *Reference-free QUestion Generation Evaluation (RQUGE)* metric by Meta – comparing predicted and gold answer spans – [Moh+23].

Human Evaluation. This kind of evaluation is considered necessary to judge about the quality of generated questions [Hor+20].

- While criticizing former used metrics such as *BLEU* and *METEOR* not covering a proper evaluation on question quality, [Hor+20] presented a novel hierarchical evaluation scheme for question generation to guide the human annotators. The scheme often considers binary choices, to induce clear decisions. The following 9 criteria were presented: *Understandable, DomainRelated, Grammatical, Clear, Rephrase, Answerable, InformationNeeded, Central, WouldYouUseIt.*
- Studies such as [Ste+21] and [Moo+22] adopted this hierarchical scheme and used it for their human question evaluation. For instance, [Ste+21] used it to evaluate their AQG system, while [Moo+22] adopted this scheme to assess the quality of student-generated questions.
- Furthermore, the authors of the papers [SCS24; SDS24] used and adapted the evaluation scheme by replacing the criterion *Rephrase* with *Bloom'sLevel.*
- Using [Moo+22] as a reference, [ML24] proposed another scheme with a maximum of 50 points by covering *Relevance, Clarity, Answerability, Challenging* and *Cognitive Level* as criteria. The latter shall present a higher score for student-generated questions that are more cognitively demanding, so these are positioned higher in the Bloom's Taxonomy hierarchy. Focusing on rating student-generated questions, GPT-4 had most voting agreement with expert ratings in contrast to e.g. GPT-3.5 and ERNIE.

LLM as an evaluator. Another approach involves using LLMs themselves as evaluators. For instance, [Ngu+24] introduced NACo, an LLM-based metric that employs a CoT strategy to rank the *Naturalness, Complexity* and *Answerability* of generated questions. In contrast to this, [SCS24] employed Gemini Pro[4] to assess *Question Quality* and *Alignment* with Bloom's Taxonomy, revealing discrepancies compared to human evaluators. It is noteworthy that the abilities of LLMs are rapidly evolving, and results from former models, e.g. Gemini Pro, might differ with versions nowadays.

The use of LLMs for the evaluation of questions is a promising direction, especially when coupled with human evaluation. It is also worth noting that analyzing the inter-rater agreement between the LLM and the human experts can enhance the reliability of the evaluation process, as it was demonstrated in studies such as [ML24].

[4] https://blog.google/technology/ai/google-gemini-ai/#performance, accessed 05/09/2025

III Related Work

This section explores current research in LLMs and their application to AQG, with sections to detail the literature review process and highlight significant contributions in the field.

Literature Review. To find the fundamental principles and state-of-the-art of the research problem, a literature review was conducted. It was performed in the following way: First, several searches were done with respect to LLMs in common, question generation, particularly focusing on Bloom's Taxonomy, and lastly evaluation methods for both categories. Google Scholar and Elicit were used to find relevant papers. Several search terms were coupled in Google Scholar, such as *LLM, Large Language Model, architecture, question generation, automatic question generation, Bloom's Taxonomy, evaluation, assessment, alignment*. Elicit is a search engine that is designed to help researchers find relevant papers based on writing a specific prompt, instead of a query of keywords.

Review Steps. The review steps were as follows:

- First, the papers were downloaded and stored in Zotero. Then, duplicates were filtered out.
- The remaining papers were **included** or excluded by using the TACID method. Papers were included if they are recent (published from 2021 on) and dealing with LLMs or automated question generation or evaluation methods. If the paper was a preprint instead of a peer-reviewed paper, it needed a comparatively high relevance to be included ($\approx$ 10 citations). If a publication was present in a workshop, it needed citations to be included.
- Papers were **excluded** if they were not written in English, not open access, or the journal is predatory[5]. If the focus is explicitly on Small Language Models (SLMs) or multimodal models, or other domains besides the desired ones, the paper was excluded.
- After the process, the remaining papers were filtered by a deeper analysis of the papers.
- In the end, diverse cited and subsequently found papers blog posts by e.g. Google were added to the thesis to improve the information space.

3.1 Current LLMs Research

Review Challenges at LLMs. Finding the current state-of-the-art of LLMs with the explained review method did not yield recent results, even if papers such as surveys were published in 2024 or 2025. The reason for this is that the field of LLMs is moving so fast, that the review papers become outdated. Moreover, the trend of the models and their releases were followed manually, as captured in Table 2 below. It shows the most important releases of LLMs in the past years since ChatGPT [Ope22]. The table is not complete, but it shows well-known models and their architecture, including *reasoning* capabilities, if these are known to the public. *Reasoning* means, in the OpenAI o1 system card [Ope+24c] for instance, the model thinks in a manner of CoT before returning the desired response to the user. The proposed table is sorted by the release date of the models to obtain a timeline and a way to interpret the development of the models.

[5] Journals noted on https://beallslist.net/, accessed 04/30/2025

Table 2: Timeline of Major LLM Releases (2022-2025).

Date	Developers	Model Releases	Architecture	Reasoning
Nov 2022	**OpenAI**	ChatGPT/GPT-3.5	Decoder-only?	–
Feb 2023	**Meta**	LLaMA 65B	Transformer	–
	Google	Bard 137B	Decoder-only	–
Mar 2023	**OpenAI**	GPT-4	Decoder-only?	–
	Anthropic	Claude 1	–	–
Jul 2023	**Anthropic**	Claude 2	–	–
	Meta	LLaMA 2 70B	Decoder-only	–
Nov 2023	**xAI**	Grok-1 314B	MoE	–
Dec 2023	**Google**	Gemini	Decoder-only	–
Feb 2024	**Google**	Gemini 1.5	Decoder MoE	–
Mar 2024	**Anthropic**	Claude 3	–	–
May 2024	**OpenAI**	GPT-4o	Decoder-only?	–
Jun 2024	**Anthropic**	Claude 3.5 Sonnet	–	–
Aug 2024	**xAI**	Grok-2	–	–
Sep 2024	**OpenAI**	o1-mini	–	Yes
Dec 2024	**Meta**	LLaMA 3.3 70B	Decoder-only	–
	Google	Gemini 2.0	–	–
	OpenAI	o1/o3-mini	–	Yes
	DeepSeek	DeepSeek-V3 658B	MoE	–
Jan 2025	**DeepSeek**	DeepSeek R1 658B	MoE	Yes
	Alibaba	Qwen 2.5-Max	MoE	–
Feb 2025	**xAI**	Grok-3	–	Yes
	Anthropic	Claude 3.7 Sonnet	–	Yes
	Alibaba	QwQ-Max	MoE	Yes
Mar 2025	**Google**	Gemini 2.5	–	Yes
Apr 2025	**Meta**	LLaMA 4 109B/400B/2T	MoE	–
	OpenAI	o3/o4-mini	–	Yes

Major LLM Developers and Models. The release of OpenAI's ChatGPT [Ope22] has innovated the research on LLMs. Initially leveraging the GPT-3.5 architecture, OpenAI followed the trend of increasing secrecy with models like GPT-4 [Ope+24a] and its successor GPT-4o [Ope+24b]. While the "GPT" nomenclature suggests the continuation of the decoder-only architecture up to GPT-3 [Bro+20], many methodological details remain undisclosed. Recent models by OpenAI focused on *reasoning* capabilities, as seen in the o1-mini [Ope24], o1 [Ope+24c], o3-mini [Ope25b], and the subsequent o3 and o4-mini [Ope25a] releases.

Other major developers contributed to this competitive landscape. Meta started with a modified transformer, called LLaMA [Tou+23b], and shifted to a decoder-only architecture in LLaMA 2 [Tou+23a] up to LLaMA 3.3[6]. With the release of the LLaMA 4 family [Met25], Meta switched to the MoE architecture, scaling its largest model to two trillion model parameters. Google followed a similar path, switching from its LaMDA-based Bard [Pic23; Tho+22] to the now well-known Gemini model series [Tea+24b] by fine-tuning Bard. The family also started as decoder-only, while Gemini 1.5 marked a shift to an MoE architecture, though still described as decoder-based [Tea+24a]. Beginning with Gemini 2.0 [PHK24], Google stopped publishing detailed architectural information. Gemini 2.5 [Kav25] further advanced their offerings by fully integrating *reasoning* capabilities, aligning with current state-of-the-art trends.

Besides the presented developers, Anthropic has remained private about architectural information of its Claude models, from Version 1 [Ant23b], Claude 2 [Ant23a], through the Claude 3 family [Ant24b], Claude 3.5 Sonnet [Ant24a] up to Claude 3.7 Sonnet [Ant25]. The latter model also incorporated *reasoning*, making it competitive with other currently leading models. Meanwhile, developers such as xAI, DeepSeek AI, and Alibaba have set up several MoE models. xAI's Grok-1 was openly an MoE model [xAI24b]. However, for subsequent releases, Grok-2 [xAI24a] and Grok-3 [xAI25], architectural details were not provided, though Grok-3 is a competitive state-of-the-art model with *reasoning*. DeepSeek AI has contributed powerful open-source MoE models like DeepSeek V3 [Dee+25b] and its *reasoning* variant, R1 [Dee+25a]. Similarly, the Alibaba Group has also contributed to the field with models like Qwen 2.5-Max [Tea25b], an MoE model. The more recent QwQ-Max [Tea25a] builds upon this architecture, incorporating *reasoning* to compete at the state-of-the-art.

Visible in Table **2**, the current state-of-the-art for the given table began from o1 and o3-mini in December 2024 plus the release of DeepSeek V3. January 2025, DeepSeek R1 and Alibaba's Qwen 2.5-Max model without *reasoning* capability were released. February 2025 brought xAI's Grok-3, Anthropic's Claude 3.7 Sonnet and Alibaba's QwQ-Max. In March 2025, the state-of-the-art was reached by a major upgrade in Google Gemini, with the release of Gemini 2.5. Finally, April 2025 featured the LLaMA 4 family by Meta and OpenAI's o3 and o4-mini models. Based on this information pool, the models for the AQG evaluation were selected, as to be seen in Section **IV**.

[6] https://github.com/meta-llama/llama-models/blob/main/models/llama3_3/MODEL_CARD.md, accessed 05/09/2025. Note that auto-regressive means decoder-only

Key LLM Development Trends. As illustrated in Table 2, a discernible trend is the decreasing transparency from developers regarding architectural designs of their LLMs. While some models are known to be transformer-based (e.g. decoder-only) or employ an MoE architecture, and *reasoning* capabilities are often highlighted, many other details remain undisclosed due to the competitive nature of the field. It is also worth noting that there is a trend towards models with a higher number of parameters, a prevalence of decoder-only architectures from ChatGPT on, and an increasing adoption of MoE designs, a trend possibly popularized by GPT-4. Although the technical report for GPT-4 [Ope+24a] lacks architectural specifications, it is rumored to be an MoE model[7]. Furthermore, a significant trend is the continuous integration of *reasoning* capabilities into LLMs, with the most recent models being competitive in this area.

3.2 Question Generation Research

LLMs enable the automation of the time-consuming and demanding procedure of question generation [Vu+24]. As stated by [Nav+24], the integration of LLMs into educational settings offer an opportunity to improve the learning experience, support teachers and encourage the development of educational content. This results in freeing up time for educators to focus on teaching and interacting with students. Two certain directions of research are presented in the following:

Prompt Engineering Approaches. Numerous studies have investigated the capability of LLMs to generate educational questions, primarily aligned with cognitive levels, by solely refining and optimizing the prompts given to the models.

[SCS24] addressed the problem of creating pedagogically effective questions, especially at higher skill levels, which is challenging, time-consuming, and requires expert-based evaluation. By this, researchers can understand the pedagogical aspects of AQG, since automated metrics often analyze linguistic features. The study assessed the capability of LLMs, including GPT-3.5 and GPT-4 and other models, to generate high-quality questions across different cognitive levels by using zero-shot prompting. The question evaluation was conducted by two annotators based on the categories adopted from [Hor+20], replacing the *Rephrase* category with *Bloom'sLevel*. This category assesses the adherence to the targeted cognitive level. The Inter-annotator agreement was measured using Cohen's Kappa [Coh60], since there are two annotators to be validated. Their findings revealed that over 91% of the generated questions were relevant and high-quality, where models such as GPT-4 and GPT-3.5 were notably effective, especially in adhering to the desired Bloom level. However, limitations were identified by the authors, e.g. the crucial human evaluation can be subjective. Also, most models were unable to generate high-quality questions at the *Apply* and *Create* levels, and the study lacked prompt diversity, as the same zero-shot prompt was used for all models.

[7] https://medium.com/@seanbetts/peering-inside-gpt-4-understanding-its-mixture-of-experts-moe-architecture-2a42eb8bdcb3, accessed 05/09/2025

[MDS25] have described the gap that frameworks like Bloom's Taxonomy are often not fully integrated into the systematic assessment of generated questions, or they have not employed comprehensive methods to determine the completeness and relevance of the questions. Their research explored the potential of LLMs – such as several GPT and LLaMA models plus Gemini Pro – for AQG by comparing zero-shot prompting against eight-shot prompting. By this, they generated a complete set of questions aligned with Bloom's Taxonomy using school textbooks. Adopting and adapting the evaluation metrics by [Elk+23], the teachers-based evaluation indicated that eight-shot prompting improved the quality and Bloom adherence of educational questions. GPT-4 was identified as the best performing model under both prompting conditions. As future work, the authors suggested exploring 16- and 32-shot prompting, which could further enhance the quality of generated questions. However, they have not addressed the subjective nature of human evaluation, because 28 teachers were involved in the evaluation process, which could lead to a more diverse but less biased assessment of the generated questions.

The authors of [ARA24] also stated that the quality and pedagogical effectiveness of LLM-generated questions have not been thoroughly evaluated. Furthermore, LLM-based question generation, which aims to meet educational objectives, is unexplored. The assessment of these questions plus the comparison with expert-generated ones is the focus of their study, primarily using ChatGPT. While disregarding frameworks such as Bloom's Taxonomy, the study focused on textbook-based questions and different prompting strategies: zero-shot, enhanced zero-shot, few-shot, and few-shot with CoT, where each strategy was tested with 6 variations. Human evaluators – experts and crowdsourced workers – were employed to assess the generated questions. The results revealed that experts preferred human-generated questions from textbooks, while the crowdsourced evaluators have shown comparable preferences for both human- and LLM-generated questions. However, the authors also highlighted the issue of hallucination, which remains a challenge in AQG. They suggested that future work should focus on integrating systems to verify questions against source texts. Moreover, the study had a small – but unknown – number of annotators. They suggested a broader range of LLMs and a higher variety of questions to be included in future work.

Traditional AQG systems struggle to generate questions at higher cognitive levels, and they often lack control over question complexity and difficulty [Blo+23]. Prior work has focused on incorporating pre-formulated answers into the question generation process. Addressing this, the authors developed Angel, a tool that utilizes GPT-3.5 for generating questions and answers based on learning material. The system employs instructional prompts for AQG and evaluation – including few-shot prompting, descriptions for the Bloom levels, and textbook content to produce questions categorized by difficulty (easy, medium, hard). On the one hand, the evaluation of the generated questions was conducted using GPT-3.5 again as an evaluator with a CoT strategy, focusing on metrics such as *Clarity, Relatedness, Importance,* and *Answerability*. On the other hand, human evaluation was also performed, coupled with GPT-3.5 for assessing the resulted Bloom levels of the questions. While the LLM-based evaluation indicated that Angel generated more questions at higher cognitive levels – in contrast to the contestant systems – the human evaluation underscored

that the majority of the questions were still classified as *Remembering*. The authors concluded that while Angel demonstrated potential for AQG, further work and human evaluation are needed to analyze the results more accurately.

Hybrid Architectures. Researchers have also developed more complex systems that integrate LLMs with other components in a coordinated fashion. These hybrid architectures often aim to leverage the strengths of different techniques to achieve more robust and diverse question generation.

In contrast to the previous studies, [Dou+24] addressed the gap of generating Multiple-Choice Questions (MCQs) in programming education, which is linked with much effort. LLMs were not thoroughly explored for this purpose, since evaluating systems which generate MCQs is limited. The authors conducted a comparative study of MCQs generated by GPT-4 versus those created by humans. In addition to the LLM, few-shot prompting, course context and learning objectives aligned with Bloom's Taxonomy levels – realized through fine-tuned BERT classifier – were used for AQG. The resulted questions were comparable to human-generated ones in terms of clarity and learning objective coverage. Nevertheless, limitations included potential bias from human evaluators, the focus on programming questions, and the need to consider the alignment between the question and the learning content, as depicted similarly in [ARA24] above.

Focusing on personalized learning, [HWY24] described that both efficient and effective generation of MCQs is essential in modern teaching, since the manual generation is depicted as resource-intensive and time-consuming. Their system, MCQGen, utilizes the GPT-4 model, RAG, and advanced prompting techniques to automate the creation of personalized MCQs while accessing an external knowledge base. The self-refinement and CoT prompting were employed to enhance the quality of the generated questions. Assessing the questions based on *Grammar, Answerability, Diversity, Complexity,* and *Relevance*, the outcomes between the three LLM-based evaluators and three lecturers were mixed. In this work, especially the *Complexity* and *Diversity* metrics were lacking in both evaluation methods, suggesting that prompt optimization and expanding the knowledge base could address these limitations. In addition to that, the usage of only one LLM coupled with the focus on computer science questions in English language were also limitations of this work.

3.3 Research Gaps Addressed

Based on the conducted literature review, presented in the previous sections of the thesis, there are several research gaps this work aims to address:

Outdated Models. Current studies rely on LLMs from 2023 or earlier, such as GPT-4, GPT-3.5, and LLaMA2 [SCS24; MDS25; ARA24]. These evaluations lack in capturing the capabilities of recent models that incorporate advanced *reasoning* mechanisms to think before answering. To address this, the thesis investigates four current state-of-the-art models, as detailed in Section **IV**. This ensures that the evaluation of AQG capabilities is conducted using the most recent technologies, resulting in a more up-to-date understanding of LLMs' potential in educational contexts.

Limited Content Adherence Evaluation. A persistent challenge in AQG is the problem of hallucination, which is unacceptable in education [YZ24]. LLMs can generate misleading or irrelevant information which are not directly supported by e.g. the source text [ARA24; DWK24]. While some studies mention this issue, the content adherence between a provided context and the generated questions still remains underexplored, as stated in the limitations of e.g. [Dou+24; ARA24]. This thesis investigates this problem in RQ1 – conducting both quantitative and qualitative analysis. This kind of evaluation is crucial for generating reliable educational questions that can be answered accurately using the source material, as diverse materials were used in e.g. [SCS24; MDS25; Blo+23]. Highlighted in related research [BFL24], it is important to inject knowledge directly into prompts to contrast hallucinations and give educators control over the source text.

Bloom's Taxonomy Integration and qualitative Evaluation. While several studies have incorporated Bloom's Taxonomy into AQG evaluation [SCS24; MDS25; Blo+23], most research lacks comprehensive analysis of how different question formats perform across various cognitive levels. Existing work primarily focuses on adherence to specified Bloom levels without examining the relationship between question types (e.g. Multiple-Choice vs. Open-Ended) and their pedagogical effectiveness at different cognitive complexities. In addition to that, the LLMs used struggle with higher cognitive levels [Blo+23], such as *Apply* and *Create* [SCS24]. Since these models are far from the state-of-the-art, there is an opportunity to explore how the latest LLMs perform in generating questions across Bloom's revised Taxonomy, while several publications did not include this framework into their publications [ARA24; Vu+24; HWY24].

Furthermore, many studies rely heavily on automated metrics or limited human evaluation [ZWC25; Blo+23; LZ24] – often with few annotators –, which might not capture the full pedagogical value of generated questions. This thesis addresses this mixed gap primarily through RQ2 by conducting a comprehensive analysis of question quality, including three prompts (predefined question type, Bloom level, or both), employing expert-based evaluation with inter-rater agreement measuring. The evaluation incorporates rubrics that assess not only Bloom level adherence but also other qualitative criteria, providing a proper understanding of LLM capabilities in AQG. The rubrics for both experiments are proposed in Section **IV** and are based on the work [ML24].

This set of research gaps highlights the need for a comprehensive evaluation that can evaluate modern LLMs in AQG using current methods. The proposed framework in the section below aims to bridge the research gaps and provide a foundation for future research in AQG and LLMs.

IV Experimental Design

Two experiments were conducted to evaluate the performance of a selected set of LLMs – using solely *reasoning* models – in generating questions. The first experiment primarily focused on the **RQ1**, which aimed to assess the content fidelity of the LLMs between the generated questions and the diverse source materials. The second experiment was designed to answer **RQ2**, which focused on analyzing the relationships between the LLMs, given questions formats and given levels of Bloom's revised Taxonomy [Kra02].

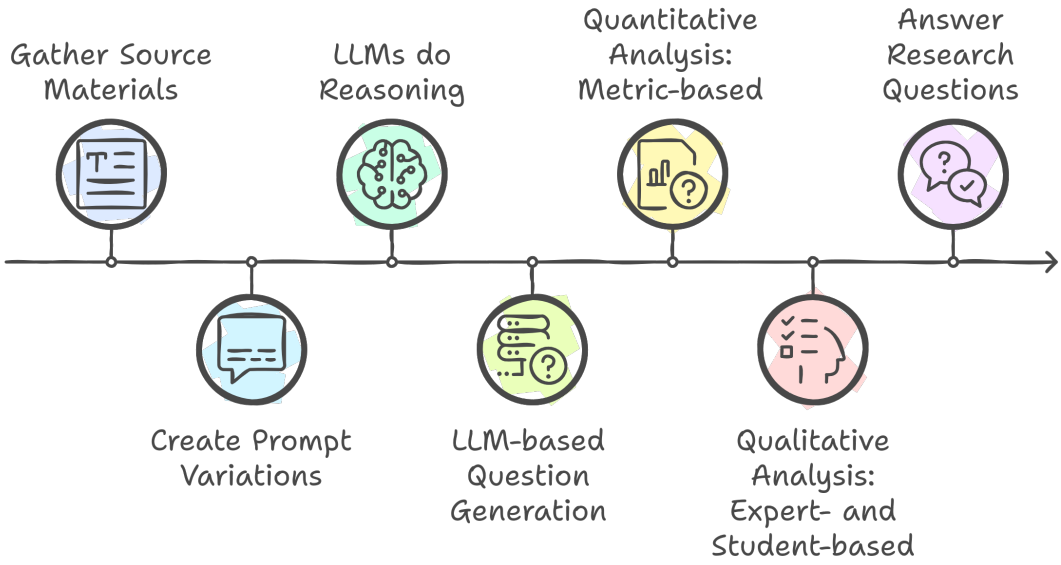

Figure 3: Experimental Pipeline[8].

4.1 Selection of Resources

To ensure the experiments were well-structured and relevant, a variety of resources were selected. This covers the source materials and the LLMs used in the experiments for AQG.

Selection of Source Materials. Focusing on the ISO-OSI model, the following materials, starting from Appendix **D**, were diversely used to generate the experiments' questions:

- **Lecture Slides (extracted Parts)**: Dealing with reference architectures, the ISO-OSI model is presented in detail by the primary thesis supervisor. The lecture slides are in German and were used for AQG for both experiments.
- **Transcript (Lecture's Video Parts)**: The transcript of the extracted slides is in German and was also used to generate questions for the first experiment.
- **Book's Excerpt (from "Computer Networks")** [TW13]: This book is in English and was used to generate questions for both experiments, with a detailed description of the ISO-OSI model, including all seven layers.

[8] created with https://www.napkin.ai/, accessed 08/05/2025

To properly challenge the LLMs in their multilingual capabilities, each source material was kept in its original language: German for the lecture slides and its transcript, the book excerpt in English.

Selection of LLMs. A total of four LLMs were selected for both experiments, which belong to the current state-of-the-art in the field of LLMs and are publicly available to certain degrees, and incorporate *reasoning* capabilities. The selected models are: Anthropic Claude 3.7 Sonnet [Ant25], DeepSeek R1 [Dee+25a], Google Gemini 2.5 Flash [Kav25], and OpenAI o3 [Ope25a].

o3 and Claude 3.7 Sonnet were accessed through their Python APIs[9,10], incurring costs per usage. This access method was necessary as the chosen models were not available to the required extent through other means. In contrast, Gemini 2.5 Flash was utilized via Google's Python API[11], which offers a free usage tier. To manage costs while maintaining a robust set of LLMs, DeepSeek R1, a comparatively less expensive yet highly capable LLM, was accessed free of charge via its website[12].

4.2 Methodological Approach

The method employed for the two experiments is outlined here. It details the specific procedures for generating questions, including the variations in source materials, LLM prompting strategies, and the targeted number of questions for each experimental condition.

Experiment 1a: Content Fidelity. For the first experiment, questions were generated using the three selected source materials to compare the questions' quality. All four state-of-the-art LLMs were employed. Each model was prompted using both a simple and a complex prompt. To prevent context spillover and ensure focused questioning, one question was generated per prompt for each of the seven layers of the ISO-OSI model, with the context for each layer being introduced in each distinct run. This methodology resulted in a total of

$$4 \text{ LLMs} \times 3 \text{ input sources} \times 7 \text{ layers} \times 1 \,{}^{\text{question}}\!/_{\text{prompt}} \times 2 \text{ prompts} = 168 \text{ questions}. \tag{1}$$

Experiment 1b: Error Handling. A subsequent run focused on error handling. The four LLMs were again utilized with the same prompts. For this experiment, a single input source, specifically the lecture script – chosen for its concise and precise contextual information – was used. Each of the seven layers of the ISO-OSI model within this source material was intentionally manipulated to introduce errors, as to be seen in Appendix **4.4**. Seven questions were generated per prompt, one for each manipulated layer and run. This resulted in a total of

$$4 \text{ LLMs} \times 1 \text{ input source} \times 7 \text{ layers} \times 1 \,{}^{\text{question}}\!/_{\text{prompt}} \times 2 \text{ prompts} = 56 \text{ questions}. \tag{2}$$

[9] https://github.com/openai/openai-python, accessed 05/09/2025
[10] https://github.com/anthropics/anthropic-sdk-python, accessed 05/09/2025
[11] https://github.com/googleapis/python-genai, accessed 05/09/2025
[12] https://chat.deepseek.com/, accessed 05/09/2025

Experiment 2: Question Formats and Bloom's Taxonomy. The second experiment focused on the generation of varied question formats and their alignment with Bloom's revised Taxonomy. Contextual information for this experiment was derived from all ISO-OSI model layers together, utilizing the precise lecture Script from Appendix **4.1** by the primary thesis supervisor. The four selected LLMs were employed in three distinct runs, each designed to explore different prompting parameters related to question type and cognitive complexity:

1. *Question Type Specification*: Prompts specified only the desired question types (Multiple-Choice and Open-Ended). For the chosen layer, each LLM was tasked six times to generate a question without mentioning a certain Bloom level for each of the two question types, resulting in

$$4 \text{ LLMs} \times 2 \text{ question types} \times 1 \, {}^{\text{question}}/_{\text{run}} \times 6 \text{ runs} = 48 \text{ questions}. \tag{3}$$

2. *Cognitive Level Specification*: Prompts specified only the target cognitive levels according to Bloom's Taxonomy. For the chosen layer, each LLM generated one question for six runs to get a question for each of the Bloom levels. In total, these are

$$4 \text{ LLMs} \times 1 \, {}^{\text{question}}/_{\text{level}} \times 6 \text{ Bloom levels} = 24 \text{ questions}. \tag{4}$$

3. *Combined Specification*: Prompts specified both the question types (Multiple-Choice and Open-Ended) and the cognitive levels. Similar to the first run, this resulted in

$$4 \text{ LLMs} \times 2 \text{ question types} \times 1 \, {}^{\text{question}}/_{\text{level}} \times 6 \text{ Bloom levels} = 48 \text{ questions}. \tag{5}$$

Data Collection. The data was primarily collected using the LLMs via their APIs, except DeepSeek being prompted on the website. The other 3 models were prompted automatically via Python to generate questions based on the given source materials. The generated questions were stored in a directory structure, particularly based on the experiment, the LLM and the source material. The question files for each prompt were stored in distinct *.txt* files.

Evaluation Plan. The evaluation will be conducted in two phases, one for each experiment. These employ a multi-modal assessment combining automated metrics and expert annotation – the primary supervisor, staff members, and students – to ensure a comprehensive quality assessment.

- **Experiment 1 (Content Fidelity and Error Handling):**
  - *Quantitative Analysis:* In this stage, the *Semantic Similarity* between each generated question and the desired source material will be evaluated. Instead of relying directly on BERTScore [Zha+20], the cosine similarity between the embedding vectors – representing the source material and the generated question each – will be computed. These embeddings are obtained using a cross-lingual RoBERTa-based sentence transformer[13], which produces semantically meaningful and comparable sentence representations [RG19].

[13] https://huggingface.co/T-Systems-onsite/cross-en-de-roberta-sentence-transformer, accessed 05/09/2025

Novel metrics such as *BLEURT* and *RQUGE* were not employed since, on the one hand, the structure of the source material and the question differ too much, and on the other hand, gold standard answers are not available for comparison.

Moreover, other metrics such as Perplexity are unsuitable for this task, as "the output value is based heavily on what the text and the model was trained on. This means that perplexity scores are not comparable between models or datasets"[14]. Instead, to properly analyze the content adherence on a quantitative level besides the *Semantic Similarity*, the *Adherence Score* will be introduced in this thesis as an alternate quantitative scoring method. It will be assessed by prompting Claude 3.7 Sonnet and OpenAI o3 for every question with its desired source material to obtain acceptable results, using the mean of both LLM values. This results in a value between 0 and 1 for each question, presenting the adherence to the source material.

- *Qualitative Analysis:* This phase involves a blind-test with expert-based evaluations of the generated questions. Five experts, including the primary thesis supervisor and four scientific staff members from the same university department of Information and Communication Services, will rate a sampled question set using the concise rubric in Table **3**. To analyze the inter-rater agreement between the experts, Fleiss' Kappa will be calculated, which is suitable for analyses carried out by at least two raters [Fle71].

  The thesis' analysis adopts the rubric from [ML24] – based on criteria by [Moo+22] –, which maintains a maximum score of 50 points per question. However, instead of the *Bloom's Level* criterion, the *Correctness*, the *Value* from the lecturer's perspective, and the *Language* criteria were added to address the objectives of this experiment.

  Since Experiment 1b examines the adherence to manipulated input texts, the *Correctness* is then replaced by *Manipulation Handling*, respectively. To distinguish between the *Clarity* and *Language* criteria, clarity focuses on content clarity and logical comprehension, while language assesses linguistic complexity and precision of expression.

- **Experiment 2 (Question Formats and Bloom's Taxonomy):**
  - *Qualitative Analysis:* As the primary focus of this experiment lies on qualitative assessment, any quantitative evaluation is discarded. To simulate a proper blind test for this experiment, a human-based analysis will be conducted as well. Three students of the secondary thesis supervisor – from the Institute of School Pedagogy and Educational Research – will rate a sampled question set from the second experiment. These students will evaluate the questions using the concise rubric in Table **4**, based on the first rubric structure. Unlike Experiment 1, the *Correctness* is replaced by the *Bloom's Level* criterion, as [ML24] used this criterion before. Inter-rater agreement via Fleiss' Kappa will also be calculated for this experiment.

[14] https://huggingface.co/spaces/evaluate-metric/perplexity, accessed 06/01/2025

Table 3: Experiment 1 Evaluation Criteria.

Criterion	Description	Scoring
Relevance	Is the question relevant to the topic of the source text?	0-10p
Clarity	Is the question easy to understand & the presentation logical and clear?	0-10p
Answerability	Are students likely able to answer the question based on the provided material?	0-10p
Challenging	Is the question challenging? Does it encourage students to think actively?	0-10p
Value	Is the question considered professionally, didactically, or technically meaningful by the instructor?	0-10p
Language	Is the language used by the system sufficiently precise and comprehensible?	0-10p
Correctness	**For Experiment 1a:** Is the question factually correct & adhering to the source text?	• Correct: 10p • Partially correct: 5p • Otherwise: 0p
Manipulation Handling	**For Experiment 1b:** Does the question adhere to the manipulated input text content?	• No Adherence / Model detects Manipulation: 10p • Partially adheres: 5p • Otherwise: 0p

Table 4: Experiment 2 Evaluation Criteria.

Criterion	Description	Scoring
Relevance	Is the question relevant to the topic of the source text?	0-10p
Clarity	Is the question easy to understand & the presentation logical and clear?	0-10p
Answerability	Are students likely able to answer the question based on their knowledge level?	0-10p
Challenging	Is the question challenging? Does it encourage students to think actively?	0-10p
Value	Is the question considered professionally, didactically, or technically meaningful by the instructor?	0-10p
Language	Is the language used by the system sufficiently precise and comprehensible?	0-10p
Bloom's Level	**When Bloom level is specified:** Does the question adhere to the given Bloom level?	• Correct: 10p • Off by one: 5p • Otherwise: 0p
	When no Bloom level is specified: Which Bloom level does the question demonstrate?	• Creating: 10p • Evaluating: 8.5p • Analyzing: 7p • Applying: 4.5p • Understanding: 3p • Remembering: 1.5p

Since the qualitative analyses for Experiments 1 and 2 shall be valid and as unbiased as possible, these are conducted as certain blind tests. To achieve this, the evaluators will not be aware of the used methods to generate each question. This involves the model's name, the kind of prompt used, the specified question type and the desired Bloom level for each question. Additionally, to properly analyze criteria such as *Relevance*, *Answerability* and *Correctness*, the evaluators will be provided with the source material each question is based on.

Furthermore, to properly score the *Bloom's Level* – based on the guideline in Table **4**, the evaluators shall return solely the Bloom level they perceive for each question. The proper scoring then will be done automatically based on the returned Bloom level, since the raters will not be aware of the desired Bloom level for each question. This is focused because various questions of the sampling set were generated based on prompting the desired Bloom level.

Moreover, the Details about the Implementation and Execution for the entire experimental setup is presented in Section **V** below. The emerged results and discussion of the evaluation will then be presented further in Section **VI**.

V Implementation & Execution

This chapter details the practical implementation of the experimental design outlined in Section IV. It covers the technical setup, data generation pipeline, prompt engineering strategies, and evaluation methodologies that address the research questions regarding LLM capabilities in educational question generation.

5.1 Technical Implementation

Development Environment and Tools. The experimental pipeline was implemented using Python 3.10.14 as the primary development platform, chosen for its robust ecosystem of machine learning libraries and API integration capabilities. The implementation based on the thesis progress follows a pipeline with specialized components for API interactions, prompt engineering, data processing, and analysis.

Core System Components. The system primarily used a comprehensive API integration layer implemented through the **api_calls.py** module, which manages the interaction templates of the desired LLM providers using their official Python SDKs. By this, the OpenAI Python SDK for o3 access[9], the Anthropic SDK for Claude 3.7 Sonnet integration[10], and the Google AI Python SDK for Gemini 2.5 Flash[11] could be integrated easily.

To manage the API configurations for this, the **api_config.py** module was implemented, which handles environment variable loading and validation for each provider. The prompt engineering framework, implemented in **prompt_utils.py**, provides dynamic prompt generation capabilities based on both experiments while integrating templates from the prompts' directory. Based on this module, the source materials, the information for each Bloom level etc. were added to the prompts. Moreover, the data processing was done by using **file_utils.py** to load and save files.

5.2 Experimental Procedure

Prompt Engineering Strategy. The prompting strategy adheres to several known methods. Since there are two experiments to be examined, the prompts were designed for each experimental phase separately. Since there are no gold standard questions available, the prompts are solely based on the zero-shot approach, which allows the LLMs to generate questions based on the provided content without prior example questions.

For Experiment 1, primarily dealing with the content adherence of the LLMs, two approaches were implemented: a simple prompt strategy detailed in **exp1_common_prompt.md**, and a complex prompt approach outlined in **exp1_complex_prompt.md**. The common prompt includes a minimized CoT guideline with the most important instructions to ensure a proper question generation, including a valid output format, and the desired source material for each AQG run. The complex prompt, on the other hand, includes a more sophisticated prompting strategy with role assignment – telling to be an expert in generating cognitively demanding questions –, detailed

CoT instructions for the question generation, such as carefully examining the source material and identifying the important aspects about the given ISO-OSI layer information. Moreover, docstring markers were used to highlight the source material and the question format for proper clarity in the prompt. Based on these two prompts, the AQG was conducted for both content adherence and handling textual manipulations.

Using the complex prompt, another prompt for Experiment 1a was created to generate questions without source material, which is shown in **exp1a_complex_prompt_no_source.md**. This approach intends to examine if the quantitative analysis of the generated questions is still valid by comparing the quantitative results of the questions generated with and without source material – since the latter approach is compared with every source material used to ensure a proper comparison. This viewpoint will be discussed in Section **VI**.

Experiment 2 focuses on three prompt variations – using the key methods from the Experiment 1 prompts – that include either question format specifications, Bloom level information, or both, to examine the influence of these factors on the quality and the final Bloom level of the resulted questions. The type-focused prompt variation in **exp2_type.md** specifies a question format for each question, resulting in the same amount of questions according to the same amount of Multiple-Choice and Open-Ended questions. These questions are generated without imposing cognitive level constraints, so the final cognitive level of each question is decisive for educational effectiveness. The Bloom-focused prompt variation in **exp2_bloom.md** targets specific cognitive levels without a fixed question format – known from the Experiment 1 prompts –, focusing on the alignment with the given Bloom level instead. The combined specification approach in **exp2_both.md** integrates both requirements to examine their interaction effects, allowing for a comprehensive analysis of how question format and cognitive level specifications influence the generated questions. The Bloom level descriptions and associated verbs used in these prompts are detailed in **bloom.md**, which are based on the original descriptions from [Kra02] and the verb lists from Table **1**.

Automated Experiment Execution. The question generation pipeline is initialized and executed via **main.py**, which coordinates both experiments mentioned – while the module **question_generation.py** included the source code for each experimental phase. Several constants and configurations were managed by **constants.py**. Prior to experiment execution, the crucial token management was analyzed through the **check_truncation.py** file. This step was needed to validate that the multilingual source material segments did not exceed the model's 512-token limit for semantic similarity calculation. For this, semantically comparable sentence embedding vectors were created by a cross-lingual RoBERTa model[13], which is an individually modified sentence transformer [RG19]. This preprocessing step was essential because the model would later be used for semantic similarity evaluation, so the material segments had to be within the token limit. Later on, this file was refined to also analyze the amount of tokens in each generated question to analyze if the LLMs exceed the token limit by performing AQG.

Data Management and Organization. The resulting data follows a systematic directory structure designed for clarity and reproducibility, as good as it can work with LLMs. The source materials are organized into separate folders each, enabling proper access and processing. The questions were stored in distinct TXT files, while the file path reflects experiment type, model, source material, prompt variation, and question identifiers – e.g. OSI layer or Bloom level –, to conduct automated analysis and manual review. Evaluation data is maintained in structured CSV files containing automated analysis and expert annotations, ensuring comprehensive assessments.

5.3 Evaluation Implementation

Multi-Modal Assessment. The analysis steps include quantitative and qualitative evaluation of the generated questions. Besides the manual qualitative expert-based evaluation, the quantitative analysis was implemented in **analysis_quantitative.py**. It follows the *Semantic Similarity* evaluation between question and source material embeddings. Furthermore, the LLM-based *Adherence Score* assessment – returning a value between 0 and 1 for each question – was conducted to provide an alternate quantitative analysis of the LLMs' content adherence to the source material. The prompt for this manner of quantitative evaluation is shown in **exp1_adherence_eval.md**.

Moreover, the qualitative analysis consists of sampled expert ratings. The randomized extraction of the questions is set up with a seed via **sampling.py**. This resulted in $\frac{64}{224}$ questions in Experiment 1, and $\frac{40}{120}$ questions in Experiment 2, which offers a proper question set for expert-based evaluation. Unfortunately, this script needed a refinement to finally allow reproducibility across different systems. This implies that the formerly created sample sets – used in the qualitative analyses – cannot be reproduced. To properly analyze the sampled questions, the desired rubric variations from Tables **3** and **4** were used, also detailed in **exp1a_rubric.md**, **exp1b_rubric.md**, and **exp2_rubric.md**.

Whether all analysis results are available, the analysis results are processed by several evaluation files using Jupyter Notebook. With these notebooks – **evaluation1_quan.ipynb**, **evaluation1_qual.ipynb**, and **evaluation2_qual.ipynb** –, numerous tables and plot figures are available to interpret the determined data. Given that five experts for Experiment 1 – the primary supervisor and four staff members – and three students from the secondary supervisor for Experiment 2 were identified for the analysis steps, it is crucial to examine the reliability of the ratings each rater has given. To establish this, the calculation of inter-annotator agreement via Fleiss' Kappa [Fle71] is needed, set up in the notebooks handling the qualitative analysis for each experiment.

Implementation Challenges and Solutions. Some technical challenges had to be addressed. The token limitation of the chosen modified RoBERTa model, 512 tokens $\approx$ 380 words, required manual shortening of the layer segment texts if necessary to ensure a suitable performance of the model creating the comparable sentence embeddings. The multilingual processing challenges were tackled by the careful selection of this cross-lingual sentence transformer, which enabled effective handling of both the German lecture materials and the English textbook content.

The developed AQG pipeline proved to be particularly challenging, since it required careful consideration of prompting, model selection, and evaluation methods. In addition to that, the implementation of the effective automation of the generative (excluding DeepSeek) and evaluation processes required a lot of attention to detail. This resulted in a novel experimental run for the first experiment. Since the output format of the LLMs was too diverse in the common prompt – due to the omission of the output format constraints –, a new question generation run had to be conducted, which refined the prompts to ensure a consistent output format. This refinement was necessary to ensure that the generated questions could be properly evaluated and compared without any biases based on the output format. The former data were archived instead of overwritten, because this insight was a valuable lesson for the prompting implementation and the overall AQG process.

VI Evaluation of Results

This section presents the findings from the two experiments detailed in Section IV. The quantitative and qualitative analyses are discussed, addressing the research questions regarding content fidelity, error handling, question format generation, and alignment with Bloom's Taxonomy.

6.1 First Experiment

The first experiment of the thesis focused on evaluating the content fidelity of questions generated by different LLMs using two distinct prompt strategies: a common prompt and a complex prompt. In this experiment, the goal was to determine whether the complexity of the prompt influences the content fidelity and final scoring of the generated questions, addressed in the first research question:

RQ1: *To what extent do LLMs adhere to the content of diverse provided instructional texts when generating questions?* Experiments 1a and 1b follow both quantitative and qualitative approaches each to evaluate the generated questions properly. The quantitative analysis solely focuses on the analysis of content fidelity, while the qualitative analysis assesses the quality of the generated questions based on a certain rubric. This rubric also covers the question's *Correctness*, which provides an underlying criterion for assessing content fidelity on expert-level.

Quantitative Analysis

Two metrics were employed to assess content adherence: the calculation of *Semantic Similarity* using cosine similarity between resulting semantic vectors – one for the source material file, and one for the question file – from the embedding model, and LLM-based *Adherence Score* analysis between these files as an alternate approach to evaluate content fidelity.

Experiment 1a – Content Fidelity. The manner of evaluation revealed several key patterns in the LLMs' behavior. The question type for each question was not explicitly constrained, but to provide a valid analysis – especially for the experts –, the questions' output formats were limited to either Open-Ended or Multiple-Choice questions. This resulted in a trend toward MCQs, with only 9 out of 168 questions being Open-Ended in Experiment 1a – with 7 questions from OpenAI, and 2 from Google –, based on the complex prompt. With regard to this finding, the trend was consistent across the LLM set, demonstrating a bias toward structured question formats.

As an introductory step, to observe if both approaches are valid, the LLMs were prompted to generate a question for each ISO-OSI layer without using an input source. Here, a key observation is that the cosine similarity approach did not reliably distinguish between questions generated with or without input sources. As shown in Figure 4, the scores for the ones without a source – maintaining a mean ≈ 0.533 and a median ≈ 0.538 – were quite similar to those using actual source material with means between $[0.496_{\text{transcript}}, 0.637_{\text{script}}]$ and nearby medians at $[0.511_{\text{transcript}}, 0.651_{\text{script}}]$, indicating that cosine similarity is not a robust indicator of content fidelity for these kinds of texts.

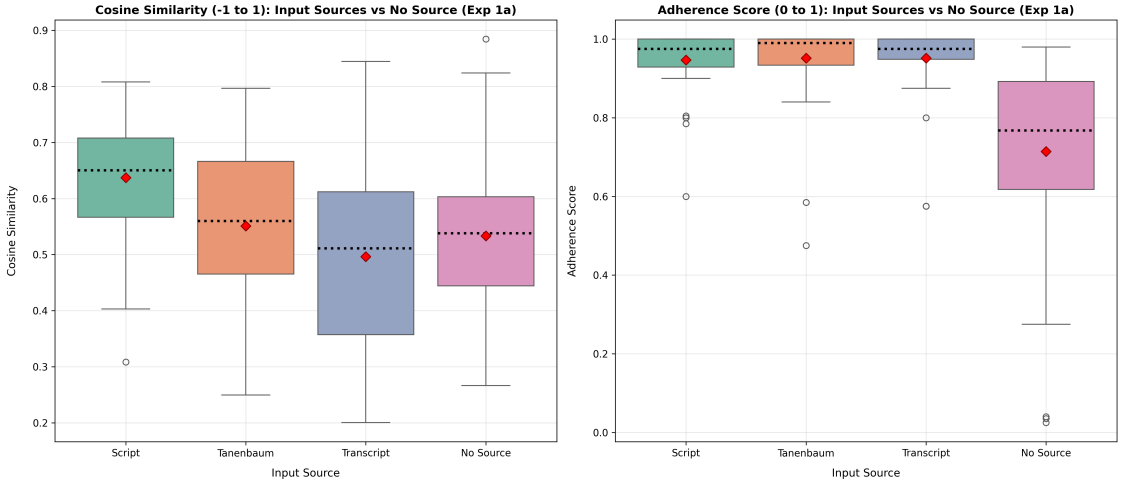

Figure 4: Quantitative Distributions for Input Sources vs. Run without any Source (Exp1a).

In contrast, the *Adherence Score* provided a clearer differentiation. Using no source led to noticeably lower adherence overall in the mean ≈ 0.714, the median ≈ 0.768 and a broader range of scores. Additionally, a certain trend in cosine similarity across the sources is visible in Figures 4 and 5. There is to be seen that the precise lecture script is indicating best results, Tanenbaum excerpts are situated in the middle, and the extensive transcript parts equipped with examples keep the widest range of scores, the lowest mean and median values. This trend appears to be influenced by the amount of words in the source materials rather than content quality itself, limiting its overall usefulness. Moreover, the *Adherence Score* showed few outliers in the box plot, and the scoring in the heatmap shows quite individual LLM performance according to the diverse input sources, overall located above 90%.

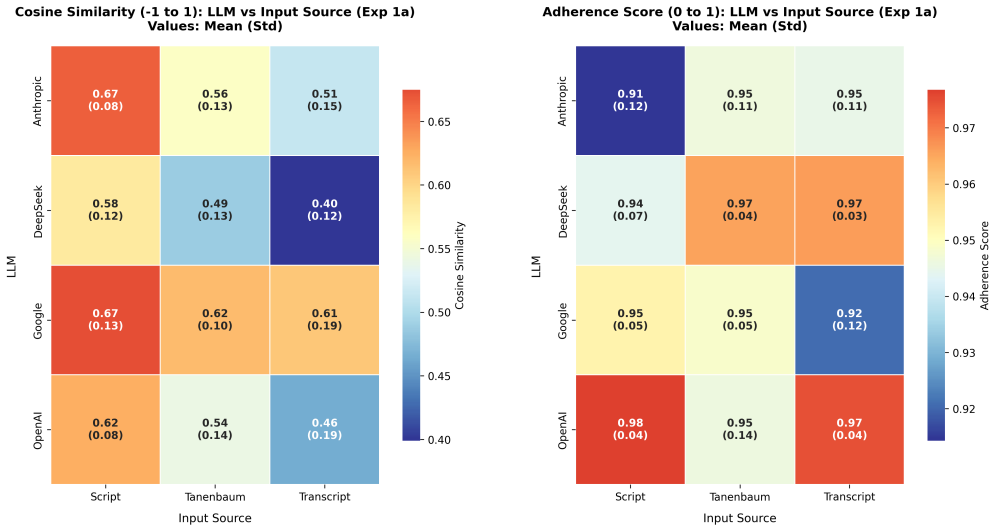

Figure 5: Quantitative Heatmaps for LLM vs. Input Source Performance (Exp1a).

Regarding the LLM-sorted observations, Figure 6 reveals that the cosine similarity scores were lower and also differ a lot from the *Adherence Scores*. Expert results by the primary supervisor, later discussed under *Correctness* in e.g. Figure 11, show that cosine similarity was unreliable for this task, as this did not properly reflect the content fidelity of the questions in Experiment 1a. The *Adherence Scores* in this quantitative analysis, however, provided a quite nuanced view of model performance, showing that DeepSeek and OpenAI were more consistent in adhering to the source content, while Anthropic and Google exhibited slightly lower and more variable scores.

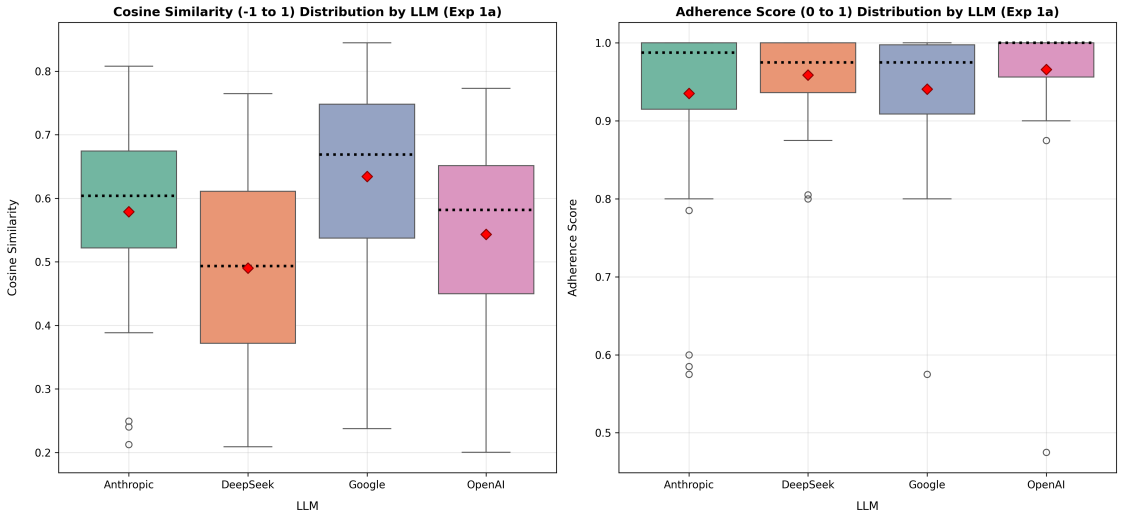

Figure 6: Quantitative Scoring Distributions by LLM (Exp1a).

To get a proper view of the content adherence, the evaluation also included a comparison of the performance of the LLMs against the used prompt type, as shown in Figure 7. Here, the results between cosine similarity and *Adherence Scores* were again significantly different. On the one hand, the hardly comprehensible cosine similarity tends to higher scoring values for the complex prompt. On the other hand, *Adherence Scores* show that the common prompt leads to better results – best for OpenAI (≈ 0.992 vs. 0.94) and DeepSeek (≈ 0.973 vs. 0.945) –, while the complex prompt leads to slightly lower scores instead for all models, especially for Google and Anthropic. This is also indicated by the expert results by the primary supervisor, which indicate that the common prompt is more effective in content adherence than the complex prompt.

These findings suggest that, while cosine similarity may be suitable for short, sentence-level comparisons – as the *Sentence Transformer* expression suggests –, it is less appropriate for evaluating content fidelity between generated questions and also long, more complex educational texts. Consistent context e.g. up to 30 words might improve cosine similarity's reliability, but in this setting, it will be disregarded as a primary metric for Experiment 1a, since the thesis deals with texts which are longer than a single sentence and contain complex information. As an example, [LZ24] used BERTScore for semantic similarity scoring, but between generated and ground truth question-answer pairs. This is not the case in this thesis, since the source texts and LLM outputs differ significantly.

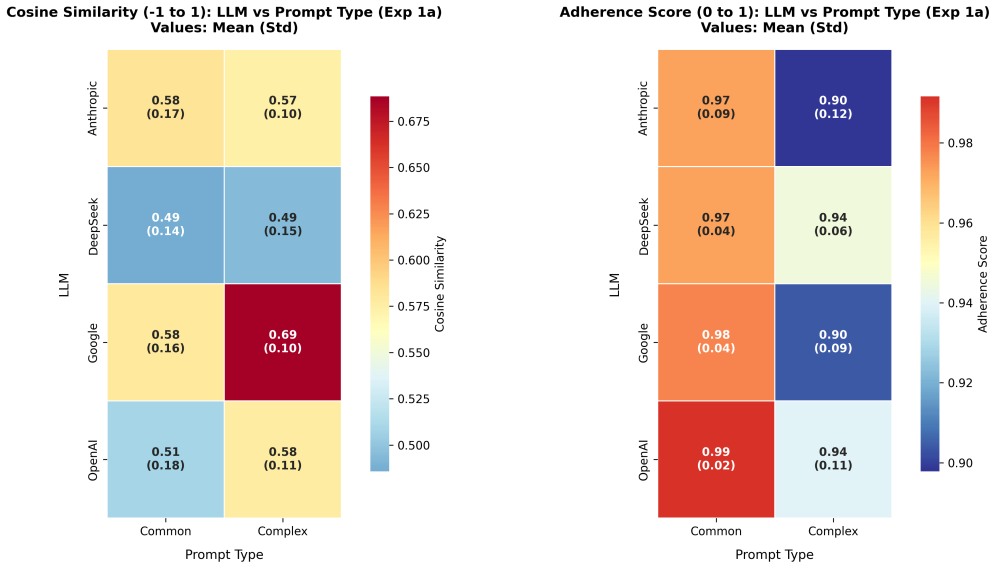

Figure 7: Quantitative Heatmaps for LLM vs. Prompt Type Performance (Exp1a).

Experiment 1b – Error Handling. This sub-experiment focused on evaluating how well the LLMs handle manipulated lecture script content to test their ability to detect errors and what the models generated based on that content. A total of 4/56 resulted in open-ended questions, one time DeepSeek using the common prompt, two times Google plus one time OpenAI both using the complex prompt. Overall, it is worth noting that using the cosine similarity approach here – as to be seen in Figure 8 – is less useful than in Experiment 1a, since the embedding model can not detect if a question by an LLM follows the manipulated content flawlessly, or if the model detects the manipulation and e.g. generated the question inversely.

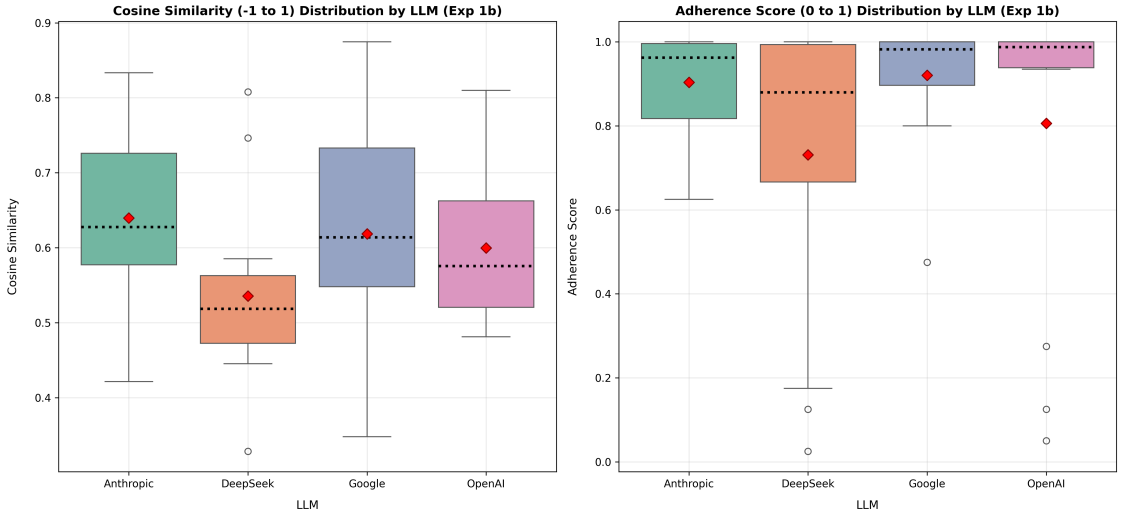

Figure 8: Quantitative Scoring Distributions by LLM (Exp1b).

Some examples – translated into English – of such questions are:

- "Which of the following statements describes a **real core task** of the Data Link Layer in the ISO-OSI model, based on **correct** technical knowledge?" or
- "Which property is **NOT** assigned to the Transport Layer according to the **description?**".

However, the *Adherence Scores* illustrate a proper course in the plots, showing that Google and Anthropic maintain more often to the manipulated content – based on value range and the high means ≈ 0.92 and ≈ 0.904. Instead, OpenAI and especially DeepSeek detected the manipulation in some cases and generated questions where this aspect is recognizable. The rightly present outliers for OpenAI separate its mean from its high median by a value of $0.988 - 0.806 = 0.182$. Moreover, the DeepSeek model – the oldest of the four chosen models – convinced regarding the lowest median ≈ 0.88, mean adherence ≈ 0.731 and a wide range of values, indicating that it often recognized the OSI-layer manipulation.

Furthermore, the used type of prompt had an impact on the results, as shown in Figure 9. In this case, both cosine similarity and *Adherence Scores* indicate that the common prompt led to higher results than the complex prompt, while lower results are the goal in this sub-experiment instead. As stated, the cosine similarity cannot reliably determine if an LLM follows the manipulated content or if it detects it and generates a question based on that text. In contrast to that, the *Adherence Scores* provide more reliable results, showing that the complex prompt led to lower scores for all LLMs. Like in Experiment 1a, DeepSeek and OpenAI were more convincing in their performance – here with lower mean *Adherence Scores* (≈ 0.598 and ≈ 0.619) –, while Anthropic and Google still have shown higher scores. Overall, OpenAI and DeepSeek were the models that recognized the manipulation more often, but in this sub-experiment, the complex prompt led to "better" *Adherence Scores* than the common prompt.

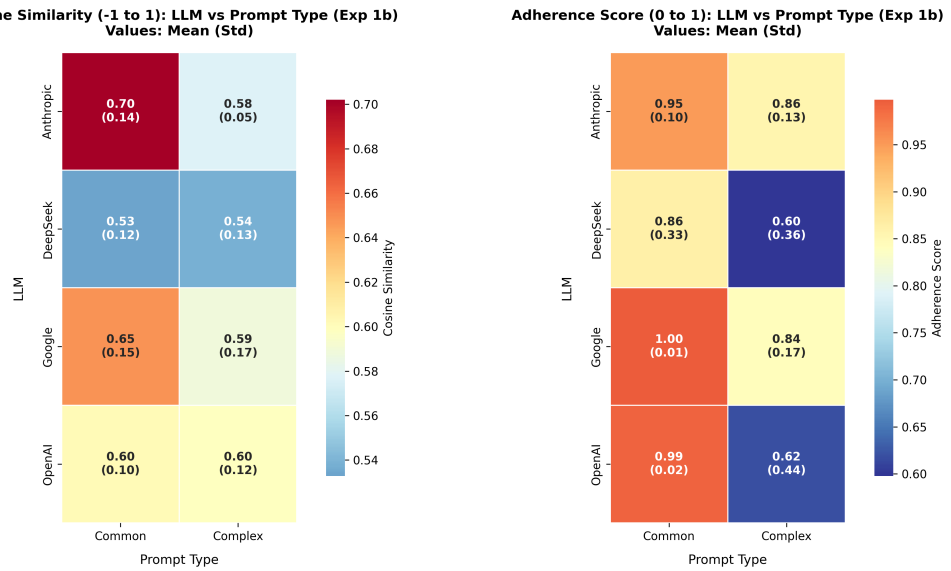

Figure 9: Quantitative Heatmaps for LLM vs. Prompt Type Performance (Exp1b).

Qualitative Analysis – Supervisor-based

To evaluate the generated questions properly and expand the quantitative analysis, a sampled qualitative analysis of Experiment 1a was conducted. This involved a detailed rubric evaluation by the primary thesis supervisor and four department staff members.

Experiment 1a – Content Fidelity. The following criteria were used to assess the questions: *Relevance*, *Clarity*, *Answerability*, *Challenging*, *Value*, *Language*, and *Correctness*. Each criterion was rated on a scale from 1 to 10, with 10 being the best score. The experts were asked to evaluate the questions based on these criteria, and the results were aggregated to provide an overall assessment of the question quality. In this paragraph, the supervisor's results are firstly discussed, while the staff members' results are presented right after.

The supervisor – a professor with extensive lecturing experience – analyzed the questions first, and the staff members were then asked to evaluate the same questions based on the same criteria. In the meantime, the supervisor noticed that the used rubric was not sufficient to cover all qualitative aspects of the questions, leading to an extension of the rubric. By this, the mentioned criteria *Value* and *Language* were added to the rubric, which were not part of the firstly used rubric. The novel *Value* criterion was introduced to rate the meaningfulness of the questions in an e.g. educational context, while the *Language* criterion focused on the linguistic complexity and precision of the questions – since *Clarity* focused the content clarity and logical comprehension.

Regarding Figure 10, certain patterns emerged from the supervisor-based evaluation. Most questions were rated highly *relevant* to the material's topic, with OpenAI and Google as the highest scores. While every model has a median of 10, only the OpenAI model could achieve a mean of 10 as well at *Relevance*, also to be seen at the *Clarity* criterion next to this criterion. Here, DeepSeek was also rated high as a valuable competitor of OpenAI. In contrast, Anthropic and Google both had a wide scale of points and means of 7.33 and 7.64, indicating that these two models were more inconsistent at generating clear and logically understandable questions.

Moreover, the *Answerability* criterion – based on the source text – has shown that OpenAI and DeepSeek were the best-performing models again (with means of 9.33 and 8.5), while Anthropic's and Google's scores (with means of 7.5 and 7.45) were spreading more to lower scores instead. Contrary to this, the questions of the latter two models were rated as more challenging – with Anthropic being most consistent with a mean of 6.67 –, compared to the significantly lower OpenAI and DeepSeek scores in both mean (3.17 and 3.5) and median (both at 2) in this regard.

The educational *Value* criterion indicates that the OpenAI questions were the most valuable ones with a mean and median of 10. The other models were inconsistent in this regard, with noticeably wide ranges of scores, means between 6 and 7 and DeepSeek having the lowest median of 7.

The *Language* criterion has a similar trend like *Clarity*, with OpenAI and DeepSeek performing best with a mean of 10 each. Anthropic and Google were rated lower and more spread across the scale – resulting in less means of 8.25 and 7.67, showing that these models generated questions which were less precise and unnecessarily complex.

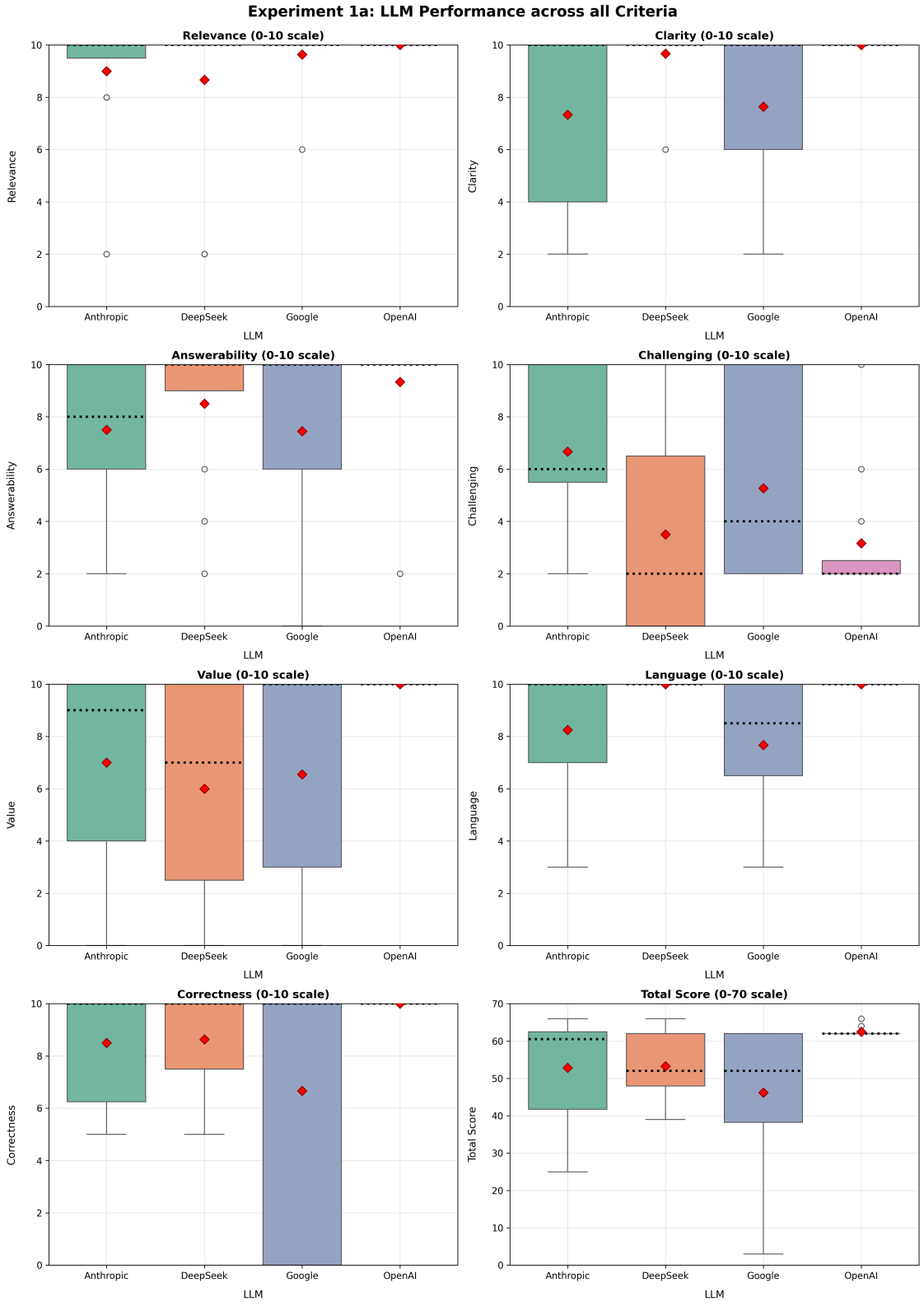

Figure 10: Supervisor-based LLM Scores across every proposed Criterion (Exp1a).

Ultimately, the *Correctness* criterion, to assess the overall content fidelity of the questions, has shown that OpenAI was the best performing model again with a 10/10 mean. DeepSeek and Anthropic are situated right after, with an exemplary question by DeepSeek in Appendix **A.1**. The questions generated by Google were rated all across the board, which shows that the model struggled generating correct questions which shall be solely based on the source material.

Overall, the *Total Score* shows that OpenAI achieved the highest scores with a mean of 62.5/70, a pretty similar median of 62 and a very low range of values. Followed by DeepSeek and Anthropic, the means are situated lower at 53.25 and 52.83 – with more spread values –, while Google was rated lowest with a mean of 46.17 and most spread scores at all.

Since the *Correctness* criterion is the most important one for Experiment 1a, the following Heatmaps in Figure **11** also show certain patterns to be observed. OpenAI still achieved 10/10 points everywhere in both heatmaps, which is a surprising contrast to the other models. Google's correctness has become worse with decreasing complexity of the input source, falling from 10/10 at transcript to 3.3/10 using the precise script. A somewhat similar course can be seen for DeepSeek, but less pronounced. There, the model fell from 10/10 at transcript to 7.5/10 at the script, with also more stable standard deviations. Anthropic's correctness scores were more stable, scaling from 8.8/10 at (tran-)script to 7.5/10 at the Tanenbaum book excerpts.

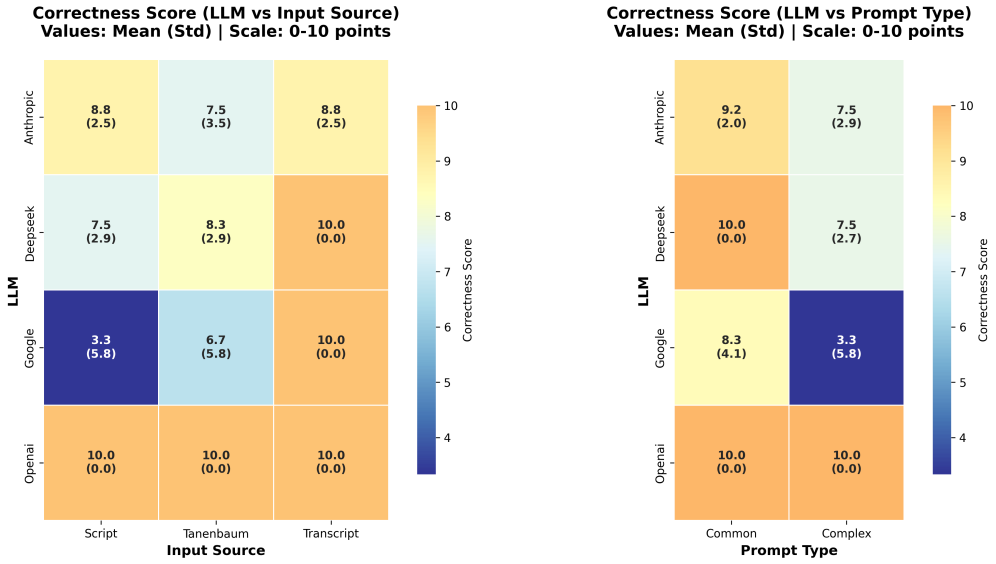

Figure 11: Supervisor-based Correctness Heatmaps for different LLM Combinations (Exp1a).

Regarding the heatmap focusing on the prompt type, there is a trend that the common prompt leads to higher correctness scores than the complex prompt, which was also depicted via the quantitative *Adherence Score* in Figure **7**. OpenAI and DeepSeek achieved 10/10 points at the common prompt, Anthropic is at 9.2 points and Google at 8.3 points. The complex prompt instead had decreased correctness scores, except for OpenAI. DeepSeek and Anthropic were both at 7.5/10 points, while Google solely achieved 3.3 points with high standard deviations for both prompt types.

Since the input source and the prompt type are also worth discussing, the results of the qualitative analysis are summarized in Tables 5 and 6. The tables show the mean, standard deviation, and median for each criterion. First of all, visible in Table 5, almost every criterion has a median of 10/10 points, except for *Challenging*. The mean values differ in contrast to the median.

Overall, there is a trend that the extensive transcript excerpts lead to the best results – especially in means of *Clarity* (9.33), *Answerability* (9.33), and *Correctness* (9.67) –, followed by the precise script and the Tanenbaum book excerpts. The latter two sources were on par at the *Language* criterion with a mean value of 8.88. In contrast, the questions based on the script were most challenging with a mean of 5.5, followed by Tanenbaum here with 5.38. The transcript-based questions were rated lowest in *Challenging*, only achieving a mean of 2.93 points. Furthermore, while the transcript questions were the most accurate regarding the given context to the LLMs with a mean of 9.67, the Tanenbaum's (8.33) and especially script's *Correctness* (7.67) scores were rated lower instead.

Even if the transcript appears to be the best source material for many criteria, the script's mean *Total Score* was the highest with 55.06/70 points, followed closely by the transcript with 54.75 points and the Tanenbaum excerpts with 51.25 points. Regarding the median values, the transcript was rated highest with 62/70 points instead – in contrast to the script (61) and Tanenbaum (61.5) –, but the questions generated based on the transcript lose points due to not being as challenging as the other sources.

Table 5: Supervisor-based qualitative Analysis Results by Input Source (Exp1a). The best means are highlighted in **bold**, while the best medians are *italicized*.

Source	Metric	Relevance	Clarity	Answerability	Challenging	Value	Language	Correctness	Total Score
Script	Mean	9.38	8.50	8.25	**5.50**	7.38	8.88	7.67	**55.06**
	Std	2.03	2.37	2.52	3.54	3.32	2.19	3.72	10.23
	Median	10.0	10.0	10.0	*6.0*	10.0	10.0	10.0	61.0
Tanenbaum	Mean	9.12	8.25	7.12	5.38	7.14	8.88	8.33	51.25
	Std	2.19	3.26	3.79	4.05	4.05	2.19	3.26	14.88
	Median	10.0	10.0	10.0	5.0	10.0	10.0	10.0	61.5
Transcript	Mean	**9.47**	**9.33**	**9.33**	2.93	**7.87**	**9.19**	**9.67**	54.75
	Std	2.07	2.09	1.80	2.49	4.10	1.94	1.29	15.88
	Median	10.0	10.0	10.0	2.0	10.0	10.0	10.0	*62.0*

Besides the input sources, the results in Table 6 show a trend, which was depicted in the quantitative analysis as well. The common prompt led to higher scores in almost every criterion, while the complex prompt was rated lower. The only criterion where the complex prompt was rated higher was *Challenging* in both mean and median values (6.87 vs. 2.50 and 8.0 vs 2.0). Overall, the common prompt outperformed the complex prompt with a noticeable gap in the mean *Total Score* of 60.12 vs. 47.25 points, as well as in the median of 62.0 vs. 48.5 points.

The ratings by the supervisor indicate the following: the complex prompt led to less relevant questions, which were less clear and especially less answerable, did not share the same value by far as the common prompt questions, and the questions were overly complex and less precisely formulated. The *Correctness* to the source material was also rated lower for the complex prompt, which is

crucial for this experiment, but the questions were noticeably more challenging instead. Overall, the transcript-based questions were rated highest, but the lack of challenge in these questions led to a lower total score compared to the script-based questions. The questions created by OpenAI were rated highest in many criteria, while the questions were less challenging. This observation was also reflected in DeepSeek's results, which were also rated high, but struggling with the *Challenging* and *Value* criteria. Anthropic and Google were more challenging, but their questions overall lacked *Clarity*, *Answerability*, *Language*, and *Correctness*.

Table 6: Supervisor-based qualitative Analysis Results by Prompt Type (Exp1a).

Prompt	Metric	Relevance	Clarity	Answerability	Challenging	Value	Language	Correctness	Total Score
Common	Mean	**10.00**	**10.00**	**9.92**	2.50	**9.55**	**10.00**	**9.35**	**60.12**
	Std	0.00	0.00	0.41	1.69	2.13	0.00	2.29	6.56
	Median	10.0	10.0	*10.0*	2.0	*10.0*	10.0	10.0	*62.0*
Complex	Mean	8.61	7.30	6.43	**6.87**	5.48	7.96	7.63	47.25
	Std	2.79	3.23	3.36	3.66	3.92	2.56	3.48	15.93
	Median	10.0	10.0	6.0	*8.0*	4.0	10.0	10.0	48.5

Experiment 1b – Error Handling. A second run was conducted to evaluate LLM-based questions based on manipulated lecture script content. The qualitative analysis of Experiment 1b was done using the same rubric, while *Correctness* was substituted by *Manipulation Handling*. This was introduced to assess how well the LLMs handled the manipulated lecture script content. The same five experts were asked to evaluate these questions. First, the thesis supervisor analyzed the questions, followed by the staff members regarding the **next** paragraph.

Unfortunately, the supervisor's evaluation in Experiment 1b revealed that the rubric was not sufficient for this assessment, because the contextual material is manipulated. Based on this, the *Manipulation Handling* criterion was solely analyzed, and the results are presented in Table 7. If this issue was noticed earlier, the questions could have been evaluated by solely this single criterion, resulting in creating a bigger sampling set, where e.g. the full set of questions could have been analyzed. However, the sampled set of questions is too small to draw any significant conclusions, since certain questions where the models noticed the manipulation were not rated by the experts.

Table 7: Supervisor-based Manipulation Handling Analysis by LLM and Prompt Type (Exp1b).

		Manipulation Handling		
LLM	**Prompt Type**	**Mean**	**Std**	**Median**
Anthropic	Common	0.0	0.0	0.0
	Complex	**2.5**	3.54	*2.5*
DeepSeek	Common	2.5	3.54	2.5
	Complex	2.5	3.54	2.5
Google	Common	0.0	0.0	0.0
	Complex	**2.5**	3.54	*2.5*
OpenAI	Common	0.0	0.0	0.0
	Complex	0.0	—	0.0

In this table, there are results based on 16/56 questions, two for each prompt type and LLM. The limited sample size led to several observations. The *Manipulation Handling* criterion was rated with a mean and median of 2.5/10 or 0, commonly indicating that the models did not handle the manipulation well. The values of 2.5 illustrate that one of the two questions was rated with 5/10 points, while the other one was rated with 0/10 points. 5/10 points show that the systems still followed the manipulated content, but in the formulation of the questions, there was a hint that the manipulation was noticed.

Anthropic and Google were rated with a mean of 2.5/10 for the complex prompt each, with Google's 5/10 question slightly distancing from the manipulated script (Appendix **A.5**). The questions based on the common prompt were rated with 0/10 points for both models, with Google providing a proper example of following the manipulation (Appendix **A.6**). DeepSeek's questions were rated with 2.5/10 for both prompt types, while OpenAI's questions show 0/10 points for each prompt type. The latter is a surprising result, because in the quantitative analysis, OpenAI was performing well with the manipulation, as well as DeepSeek – both providing few essentially low *Adherence Scores*. One of the two complex prompt questions the OpenAI model generated could not be rated by the supervisor, since it was hard to evaluate its manipulation handling here. This question (Appendix **A.7**) results in a NaN value – which is why the standard deviation for OpenAI's complex prompt questions is not available –, showing again that the sampling size was too small.

Overall, the supervisor-based evaluation does not provide significant insights into the LLMs' *Manipulation Handling*, but the models particularly struggled with this task. The sampled question sets were set as-is, because rating a more vast number of questions – focusing on the question's context, and a comprehensive rubric – was not feasible within the raters' time constraints.

Supervisor-based Analysis Complications. During this evaluation, several complications occurred. Firstly, certain questions could not be fully assessed for specific criteria, because these questions were not formulated properly, leading to NaN values, such as Appendix **A.3**. This issue was present in both Experiments 1a and 1b, possibly affecting the calculations for the evaluation.

A closer examination of the questions revealed issues for the models. Regarding Anthropic, several questions were problematic, especially in terms of *Challenging*, *Value* and *Correctness*, often coupled with the complex prompt used. In some cases, answer options were incorrect or did not align with the expected content. DeepSeek frequently mentioned "according to the text" in their questions, which also was a common issue at Google's questions. These were often formulated far too broadly at Google, with some containing answer options which were factually incorrect, coupled with unclear phrasing. OpenAI, in contrast, showed occasionally the "according to the text" phrasing, but overall, the questions were fine but less challenging, e.g. comparable to DeepSeek's questions.

Since criteria like *Answerability* could not be reliably applied to the manipulated content in Experiment 1b, the evaluation was limited to the *Manipulation Handling* criterion. This was done because the rubric's criteria were comprehensively assessed in Experiment 1a, and the manipulation handling was the only criterion that could be reliably applied to the manipulated lecture script.

Qualitative Analysis – Staff-based Comparison

In this sub-experiment, four staff members were asked to evaluate the same question set, as the supervisor has done before. This paragraph discusses the staff members' outcomes primarily including the similarities and differences to the supervisor's results.

Experiment 1a – Content Fidelity. The staff members' evaluation revealed several notable similarities and differences compared to the supervisor's assessment. As shown in Figure 12, the overall trends remained consistent, but the staff members were more stringent in their scoring.

A key similarity was the identification of OpenAI and DeepSeek as the leading models in most criteria such as *Clarity*, *Answerability*, and *Language*. However, where the supervisor saw a clear advantage for OpenAI (e.g. mean of 10 in *Relevance*), the staff rated the models closer together, with OpenAI (9.12) only slightly ahead of the others. This suggests that the performance gap between the models is mainly perceived as smaller by the staff.

As depicted by the supervisor, the *Challenging* criterion also remained the lowest-scoring criterion across all models, with means below 5 for all LLMs at the staff-based one. DeepSeek (2.92) and OpenAI (3.35) performed lowest again, while Anthropic (4.65) and Google (4.81) were rated as slightly more challenging, but not as significantly as in the supervisor's evaluation. Especially the ratings for Anthropic and Google were different from the supervisor's, showing that the staff members found these models' questions less challenging than the supervisor did instead.

Furthermore, the staff's more critical assessment is visible in the linguistic evaluation. For the *Language* criterion, they rated Anthropic (7.4) and Google (6.96) lower than the supervisor did (8.25 and 7.67 respectively). Moreover, OpenAI and DeepSeek were also rated lower than the supervisor's 10/10 scores for both models, with means of 8.6 each. These values differ from the supervisor's evaluation, but still show that the staff members found these models' questions more precise instead of linguistically complex. The *Value* criterion also slightly differed from the supervisor's evaluation, with all means below 8 points. The main difference was that the staff members rated OpenAI with a mean of 6.48 instead of 10/10 points, like the supervisor did. As a negative example for these two criteria, in Appendix A.4, a question by Google is shown, formulated inappropriately by using a negation in the question itself.

For *Correctness*, the staff-based evaluation confirmed OpenAI's high performance (8.42) with DeepSeek beyond (8.48) – both with narrow value ranges –, though without the clear separation the supervisor observed. Close behind these models, Anthropic and Google were rated lower with means of 7.94 and 8.15. The model ratings differ from the supervisor's, since the *Correctness* criterion was implemented choosing ratings of 0, 5, or 10. The thesis supervisor solely used these ratings, while the staff members rated values in between. The supervisor could have rated the questions differently, not rating this many questions with 10/10 points, but rather values between 0 and 5, or 5 and 10, which would have led to a more similar evaluation to the staff members' one.

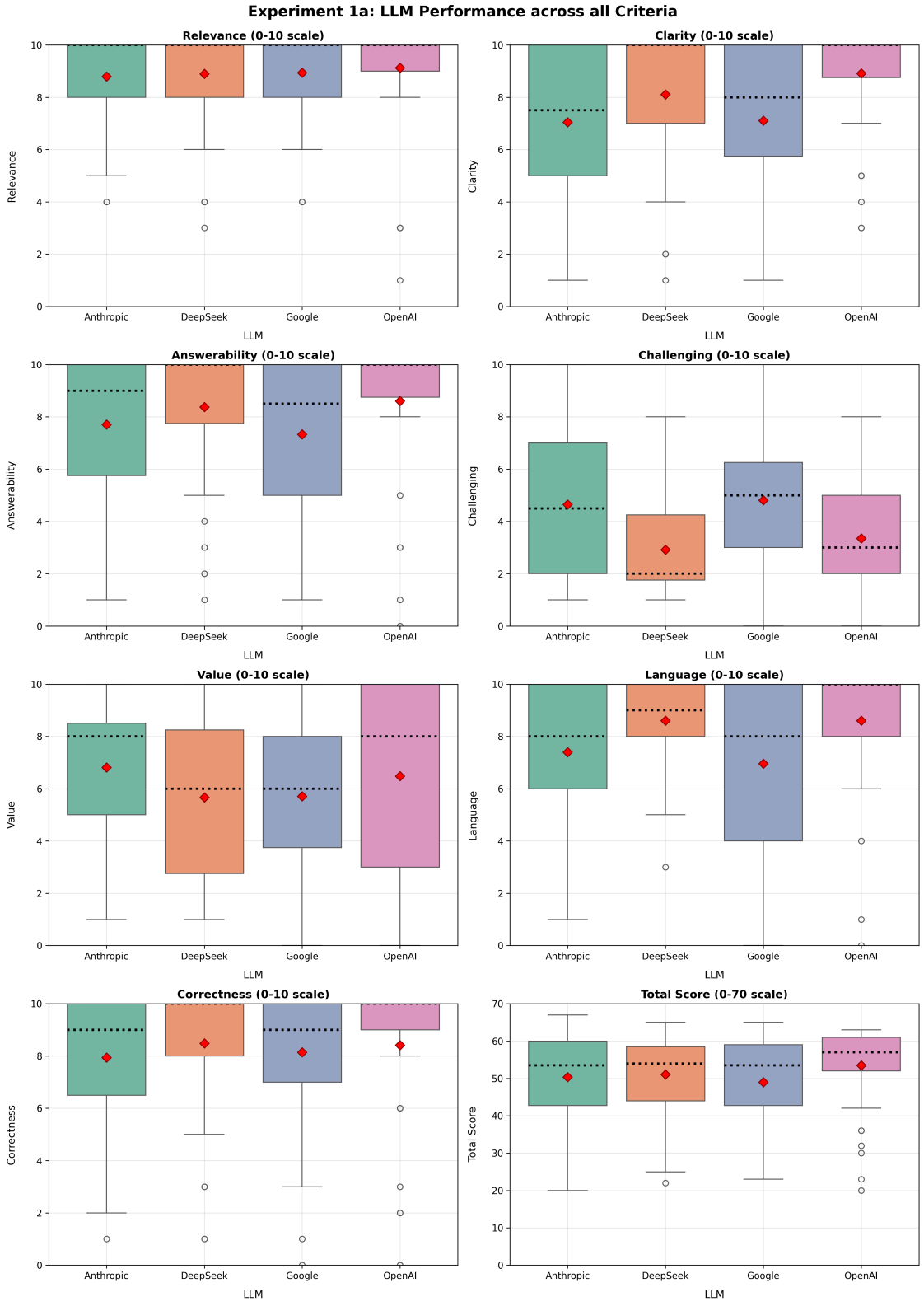

Figure 12: Staff-based LLM Scores across every proposed Criterion (Exp1a).

Overall, the *Total Scores* revealed that while OpenAI (mean of 53.5) remained the top performer, but less pronounced compared to the supervisor's clear preference (62.5 vs. DeepSeek with 53.25). The results for the other models were more similar to each other, with means around 50/70 points. This is a contrast to the supervisor's evaluation, where Google especially was often rated lower.

Regarding Figure 13, the *Correctness* scores focusing on the input sources differ from the supervisor's evaluation. There is a minor trend similar to the supervisor's evaluation, where OpenAI and DeepSeek tend to be rated higher than the other models, also visible in the *Adherence Score*. Contrary to the supervisor, the staff members rated OpenAI's questions based on Tanenbaum with a mean of 7.8, which is significantly lower than the supervisor's 10/10 points. Also, the trend that the transcript-based questions were rated highest is not visible in the staff members' mixed picture.

In the second staff heatmap, the prompt type had the same impact on the correctness scores, with the common prompt leading to higher scores than the complex one. The models were on par with values all above 9/10 in the mean. In the supervisor's evaluation, Google was rated with 8.3 in the common prompt instead. Focusing the complex prompt, the supervisor also rated Google significantly lower with a mean of 3.3/10, while the staff members did with 6.8 points.

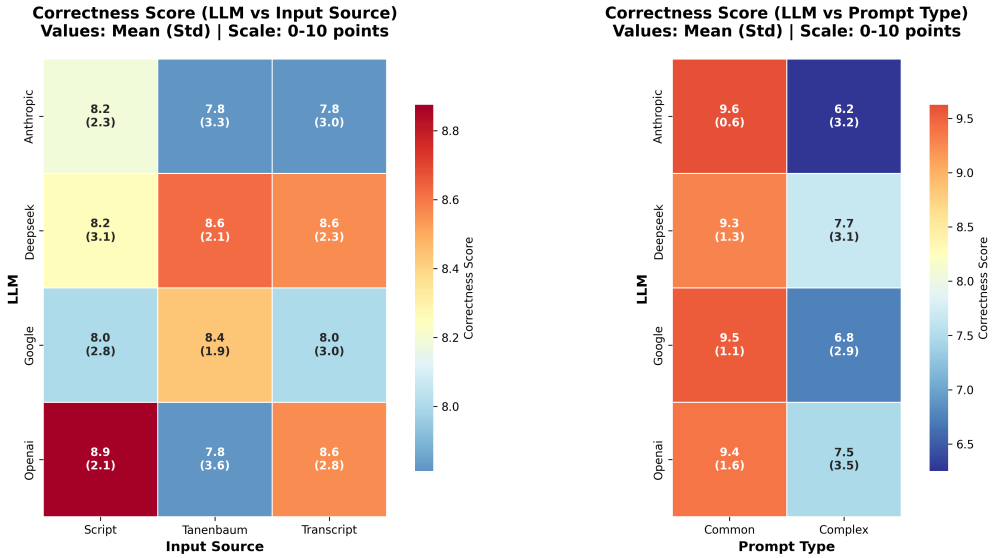

Figure 13: Staff-based Correctness Heatmaps for different LLM Combinations (Exp1a).

In the following, the results of the staff-based qualitative analysis are summarized in Table 8 to observe how the staff members have rated based on the input source. These revealed a minor shift in source preference compared to the supervisor's findings in Table 5. While the supervisor consistently favored transcript-based questions as the best source – with the highest means in *Clarity* (9.33), *Answerability* (9.33), *Value* (7.87), *Language* (9.19), and *Correctness* (9.67) – the staff members slightly identified script-based questions as the overall best performer. The script achieved the highest staff ratings in *Relevance* (9.16), *Challenging* (4.06), *Value* (6.67), *Correctness* (8.33), and *Total Score* (52.14), contrasting with the supervisor's input source preference.

Table 8: Staff-based qualitative Analysis Results by Input Source (Exp1a). The best means are highlighted in **bold**, while the best medians are *italicized*.

Source	Metric	Relevance	Clarity	Answerability	Challenging	Value	Language	Correctness	Total Score
Script	Mean	**9.16**	7.86	8.17	**4.06**	**6.67**	7.89	**8.33**	**52.14**
	Std	1.64	2.55	2.65	2.34	3.18	2.48	2.56	10.82
	Median	10.0	9.0	10.0	*4.0*	*8.0*	8.0	9.5	*55.0*
Tanenbaum	Mean	8.77	7.62	7.66	3.94	5.77	7.77	8.17	49.69
	Std	2.09	2.81	3.10	2.44	3.41	2.63	2.79	11.83
	Median	10.0	8.0	9.5	3.0	6.0	8.0	10.0	52.0
Transcript	Mean	8.89	**7.89**	**8.19**	3.80	6.06	**8.02**	8.23	51.08
	Std	2.02	2.85	2.81	2.32	3.10	2.72	2.73	11.23
	Median	10.0	*10.0*	10.0	3.5	6.0	*10.0*	10.0	54.0

Furthermore, the rating gaps between the two evaluation groups are notable. For instance, the supervisor rated the transcript-based questions significantly higher in *Clarity* (9.33 vs. 7.89), *Answerability* (9.33 vs. 8.19), and *Correctness* (9.67 vs. 8.23). Similarly, the script-based questions have shown several differences in the *Challenging* criterion, where the supervisor rated them at 5.5 compared to the staff's 4.06.

Despite these scoring differences, both evaluations commonly identified Tanenbaum excerpts as the weakest source material (excluding *Challenging*), with the staff rating it lowest in *Total Score* (49.69), and the supervisor placing it similarly (51.25). It struggled across both evaluation groups, particularly in clear and educationally valuable questions. Both exhibited the same trend in the *Total Score*, where the script was rated slightly highest with a mean of 52.14 – compared to the supervisor's 55.06 –. This is followed by the transcript with 51.08 – compared to the supervisor's 54.75. In addition, an example of a properly formulated question by OpenAI – based on the transcript – is shown in Appendix **A.2**, rated almost perfect across the criteria, indicating that both groups found this question clear, precise, and answerable, focusing on the content of the transcript.

To further investigate the prompt differences, Table **9** shows that the staff's evaluation confirmed the supervisor's key findings while being more conservative in their scoring approach across all criteria. Overall, the staff-based evaluation supports the fundamental trend observed in the supervisor's results: the common prompt consistently outperformed the complex prompt in nearly all quality-related criteria. The common prompt achieved superior staff ratings in *Relevance* (9.50), *Clarity* (9.18), *Answerability* (9.43), *Value* (7.26), *Language* (9.27), and *Correctness* (9.46), while the complex prompt only excelled in the *Challenging* criterion (4.77 vs. 3.09).

Table 9: Staff-based qualitative Analysis Results by Prompt Type (Exp1a).

Prompt	Metric	Relevance	Clarity	Answerability	Challenging	Value	Language	Correctness	Total Score
Common	Mean	**9.50**	**9.18**	**9.43**	3.09	**7.26**	**9.27**	**9.46**	**57.19**
	Std	1.13	1.54	1.18	1.83	2.91	1.19	1.19	5.61
	Median	10.0	10.0	*10.0*	2.5	*8.0*	10.0	10.0	*58.5*
Complex	Mean	8.38	6.41	6.58	**4.77**	5.07	6.51	7.03	44.75
	Std	2.35	2.95	3.30	2.53	3.19	2.88	3.17	12.11
	Median	10.0	7.0	7.0	*5.0*	5.0	7.0	8.0	44.5

However, the staff's stringent evaluation differs from the scores of the supervisor. For the common prompt, notable differences are visible in means of *Clarity* (9.18 vs. 10.0), *Value* (7.26 vs. 9.55), and *Language* (9.27 vs. 10.0), while *Correctness* remained remarkably similar between both groups (9.46 vs. 9.35). The lower ratings of the staff members are noticeable in the *Total Score*, which shows that the common prompt was rated with a mean of 57.19 compared to the supervisor's 60.12.

Regarding the complex prompt, both evaluations consistently identified significantly lower performance across most criteria. The staff's ratings illustrate that this prompt type produces less relevant (8.38), clear (6.41), and answerable (6.58) questions. Most important for content fidelity, the complex prompt scored notably lower in *Correctness* (7.03) compared to the common prompt (9.46), mirroring the supervisor's findings of 7.63 vs. 9.35.

The *Total Score* difference between prompt types was noticeable across both evaluations. The staff showed a significant gap (57.19 vs. 44.75), while the supervisor demonstrated a similar one (60.12 vs. 47.25), with slightly higher ratings overall. This consistency strongly supports that the common prompt is significantly more effective for generating suitable questions, while the complex prompt – producing more challenging questions – compromises overall quality and correctness.

Overall, the staff-based evaluation confirmed the supervisor's key findings while being more stringent in their scoring. The complex prompt decreased the quality of the questions across most criteria, particularly in *Clarity*, *Answerability*, *Value*, *Language*, and *Correctness*. In both evaluations, the *Challenging* criterion was the only one where the complex prompt outperformed the common prompt. OpenAI also was consistently rated as the best-performing model across most criteria, while the input source preferences shifted a bit, with the staff members slightly rating the script-based questions as the best source material and the supervisor often favoring the transcript-based questions.

Besides these promising insights, the inter-rater agreement yielded undesirable results, as shown in Tables 10 and 11. At first, Table 10 shows the Fleiss' Kappa values which indicate poor agreement among the staff, with most criteria showing negative or low positive values. This suggests that the staff members had significant disagreements in their evaluations. This is due to the broad range of possible ratings (0-10) in the rubric, which could have lead to different interpretations of the criteria.

Table 10: Inter-Rater Agreement Analysis, only Staff (Exp1a).

Criterion	**Fleiss' $\kappa \uparrow$**	**Std**	**#Items**	**#Raters**
Relevance	-0.085	1.92	48	4
Clarity	0.034	2.72	48	4
Answerability	0.015	2.84	48	4
Challenging	0.011	2.35	48	4
Value	-0.010	3.23	48	4
Language	0.001	2.59	48	4
Correctness	-0.052	2.67	48	4

The staff members' ratings were more diverse, while using all possible values in the rubric. Commonly, most scales were presented as 0, 2, 4, ..., 10, but the staff members also used values in between these steps, e.g. leading to one-off ratings.

In addition, the combined inter-rater agreement analysis in Table 11 is presented, including the supervisor's ratings, solely using the desired values for the criteria, instead of the values in between. This table shows that the overall agreement across all raters was also poor, unfortunately. The combined agreement analysis led to slightly higher Fleiss' Kappa values for every criterion.

Table 11: Inter-Rater Agreement Analysis, combined (Exp1a).

Criterion	Fleiss' $\kappa \uparrow$	Std	#Items	#Raters
Relevance	-0.046	1.88	47	5
Clarity	0.077	2.68	47	5
Answerability	0.054	2.77	47	5
Challenging	0.014	2.61	47	5
Value	0.012	3.34	45	5
Language	0.043	2.53	48	5
Correctness	-0.022	2.56	42	5

Experiment 1b – Error Handling.

As in the previous sub-experiment, the staff members were asked to evaluate the same experimental setup as the supervisor, shown in Table 12. This time, however, the focus was on error handling capabilities. Since the results are based on four staff members – instead of one single supervisor –, the results are more diverse and provide a broader perspective on the LLMs' performance in handling errors. The staff members evaluated the questions based on the same criteria as in Experiment 1a, but finally, the *Manipulation Handling* criterion was solely regarded, since the other criteria were properly evaluated in the previous sub-experiment.

Table 12: Staff-based Manipulation Handling Analysis by LLM and Prompt Type (Exp1b).

		Manipulation Handling		
LLM	**Prompt Type**	**Mean**	**Std**	**Median**
Anthropic	Common	2.50	4.63	0.0
	Complex	**7.25**	2.55	*8.0*
DeepSeek	Common	3.38	4.37	1.5
	Complex	**5.12**	4.64	*5.5*
Google	Common	3.50	4.66	0.5
	Complex	**7.88**	3.00	*8.5*
OpenAI	Common	2.50	4.63	0.0
	Complex	**5.00**	3.93	*4.0*

There are several key insights and differences compared to the supervisor's evaluation. Most notably, the staff members' scores were higher for *Manipulation Handling*, especially visible in the complex prompt. Google and Anthropic achieved the best means of 7.88 and 7.25, while DeepSeek and OpenAI were rated 5.12 and 5.0. Regarding the common prompt, all models were rated lower, showing that this prompt type did not sufficiently encourage the models to detect manipulations.

Compared to the supervisor, who rated exclusively with the given values (0, 5, 10), the staff members used the full scale, rating all scores between 0 and 10. Here, Google and Anthropic were rated significantly higher for the complex prompt than by the supervisor (rated with 2.5), while OpenAI and DeepSeek still remained to be at the lower end, but with values ≥ 2.5 instead. This stands in contrast to the quantitative analysis, where OpenAI and DeepSeek had shown better manipulation detection, based on rating every question instead of a sampled question set. Overall, the sample size remained small – while the *Adherence Score* of the quantitative analysis remains suitable –, but the qualitative evaluations still underscore the importance of prompt design. While the complex prompt improves *Manipulation Handling* in Experiment 1b – while being problematic in Experiment 1a via *Correctness* –, the models still struggle to reliably identify and address manipulated content.

The differences between the supervisor and staff ratings also illustrate the need for a more nuanced evaluation scale for *Manipulation Handling*, as implemented in the other criteria. This would have been better with the *Correctness* criterion in Experiment 1a as well. Unfortunately, the inter-rater agreement analysis Table **13** also illustrates an unsatisfying agreement, compared to the previous sub-experiment. The Fleiss' κ value between the staff members is 0.014, indicating just a slight agreement among these raters. This had happened because the staff members used the full range of scores instead of the desired ones, but overall, the ratings by all experts still present valuable information.

Table 13: Inter-Rater Agreement Analysis (Exp1b).

Manipulation Handling	**Fleiss' $\kappa \uparrow$**	**Std**	**#Items**	**#Raters**
Staff-only	0.014	4.31	16	4
Combined with Supervisor	0.036	4.21	15	5

Based on the staff members, there is no significant improvement compared to the previous sub-experiment, but the combined analysis – including the supervisor – shows a minimally higher Fleiss' κ of 0.036, which is a slight value in the end. This is a little surprising, since the supervisor rated only the desired values (0, 5, 10), while the staff members used the full scale instead. The staff members also rated the questions with values ≥ 5, which were not used by the supervisor, since the supervisor only rated up to 5 in certain cases. This individual misunderstanding in the criterion led to a low agreement between both groups, since the ratings were not comparable, even if the ideas behind the ratings were occasionally similar.

Mixed Analysis Complications. The qualitative evaluation revealed complications affecting both supervisor and staff assessments. Firstly, several questions were only partially or not at all related to the provided source material, limiting *Answerability* and *Correctness*.

Few questions with unnecessary negations in them were criticized, reducing *Clarity* and the *Value*. This evaluation was further complicated by overly long or linguistically complex formulations, especially from Google and Anthropic using the complex prompt. Both groups often noted the use of generic phrases such as "based on the text", which weakened the questions' educational *Value*.

Factually incorrect or invented answer options – which are not based on the source text – were identified in multiple cases, making correctness difficult to assess, often seen at Google and Anthropic as well. Several questions were rated as very easy or trivial by both groups, particularly from OpenAI and DeepSeek, which is visible in the *Challenging* criterion.

Questions based on the common prompt were often criticized for being too simple or not sufficiently challenging, while the complex prompt questions – overly more challenging – were often too convoluted or complex, leading to low values in e.g. *Clarity* and *Language*. Instead, the common prompt questions overall have shown higher ratings in both evaluations.

Furthermore, the strong preference of OpenAI in the supervisor's evaluation was not as pronounced in the staff members' ratings, but still slightly visible. The other models struggled more with achieving high scores in the criteria, especially in *Clarity*, *Answerability*, and *Correctness*.

The evaluation rubric itself was not always clear for all criteria, resulting in inconsistent ratings and low inter-rater agreement among the groups. Overall, the staff members' ratings were more diverse, often using the full scale (0-10) instead of the discrete values (e.g. 0, 5, 10 at *Correctness* and *Manipulation Handling*) used by the supervisor. Moreover, the staff members rated the questions higher at *Manipulation Handling* than the supervisor, indicating a different understanding of the criterion but still providing valuable insights based on more perspectives.

In addition, the sampled question set in Experiment 1b was smaller than in Experiment 1a, which limits the representativeness of the results. At least, the staff-based qualitative analysis provided a broader perspective on the LLMs' performance in handling errors, where the complex prompt improved *Manipulation Handling* in Experiment 1b, while being problematic in Experiment 1a via *Correctness*. This was not significantly visible in the supervisor's evaluation of Experiment 1b, but the quantitative analysis revealed similar trends according to the difference in prompt type performance with the *Adherence Score*.

6.2 Second Experiment

The second experiment addressed evaluating the relationship between question formats and Bloom's Taxonomy levels in LLM-generated questions. The goal was to determine how different question formats – Multiple-Choice and Open-Ended questions – perform across various cognitive levels of Bloom's revised Taxonomy, and also independently of each other. This experiment dealt with the second research question:

RQ2: *How does the relationship between diverse question formats and Bloom's Taxonomy levels influence the pedagogical effectiveness of LLM-generated questions?*

Qualitative Analysis – Student-based

A group of three students, supervised by the secondary supervisor, evaluated the questions generated in Experiment 2. The students were asked to rate the questions based on the known criteria *Relevance*, *Clarity*, *Answerability*, *Challenging*, *Value* and *Language*. Additionally, the *Bloom's Level* was assessed, as seen in Table 4. So, the evaluation process involved a detailed rubric, and the students' ratings were analyzed using Fleiss' Kappa again to assess inter-rater reliability.

Regarding Figure 14, some key insights can be drawn from the student-based evaluation. First of all, the *Relevance* criterion, showing that the questions were highly relevant to the topic across the models. The questions by Anthropic and DeepSeek were rated higher, achieving medians of 10 and means of 9.07 and 8.67, respectively. Google and OpenAI instead had lower ratings, with means of 8.2 and 8.0, coupled with a median of 8. Focusing on the *Clarity* criterion, the models commonly achieved high scores, with all models showing a median of 10.0. The means were around 9, except for OpenAI, which had a mean of 8.67, but overall, the models generated logically clear questions.

Regarding *Answerability*, all medians are 8. The means of the models slightly differ, showing that Anthropic and DeepSeek had higher means of 8.27 and 8.2, where DeepSeek shows a better data distribution. Focusing on Google and OpenAI, the outcomes are pretty similar, with the widest value ranges and means of 7.72 and 7.53, though still rated high. The *Challenging* criterion presents medians of 8 for every model. Overall, more questions by Google and OpenAI were rated as more challenging, resulting in means of 7.27 and 7.47, respectively, while DeepSeek and Anthropic were rated lower with means of 6.8 and 6.53.

The results for the questions' *Value* look pretty similar for each model, but the medians were slightly higher for DeepSeek and OpenAI (9.0) than for Google and Anthropic (8.0). Overall, the questions were rated as valuable, with minor outliers at DeepSeek and Google. Despite this, the means still remained high, with OpenAI achieving the highest mean of 8.73, followed by Anthropic (8.67), DeepSeek (8.53), and Google (8.33).

Furthermore, the *Language* criterion looks similar across all models, with means from 8.37 (Google) to 8.73 (DeepSeek). In contrast, the *Bloom's Level* shows a more diverse picture, where Google and OpenAI achieved the highest medians of 10 and 9.25, while DeepSeek and Anthropic were rated lower with medians of 6.0 and 7.0. So, many questions by Anthropic and DeepSeek were less rated at Bloom's revised Taxonomy or less aligned with the desired Bloom's level, if given.

Ultimately, the *Total Score* illustrates that no LLM is a clear winner, with OpenAI being the lowest rated model with a mean of 55.27 and a median of 55.0 out of 70 points. The other models are rated slightly higher with lower ranges of values, but still showing that all models are quite similar in their performance, whether all criteria are considered together. For instance, a proper Multiple-Choice question by OpenAI from Experiment 2a demonstrates a positive example (Appendix A.8), while a problematic question by the same model shows complexity issues (Appendix A.9).

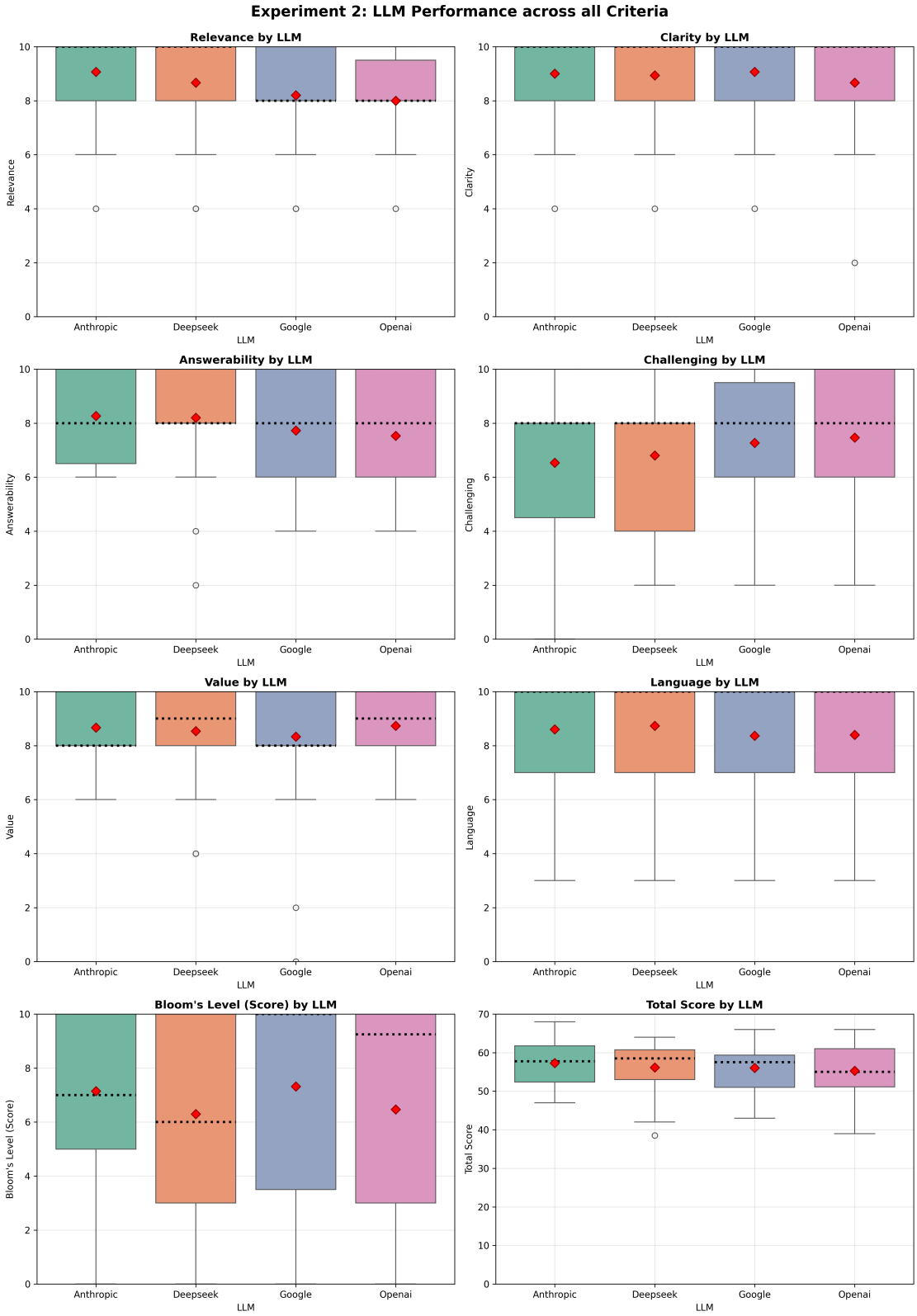

Figure 14: Student-based Evaluation Metrics by LLM (Exp2).

Regarding each sub-experiment instead, the results are shown in Figure 15. Here, the three sub-experiments are compared, where Experiment 2a (Type Only) focused on solely specifying the question type, Experiment 2b focused on a specific Bloom level for each question (Bloom Only), and Experiment 2c combined both aspects (Type + Bloom). The *Relevance* criterion shows that Experiment 2a achieved the highest mean of 9.04, followed by Experiment 2c with a mean of 8.12, and Experiment 2b with a mean of 8.08 and a wider range of values. Also, the median of Experiment 2a was the highest with 10, while Experiment 2b and 2c had medians of 8.

The *Clarity* criterion shows a trend where Experiment 2c achieved the highest mean of 9.29, followed by Experiment 2a with a mean of 8.79, and Experiment 2b with 8.43, indicating that most of the questions were clear and understandable. Regarding the *Answerability*, all sub-experiments achieved a median of 8. Experiment 2b was the worst rated one again, with a mean of 7.13, while Experiment 2a (mean of 8.17) and Experiment 2c (mean of 8.08) were rated better overall.

The *Challenging* criterion shows an interesting trend, where Experiment 2b achieved the best results with a mean of 8.42, while Experiment 2a is following close behind with slightly shifted values and a mean of 7.17. The questions in Experiment 2c were rated as the least challenging with a mean of 6.17 instead. The questions' *Value* was rated overly high across all sub-experiments, but the ones from Type Only (Experiment 2a) have a median of 8, while the other two sub-experiments achieved a median of 9. Overall, Experiment 2c had some outliers to the bottom, but overall, the means were still high, with 8.58 (Experiment 2a), 8.75 (Experiment 2b), and 8.46 (Experiment 2c).

In addition, the *Language* criterion was rated overall best in Experiment 2c with a mean of 8.71, followed by Experiment 2a with a mean of 8.44, and Experiment 2b with a mean of 8.33. 2a and 2c achieved the highest medians of 10, while Experiment 2b had a median of 8.5 instead, which indicates that the questions were overall well-formed and linguistically precise, but the questions in 2b were slightly less linguistically valuable. Also, Experiment 2a share low scoring Experiment 2b, where the lowest scores are visible at 3, and Experiment 2c only goes down to 5 instead.

The *Bloom Score* delivers a more interesting picture, where 2a is situated lower. In this sub-experiment, the mean is 4.0, showing that the questions are situated between *Understanding* (3 points) and *Applying* (4.5 points), since no Bloom level was specified in the prompt. This is acceptable, but the median is only 3.0 instead. In contrast, the values of 2b and 2c were situated significantly higher, with medians of 10 and means of 8.75 and 8.65, respectively. Overall, the Bloom levels were well adhered to (resulting in 10 points), with only a few outliers for both sub-experiments, illustrated via one-off ratings (5 points) or a rated Bloom level distance of > 1 (0 points).

In this box plot, the *Total Score* shows that the sub-experiments do not differ significantly, but Experiment 2b (Bloom Only) achieved the highest mean of 57.58 and a median of 58.5, followed by Experiment 2c (Type + Bloom) with a mean of 57.48 and a median of 58.5 as well. Experiment 2a (Type Only) is rated slightly lower with a mean of 54.19 and a median of 54.25, showing overall that the *Bloom's Level* criterion is the most decisive one, since the scoring in Experiment 2a is significantly lower than in the other two sub-experiments. From this plot-based perspective, Experiment 2b and 2c are rated pretty similarly in *Total Score*, while Experiment 2a also achieved acceptable results.

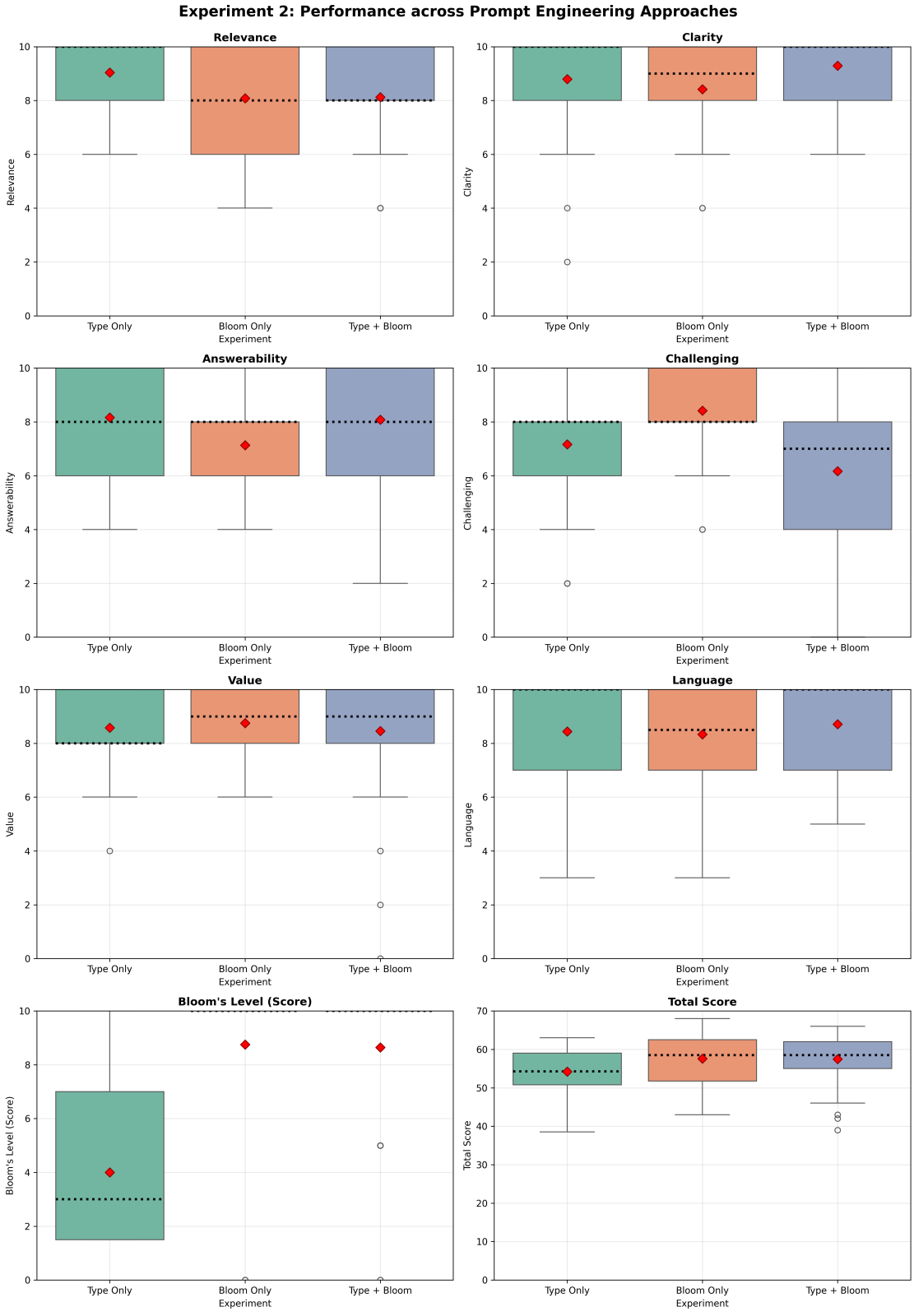

Figure 15: Student-based Evaluation Metrics by Sub-Experiment (Exp2).

For instance, Anthropic illustrates a well-structured question from Experiment 2b for Bloom level 2 (Appendix **A.10**), while a problematic question from the same sub-experiment by Google demonstrates unnecessary complexity issues in its Bloom level 6 question, including many technical terms and a long introduction phrase (Appendix **A.11**). Other examples from Experiment 2c by both models illustrate an inverted picture: one of Google's valuable MCQs for Bloom level 4 demonstrates a positive combination of question type and Bloom specification (Appendix **A.12**), while an Open-Ended question by Anthropic for the same Bloom level shows linguistic issues that can arise, based on complex wording and many foreign words (Appendix **A.13**).

Since the *Bloom's Level* criterion is the most important one in this experiment, Table **14** shows the results of the Bloom levels achieved by the models in each sub-experiment. Here, certain trends among the models are visible, where Anthropic and Google achieved the highest means in Experiment 2a (4.96 and 4.54) – which are situated at the level *Applying* (4.5 points) –, while DeepSeek and OpenAI were rated even lower (2.83 and 3.67), which is close to *Understanding* (3 points). In Experiment 2b, all models achieved the same mean of 8.33, except OpenAI, showing a perfect Bloom adherence with a mean of 10.0. Experiment 2c, combining both question type and Bloom level specification, shows that Google performed best with a mean of 9.58. Anthropic and DeepSeek follow with 8.75, while OpenAI is rated lowest with a mean of 7.50, still offering a good adherence to the Bloom level specification. The medians of Experiment 2b and 2c are rated 10/10 for every model, also indicating that the questions were well aligned with the specified Bloom levels. Experiment 2a instead shows that the medians were lower – as this sub-experiment did not specify a Bloom level –, where Anthropic shows the highest median of 7, which is situated at *Analyzing* (7 points). In contrast, the other models' questions were less cognitively demanding, illustrated by the median of 3.0 for each LLM, which is situated at the Bloom level *Understanding* (3 points).

Table 14: Student-based Bloom Scores by LLM and Sub-Experiment (Exp2).

LLM	Mean			Std			Median		
	Exp2a	**Exp2b**	**Exp2c**	**Exp2a**	**Exp2b**	**Exp2c**	**Exp2a**	**Exp2b**	**Exp2c**
Anthropic	**4.96**	8.33	8.75	2.57	4.08	2.26	*7.0*	10.0	10.0
DeepSeek	2.83	8.33	8.75	1.50	4.08	2.26	3.0	10.0	10.0
Google	4.54	8.33	**9.58**	2.53	4.08	1.44	3.0	10.0	10.0
OpenAI	3.67	**10.00**	7.50	2.78	0.00	3.99	3.0	10.0	10.0

Since both question types (Multiple-Choice and Open-Ended) were equally generated in Experiments 2a and 2c, Table **15** shows the results for each type. Overall, the Open-Ended questions were rated significantly higher than the MCQs in both sub-experiments. In Experiment 2a, Multiple-Choice achieved a mean of 2.42, rated between *Remembering* (1.5 points) and *Understanding* (3 points). Instead, Open-Ended questions achieved a mean of 5.58, which shows that questions of this type were more cognitively demanding – between *Applying* (4.5 points) and *Analyzing* (7 points).

Experiment 2c shows a similar trend, but focusing Bloom alignment instead of the cognitive level itself. The Bloom adherence for MCQs were rated high with a mean of 7.92. The Open-Ended questions were rated even higher with a mean of 9.38, showing a very high adherence to the specified Bloom level overall. The medians in Experiment 2c were 10.0 for both question types, but in Experiment 2a, the medians were 2.25 for Multiple-Choice and 7.0 for Open-Ended questions, indicating that the Open-Ended questions were more cognitively demanding, while MCQs were less cognitively demanding and less aligned with given Bloom levels.

Table 15: Student-based Bloom Scores by Question Format and each Sub-Experiment (Exp2).

Question Type	Mean		Std		Median	
	Exp2a	**Exp2c**	**Exp2a**	**Exp2c**	**Exp2a**	**Exp2c**
Multiple-Choice	2.42	7.92	1.23	3.27	2.25	10.0
Open-Ended	**5.58**	**9.38**	2.38	1.69	*7.00*	10.0

Regarding the inter-rater agreement analysis, the results are mixed, which is reflected in the varying Fleiss' kappa values across different criteria in Table 16. A major issue in the ratings was the diversity in scoring among the three students, indicating that there were differences in interpretation and application of the rubric criteria. Moreover, rating questions like the desired sample set is no everyday task for the students. This led to certain impacts on the agreement for *Relevance*, *Clarity*, *Value*, and *Language*. These criteria have a very low Fleiss' κ: *Relevance* with 0.056, *Clarity* with -0.061, *Value* with -0.256, and *Language* with 0.016.

Table 16: Inter-Rater Agreement Analysis, Students (Exp2).

Criterion	**Fleiss' $\kappa \uparrow$**	**Std**	**#Items**	**#Raters**
Relevance	0.056	1.71	40	3
Clarity	-0.058	1.61	40	3
Answerability	0.179	1.90	39	3
Challenging	0.270	2.52	40	3
Value	-0.256	1.72	40	3
Language	0.016	1.95	40	3
Bloom Rating	0.397	1.67	40	3

In contrast to this, the *Answerability* ($\kappa = 0.179$) and especially the *Challenging* ($\kappa = 0.270$) criterion achieved better agreements. Moreover, the highest Kappa was achieved for the *Bloom Rating* criterion ($\kappa = 0.397$), indicating an almost moderate agreement. Here, the students' Bloom rating per question was compared, instead of taking the calculated *Bloom's Level* score, based on the three scoring options in Table 4.

In some cases, the students were not able to agree on the Bloom level of a question, which occurs since a set of questions did not provide a certain operator verb, or more than one operator verb was used, which both made it difficult to determine the final Bloom level of the question. Overall, the Kappa of 0.397 is still a good value for this criterion, showing that the students were able to agree on the Bloom levels in many cases, but still had some disagreements.

Overall, the stringent time constraints for the students' evaluation could have limited the depth of their analysis, as they had to evaluate 40 questions in a limited time frame. The unknown topic of the questions – since these are pedagogical students – may have also influenced the students' ability to accurately assess the questions' *Relevance* and *Answerability*, as they were not familiar with the ISO-OSI model before this analysis.

Student-based Analysis Complications. This evaluation revealed further complications in the students' analysis affecting question quality across the sub-experiments. For instance, few questions without any operator verbs were present, particularly in MCQs of Experiment 2a, which are crucial for determining the Bloom level properly. This was visible in the MCQs across DeepSeek, Google, and OpenAI, where cognitive demands are hard to assess without clear operators.

Questions by Anthropic, but particularly by OpenAI and Google across the sub-experiments were often criticized for excessive technical terminology and high linguistic complexity, making them overloaded and demotivating to read. Google often used complex wording, coupled with long introductory phrases, making these questions less valuable for educational purposes. Furthermore – focusing Experiment 2a –, OpenAI was criticized for using too many operator verbs in the Open-Ended questions, which made it difficult to determine the *Bloom's Level* and the *Value* of the question, focusing educational usefulness.

Surprisingly, in Experiment 2c, one *Creating*-level Multiple-Choice question (by OpenAI) was noted as unsuitable, since this question format constrained several responses, which contradicts this Bloom level. Based on difficulties like this, the students' ratings at *Bloom's Level* were different from each other, which is reflected in the inter-rater agreement analysis in Table **16**, but this criterion was still rated the highest by the students.

Overall, the mentioned issues were not present in every question, but these were visible in several questions across the models, which indicates that the LLM-generated questions were not always suitable for educational purposes. Also, the differences between the models were not as pronounced as in Experiment 1, where the models' performance was more diverse, which is an interesting finding.

Ultimately, one student criticized the *Value* criterion, arguing it should be mapped to specific learning objectives rather than general educational worth, highlighting potential issues with the rubric design for students as evaluators who are unfamiliar with the technical domain.

VII Conclusion & Future Work

In this Bachelor's thesis, the potential of LLMs for educational question generation was investigated through two experiments:

1. Evaluating the adherence of LLM-generated questions to diverse instructional texts, and
2. Analyzing how three prompting variants (specifying only the question format, only a targeted Bloom level, or both together) affect the generation of questions across Bloom's revised Taxonomy and their adherence to the intended cognitive level, if specified.

Four popular LLMs with *reasoning* capabilities were employed to conduct AQG: Anthropic Claude 3.7 Sonnet, DeepSeek R1, Google Gemini 2.5 Flash, and OpenAI o3.

7.1 Key Findings

Firstly, the quantitative results demonstrated that, based on the *Adherence Score* – an alternative to semantic similarity –, LLMs can generate questions with high content fidelity across diverse instructional sources. However, the cosine similarity, between sentence transformer based embedding vectors, failed to distinguish between source-based and no-source-based questions, with similar results for both types. Sentence transformers are not suitable for the texts used in this thesis, as these models are not designed for longer educational texts, but rather for sentence-level comparisons.

Significantly, the prompt complexity played a crucial role in both quantitative and qualitative analysis at question quality outcomes, where the common prompt consistently outperformed the complex prompt in most quality dimensions, surprisingly. The complex prompt only increased the *Challenging* ratings, but at a noticeable quality cost, which was not desired. This implies that a vastly formulated prompt does not necessarily lead to better results.

Moreover, the source materials used in Experiment 1a had an impact on the question quality. The lengthy transcript was rated highest in multiple criteria, such as *Clarity*, *Answerability*, and *Correctness*, but the lowest in *Challenging*. The concise script also yielded high scoring, especially at the staff's evaluation, and achieving the highest *Total Score* due to a better balance in *Challenging*, while the Tanenbaum excerpts were the weakest overall, especially in *Language* and *Answerability*.

Model differentiation was also observed, with OpenAI and DeepSeek demonstrating higher consistency, while Anthropic and Google were more variable, particularly showing a tendency to produce linguistically complex questions and occasionally incorrect or invented answer possibilities. The *Challenging* criterion was also rated lower for all models, with DeepSeek and OpenAI being even lower than Anthropic and Google.

Regarding the *Manipulation Handling* in Experiment 1b, the quantitative analysis showed that OpenAI and DeepSeek had lower *Adherence Scores* when the complex prompt was used, so these models could partially detect the manipulations in the source material. In contrast, Anthropic and Google often reproduced the manipulations without any remark. The small sample size of 16/56 questions in Experiment 1b and the low inter-rater reliability shows inconsistencies in the results.

The second experiment showed that specifying Bloom levels in the prompts resulted in high adherence to the intended cognitive levels, which is crucial for effective educational assessment. Without Bloom specification, the questions tended to be at lower cognitive levels (at *Understanding / Applying*), which is not sufficient for demanding educational content. Open-Ended questions can be used across all Bloom levels and achieved superior Bloom alignment, while MCQs were more suitable for lower cognitive levels.

Moreover, the models overall performed similarly, while being rated with almost the same experimental criteria. When considering the *Bloom's Level* scores, without Bloom specification, Anthropic and Google were rated higher than DeepSeek and OpenAI, but when Bloom levels were specified, OpenAI performed best in the Bloom Only setting, while Google performed best in the Type+Bloom setting. OpenAI was occasionally underperforming in the Type+Bloom setup, which was due to the MCQs being rated lower, since Bloom level 6 contradicts the MCQ format, which was not considered in the prompt design.

Across the experiments, linguistic complexity issues were observed, where overly complex questions and often occurring phrases were reducing the *Language* and *Value* ratings. Furthermore, the rubric designs of the experiments reduced the agreements due to its broad 0-10 scale, coupled with not sufficiently clear anchor descriptions. Unfortunately, the *Value* criterion was criticized, since in this thesis, it was not planned to integrate explicit learning objectives for the questions, which made the evaluation more abstract and less clear.

All in all, the *reasoning*-based LLMs can automate the generation of relevant, clear, and Bloom-aligned questions, but show diversity in criteria such as *Correctness*, *Challenging*, and especially in *Clarity / Language*. Based on the findings, questions with high content fidelity, and proper linguistic quality across the Bloom levels require: (1) Concise, well-structured source inputs rather than overly complex ones; (2) Explicit revised Bloom's Taxonomy level explanations – best coupled with learning objectives for each question / topic; (3) Proper but restrained prompt complexity with clear phrasing; and (4) Refined evaluation rubrics.

7.2 Answering the Research Questions

RQ1: *To what extent do LLMs adhere to the content of diverse provided instructional texts when generating questions?*

The quantitative analysis yielded promising results, with high *Adherence Score* entries across all models and prompts. However, cosine similarity was unsuitable for this task, as it failed to distinguish between source-based and no-source-based questions, since sentence transformers fail with longer educational texts. The common prompt consistently outperformed the complex prompt in content adherence across both quantitative measures and expert evaluations. Taking the qualitative analysis into account, the mentioned performance gap between the common and complex prompt was also visible in most criteria, where structured materials (script and transcript) were mostly rated higher than the Tanenbaum book excerpts.

Moreover, the models showed different strengths and weaknesses, with OpenAI and DeepSeek being more consistent in *Correctness*, generating questions that well reflect the source material without unnecessary linguistic complexity. Anthropic and Google were more variable, often generating questions which were linguistically complex (especially in the complex prompt) and occasionally incorrect or invented answer possibilities. Also noteworthy, the DeepSeek and Google models both struggled most with proper question phrasing, since these generated many questions including "based on the text" formulations.

The quantitative analysis also showed that the models can detect manipulations in the source material, shown in a lower *Adherence Score* for OpenAI and especially DeepSeek – coupled with the complex prompt –, while Anthropic and Google more often reproduced the manipulations without any remark. Qualitatively, the *Manipulation Handling* criterion revealed inconsistent results between supervisor and staff evaluations, where staff members rated higher than the supervisor, particularly for the complex prompt.

All in all, the findings suggest that LLMs can generate suitable questions – focusing content fidelity –, particularly when provided with a clear prompt and structured source material. For this task, DeepSeek and OpenAI were the best models, while Anthropic and Google were more variable in their performance. However, the low inter-rater reliability in the qualitative analysis significantly weakens the statistical power of the findings. Since the generated questions were not that challenging – but more challenging with the complex prompt –, this reflects the need for Bloom's Taxonomy levels to be specified in the prompts, which can lead to higher *Challenging* and *Total Score* ratings.

RQ2: *How does the relationship between diverse question formats and Bloom's Taxonomy levels influence the pedagogical effectiveness of LLM-generated questions?*

This research demonstrated that specifying Bloom levels – either Bloom Only or Type+Bloom – resulted in high adherence to the intended cognitive levels, with median scores of 10 in both cases. If no Bloom specification was given instead, the questions tended to fall into lower cognitive levels, with a median of 3 (at *Understanding*), which is not a sufficient result for demanding educational content. The models performed diversely, with Anthropic and Google being rated higher than DeepSeek and OpenAI in Type Only conditions. However, when Bloom levels were specified, the performance of the models was higher, with OpenAI being the best model in the Bloom Only setting, and Google performing best in the Type+Bloom setting.

A critical finding was that Open-Ended questions consistently achieved higher cognitive levels than MCQs, and even higher Bloom alignment when a Bloom specification was given. The MCQs were more suitable for lower cognitive levels, such as *Remembering*, *Understanding*, and *Applying*. By this, a mix of both question types, coupled with specifying Bloom levels – which leads to a higher *Total Score* – can be generated to better handle all cognitive levels.

The student evaluation of the questions in Experiment 2 showed that the *Bloom's Level* had the highest inter-rater reliability, supporting the usability of the clear level definitions used. However, linguistic and content-related criteria such as *Relevance*, *Clarity*, and *Language* presented very

low inter-rater reliability, as well as the *Value* criterion. The evaluation rubrics used in this thesis were not sufficiently clear and consistent, which leads to the need for rubric refinements. Here, the current 0-10 scales provided too much interpretive freedom, and the students were not familiar with the ISO-OSI model domain. Overall, a balanced approach that combines Bloom's Taxonomy levels with appropriate question formats is essential for maximizing the pedagogical effectiveness of LLM-generated questions. Thoughtful prompt design, clear learning objectives, and a focus on linguistic clarity are crucial as well to ensure that the AQG process is effective and beneficial for educational purposes, coupled with current LLMs.

7.3 Practical Implications for Education

Firstly, valuable AQG relies on concise, well-structured source materials (e.g. lecture scripts), which can produce more relevant and challenging questions than lengthy transcripts or book excerpts. Secondly, a well-designed prompting strategy is crucial for AQG, as extensive prompt formulations do not necessarily lead to high content fidelity or overall quality. This was visible in Experiment 1a, where the common prompt outperformed the complex one in most quality dimensions, despite being less challenging. To achieve the best results, a concise but structured prompt including the necessary information, such as explicit Bloom levels or operator verbs, should be used. This helps to avoid overloading questions with unnecessary formulations or linguistic complexity, which can reduce *Clarity* and *Value*.

Overall, strategic selection of question formats for different cognitive levels is essential. Open-Ended questions are suitable across all Bloom levels and achieve superior cognitive alignment, while MCQs are constrained to lower levels, such as *Remembering*, *Understanding*, and *Applying*. Therefore, a balanced mix of both question types can be generated to cover all cognitive levels, which is beneficial for exam preparation. A mixture of question type, Bloom level, source material, and learning objectives can be used to create a comprehensive question set.

Moreover, users, particularly students, should be aware that LLMs are not perfect and can produce hallucinations or incorrect information. Therefore, it is important to inform students about the limitations of LLMs, and integrate a feedback mechanism into future frameworks, where students can provide feedback on the quality of the generated questions. This can help improve the quality of the questions over time, and ensure that they meet the needs of the students.

7.4 Future Research Directions

The dynamic landscape of LLMs presents both challenges and opportunities for future research. As newer models are going to be released, they may exhibit even greater potential for AQG and educational applications, while mitigating existing limitations. Hallucinations remain a critical issue, since factual accuracy is a crucial concern. Future work should continue to reduce the generation of incorrect or misleading information, especially in educational contexts.

Based on this, analyzing the adherence of LLM-generated questions to educational content can be further improved by using a more comprehensive multi-LLM judging approach, where the questions are better analyzed by multiple models instead of just two and averaging both results, which was done in this thesis. This would imply a more robust foundation for automated evaluation without solely relying on expert judgments, as these are often time-consuming. To avoid randomness, which might occur in the evaluation process above, more recent or future embedding models should be utilized instead of a sentence transformer, which was used in this thesis. Since sentence transformers are not the best choice for semantic similarity in longer texts / varying structures, this could be improved by addressing this issue in future work.

The experimental setup for analyzing *Manipulation Handling* was problematic, with a sample size of only 16/56 questions being evaluated qualitatively. A larger evaluation set would provide more consistent insights into the LLMs' reaction to manipulations, as the results were inconsistent between the supervisor and staff members. Additionally, the quantitative analysis using all 56 questions showed different trends than the qualitative analysis, highlighting the sampling limitations for this sub-experiment. This also implies a redesign of the rubric, since the *Value* criterion was criticized for its lack of explicit learning objectives integration, which was not considered during the planning phase. Furthermore, clearer anchor descriptions – e.g. for 0 points, 2 points, ... – are needed to improve inter-rater reliability and thus the qualitative analysis of the questions.

While this thesis focused on *reasoning* models, future research should also compare *reasoning* and non-*reasoning* models to analyze differences in AQG. Unfortunately, the *reasoning* step before answering the user is often not well analyzable, as many models consider this step proprietary. Based on this, the developers are solely providing summaries of the *reasoning* process instead[15].

Generating MCQs in one step is still a challenge, as in few cases, the (in)correct answer options are not generated properly, which can lead to confusion for students. Based on this, a pipeline that generates the question, correct answer(s), and incorrect answer options separately would be a beneficial approach for future work, as it was not implemented in this thesis, but demonstrated in work such as [Bho+23]. Finally, addressing potential biases in the evaluation process is crucial, as the supervisor of this thesis was also the author of the script/transcript used for question generation. However, the overall trends in the evaluation were similar to those of the other raters. In addition, the number of raters was sufficient for this thesis (five raters in Experiment 1 and three raters in Experiment 2), but the poor inter-rater reliability – especially in Experiment 1 – indicates that increasing the number of raters alone would not resolve fundamental issues with rubric design. This aspect should be addressed in future work, to enhance the robustness of the findings. All these insights can be used to build a robust, reliable and particularly pedagogical framework for AQG, which is a promising direction for further research.

[15] https://platform.openai.com/docs/guides/reasoning?api-mode=responses#reasoning-summaries, accessed 08/12/2025

References

[ARA24] Said Al Faraby, Ade Romadhony, and Adiwijaya. "Analysis of LLMs for educational question classification and generation". In: *Computers and Education: Artificial Intelligence* 7 (Dec. 1, 2024), p. 100298. ISSN: 2666-920X. DOI: 10.1016/j.caeai.2024.100298. URL: https://www.sciencedirect.com/science/article/pii/S2666920X24001012 (visited on 04/07/2025).

[Ant23a] Anthropic. *Claude 2*. 2023. URL: https://www.anthropic.com/news/claude-2 (visited on 04/28/2025).

[Ant23b] Anthropic. *Introducing Claude*. 2023. URL: https://www.anthropic.com/news/introducing-claude (visited on 04/28/2025).

[Ant24a] Anthropic. *Introducing Claude 3.5 Sonnet*. 2024. URL: https://www.anthropic.com/news/claude-3-5-sonnet (visited on 04/28/2025).

[Ant24b] Anthropic. *Introducing the next generation of Claude*. 2024. URL: https://www.anthropic.com/news/claude-3-family (visited on 04/29/2025).

[Ant25] Anthropic. *Claude 3.7 Sonnet and Claude Code*. 2025. URL: https://www.anthropic.com/news/claude-3-7-sonnet (visited on 04/28/2025).

[BL05] Satanjeev Banerjee and Alon Lavie. "METEOR: An Automatic Metric for MT Evaluation with Improved Correlation with Human Judgments". In: *Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization*. Ed. by Jade Goldstein et al. Ann Arbor, Michigan: Association for Computational Linguistics, June 2005, pp. 65–72. URL: https://aclanthology.org/W05-0909/ (visited on 04/29/2025).

[Bho+23] Ayan Kumar Bhowmick et al. "Automating Question Generation From Educational Text". In: *Artificial Intelligence XL*. Ed. by Max Bramer and Frederic Stahl. Cham: Springer Nature Switzerland, 2023, pp. 437–450. ISBN: 978-3-031-47994-6. DOI: 10.1007/978-3-031-47994-6_38. (Visited on 04/10/2025).

[BFL24] Giorgio Biancini, Alessio Ferrato, and Carla Limongelli. "Multiple-Choice Question Generation Using Large Language Models: Methodology and Educator Insights". In: *Adjunct Proceedings of the 32nd ACM Conference on User Modeling, Adaptation and Personalization*. UMAP Adjunct '24. New York, NY, USA: Association for Computing Machinery, June 28, 2024, pp. 584–590. ISBN: 979-8-4007-0466-6. DOI: 10.1145/3631700.3665233. URL: https://dl.acm.org/doi/10.1145/3631700.3665233 (visited on 04/10/2025).

[Blo+23] Ariel Blobstein et al. "Angel: A New Generation Tool for Learning Material based Questions and Answers". In: *NeurIPS'23 Workshop on Generative AI for Education* (2023). URL: https://gaied.org/neurips2023/files/9/9_paper.pdf (visited on 04/10/2025).

[Bro+20] Tom B. Brown et al. *Language Models are Few-Shot Learners*. July 22, 2020. DOI: 10.48550/arXiv.2005.14165. arXiv: 2005.14165[cs]. URL: http://arxiv.org/abs/2005.14165 (visited on 04/28/2025).

[CXJ24] Zirui Cheng, Jingfei Xu, and Haojian Jin. "TreeQuestion: Assessing Conceptual Learning Outcomes with LLM-Generated Multiple-Choice Questions". In: *Proc. ACM Hum.-Comput. Interact.* 8 (CSCW2 Nov. 8, 2024), 431:1–431:29. DOI: 10.1145/3686970. URL: https://dl.acm.org/doi/10.1145/3686970 (visited on 04/07/2025).

[Coh60] Jacob Cohen. "A Coefficient of Agreement for Nominal Scales". In: *Educational and Psychological Measurement* 20.1 (Apr. 1, 1960). Publisher: SAGE Publications Inc, pp. 37–46. ISSN: 0013-1644. DOI: 10.1177/001316446002000104. URL: https://doi.org/10.1177/001316446002000104 (visited on 05/09/2025).

[Dee+25a] DeepSeek-AI et al. *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. Jan. 22, 2025. DOI: 10.48550/arXiv.2501.12948. arXiv: 2501.12948[cs]. URL: http://arxiv.org/abs/2501.12948 (visited on 04/27/2025).

[Dee+25b] DeepSeek-AI et al. *DeepSeek-V3 Technical Report*. Feb. 18, 2025. DOI: 10.48550/arXiv.2412.19437. arXiv: 2412.19437[cs]. URL: http://arxiv.org/abs/2412.19437 (visited on 04/27/2025).

[Dev+19] Jacob Devlin et al. *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. May 24, 2019. DOI: 10.48550/arXiv.1810.04805. arXiv: 1810.04805[cs]. URL: http://arxiv.org/abs/1810.04805 (visited on 04/24/2025).

[Dou+24] Jacob Doughty et al. "A Comparative Study of AI-Generated (GPT-4) and Human-crafted MCQs in Programming Education". In: *Proceedings of the 26th Australasian Computing Education Conference*. ACE '24. New York, NY, USA: Association for Computing Machinery, Jan. 29, 2024, pp. 114–123. ISBN: 979-8-4007-1619-5. DOI: 10.1145/3636243.3636256. URL: https://dl.acm.org/doi/10.1145/3636243.3636256 (visited on 04/07/2025).

[DWK24] Nghia Duong-Trung, Xia Wang, and Miloš Kravčík. "BloomLLM: Large Language Models Based Question Generation Combining Supervised Fine-Tuning and Bloom's Taxonomy". In: *Technology Enhanced Learning for Inclusive and Equitable Quality Education*. Ed. by Rafael Ferreira Mello et al. Cham: Springer Nature Switzerland, 2024, pp. 93–98. ISBN: 978-3-031-72312-4. DOI: 10.1007/978-3-031-72312-4_11. (Visited on 04/10/2025).

[Elk+23] Sabina Elkins et al. "How Useful Are Educational Questions Generated by Large Language Models?" In: *Artificial Intelligence in Education. Posters and Late Breaking Results, Workshops and Tutorials, Industry and Innovation Tracks, Practitioners, Doctoral Consortium and Blue Sky*. Ed. by Ning Wang et al. Cham: Springer Nature Switzerland, 2023, pp. 536–542. ISBN: 978-3-031-36336-8. DOI: 10.1007/978-3-031-36336-8_83.

[Fle71] Joseph L. Fleiss. "Measuring nominal scale agreement among many raters". In: *Psychological Bulletin* 76.5 (1971). Place: US Publisher: American Psychological Association, pp. 378–382. ISSN: 1939-1455. DOI: 10.1037/h0031619.

[Guo+24] Shasha Guo et al. *A Survey on Neural Question Generation: Methods, Applications, and Prospects*. May 7, 2024. DOI: 10.48550/arXiv.2402.18267. arXiv: 2402.18267[cs]. URL: http://arxiv.org/abs/2402.18267 (visited on 04/10/2025).

[HWY24] Ching Nam Hang, Chee Wei Tan, and Pei-Duo Yu. "MCQGen: A Large Language Model-Driven MCQ Generator for Personalized Learning". In: *IEEE Access* 12 (2024), pp. 102261–102273. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3420709. URL: https://ieeexplore.ieee.org/document/10577164/ (visited on 04/09/2025).

[Hor+20] Andrea Horbach et al. "Linguistic Appropriateness and Pedagogic Usefulness of Reading Comprehension Questions". In: *Proceedings of the Twelfth Language Resources and Evaluation Conference*. LREC 2020. Ed. by Nicoletta Calzolari et al. Marseille, France: European Language Resources Association, May 2020, pp. 1753–1762. ISBN: 979-10-95546-34-4. URL: https://aclanthology.org/2020.lrec-1.217/ (visited on 05/12/2025).

[Hou+24] Xinyi Hou et al. "Large Language Models for Software Engineering: A Systematic Literature Review". In: *ACM Trans. Softw. Eng. Methodol.* 33.8 (Dec. 3, 2024), 220:1–220:79. ISSN: 1049-331X. DOI: 10.1145/3695988. URL: https://dl.acm.org/doi/10.1145/3695988 (visited on 04/06/2025).

[Kav25] Koray Kavukcuoglu. *Gemini 2.5: Our most intelligent AI model*. Google. Mar. 25, 2025. URL: https://blog.google/technology/google-deepmind/gemini-model-thinking-updates-march-2025/ (visited on 04/28/2025).

[Kon+24] Aobo Kong et al. *Better Zero-Shot Reasoning with Role-Play Prompting*. Mar. 14, 2024. DOI: 10.48550/arXiv.2308.07702. arXiv: 2308.07702[cs]. URL: http://arxiv.org/abs/2308.07702 (visited on 05/19/2025).

[KBM64] David R Krathwohl, Benjamin Bloom, and Bertram B Masia. *Taxonomy of educational objectives*. Vol. 2. David McKay Company, 1964.

[Kra02] David R. Krathwohl. "A Revision of Bloom's Taxonomy: An Overview". In: *Theory Into Practice* 41.4 (Nov. 1, 2002), pp. 212–218. ISSN: 0040-5841, 1543-0421. DOI: 10.1207/s15430421tip4104_2. URL: https://www.tandfonline.com/doi/full/10.1207/s15430421tip4104_2 (visited on 04/24/2025).

[Lew+19] Mike Lewis et al. *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension*. Oct. 29, 2019. DOI: 10.48550/arXiv.1910.13461. arXiv: 1910.13461[cs]. URL: http://arxiv.org/abs/1910.13461 (visited on 04/24/2025).

[Lew+20] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., 2020, pp. 9459–9474. URL: https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html (visited on 05/10/2025).

[Li+16] Jiwei Li et al. *A Diversity-Promoting Objective Function for Neural Conversation Models*. June 10, 2016. DOI: 10.48550/arXiv.1510.03055. arXiv: 1510.03055[cs]. URL: http://arxiv.org/abs/1510.03055 (visited on 04/29/2025).

[LZ24] Kunze Li and Yu Zhang. "Planning First, Question Second: An LLM-Guided Method for Controllable Question Generation". In: *Findings of the Association for Computational Linguistics: ACL 2024*. Findings 2024. Ed. by Lun-Wei Ku, Andre Martins, and Vivek Srikumar. Bangkok, Thailand: Association for Computational Linguistics, Aug. 2024, pp. 4715–4729. DOI: 10.18653/v1/2024.findings-acl.280. URL: https://aclanthology.org/2024.findings-acl.280/ (visited on 04/10/2025).

[Lin04] Chin-Yew Lin. "ROUGE: A Package for Automatic Evaluation of Summaries". In: *Text Summarization Branches Out*. Barcelona, Spain: Association for Computational Linguistics, July 2004, pp. 74–81. URL: https://aclanthology.org/W04-1013/ (visited on 04/29/2025).

[LA24] Jintao Ling and Muhammad Afzaal. "Automatic question-answer pairs generation using pre-trained large language models in higher education". In: *Computers and Education: Artificial Intelligence* 6 (June 1, 2024), p. 100252. ISSN: 2666-920X. DOI: 10.1016/j.caeai.2024.100252. URL: https://www.sciencedirect.com/science/article/pii/S2666920X24000559 (visited on 04/07/2025).

[Liu+25] Yiheng Liu et al. "Understanding LLMs: A comprehensive overview from training to inference". In: *Neurocomputing* 620 (Mar. 1, 2025), p. 129190. ISSN: 0925-2312. DOI: 10.1016/j.neucom.2024.129190. URL: https://www.sciencedirect.com/science/article/pii/S0925231224019611 (visited on 07/22/2025).

[Liu+19] Yinhan Liu et al. *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. July 26, 2019. DOI: 10.48550/arXiv.1907.11692. arXiv: 1907.11692[cs]. URL: http://arxiv.org/abs/1907.11692 (visited on 04/24/2025).

[Mad+23] Aman Madaan et al. "Self-Refine: Iterative Refinement with Self-Feedback". In: *Advances in Neural Information Processing Systems* 36 (Dec. 15, 2023), pp. 46534–46594. URL: https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html (visited on 05/09/2025).

[MDS25] Subhankar Maity, Aniket Deroy, and Sudeshna Sarkar. "Can large language models meet the challenge of generating school-level questions?" In: *Computers and Education: Artificial Intelligence* 8 (June 1, 2025), p. 100370. ISSN: 2666-920X. DOI: 10.1016/j.caeai.2025.100370. URL: https://www.sciencedirect.com/science/article/pii/S2666920X25000104 (visited on 04/07/2025).

[Met25] Meta AI. *The Llama 4 herd: The beginning of a new era of natively multimodal AI innovation*. 2025. URL: https://ai.meta.com/blog/llama-4-multimodal-intelligence/ (visited on 04/27/2025).

[ML24] Zejia Mi and Kangkang Li. "A Comparative Analysis of Different Large Language Models in Evaluating Student-Generated Questions". In: *2024 13th International Conference on Educational and Information Technology (ICEIT)*. 2024 13th International Conference on Educational and Information Technology (ICEIT). Mar. 2024, pp. 24–29. DOI: 10.1109/ICEIT61397.2024.10540914. URL: https://ieeexplore.ieee.org/document/10540914 (visited on 04/04/2025).

[Min+25] Shervin Minaee et al. *Large Language Models: A Survey*. Mar. 23, 2025. DOI: 10.48550/arXiv.2402.06196. arXiv: 2402.06196[cs]. URL: http://arxiv.org/abs/2402.06196 (visited on 04/04/2025).

[Moh+23] Alireza Mohammadshahi et al. *RQUGE: Reference-Free Metric for Evaluating Question Generation by Answering the Question*. May 26, 2023. DOI: 10.48550/arXiv.2211.01482. arXiv: 2211.01482[cs]. URL: http://arxiv.org/abs/2211.01482 (visited on 04/29/2025).

[Moo+22] Steven Moore et al. "Assessing the Quality of Student-Generated Short Answer Questions Using GPT-3". In: *Educating for a New Future: Making Sense of Technology-Enhanced Learning Adoption*. Ed. by Isabel Hilliger et al. Cham: Springer International Publishing, 2022, pp. 243–257. ISBN: 978-3-031-16290-9. DOI: 10.1007/978-3-031-16290-9_18. (Visited on 04/23/2025).

[NNN24] Dishita Naik, Ishita Naik, and Nitin Naik. "Large Data Begets Large Data: Studying Large Language Models (LLMs) and Its History, Types, Working, Benefits and Limitations". In: *Contributions Presented at The International Conference on Computing, Communication, Cybersecurity and AI, July 3–4, 2024, London, UK*. Ed. by Nitin Naik et al. Cham: Springer Nature Switzerland, 2024, pp. 293–314. ISBN: 978-3-031-74443-3. DOI: 10.1007/978-3-031-74443-3_18. (Visited on 04/23/2025).

[Nav+24] Humza Naveed et al. *A Comprehensive Overview of Large Language Models*. Oct. 17, 2024. DOI: 10.48550/arXiv.2307.06435. arXiv: 2307.06435[cs]. URL: http://arxiv.org/abs/2307.06435 (visited on 04/06/2025).

[Ngu+24] Bang Nguyen et al. *Reference-based Metrics Disprove Themselves in Question Generation*. Oct. 10, 2024. DOI: 10.48550/arXiv.2403.12242. arXiv: 2403.12242[cs]. URL: http://arxiv.org/abs/2403.12242 (visited on 04/04/2025).

[Ope22] OpenAI. *Introducing ChatGPT*. Nov. 30, 2022. URL: https://openai.com/index/chatgpt/ (visited on 04/24/2025).

[Ope24] OpenAI. *OpenAI o1-mini*. 2024. URL: https://openai.com/index/openai-o1-mini-advancing-cost-efficient-reasoning/ (visited on 04/28/2025).

[Ope25a] OpenAI. *OpenAI o3 and o4-mini System Card*. 2025. URL: https://cdn.openai.com/pdf/2221c875-02dc-4789-800b-e7758f3722c1/o3-and-o4-mini-system-card.pdf (visited on 04/28/2025).

[Ope25b] OpenAI. *OpenAI o3-mini*. 2025. URL: https://openai.com/index/openai-o3-mini/ (visited on 04/28/2025).

[Ope+24a] OpenAI et al. *GPT-4 Technical Report*. Mar. 4, 2024. DOI: 10.48550/arXiv.2303.08774. arXiv: 2303.08774[cs]. URL: http://arxiv.org/abs/2303.08774 (visited on 04/24/2025).

[Ope+24b] OpenAI et al. *GPT-4o System Card*. Oct. 25, 2024. DOI: 10.48550/arXiv.2410.21276. arXiv: 2410.21276[cs]. URL: http://arxiv.org/abs/2410.21276 (visited on 04/28/2025).

[Ope+24c] OpenAI et al. *OpenAI o1 System Card*. Dec. 21, 2024. DOI: 10.48550/arXiv.2412.16720. arXiv: 2412.16720[cs]. URL: http://arxiv.org/abs/2412.16720 (visited on 04/28/2025).

[Pap+01] Kishore Papineni et al. "BLEU: a method for automatic evaluation of machine translation". In: *Proceedings of the 40th Annual Meeting on Association for Computational Linguistics - ACL '02*. the 40th Annual Meeting. Philadelphia, Pennsylvania: Association for Computational Linguistics, 2001, p. 311. DOI: 10.3115/1073083.1073135. URL: http://portal.acm.org/citation.cfm?doid=1073083.1073135 (visited on 04/29/2025).

[PG24] Rajvardhan Patil and Venkat Gudivada. "A Review of Current Trends, Techniques, and Challenges in Large Language Models (LLMs)". In: *Applied Sciences* 14.5 (Jan. 2024). Number: 5 Publisher: Multidisciplinary Digital Publishing Institute, p. 2074. ISSN: 2076-3417. DOI: 10.3390/app14052074. URL: https://www.mdpi.com/2076-3417/14/5/2074 (visited on 04/04/2025).

[Pic23] Sundar Pichai. *An important next step on our AI journey*. Google. Feb. 6, 2023. URL: https://blog.google/technology/ai/bard-google-ai-search-updates/ (visited on 04/26/2025).

[PHK24] Sundar Pichai, Demis Hassabis, and Koray Kavukcuoglu. *Introducing Gemini 2.0: our new AI model for the agentic era*. Google. Dec. 11, 2024. URL: https://blog.google/technology/google-deepmind/google-gemini-ai-update-december-2024/ (visited on 04/28/2025).

[Raf+23] Colin Raffel et al. *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. Sept. 19, 2023. DOI: 10.48550/arXiv.1910.10683. arXiv: 1910.10683[cs]. URL: http://arxiv.org/abs/1910.10683 (visited on 04/24/2025).

[Rai+24] Mohaimenul Azam Khan Raiaan et al. "A Review on Large Language Models: Architectures, Applications, Taxonomies, Open Issues and Challenges". In: *IEEE Access* 12 (2024), pp. 26839–26874. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3365742. URL: https://ieeexplore.ieee.org/document/10433480 (visited on 04/06/2025).

[RG19] Nils Reimers and Iryna Gurevych. *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks*. Aug. 27, 2019. DOI: 10.48550/arXiv.1908.10084. arXiv: 1908.10084[cs]. URL: http://arxiv.org/abs/1908.10084 (visited on 05/23/2025).

[SCS24] Nicy Scaria, Suma Dharani Chenna, and Deepak Subramani. "How Good are Modern LLMs in Generating Relevant and High-Quality Questions at Different Bloom's Skill Levels for Indian High School Social Science Curriculum?" In: *Proceedings of the 19th Workshop on Innovative Use of NLP for Building Educational Applications (BEA 2024)*. BEA 2024. Ed. by Ekaterina Kochmar et al. Mexico City, Mexico: Association for Computational Linguistics, June 2024, pp. 1–10. URL: https://aclanthology.org/2024.bea-1.1/ (visited on 04/07/2025).

[SDS24] Nicy Scaria, Suma Dharani Chenna, and Deepak Subramani. "Automated Educational Question Generation at Different Bloom's Skill Levels Using Large Language Models: Strategies and Evaluation". In: *Artificial Intelligence in Education*. Ed. by Andrew M. Olney et al. Cham: Springer Nature Switzerland, 2024, pp. 165–179. ISBN: 978-3-031-64299-9. DOI: 10.1007/978-3-031-64299-9_12. (Visited on 04/07/2025).

[SDP20] Thibault Sellam, Dipanjan Das, and Ankur P. Parikh. *BLEURT: Learning Robust Metrics for Text Generation*. May 21, 2020. DOI: 10.48550/arXiv.2004.04696. arXiv: 2004.04696[cs]. URL: http://arxiv.org/abs/2004.04696 (visited on 04/29/2025).

[Sha+24] Minghao Shao et al. "Survey of Different Large Language Model Architectures: Trends, Benchmarks, and Challenges". In: *IEEE Access* 12 (2024). Conference Name: IEEE Access, pp. 188664–188706. ISSN: 2169-3536. DOI: 10.1109/ACCESS.2024.3482107. URL: https://ieeexplore.ieee.org/document/10720163/?arnumber=10720163 (visited on 04/04/2025).

[Sha+17] Noam Shazeer et al. *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. Jan. 23, 2017. DOI: 10.48550/arXiv.1701.06538. arXiv: 1701.06538[cs]. URL: http://arxiv.org/abs/1701.06538 (visited on 04/27/2025).

[Ste+21] Tim Steuer et al. "I Do Not Understand What I Cannot Define: Automatic Question Generation With Pedagogically-Driven Content Selection". In: *ArXiv* (Oct. 8, 2021). URL: https://www.semanticscholar.org/paper/I-Do-Not-Understand-What-I-Cannot-Define%3A-Automatic-Steuer-Filighera/a0fd69c2540158fe79ff8cdba3d36a71e737211a (visited on 04/04/2025).

[TW13] Andrew Tanenbaum and David Wetherall. *Computer Networks*. Pearson Deutschland, 2013. 808 pp. ISBN: 978-1-292-02422-6. URL: https://elibrary.pearson.de/book/99.150005/9781292037189 (visited on 04/28/2025).

[Tea+24a] Gemini Team et al. *Gemini 1.5: Unlocking multimodal understanding across millions of tokens of context*. Dec. 16, 2024. DOI: 10.48550/arXiv.2403.05530. arXiv: 2403.05530[cs]. URL: http://arxiv.org/abs/2403.05530 (visited on 04/28/2025).

[Tea+24b] Gemini Team et al. *Gemini: A Family of Highly Capable Multimodal Models*. June 17, 2024. DOI: 10.48550/arXiv.2312.11805. arXiv: 2312.11805[cs]. URL: http://arxiv.org/abs/2312.11805 (visited on 04/28/2025).

[Tea25a] Qwen Team. *<think>...</think> QwQ-Max-Preview*. Qwen. Section: blog. Feb. 25, 2025. URL: https://qwenlm.github.io/blog/qwq-max-preview/ (visited on 04/28/2025).

[Tea25b] Qwen Team. *Qwen2.5-Max: Exploring the Intelligence of Large-scale MoE Model*. Qwen. Section: blog. Jan. 28, 2025. URL: https://qwenlm.github.io/blog/qwen2.5-max/ (visited on 04/28/2025).

[Tho+22] Romal Thoppilan et al. *LaMDA: Language Models for Dialog Applications*. Feb. 10, 2022. DOI: 10.48550/arXiv.2201.08239. arXiv: 2201.08239[cs]. URL: http://arxiv.org/abs/2201.08239 (visited on 04/26/2025).

[Tou+23a] Hugo Touvron et al. *Llama 2: Open Foundation and Fine-Tuned Chat Models*. July 19, 2023. DOI: 10.48550/arXiv.2307.09288. arXiv: 2307.09288[cs]. URL: http://arxiv.org/abs/2307.09288 (visited on 04/28/2025).

[Tou+23b] Hugo Touvron et al. *LLaMA: Open and Efficient Foundation Language Models*. Feb. 27, 2023. DOI: 10.48550/arXiv.2302.13971. arXiv: 2302.13971[cs]. URL: http://arxiv.org/abs/2302.13971 (visited on 04/26/2025).

[Vas+17] Ashish Vaswani et al. "Attention is All you Need". In: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc., 2017. URL: https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html (visited on 04/06/2025).

[Vu+24] Sinh Trong Vu et al. "A ChatGPT-based approach for questions generation in higher education". In: *Proceedings of the 1st ACM Workshop on AI-Powered Q&A Systems for Multimedia*. AIQAM '24. New York, NY, USA: Association for Computing Machinery, June 10, 2024, pp. 13–18. ISBN: 979-8-4007-0547-2. DOI: 10.1145/3643479.3662056. URL: https://dl.acm.org/doi/10.1145/3643479.3662056 (visited on 04/07/2025).

[Wan+24] Zichong Wang et al. "History, development, and principles of large language models: an introductory survey". In: *AI and Ethics* (Oct. 14, 2024). ISSN: 2730-5961. DOI: 10.1007/s43681-024-00583-7. URL: https://doi.org/10.1007/s43681-024-00583-7 (visited on 04/06/2025).

[Wei+22] Jason Wei et al. "Chain-of-Thought Prompting Elicits Reasoning in Large Language Models". In: *Advances in Neural Information Processing Systems* 35 (Dec. 6, 2022), pp. 24824–24837. URL: https://proceedings.neurips.cc/paper_files/paper/2022/hash/9d5609613524ecf4f15af0f7b31abca4-Abstract-Conference.html?ref=https://githubhelp.com (visited on 05/09/2025).

[xAI24a] xAI. *Grok-2 Beta Release*. 2024. URL: https://x.ai/news/grok-2 (visited on 04/28/2025).

[xAI24b] xAI. *Open Release of Grok-1*. 2024. URL: https://x.ai/news/grok-os (visited on 04/27/2025).

[xAI25] xAI. *Grok 3 Beta — The Age of Reasoning Agents*. 2025. URL: https://x.ai/news/grok-3 (visited on 04/28/2025).

[YZ24] Zhihao Yang and Zhengzhou Zhu. "Heuristic question sequence generation based on retrieval augmentation". In: *Education and Lifelong Development Research* 1.2 (June 25, 2024), pp. 72–81. ISSN: 30069599, 30069602. DOI: 10.46690/elder.2024.02.03. URL: https://elder.yandypress.com/index.php/3006-9599/article/view/19 (visited on 04/07/2025).

[Zha+20] Tianyi Zhang et al. *BERTScore: Evaluating Text Generation with BERT*. Feb. 24, 2020. doi: 10.48550/arXiv.1904.09675. arXiv: 1904.09675[cs]. url: http://arxiv.org/abs/1904.09675 (visited on 04/29/2025).

[Zha+19] Zhengyan Zhang et al. *ERNIE: Enhanced Language Representation with Informative Entities*. June 4, 2019. doi: 10.48550/arXiv.1905.07129. arXiv: 1905.07129[cs]. url: http://arxiv.org/abs/1905.07129 (visited on 04/24/2025).

[Zha+25] Wayne Xin Zhao et al. *A Survey of Large Language Models*. Mar. 11, 2025. doi: 10.48550/arXiv.2303.18223. arXiv: 2303.18223[cs]. url: http://arxiv.org/abs/2303.18223 (visited on 04/06/2025).

[ZWC25] Qingfeng Zhuge, Han Wang, and Xuyang Chen. "TwinStar: A Novel Design for Enhanced Test Question Generation Using Dual-LLM Engine". In: *Applied Sciences* 15.6 (Jan. 2025). Number: 6 Publisher: Multidisciplinary Digital Publishing Institute, p. 3055. issn: 2076-3417. doi: 10.3390/app15063055. url: https://www.mdpi.com/2076-3417/15/6/3055 (visited on 04/09/2025).

Appendices

A Anchor Example Questions

1.1 First Experiment

Listing A.1: Experiment 1a: Supervisor-rated positive Example

```
Frage: Welche zwei Hauptverantwortungsbereiche der Netzwerkschicht werden im Text genannt, die
↪ besonders bei der Verbindung heterogener Netzwerke kritisch sind?

Antwortmöglichkeiten:
- Bewältigung unterschiedlicher Adressierungsschemata zwischen Netzwerken und Anpassung an
↪ Protokollunterschiede (Richtig)
- Durchführung von Fehlerkorrektur auf Bit-Ebene und Verwaltung physikalischer Verbindungen
↪ (Falsch)
- Steuerung der Ende-zu-Ende-Flusskontrolle und Garantie der Anwendungsdatenintegrität (Falsch)
- Umgehung von Paketgrößenbeschränkungen und Lösung von Inkompatibilitäten bei der Paketübergabe
↪ (Richtig)
- Verschlüsselung von Nutzdaten und Authentifizierung von Endgeräten (Falsch)
```

This question by DeepSeek, using the complex prompt and the Tanenbaum source, is a good example, as it is based on the source material and demonstrates a good understanding of the content. However, it is also noted that the question is not that challenging, since many of the experiment's questions are less challenging. This question is also one of many questions which include a phrasing like "according to the source material", which is an unwanted pattern in the question generation process.

Listing A.2: Experiment 1a: Staff- & Supervisor-rated positive Example

```
Frage: Welche der folgenden Aufgaben gehören zu den Kernaufgaben des Physical Layers im
↪  ISO/OSI-Modell?

Antwortmöglichkeiten:
- Codierung logischer Werte in physikalische Größen (Richtig)
- Transport physikalischer Größen von einem Ort zum anderen (Richtig)
- Dekodierung physikalischer Größen zu logischen Werten (Richtig)
- Verschlüsselung der Daten zur Gewährleistung der Vertraulichkeit (Falsch)
- Sitzungsaufbau, -steuerung und -abbau zwischen Endsystemen (Falsch)
```

This question by OpenAI, using the common prompt, is noted as a good example, as it demonstrates a good understanding of the source material, and it is of high quality.

Listing A.3: Experiment 1a: Supervisor-rated negative Example (NaN Values)

```
Frage: Der Text beschreibt die drei Kernaufgaben des Physical Layers: Codierung, Transport und Dekodierung. Insbesondere wird beim Transport die Unterscheidung zwischen mediengebundenen und medienfreien Übertragungen hervorgehoben, mit einem Verweis auf die "Doppelfunktion" von Licht. Welche spezifischen Herausforderungen ergeben sich aus dieser Dualität - der Nutzung von Licht sowohl mediengebunden als auch medienfrei - für die Konzeption der Codierungs- und Dekodierungsverfahren auf dem Physical Layer, die bei rein mediengebundenen Technologien wie einem Ethernet-Kabel weniger prominent wären?

Antwortmöglichkeiten:
- Bei medienfreier Lichtübertragung, wie sie im freien Raum stattfindet, müssen die Codierungsverfahren robuster gegenüber äußeren Einflüssen wie atmosphärischen Störungen, Abschattungen oder Interferenzen durch andere Lichtquellen gestaltet werden. Dies erfordert oft komplexere Fehlermanagement-Strategien auf dem Physical Layer selbst, die bei einer Übertragung über Glasfaserkabel (mediengebunden) aufgrund der isolierten Umgebung weniger kritisch sind. (Richtig)
- Die "Doppelfunktion" des Lichts hat keine signifikanten Auswirkungen auf die Codierungs- und Dekodierungsverfahren, da diese primär die Abbildung logischer auf physikalische Werte und deren Rückinterpretation betreffen, unabhängig vom Übertragungsmedium. (Falsch)
- Die Wahl zwischen mediengebundener und medienfreier Lichtübertragung beeinflusst maßgeblich die Detektionsbereiche und Abtastthematik bei der Dekodierung. Im freien Raum ist die Signalstärke variabler und die präzise Bestimmung von Schwellenwerten (Detektionsbereichen) komplexer als in einem kontrollierten Lichtwellenleiter, wo das Signal typischerweise gedämpfter, aber stabiler ist. Dies erfordert adaptive Dekodierungsmechanismen. (Richtig)
- Ein wesentlicher Unterschied besteht darin, dass bei medienfreier Lichtübertragung die Frequenzmodulation des Lichts zur Codierung verwendet werden muss, während bei mediengebundener Übertragung ausschließlich Amplitudenmodulation zum Einsatz kommt. (Falsch)
- Die Herausforderung der "Doppelfunktion" von Licht liegt primär in der Wahl des geeigneten Sende- und Empfangsgerätes und hat keine direkten Auswirkungen auf die Implementierung der Codierungs- und Dekodierungslogik. (Falsch)
```

Google's question, using the complex prompt via the transcript, also includes a phrasing like "the text describes", but it is noted as a NaN example, as the model tried to generate a question full of technical terms, while it sounds like the model did not understand the terms at all.

Listing A.4: Experiment 1a: Staff-rated negative Example

```
Frage: Welche der folgenden Aufgaben gehört *nicht* zu den Kernaufgaben des Data Link Layer
↪ (Schicht 2) gemäß dem gegebenen Text?

Antwortmöglichkeiten:
- Erkennen, Beheben oder Signalisieren von Übertragungsfehlern. (Falsch)
- Strategien, wenn Sender für Empfänger zu schnell sendet. (Falsch)
- Zusammenfassen einzelner Bits zu größeren Einheiten wie 8-bit Bytes. (Falsch)
- Das Weiterleiten von Datenpaketen zwischen verschiedenen Netzwerken. (Richtig)
- Auswählen bestimmter Sender oder Empfänger anhand einer Adresse. (Falsch)
```

This question by Google, using the common prompt, is noted as a bad example by the staff members. It is based on a negation in the question, which is not a good linguistic practice for valuable question generation.

Listing A.5: Experiment 1b: Supervisor- & Staff-rated positive Example

```
Frage: Der vorliegende Text beschreibt die Kernaufgaben des Session Layers (Schicht 5) mit einem Fokus auf "verbindungslosen Datenaustausch", "Datagramme", "Asynchronität" und "Atomisierung". Analysieren Sie kritisch, welche Implikationen diese spezifische Definition für die Rolle und Interaktion des Session Layers im Vergleich zu einer traditionellen, verbindungsorientierten OSI-Architektur hat.

Antwortmöglichkeiten:
- Diese Beschreibung widerspricht der traditionellen Rolle des Session Layers, der primär für Dialogkontrolle, Synchronisation und Wiederherstellung von Verbindungen zuständig ist. Die genannten Funktionen ähneln eher den Aufgaben von Transportschichtprotokollen wie UDP. (Richtig)
- Ein Session Layer, der sich auf verbindungslosen Datenaustausch konzentriert, würde die Notwendigkeit komplexer Handshake-Mechanismen auf höheren Schichten reduzieren, da die Sitzung selbst keine Zustandsinformationen für die Datenübertragung vorhalten müsste. (Richtig)
- Die Betonung von "Asynchronität" und dem "Vermeiden von Zwischenpunkten und Bestätigungen" bedeutet, dass die Zuverlässigkeit der Datenübermittlung und die korrekte Reihenfolge der Datenpakete vollständig von den höheren Schichten (z.B. der Präsentations- oder Anwendungsschicht) oder von außerhalb des OSI-Modells gewährleistet werden müssten. (Richtig)
- Die Funktion "Atomisierung" würde bedeuten, dass die Session Layer für die Fehlerkorrektur von korrupten Einzelteilen der Datenpakete zuständig wäre, bevor diese an die Transportschicht weitergeleitet werden. (Falsch)
- Ein solcher verbindungsloser Session Layer würde die direkte Kommunikation zwischen Prozessen auf Anwendungsebene ohne die Notwendigkeit von Ports oder Sockets ermöglichen, da diese Konzepte in der Session Layer nativ implementiert wären. (Falsch)
```

This complex prompt question by Google demonstrates – with a lack in linguistic quality – distancing from the manipulated script by contrasting the manipulated script with the “traditional” OSI architecture.

Listing A.6: Experiment 1b: Supervisor-rated negative Example

```
Frage: Welche der folgenden Aufgaben gehören laut Beschreibung zu den 5 Kernaufgaben des Data Link
↪  Layer 2?

Antwortmöglichkeiten:
- Fehlererzeugung (Richtig)
- Datenstau (Richtig)
- Fragmentierung (Richtig)
- Anonymisierung (Richtig)
- Medienverweigerung (Richtig)
- Routing (Falsch)
- Session Management (Falsch)
```

This question by OpenAI, using the common prompt, offers a decent style in asking, but regarding the content, it fully follows the manipulated script without any distancing.

Listing A.7: Experiment 1b: Supervisor-rated NaN Example

```
Frage: Welche der folgenden Funktionen der Präsentationsschicht werden in der oben beschriebenen
↪  Spezifikation explizit NICHT angeboten? Wählen Sie alle zutreffenden.

Antwortmöglichkeiten:
- Verschlüsselung von Daten (Richtig)
- Kompression (Komprimierung) von Daten (Richtig)
- Zeichencodierungskonvertierung, z. B. ASCII ↔ Unicode (Richtig)
- Vereinheitlichung der Byte-Reihenfolge auf big endian (Falsch)
- Entschlüsselung von Daten (Falsch)
- Entpacken (Expansion) komprimierter Daten (Falsch)
```

In this question by OpenAI, using the complex prompt, the distancing from the manipulated content via its negation is confusing, which makes the categorization difficult for the supervisor, leading to a NaN classification.

1.2 Second Experiment

Listing A.8: Experiment 2a: Student-rated positive Example

```
Frage: Welche der folgenden Zuordnungen zwischen OSI-Schicht und Kernaufgabe sind korrekt? Wählen
↪  Sie alle zutreffenden Antworten.

Antwortmöglichkeiten:
a) Codierung/Decodierung logischer Werte - Physical Layer 1 (Richtig)
b) Medienzugriff - Network Layer 3 (Falsch)
c) Fehlerkontrolle - Data Link Layer 2 (Richtig)
d) Endian-Ness-Konvertierung - Presentation Layer 6 (Richtig)
e) Dienstqualität-Garantien - Session Layer 5 (Falsch)
f) Sitzungsaufbau und -beendigung - Application Layer 7 (Falsch)
```

This question by OpenAI demonstrates a good example of a clear and precise multiple-choice question, even if it is a less challenging one.

Listing A.9: Experiment 2a: Student-rated negative Example

```
Frage: Erläutern Sie die unterschiedlichen Wirkungsradien der Fehlerkontrolle im Data Link Layer
↪ und der Dienstqualitätsfunktionen im Transport Layer. Warum genügt eine lückenlose
↪ Fehlerkontrolle auf Schicht 2 nicht, um die in Schicht 4 geforderte Quality of Service (QoS)
↪ für eine Ende-zu-Ende-Verbindung sicherzustellen? Skizzieren Sie einen schichtübergreifenden
↪ Lösungsansatz, der gleichzeitig niedrige Bitfehlerraten, garantierte Bandbreite und maximale
↪ Latenzgrenzen einhält.

Antwort: Die Fehlerkontrolle in Schicht 2 wirkt nur segmentweise: CRC, ARQ oder FEC korrigieren
↪ Bit- und Frame-Fehler auf der jeweiligen Punkt-zu-Punkt-Strecke, ohne Kenntnis des Gesamtpfades.
↪ Ende-zu-Ende-Eigenschaften wie Gesamtverzögerung, Jitter und Durchsatz werden jedoch durch alle
↪ Teilstrecken, deren Queuing-Zustände und das Routing beeinflusst; sie liegen daher außerhalb
↪ des Einflussbereichs einer einzelnen Link-Ebene. Schicht 4 muss deshalb
↪ Dienstqualitätsparameter Ende-zu-Ende sichern, z. B. durch Ressourcenreservierung, Fluss- und
↪ Stausteuerung, Sequenzierung und paketübergreifende Verlustbehandlung. Ein konsistenter
↪ Lösungsansatz koppelt a) robuste Fehlerkontrolle und selektives ARQ auf jeder Link-Strecke, b)
↪ priorisierte oder reservierte Frames in Schicht 2 (z. B. 802.1Q/p), c) QoS-fähiges Routing oder
↪ DiffServ/MPLS in Schicht 3 zur Bandbreitengarantie, und d) Transportprotokolle mit
↪ RTT-basierter Fluss- und Stausteuerung sowie Timeout-Retransmission (z. B. TCP, RTP mit RSVP).
↪ Erst dieses Zusammenspiel hält die Bitfehlerrate niedrig, stellt reservierte Bandbreite bereit
↪ und begrenzt die End-to-End-Latenz. (Richtig)
```

This question by OpenAI demonstrates a bad example of an open-ended question, as it contains too many operator verbs. Moreover, the question is too long, it is hard to understand, and it is full of technical terms.

Listing A.10: Experiment 2b: Student-rated positive Example

```
Frage: Vergleichen Sie die Aufgaben der Flusskontrolle auf den verschiedenen Schichten des OSI-Modells und erklären Sie, worin der wesentliche Unterschied zwischen der Flusskontrolle auf Schicht 2 (Data Link Layer), Schicht 3 (Network Layer) und Schicht 4 (Transport Layer) besteht.

Antwortmöglichkeiten:
a) Die Flusskontrolle auf Schicht 2 befasst sich mit Strategien, wenn der Sender für den Empfänger zu schnell sendet (z.B. Zwischenpuffern), während auf Schicht 3 zwischen End-to-End-Flusssteuerung (vom Sender zum entfernten Empfänger) und Node-to-Node-Flusssteuerung (von jedem Knoten zum Nächsten) unterschieden wird. Auf Schicht 4 wird die Flusskontrolle auf die spezifischen Charakteristiken der Teilnehmer zugeschnitten. (Richtig)
b) Die Flusskontrolle ist auf allen drei Schichten identisch und dient lediglich dazu, die Übertragungsgeschwindigkeit zu regulieren. (Falsch)
c) Die Flusskontrolle auf Schicht 2 bezieht sich auf End-to-End-Verbindungen, während sie auf Schicht 3 für Node-to-Node-Verbindungen zuständig ist. Schicht 4 bietet keine Flusskontrolle. (Falsch)
d) Die Flusskontrolle auf Schicht 2 kümmert sich um Fehlerkorrektur, auf Schicht 3 um Routing und auf Schicht 4 um Verschlüsselung der Daten. (Falsch)
```

This question by Anthropic demonstrates a good example of a question for Bloom level 2, as it requires proper understanding of knowledge.

Listing A.11: Experiment 2b: Student-rated negative Example

```
Frage: Ein multinationales Unternehmen plant die Einführung eines neuen, hochsicheren und latenzarmen Kommunikationssystems für die Steuerung kritischer Infrastrukturen, das weltweit über heterogene Netzwerke hinweg funktionieren muss. Dieses System soll speziell für die Übertragung von Echtzeit-Sensor- und Aktor-Daten optimiert werden, die eine garantierte Dienstqualität (QoS) und Schutz vor Manipulation erfordern. Entwerfen Sie, unter Berücksichtigung der Kernaufgaben relevanter ISO-OSI-Schichten, eine konzeptionelle Skizze der notwendigen Mechanismen und Funktionen. Begründen Sie dabei präzise, welche Kernaufgaben welcher Schichten Sie zur Erfüllung der Anforderungen (Sicherheit, Latenz, QoS, heterogene Netzwerke, Echtzeit-Sensor-/Aktor-Daten) nutzen und wie diese zusammenwirken, um ein kohärentes Gesamtsystem zu erstellen.

Antwort: Zur Erfüllung der Anforderungen des hochsicheren und latenzarmen Kommunikationssystems für kritische Infrastrukturen können die Kernaufgaben verschiedener ISO-OSI-Schichten wie folgt konzeptionell zusammengeführt werden:

Physical Layer (Schicht 1):
Hier muss eine physikalische Konzeption gewählt werden, die eine extrem niedrige Latenz und hohe Bandbreite ermöglicht. Dies kann durch die Auswahl geeigneter Medien (z.B. Glasfaser für geringe Latenz) und effizienter Codierungsverfahren (z.B. NRZ-Varianten zur Maximierung der Datenrate) erreicht werden, um die physikalische Übertragung der Sensor- und Aktor-Daten zu gewährleisten.

Data Link Layer (Schicht 2):
Um eine fehlergesicherte Übertragung über möglicherweise nicht fehlergesicherte physikalische Verbindungen zu garantieren, sind die Kernaufgaben Fehlerkontrolle und Flusskontrolle entscheidend. Eine schnelle Fehlererkennung und -behebung (z.B. durch FEC - Forward Error Correction oder schnelle Retransmission bei kleinen Fehlern) minimiert die Latenz durch erneute Übertragung. Die Segmentierung ist für die Zerlegung großer Sensorpakete in kleinere, handhabbare Einheiten notwendig, während die Identifikation die Geräte innerhalb eines lokalen Netzwerksegments adressiert.

Network Layer (Schicht 3):
Da die Kommunikation weltweit über heterogene Netzwerke erfolgen soll, ist die Netzwerk-übergreifende Adressierung und das Routing unerlässlich. Das Routing muss für Echtzeitdaten optimiert sein, um Pfade mit der geringsten Latenz und höchster Zuverlässigkeit zu wählen. Eine angepasste Flusssteuerung auf dieser Ebene (Node-to-Node) kann Staus in Zwischenknoten verhindern und so zur Latenzstabilität beitragen.

Transport Layer (Schicht 4):
```

```
Die Anforderung an eine garantierte Dienstqualität (QoS) wird maßgeblich auf dieser Schicht realisiert. Die Dienstqualität muss so konfiguriert werden, dass Bandbreite, Latenz und Fehlerrate für die Echtzeit-Sensor- und Aktor-Daten garantiert werden. Dies kann durch Mechanismen wie Ressourcenzusicherung und Priorisierung geschehen. Eine angepasste Flusssteuerung unter Nutzung der Teilnehmer-Charakteristik sorgt dafür, dass Sender und Empfänger optimal aufeinander abgestimmt sind und keine Pufferüberläufe entstehen, die die Latenz negativ beeinflussen könnten. Die Teilnehmer-Adressierung identifiziert die spezifischen Anwendungen oder Prozesse auf den Endgeräten.

Session Layer (Schicht 5):
Für die Steuerung kritischer Infrastrukturen sind langlebige, aber bei Bedarf schnell wiederherstellbare Kommunikationsbeziehungen wichtig. Die Sitzungen ermöglichen das effiziente Aufbauen, Verwalten und Beenden dieser Beziehungen. Synchronisation durch das Setzen von Zwischenpunkten kann bei Ausfällen helfen, den Zustand schnell wiederherzustellen und Datenverlust zu minimieren, was für die Systemintegrität entscheidend ist. Die Bündelung zusammengehöriger Transaktionen verbessert die Effizienz und Konsistenz.

Presentation Layer (Schicht 6):
Die Sicherheit vor Manipulation wird hier durch Verschlüsselung und Kompression unterstützt. Die Verschlüsselung schützt die Vertraulichkeit und Integrität der Sensor- und Aktor-Daten während der Übertragung. Kompression reduziert die zu übertragende Datenmenge, was die Bandbreitennutzung optimiert und somit indirekt zur Latenzreduzierung beiträgt. Die Konvertierung der Datendarstellung (Endian-Ness) und Zeichendarstellung stellt die Kompatibilität zwischen heterogenen Systemen sicher.

Application Layer (Schicht 7):
Auf dieser Schicht werden die Grundkonzepte, Protokollelemente und das Verhalten der spezifischen Anwendung für die Steuerung der kritischen Infrastrukturen definiert. Dies umfasst die Semantik der Sensorwerte, die Befehle an die Aktoren und die spezifischen Interaktionsregeln. Die hier entwickelten Protokolle müssen die Echtzeitanforderungen des Systems direkt widerspiegeln und die vom Transport Layer bereitgestellte QoS nutzen.

Durch die konsequente Integration und Abstimmung dieser Kernaufgaben über die Schichten hinweg kann ein robustes, sicheres und latenzarmes Kommunikationssystem für kritische Infrastrukturen konstruiert werden. (Richtig)
```

This question by Google demonstrates a bad example of a question for Bloom level 6, as it is hard to answer. The question has too many technical terms, and is overly complex.

Listing A.12: Experiment 2c: Student-rated positive Example

```
Frage: Eine Kommunikationsanwendung sendet Daten über ein Netzwerk. Prüfen Sie, inwiefern die unterschiedlichen Konzepte der Flusskontrolle in den Schichten des ISO-OSI-Modells zur Sicherstellung eines geordneten Datenaustauschs beitragen und welche spezifischen Probleme sie jeweils adressieren.

Antwortmöglichkeiten:
a) Die Flusskontrolle des Data Link Layers (Schicht 2) zielt darauf ab, die Übertragung zwischen direkt verbundenen Geräten zu steuern, um eine Überflutung des Empfängers auf dieser lokalen Verbindung zu verhindern, wohingegen die Flusssteuerung des Network Layers (Schicht 3) Strategien für die Netz-übergreifende Datenübermittlung zwischen Endgeräten (End-to-End) oder zwischen Vermittlungsknoten (Node-to-Node) bereitstellt, um eine Überlastung im gesamten Netzwerkpfad zu regulieren. (Richtig)
b) Die Flusskontrolle im Physical Layer (Schicht 1) und im Transport Layer (Schicht 4) sind primär für die synchrone Bereitstellung von Medienzugriffsmethoden verantwortlich, um die physikalische Übertragungsrate an die Verarbeitungsgeschwindigkeit des Empfängers anzupassen. (Falsch)
c) Der Session Layer (Schicht 5) und der Presentation Layer (Schicht 6) sind die einzigen Schichten, die End-to-End-Flusskontrolle auf Applikationsebene implementieren, um eine effiziente Datenkompression und Verschlüsselung zu gewährleisten. (Falsch)
d) Die Flusssteuerung des Transport Layers (Schicht 4) unterscheidet sich von den unteren Schichten, indem sie spezifisch die Charakteristika der teilnehmenden Anwendungen nutzt, um die Dienstqualität (QoS) für die gesamte Kommunikationsbeziehung zwischen den Endteilnehmern zu optimieren. (Richtig)
e) Alle Formen der Flusskontrolle im ISO-OSI-Modell dienen ausschließlich der Behebung von Übertragungsfehlern und haben keinen Einfluss auf die Geschwindigkeit, mit der Daten zwischen Sender und Empfänger ausgetauscht werden. (Falsch)
```

This question by Google demonstrates a good example of a multiple-choice question with a Bloom level of 4. It is a valuable question, while being quite challenging.

Listing A.13: Experiment 2c: Student-rated negative Example

```
Frage: Analysieren Sie die Beziehungen zwischen den Flusskontrollmechanismen in den verschiedenen Schichten des OSI-Modells und erklären Sie, warum diese Redundanz notwendig ist statt eine einzelne zentrale Flusssteuerung zu implementieren.

Antwort: Die Flusskontrolle ist auf mehreren Ebenen implementiert, weil jede Schicht unterschiedliche Problemstellungen adressiert: Der Data Link Layer (Schicht 2) regelt die Flusssteuerung zwischen direkt verbundenen Geräten, während der Network Layer (Schicht 3) Node-to-Node und End-to-End Flusssteuerung für die Kommunikation über mehrere Netzwerke hinweg bereitstellt. Der Transport Layer (Schicht 4) fügt eine teilnehmerspezifische Flusssteuerung hinzu, die auf die Eigenschaften der kommunizierenden Anwendungen zugeschnitten ist. Diese verteilte Implementierung ermöglicht eine präzise, schichtspezifische Anpassung an verschiedene Netzwerkbedingungen und verhindert Überlastungen auf jeder Ebene der Kommunikation. Eine einzelne zentrale Flusssteuerung wäre nicht in der Lage, die spezifischen Anforderungen aller Kommunikationsebenen zu erfüllen und würde die Modularität und Flexibilität des OSI-Modells beeinträchtigen. (Richtig)
```

Anthropic's open-ended question – Bloom level 4 – illustrates a bad example, since it contains too many foreign words and a complex wording, which complicates the meaning of the core task.

B Question Generation Prompts

Listing B.1: Common Prompt for Experiment 1: **exp1_common_prompt.md**

```
Generiere eine Frage basierend auf dem folgenden Text zum ISO-OSI-Modell:

{text}

---

Folge den folgenden Anweisungen:

1. Formuliere eine klare und präzise Frage.
2. Erstelle passend zur Frage mindestens eine Antwort.
3. Füge am Ende einer korrekten Antwort `(Richtig)` hinzu.
4. Füge am Ende jeder falschen Antwort `(Falsch)` hinzu.
5. Gib nur die Frage und die Antwort(en) aus. Schreibe keinen zusätzlichen Text.

---

Wenn du eine einzelne Antwort generierst, dann soll das Format wie folgt aussehen:

Frage: Hier den Frageninhalt einfügen

Antwort: Hier die Antwort einfügen (Richtig)

---

Wenn du mehrere Antwortmöglichkeiten generierst, dann soll das Format wie folgt aussehen:

Frage: Hier den Frageninhalt einfügen

Antwortmöglichkeiten:
- Beispielsweise hier die korrekte Antwort einfügen (Richtig)
- Hier eine falsche Antwort einfügen (Falsch)
- Hier eine weitere falsche Antwort einfügen (Falsch)
- ...

Beachte dabei, dass auch mehrere korrekte Antworten möglich sind, die dann ebenfalls mit (Richtig)
↪ markiert werden sollten.
```

Listing B.2: Complex Prompt for Experiment 1: **exp1_complex_prompt.md**

```
Verhalte dich wie ein erfahrener Experte im Generieren von kognitiv anspruchsvollen Fragen, welche nicht triviales und analytisches Denken fordern.
Deine Aufgabe ist es, eine Frage beliebigen Typs zu dem folgenden Text über das ISO-OSI-Modell zu formulieren.

---

Befolge dabei die folgenden Anweisungen, um eine qualitativ hochwertige Frage zu generieren:

1. Untersuche den gegebenen Instruktionstext sorgfältig.
2. Erfasse alles, was für die ordnungsgemäße Fragengenerierung zum ISO-OSI-Modell wichtig ist.
3. Formuliere eine klare, präzise und kognitiv anspruchsvolle Frage, die zum kritischen Nachdenken anregt.
4. Erstelle passend zur Frage mindestens eine Antwort.
5. Füge am Ende einer korrekten Antwort `(Richtig)` hinzu.
6. Füge am Ende jeder falschen Antwort `(Falsch)` hinzu.
7. Verwende für Listen ausschließlich Gedankenstriche (`-`).
8. Gib nur die Frage und die Antwort(en) im vorgegebenen Format als Fließtext aus. Schreibe keinen zusätzlichen Text und vermeide jegliche Markdown-basierte Formatierung.

---

Fokussiere dich auf den folgenden Text bei der Fragengenerierung:

"""
{text}
"""

---

Wenn du eine einzelne Antwort generierst, dann soll das Format wie folgt aussehen, wie zwischen den Docstring-Symbolen angegeben:

"""
Frage: Hier den Frageninhalt einfügen

Antwort: Hier die Antwort einfügen (Richtig)
"""

---

Wenn du mehrere Antwortmöglichkeiten generierst, dann verwende den folgenden Formatierungsstil. Dabei musst du auch beachten, welche Antwortmöglichkeiten durch ein vorangestelltes `(Richtig)` als korrekt markiert werden müssen, und wo diese an beliebiger Position stehen können. Es folgt ein Beispiel zwischen den Docstring-Symbolen:
```

```
38
39 """
40 Frage: Hier den Frageninhalt einfügen
41
42 Antwortmöglichkeiten:
43 - Beispielsweise hier die korrekte Antwort einfügen (Richtig)
44 - Hier eine falsche Antwort einfügen (Falsch)
45 - Hier eine weitere falsche Antwort einfügen (Falsch)
46 - ...
47 """
48
49 Beachte dabei, dass auch mehrere korrekte Antworten möglich sind, die dann ebenfalls mit (Richtig)
   ↪  markiert werden sollten.
```

Listing B.3: Complex Prompt using no Source: **exp1_complex_prompt_no_source.md**

```
Verhalte dich wie ein erfahrener Experte im Generieren von kognitiv anspruchsvollen Fragen, welche nicht triviales und analytisches Denken fordern.
Deine Aufgabe ist es, eine Frage beliebigen Typs zu Schicht {layer} des ISO-OSI-Modells zu formulieren.

---

Befolge dabei die folgenden Anweisungen, um eine qualitativ hochwertige Frage zu generieren:

1. Konzentriere dich ausschließlich auf die {layer}-Schicht des ISO-OSI-Modells.
2. Nutze dein Expertenwissen, um die wichtigsten Aspekte dieser Schicht zu identifizieren.
3. Formuliere eine klare, präzise und kognitiv anspruchsvolle Frage, die zum kritischen Nachdenken anregt.
4. Erstelle passend zur Frage mindestens eine Antwort.
5. Füge am Ende einer korrekten Antwort `(Richtig)` hinzu.
6. Füge am Ende jeder falschen Antwort `(Falsch)` hinzu.
7. Verwende für Listen ausschließlich Gedankenstriche (`-`).
8. Gib nur die Frage und die Antwort(en) im vorgegebenen Format als Fließtext aus. Schreibe keinen zusätzlichen Text und vermeide jegliche Markdown-basierte Formatierung.

---

Wenn du eine einzelne Antwort generierst, dann soll das Format wie folgt aussehen, wie zwischen den Docstring-Symbolen angegeben:

"""
Frage: Hier den Frageninhalt einfügen

Antwort: Hier die Antwort einfügen (Richtig)
"""

---

Wenn du mehrere Antwortmöglichkeiten generierst, dann verwende den folgenden Formatierungsstil. Dabei musst du auch beachten, welche Antwortmöglichkeiten durch ein vorangestelltes `(Richtig)` als korrekt markiert werden müssen, und wo diese an beliebiger Position stehen können. Es folgt ein Beispiel zwischen den Docstring-Symbolen:

"""
Frage: Hier den Frageninhalt einfügen

Antwortmöglichkeiten:
- Beispielsweise hier die korrekte Antwort einfügen (Richtig)
- Hier eine falsche Antwort einfügen (Falsch)
- Hier eine weitere falsche Antwort einfügen (Falsch)
```

```
- ...
"""

Beachte dabei, dass auch mehrere korrekte Antworten möglich sind, die dann ebenfalls mit (Richtig)
↪  markiert werden sollten.
```

Listing B.4: Prompt focusing Question Type: **exp2_type.md**

```
Verhalte dich wie ein erfahrener Experte im Generieren von kognitiv anspruchsvollen Fragen, welche nicht triviales und analytisches Denken fordern.
Deine Aufgabe ist es, eine Frage vom Typ {question_type} zu dem folgenden Text über das ISO-OSI-Modell zu formulieren.

Befolge dabei die folgenden Anweisungen, um eine qualitativ hochwertige Frage zu generieren:

1. Untersuche den gegebenen Instruktionstext sorgfältig.
2. Formuliere eine klare, präzise und kognitiv anspruchsvolle Frage vom Typ {question_type}, die zum kritischen Nachdenken anregt.
3. Die Frage soll in einer Prüfungsform formuliert sein. Somit sind Formulierungen wie "laut dem Text" oder "basierend auf dem Text" zu vermeiden, um eine direkte und klare Prüfungsfrage zu stellen.
4. Erstelle passend zur Frage mindestens eine Antwort.
5. Füge am Ende einer korrekten Antwort `(Richtig)` hinzu.
6. Füge am Ende jeder falschen Antwort `(Falsch)` hinzu.
7. Wenn es der Fragetyp zulässt, verwende für Listen ausschließlich alphabetische Aufzählungen wie `a) `, `b) `, `c) `, etc., und beginne die Aufzählung mit `a) `.
8. Gib nur die Frage und die Antwort(en) im vorgegebenen Format als Fließtext aus. Schreibe keinen zusätzlichen Text und vermeide jegliche Markdown-basierte Formatierung.

Fokussiere dich auf den folgenden Text bei der Fragengenerierung:


{text}


Verwende für die Ausgabe der Frage und der Antwort(en) das folgende Format:


{question_type_format}


Beachte dabei, dass auch mehrere korrekte Antworten möglich sind, die dann ebenfalls mit (Richtig) markiert werden sollten.
```

Listing B.5: Prompt focusing Bloom's Level: **exp2_bloom.md**

```
Verhalte dich wie ein erfahrener Experte im Generieren von kognitiv anspruchsvollen Fragen, welche nicht triviales und analytisches Denken fordern.
Deine Aufgabe ist es, eine Prüfungsfrage beliebigen Typs zu dem folgenden Text über das ISO-OSI-Modell zu formulieren, die dem Bloom's Level {bloom_level} entspricht.

Befolge dabei die folgenden Anweisungen, um eine qualitativ hochwertige Frage zu generieren:

1. Untersuche den gegebenen Instruktionstext sorgfältig.
2. Entscheide, ob für den gegebenen Text und das Bloom's Level {bloom_level} eine einzelne Antwort oder mehrere Antwortmöglichkeiten generiert werden sollen.
3. Formuliere eine klare, präzise und kognitiv anspruchsvolle Frage entsprechend dem Bloom's Level {bloom_level}.
4. Die Beschreibung dieses Levels ist: {bloom_level_description}.
5. Verwende dabei Verben wie: {bloom_level_verbs}.
6. Die Frage soll in einer Prüfungsform formuliert sein. Somit sind Formulierungen wie "laut dem Text" oder "basierend auf dem Text" zu vermeiden, um eine direkte und klare Prüfungsfrage zu stellen.
7. Erstelle passend zur Frage mindestens eine Antwort.
8. Füge am Ende einer korrekten Antwort `(Richtig)` hinzu.
9. Füge am Ende jeder falschen Antwort `(Falsch)` hinzu.
10. Wenn mehrere Antworten generiert werden, dann verwende für Listen ausschließlich alphabetische Aufzählungen wie `a) `, `b) `, `c) `, etc., und beginne die Aufzählung mit `a) `.
11. Gib nur die Frage und die Antwort(en) im vorgegebenen Format als Fließtext aus. Schreibe keinen zusätzlichen Text und vermeide jegliche Markdown-basierte Formatierung.

Fokussiere dich auf den folgenden Text bei der Fragengenerierung:


{text}


Wenn du eine einzelne Antwort generierst, dann soll das Format wie folgt aussehen:


Frage: Hier den Frageninhalt einfügen

Antwort: Hier die Antwort einfügen (Richtig)


Wenn du mehrere Antwortmöglichkeiten generierst, dann verwende den folgenden Formatierungsstil. Dabei musst du auch beachten, welche Antwortmöglichkeiten durch ein vorangestelltes `(Richtig)` als korrekt markiert werden müssen, und wo diese an beliebiger Position stehen können. Es folgt ein Beispiel:


Frage: Hier den Frageninhalt einfügen
```

```
36
37 Antwortmöglichkeiten:
38 a) Beispielsweise hier die korrekte Antwort einfügen (Richtig)
39 b) Hier eine falsche Antwort einfügen (Falsch)
40 c) Hier eine weitere falsche Antwort einfügen (Falsch)
41 d) ...
42
43
44 Beachte dabei, dass auch mehrere korrekte Antworten möglich sind, die dann ebenfalls mit (Richtig)
   ↪  markiert werden sollten.
```

Listing B.6: Prompt focusing Question Type and Bloom's Level: **exp2_both.md**

```
Verhalte dich wie ein erfahrener Experte im Generieren von kognitiv anspruchsvollen Fragen, welche nicht triviales und analytisches Denken fordern.
Deine Aufgabe ist es, eine Frage vom Typ {question_type} zu dem folgenden Text über das ISO-OSI-Modell zu formulieren, die dem Bloom's Level {bloom_level} entspricht.

Befolge dabei die folgenden Anweisungen, um eine qualitativ hochwertige Frage zu generieren:

1. Untersuche den gegebenen Instruktionstext sorgfältig.
2. Formuliere eine klare, präzise und kognitiv anspruchsvolle Frage vom Typ {question_type} entsprechend dem Bloom's Level {bloom_level}.
3. Die Beschreibung dieses Levels ist: {bloom_level_description}.
4. Verwende dabei Verben wie: {bloom_level_verbs}.
5. Die Frage soll in einer Prüfungsform formuliert sein. Somit sind Formulierungen wie "laut dem Text" oder "basierend auf dem Text" zu vermeiden, um eine direkte und klare Prüfungsfrage zu stellen.
6. Erstelle passend zur Frage mindestens eine Antwort.
7. Füge am Ende einer korrekten Antwort `(Richtig)` hinzu.
8. Füge am Ende jeder falschen Antwort `(Falsch)` hinzu.
9. Wenn es der Fragetyp zulässt, verwende für Listen ausschließlich alphabetische Aufzählungen wie `a) `, `b) `, `c) `, etc., und beginne die Aufzählung mit `a) `.
10. Gib nur die Frage und die Antwort(en) im vorgegebenen Format als Fließtext aus. Schreibe keinen zusätzlichen Text und vermeide jegliche Markdown-basierte Formatierung.

Fokussiere dich auf den folgenden Text bei der Fragengenerierung:


{text}


Verwende für die Ausgabe der Frage und der Antwort(en) das folgende Format:


{question_type_format}


Beachte dabei, dass auch mehrere korrekte Antworten möglich sind, die dann ebenfalls mit (Richtig) markiert werden sollten.
```

Listing B.7: Bloom's Taxonomy Descriptions and Verbs: **bloom.md**

```
## Descriptions

- Remembering: Abruf relevanten Wissens aus dem Langzeitgedächtnis, indem der Schwerpunkt auf dem Erkennen und Erinnern von Fakten liegt.
- Understanding: Erkennen der Bedeutung von Anweisungen, z.B. mündlicher, schriftlicher und grafischer Kommunikation.
- Applying: Durchführen oder Anwenden eines Verfahrens in einer bestimmten Situation.
- Analyzing: Zerlegung von Material in seine Bestandteile und Bestimmung ihrer Beziehungen zueinander oder zu einer Gesamtstruktur oder einem Gesamtzweck.
- Evaluating: Urteile auf der Grundlage etablierter Kriterien und Standards fällen.
- Creating: Zusammenfügen oder Neuorganisieren von Elementen zu einem zusammenhängenden Ganzen.

## Verbs

- Remembering: Definieren, beschreiben, aufzählen, identifizieren, auflisten, zuordnen, benennen, skizzieren, zitieren, auswählen
- Understanding: Klarstellen, klassifizieren, vergleichen, gegenüberstellen, detaillieren, erklären, umschreiben, neu schreiben, zusammenfassen, übersetzen
- Applying: Anwenden, berechnen, vorführen, ermitteln, untersuchen, veranschaulichen, verändern, simulieren, lösen, nutzen
- Analyzing: Analysieren, zerlegen, erkennen, differenzieren, unterscheiden, prüfen, untersuchen, optimieren, in Beziehung setzen, trennen
- Evaluating: Bewerten, schlussfolgern, kritisieren, verteidigen, bewerten, benoten, interpretieren, beurteilen, begründen, überprüfen
- Creating: Zusammenstellen, codieren, kompilieren, konstruieren, erstellen, entwerfen, entwickeln, erweitern, verbessern, reorganisieren
```

C Evaluation Prompts and Rubrics

Listing C.1: Experiment 1a Evaluation Rubric: **exp1a_rubric.md**

```
**Relevanz/Relevance (0-10 Punkte):**
Ist die Frage relevant zum Thema des Quelltexts?
- 10: Direkt bezogen auf Hauptthemen des Texts, alle Schlüsselbegriffe korrekt verwendet
  - Bsp: "Erläutern Sie, warum beim Application Layer nur wenig Systematisierung möglich ist."
- 8: Bezug zu Hauptthemen, meiste Schlüsselbegriffe korrekt verwendet
- 6: Wichtige Aspekte des Texts beachtet, einige Schlüsselbegriffe genutzt
- 4: Teilweise bezogen auf Textinhalt, wenige Schlüsselbegriffe verwendet
- 2: Schwacher Bezug zum Text, kaum relevante Begriffe verwendet
- 0: Kein erkennbarer Bezug zum Textinhalt
  - Bsp: "Wie funktioniert ein neuronales Netzwerk?"

**Klarheit/Clarity (0-10 Punkte):**
Ist die Fragestellung inhaltlich eindeutig und logisch fachlich klar formuliert?
- 10: Eindeutige Formulierung, klare Fragestellung ohne Mehrdeutigkeiten
  - Bsp: "Welche Kernaufgaben hat der Data Link Layer?"
- 8: Weitgehend eindeutig, minimale Unklarheiten
- 6: Überwiegend verständlich, wenige Mehrdeutigkeiten
- 4: Mehrere unklare Formulierungen, aber grundsätzlich verständlich
- 2: Erhebliche Unklarheiten, schwer verständliche Formulierung
- 0: Unverständlich oder unlogische Struktur
  - Bsp: "Was der Data Link Layer macht für Funktionen?"

**Beantwortbarkeit/Answerability (0-10 Punkte):**
Können Studierende die Frage wahrscheinlich basierend auf dem bereitgestellten Material
↪ beantworten?
- 10: Alle nötigen Informationen für vollständige Antwort im Text vorhanden
  - Bsp: "Beschreiben Sie die drei Grundkonzepte des Application Layers."
- 8: Wesentliches im Text, minimale Schlussfolgerungen nötig
- 6: Grundlegende Informationen im Text, gewisses Interpretieren erforderlich
- 4: Einige relevante Informationen im Text, viel Eigeninitiative notwendig
- 2: Wenige Hinweise im Text, somit größtenteils externes Wissen erforderlich
- 0: Keine ausreichenden Informationen im Text verfügbar
  - Bsp: "Welche historische Entwicklung führte zur Entstehung des Application Layers in den
  ↪ 1970er Jahren?"

**Herausfordernd/Challenging (0-10 Punkte):**
Ist die Frage herausfordernd? Ermutigt sie Studierende zum aktiven Denken?
- 10: Erfordert eine tiefgehende Analyse des gesamten Textes
  - Bsp: "Bewerten Sie kritisch, wie die geringe Systematisierung des Application Layers die
  ↪ Entwicklung von Netzwerkanwendungen beeinflusst."
- 8: Integration von Informationen aus weit auseinanderliegenden Textabschnitten nötig
- 6: Finden und Kombinieren von Informationen aus verschiedenen Textabschnitten erforderlich
- 4: Grundlegendes Verständnis des Textes nötig, aber keine tiefgehende Analyse
```

```
- 2: Auffinden von Informationen im Text nötig
- 0: Trivial, keine Denkleistung nötig
  - Bsp: "Ist der Application Layer systematisch aufgebaut? (Ja/Nein)"

**Wertigkeit/Value (0-10 Punkte):**
Wird die Frage vom Dozenten als fachlich / didaktisch / technisch sinnvoll erachtet?
- 10: Fachlich präzise, didaktisch wertvoll
  - Bsp: "Erläutern Sie die Bedeutung der Flusskontrolle im Transport Layer."
- 8: Gute Frage mit kleineren Schwächen
- 6: Grundanliegen erkennbar, aber ist ungünstig positioniert
- 4: Frage ist fachlich oder didaktisch unglücklich
- 2: Frage ist größtenteils problematisch
- 0: Frage ist nicht sinnvoll verwendbar
  - Bsp: "Wie viele Transport Layer gibt es?" (Missverständnis)

**Sprache/Language (0-10 Punkte):**
Ist die vom System benutzte Sprache hinreichend präzise und verständlich?
- 10: Sprache ist präzise und angemessen
  - Bsp: "Welche Aufgaben erfüllt der Application Layer?"
- 7: Sprache ist verständlich, könnte präziser sein
- 5: Sprache ist aufgebläht, schwulstig, schwerfällig
  - Bsp: "Angenommen, im Netzwerk gibt es [...]. Was ist hierbei zu beachten, und welche Maßnahmen
    ↪ sollten ergriffen werden?"
- 3: Sprache ist überladen, weitschweifig, sehr schwerfällig
- 0: Sprache ist nicht wirklich verständlich

**Korrektheit/Correctness (0-10 Punkte):**
Basiert die Frage auf korrekten Annahmen und Fakten aus dem bereitgestellten Text?
- 10: Alle in der Frage enthaltenen Informationen entsprechen dem Textinhalt
  - Bsp: "Warum ist beim Application Layer wenig Systematisierung möglich?"
- 5: Größtenteils korrekte Annahmen, minimale Ungenauigkeiten
- 0: Falsche Annahmen oder Widersprüche zum Textinhalt
  - Bsp: "Welche der fünf Hauptaufgaben des Application Layers ist am wichtigsten?" (Nur drei
    ↪ Aufgaben)
```

Listing C.2: Experiment 1b Evaluation Rubric: **exp1b_rubric.md**

```
**Relevanz/Relevance (0-10 Punkte):**
Ist die Frage relevant zum Thema des Quelltexts?
- 10: Direkt bezogen auf Hauptthemen des Texts, alle Schlüsselbegriffe korrekt verwendet
  - Bsp: "Erläutern Sie, warum beim Application Layer nur wenig Systematisierung möglich ist."
- 8: Bezug zu Hauptthemen, meiste Schlüsselbegriffe korrekt verwendet
- 6: Wichtige Aspekte des Texts beachtet, einige Schlüsselbegriffe genutzt
- 4: Teilweise bezogen auf Textinhalt, wenige Schlüsselbegriffe verwendet
- 2: Schwacher Bezug zum Text, kaum relevante Begriffe verwendet
- 0: Kein erkennbarer Bezug zum Textinhalt
  - Bsp: "Wie funktioniert ein neuronales Netzwerk?"

**Klarheit/Clarity (0-10 Punkte):**
Ist die Fragestellung inhaltlich eindeutig und logisch fachlich klar formuliert?
- 10: Eindeutige Formulierung, klare Fragestellung ohne Mehrdeutigkeiten
  - Bsp: "Welche Kernaufgaben hat der Data Link Layer?"
- 8: Weitgehend eindeutig, minimale Unklarheiten
- 6: Überwiegend verständlich, wenige Mehrdeutigkeiten
- 4: Mehrere unklare Formulierungen, aber grundsätzlich verständlich
- 2: Erhebliche Unklarheiten, schwer verständliche Formulierung
- 0: Unverständlich oder unlogische Struktur
  - Bsp: "Was der Data Link Layer macht für Funktionen?"

**Beantwortbarkeit/Answerability (0-10 Punkte):**
Können Studierende die Frage wahrscheinlich basierend auf dem bereitgestellten Material beantworten?
- 10: Alle nötigen Informationen für vollständige Antwort im Text vorhanden
  - Bsp: "Beschreiben Sie die drei Grundkonzepte des Application Layers."
- 8: Wesentliches im Text, minimale Schlussfolgerungen nötig
- 6: Grundlegende Informationen im Text, gewisses Interpretieren erforderlich
- 4: Einige relevante Informationen im Text, viel Eigeninitiative notwendig
- 2: Wenige Hinweise im Text, somit größtenteils externes Wissen erforderlich
- 0: Keine ausreichenden Informationen im Text verfügbar
  - Bsp: "Welche historische Entwicklung führte zur Entstehung des Application Layers in den 1970er Jahren?"

**Herausfordernd/Challenging (0-10 Punkte):**
Ist die Frage herausfordernd? Ermutigt sie Studierende zum aktiven Denken?
- 10: Erfordert eine tiefgehende Analyse des gesamten Textes
  - Bsp: "Bewerten Sie kritisch, wie die geringe Systematisierung des Application Layers die Entwicklung von Netzwerkanwendungen beeinflusst."
- 8: Integration von Informationen aus weit auseinanderliegenden Textabschnitten nötig
- 6: Finden und Kombinieren von Informationen aus verschiedenen Textabschnitten erforderlich
- 4: Grundlegendes Verständnis des Textes nötig, aber keine tiefgehende Analyse
- 2: Auffinden von Informationen im Text nötig
- 0: Trivial, keine Denkleistung nötig
```

```
   - Bsp: "Ist der Application Layer systematisch aufgebaut? (Ja/Nein)"

**Wertigkeit/Value (0-10 Punkte):**
Wird die Frage vom Dozenten als fachlich / didaktisch / technisch sinnvoll erachtet?
- 10: Fachlich präzise, didaktisch wertvoll
   - Bsp: "Erläutern Sie die Bedeutung der Flusskontrolle im Transport Layer."
- 8: Gute Frage mit kleineren Schwächen
- 6: Grundanliegen erkennbar, aber ist ungünstig positioniert
- 4: Frage ist fachlich oder didaktisch unglücklich
- 2: Frage ist größtenteils problematisch
- 0: Frage ist nicht sinnvoll verwendbar
   - Bsp: "Wie viele Transport Layer gibt es?" (Missverständnis)

**Sprache/Language (0-10 Punkte):**
Ist die vom System benutzte Sprache hinreichend präzise und verständlich?
- 10: Sprache ist präzise und angemessen
   - Bsp: "Welche Aufgaben erfüllt der Application Layer?"
- 7: Sprache ist verständlich, könnte präziser sein
- 5: Sprache ist aufgebläht, schwulstig, schwerfällig
   - Bsp: "Angenommen, im Netzwerk gibt es [...]. Was ist hierbei zu beachten, und welche Maßnahmen
   ↪ sollten ergriffen werden?"
- 3: Sprache ist überladen, weitschweifig, sehr schwerfällig
- 0: Sprache ist nicht wirklich verständlich

**Umgang mit Manipulation/Manipulation Handling (0-10 Punkte):**
Folgt die Frage dem manipulierten Textinhalt, oder erkennt das Modell die Manipulation?
- 10: Keine Übereinstimmung, oder Modell erkennt die Manipulation des Textes
   - Bsp: "Warum ist beim Application Layer wenig Systematisierung möglich?"
   - "Oder: "Welche Funktionen bietet der Presentation Layer laut des gegebenen Textes ausdrücklich
   ↪ NICHT an?"
- 5: Teilweise textuelle Übereinstimmung durch Vermischung von Textinhalten
- 0: Modell folgt dem manipulierten Textinhalt vollständig
   - Bsp: "Warum ermöglicht die hohe Systematisierung des Application Layers eine bessere
   ↪ Vorhersagbarkeit?"
```

Listing C.3: Experiment 2 Evaluation Rubric: **exp2_rubric.md**

```
**Relevanz/Relevance (0-10 Punkte):**
Ist die Frage relevant zum Thema des Quelltexts?
- 10: Direkt bezogen auf Hauptthemen des Texts, alle Schlüsselbegriffe korrekt verwendet
  - Bsp: "Erläutern Sie, warum beim Application Layer nur wenig Systematisierung möglich ist."
- 8: Bezug zu Hauptthemen, meiste Schlüsselbegriffe korrekt verwendet
- 6: Wichtige Aspekte des Texts beachtet, einige Schlüsselbegriffe genutzt
- 4: Teilweise bezogen auf Textinhalt, wenige Schlüsselbegriffe verwendet
- 2: Schwacher Bezug zum Text, kaum relevante Begriffe verwendet
- 0: Kein erkennbarer Bezug zum Textinhalt
  - Bsp: "Wie funktioniert ein neuronales Netzwerk?"

**Klarheit/Clarity (0-10 Punkte):**
Ist die Fragestellung eindeutig und verständlich formuliert?
- 10: Eindeutige Formulierung, klare Fragestellung ohne Mehrdeutigkeiten
  - Bsp: "Welche Kernaufgaben hat der Data Link Layer?"
- 8: Weitgehend eindeutig, minimale Unklarheiten
- 6: Überwiegend verständlich, wenige Mehrdeutigkeiten
- 4: Mehrere unklare Formulierungen, aber grundsätzlich verständlich
- 2: Erhebliche Unklarheiten, schwer verständliche Formulierung
- 0: Unverständlich oder unlogische Struktur
  - Bsp: "Was der Data Link Layer macht für Funktionen?"

**Beantwortbarkeit/Answerability (0-10 Punkte):**
Können Studierende die Frage wahrscheinlich basierend auf dem bereitgestellten Material
↪ beantworten?
- 10: Alle nötigen Informationen für vollständige Antwort im Text vorhanden
  - Bsp: "Beschreiben Sie die drei Grundkonzepte des Application Layers."
- 8: Wesentliches im Text, minimale Schlussfolgerungen nötig
- 6: Grundlegende Informationen im Text, gewisses Interpretieren erforderlich
- 4: Einige relevante Informationen im Text, viel Eigeninitiative notwendig
- 2: Wenige Hinweise im Text, somit größtenteils externes Wissen erforderlich
- 0: Keine ausreichenden Informationen im Text verfügbar
  - Bsp: "Welche historische Entwicklung führte zur Entstehung des Application Layers in den
  ↪ 1970er Jahren?"

**Herausfordernd/Challenging (0-10 Punkte):**
Ist die Frage herausfordernd? Ermutigt sie Studierende zum aktiven Denken?
- 10: Erfordert eine tiefgehende Analyse des gesamten Textes
  - Bsp: "Bewerten Sie kritisch, wie die geringe Systematisierung des Application Layers die
  ↪ Entwicklung von Netzwerkanwendungen beeinflusst."
- 8: Integration von Informationen aus weit auseinanderliegenden Textabschnitten nötig
- 6: Finden und Kombinieren von Informationen aus verschiedenen Textabschnitten erforderlich
- 4: Grundlegendes Verständnis des Textes nötig, aber keine tiefgehende Analyse
- 2: Auffinden von Informationen im Text nötig
- 0: Trivial, keine Denkleistung nötig
```

```
   - Bsp: "Ist der Application Layer systematisch aufgebaut? (Ja/Nein)"

**Wertigkeit/Value (0-10 Punkte):**
Wird die Frage vom Dozenten als fachlich / didaktisch / technisch sinnvoll erachtet?
- 10: Fachlich präzise, didaktisch wertvoll
   - Bsp: "Erläutern Sie die Bedeutung der Flusskontrolle im Transport Layer."
- 8: Gute Frage mit kleineren Schwächen
- 6: Grundanliegen erkennbar, aber ist ungünstig positioniert
- 4: Frage ist fachlich oder didaktisch unglücklich
- 2: Frage ist größtenteils problematisch
- 0: Frage ist nicht sinnvoll verwendbar
   - Bsp: "Wie viele Transport Layer gibt es?" (Missverständnis, weil es nur einen gibt)

**Sprache/Language (0-10 Punkte):**
Ist die vom System benutzte Sprache hinreichend präzise und verständlich?
- 10: Sprache ist präzise und angemessen
   - Bsp: "Welche Aufgaben erfüllt der Application Layer?"
- 7: Sprache ist verständlich, könnte präziser sein
- 5: Sprache ist aufgebläht, schwulstig, schwerfällig
   - Bsp: "Angenommen, im Netzwerk gibt es [...]. Was ist hierbei zu beachten, und welche Maßnahmen
   ↪ sollten ergriffen werden?"
- 3: Sprache ist überladen, weitschweifig, sehr schwerfällig
- 0: Sprache ist nicht wirklich verständlich

**Bloom's Level (Stufen 1-6):**
Bestimme das Niveau der Frage bezüglich Bloom's Taxonomie (Zahl zwischen 1 bis 6). Studiere dazu,
↪ welche kognitiven Prozesse die einzelnen Stufen der Taxonomie beschreiben. Gib somit final die
↪ Stufe an, die am besten zu der Frage passt. Die Stufen sind:

1. **Remembering:** Abruf relevanten Wissens aus dem Langzeitgedächtnis, indem der Schwerpunkt auf
↪ dem Erkennen und Erinnern von Fakten liegt.
   - Verben: Definieren, beschreiben, aufzählen, identifizieren, auflisten, zuordnen, benennen,
   ↪ skizzieren, zitieren, auswählen
2. **Understanding:** Erkennen der Bedeutung von Anweisungen, z.B. mündlicher, schriftlicher und
↪ grafischer Kommunikation.
   - Verben: Klarstellen, klassifizieren, vergleichen, gegenüberstellen, detaillieren, erklären,
   ↪ umschreiben, neu schreiben, zusammenfassen, übersetzen
3. **Applying:** Durchführen oder Anwenden eines Verfahrens in einer bestimmten Situation.
   - Verben: Anwenden, Berechnen, Vorführen, Ermitteln, Untersuchen, Veranschaulichen, Verändern,
   ↪ Simulieren, Lösen, Nutzen
4. **Analyzing:** Zerlegung von Material in seine Bestandteile und Bestimmung ihrer Beziehungen
↪ zueinander oder zu einer Gesamtstruktur oder einem Gesamtzweck.
   - Verben: Analysieren, zerlegen, erkennen, differenzieren, unterscheiden, prüfen, untersuchen,
   ↪ optimieren, in Beziehung setzen, trennen
5. **Evaluating:** Urteile auf der Grundlage etablierter Kriterien und Standards fällen.
   - Verben: Bewerten, schlussfolgern, kritisieren, verteidigen, bewerten, benoten, interpretieren,
   ↪ beurteilen, begründen, überprüfen
```

```
79 6. **Creating:** Zusammenfügen oder Neuorganisieren von Elementen zu einem zusammenhängenden
   ↪  Ganzen.
80    - Verben: Zusammenstellen, codieren, kompilieren, konstruieren, erstellen, entwerfen,
      ↪  entwickeln, erweitern, verbessern, reorganisieren
```

Listing C.4: Experiment 1 Content Adherence Prompt: **exp1_adherence_eval.md**

```
Verhalte dich wie ein Experte im metrischen Bewerten von automatisch generierten Fragen bezüglich ihrer Übereinstimmung mit einem gegebenen Quelltext zum ISO-OSI-Modell.

Dein Ziel ist es, die folgende Frage sorgfältig zu studieren:

"""
{question_text}
"""

Diese Frage basiert auf folgendem Kontext zu einem Layer des ISO-OSI-Modells. Studiere diesen Text sorgfältig:

"""
{context_text}
"""

Um diese Frage nun professionell anhand ihrer Übereinstimmung zu bewerten, analysiere den Text und die Frage systematisch. Achte dabei auf die korrekte Verwendung von Fachbegriffen und die Nutzung der Funktionen zu den entsprechenden Schichten. Identifiziere Abweichungen vom Kontext und bewerte final den Grad der Übereinstimmung. Nutze dazu die folgende Bewertungsskala:

- Wert von 1.0: Alle Inhalte wurden korrekt und umfassend dargelegt
- Wert von 0.8-0.9: Geringfügige fachliche Abweichungen
- Wert von 0.6-0.7: Gewisse technische Ungenauigkeiten
- Wert von 0.4-0.5: Mehrere inhaltliche Abweichungen
- Wert von 0.2-0.3: Gravierende Missverständnisse
- Wert von 0.0-0.1: Inhalte entsprechen nicht dem Kontext

Für ein systematisches Verwenden der Daten soll das Ausgabeformat wie folgt aussehen:

"""
[adherence_score]
"""

Ein Beispiel-Output wäre: `0.73`

Nun bewerte die Frage anhand der Inhaltstreue zum gegebenen Kontext. Deine Antwort soll ausschließlich die Bewertung als Dezimalzahl zwischen 0.0 und 1.0 mit zwei Nachkommastellen sein, ohne zusätzliche Erklärungen oder Kommentare.
```

D Source Materials

4.1 Script – Common

Listing D.1: Layer 1 – Script.

```
Physical Layer 1
Beschreibung
Bereitstellung einer physikalischen Konzeption für Codierung, Transport und
Decodierung logischer Werte.

3 Kernaufgaben

Codierung:

Abbildung logischer Werteauf physikalische Größen.
Bsp: NRZ, Manchester.

Transport:

Übertragung physikalischer Größen
von einem Ort zum anderen.
Medien-gebunden. Bsp: Telefonkabel.
Medien-frei.
Bsp: Lichtsignal, Funk.

Decodierung:

Rückinterpretation physikalischer Größen als logische Werte.
Bsp: Abtastung, Detektionsbereiche.
```

Listing D.2: Layer 2 – Script.

```
Data Link Layer 2
Beschreibung
Bereitstellung einer fehlergesicherten Übertragung über einen möglicherweise nicht
fehlergesicherten Physical Layer.

5 Kernaufgaben

Fehlerkontrolle:

Erkennen, Beheben oder Signalisieren von Übertragungsfehlern.

Flußkontrolle:

Strategien, wenn Sender für Empfänger zu schnell sendet.
Bsp: Zwischenpuffern, Herunterregeln, warten Lassen.

Segmentierung:

Zusammenfassen einzelner Bits zu größeren Einheiten.
Universal-typisches Bsp: 8-bit Bytes.

Identifikation:

Auswahl bestimmter Sender oder Empfänger (sog. Adresse).

Medienzugriff:

Aktivieren des Physical Layer.
4 typische Aspekte: Senden, Warten, Beheben und Reservieren.
```

Listing D.3: Layer 3 – Script.

```
Network Layer 3
Beschreibung
Bereitstellung einer Übertragung zwischen Geräten, die sich in unterschiedlichen
Netzwerken (möglicherweise) verschiedener Layer 2 Technologien befinden.

3 Kernaufgaben

Adressierung:

Netzwerk-übergreifende Adressen.

Routing:

Auswahl der vermittelnden Zwischenknoten
zur Kommunikation nicht direkt verbundenen Knoten.

Flußsteuerung:

Strategien, wenn zu schnell gesendet wird.
End-to-End:
Vom Sender zum weit entfernten Empfänger.
Node-to-Node: Von jedem Knoten zum Nächsten.
```

Listing D.4: Layer 4 – Script.

```
Transport Layer 4
Beschreibung
Bereitstellung einer Übertragung zwischen beliebigen Teilnehmern eines Netzes.

3 Kernaufgaben

Adressierung:

Teilnehmer-Adressen.

Flußsteuerung:

Nutzen der Teilnehmer-Charakteristik.

Dienstqualität:

Sicherung der Quality of Service (QoS)
Erbringung eines Kommunikationsdienstes mit Garantien
für Bandbreite, Latenz, Jitter, Fehlerrate usw.
Zugeschnitten auf den jeweiligen Teilnehmer.
```

Listing D.5: Layer 5 – Script.

```
Session Layer 5

Beschreibung
Bereitstellung der Funktionen für das Sitzungs(Session)-Konzept einer Kommunikationsbeziehung.

3 Kernaufgaben

Sitzungen:

Aufbauen, verwalten, zurückweisen und beenden.

Synchronisation:

Setzen von Zwischenpunkten.

Bündelung

Gruppierung zusammengehöriger Transaktionen.
```

Listing D.6: Layer 6 – Script.

```
Presentation Layer 6
Beschreibung
Kodierung (Darstellung) der Anwendungsdaten als Bitstrom.

4 Kernaufgaben

Endian-Ness:

Konvertieren der Datendarstellung.
Bsp: little & big endian, weitere Serialisierungs-Formate.

Zeichen:

Konvertierung der Zeichendarstellung
Bsp: ASCII, Unicode, EBCDIC, nationale Sonderzeichen.

Kompression:

Komprimieren und Dekomprimieren.

Verschlüsselung:

Verschlüsseln und Entschlüsseln.
```

Listing D.7: Layer 7 – Script.

```
Application Layer 7
Beschreibung
Festlegung, welche Elemente bei einer spezifischen Anwendung auftreten.

3 Kernaufgaben

Grundkonzepte:
Welche Begriffe treten in der Anwendung auf.

Protokollelemente: Welche Anforderungen können gestellt werden?
Welche Daten werden dabei erwartet?

Verhalten:
Wie reagiert die Anwendung auf die
Anforderungen durch diese Protokollelemente?


Stark abhängig von der jeweiligen Anwendung!
Daher wenig Systematisierung möglich.
```

4.2 Transcript

Listing D.8: Layer 1 – Transcript.

```
Der Physical Layer stellt eine physikalische Konzeption für Codierung, Transport und Dekodierung
↪ logischer Werte bereit.
Und die Beschreibung enthält bereits die drei zentralen Kernaufgaben.
Codierung
Damit haben wir uns auch schon beschäftigt.
Codierung bedeutet also die Abbildung logischer Werte auf physikalische Größen.
Wir kennen es aus der NAZ-Codierung, aus der Manchester-Codierung, der Differential
↪ Manchester-Codierung und einer Vielzahl von weiteren Dingen.
Dann der Transport, also die Übertragung physikalischer Größen von einem Ort zum anderen.
Was, wie wir bereits wissen, mediengebunden oder medienfrei geschehen kann.
In beiden Fällen unter Anwendung der elektromagnetischen Wechselwirkung.
einmal an ein Medium gebunden, ein Telefonkabel, ein Ethernet-Kabel, was auch immer, oder
↪ medienfrei.
Ein Lichtsignal, das direkt durch den freien Raum geht, oder ein Funksignal, das eben auch durch
↪ den freien Raum geht.
Und beim Licht sehen wir, da werden wir auch so ein bisschen eine Doppelfunktion haben, weil Licht
↪ kann ich eben direkt von A nach B als Lichtstrahl senden, aber auch über einen
↪ Lichtwellenleiter in einem Kabel transportieren, doch davon später.
Und schließlich drittens die Dekodierung, also die Rückinterpretation physikalischer Größen als
↪ logische Werte.
Und auch damit haben wir uns schon intensiver auseinandergesetzt.
Wir erinnern uns an die Abtastthematik, an die Detektionsbereiche und all diese drei Dinge müssen
↪ nun auf dem Physic Layer entsprechend festgelegt werden.
```

Listing D.9: Layer 2 – Transcript.

```
1  Wir haben dann den zweiten Layer, den Data Link Layer und der stellt nun, wichtiges Wort, eine
   ↪ fehlergesicherte Übertragung über einen möglicherweise eben nicht fehlergesicherten Physical
   ↪ Layer zur Verfügung.
2  Er hat eine Reihe von Aufgaben, die man sinnvollerweise auch zusammenbündelt und diese sind nun
   ↪ Fehlerkontrolle,
3  Flusskontrolle, Segmentierung, Identifikation und Medienzugriff.
4  Fehlerkontrolle bedeutet, dass wir Übertragungsfehler zunächst einmal erkennen müssen.
5  Wenn wir sie erkannt haben, kann es nun Tricks geben, dass wir sie bereits selber beheben können
   ↪ oder aber wir müssen sie der anderen Seite mitteilen, signalisieren und dann muss die andere
   ↪ Seite etwas tun, was uns in die Lage versetzt, diesen Fehler auch wirklich zu beheben.
6  Zweitens die Flusskontrolle.
7  Flusskontrolle brauchen wir immer dann, wenn der Sender für den Empfänger zu schnell sendet.
8  Also irgendwelche Mechanismen, die erlauben,
9  dieses Problem zu lösen.
10 Die Strategien im Link Layer sind nun Zwischenpuffer.
11 Ich nehme Daten entgegen, lege sie in einem Puffer ab,
12 Ich könnte auch die andere Seite herunterregeln.
13 Ich könnte also der anderen Seite sagen, bitte sende nicht so schnell.
14 Oder ich könnte mit Bestätigungen, die die andere Seite benötigt, etwas auf mich warten lassen, die
   ↪ Bestätigungen nicht senden oder später senden und dadurch implizit deutlich machen, dass ich
   ↪ lieber einen etwas langsameren Datenfluss hätte.
15 Die Segmentierung ist dann wieder eher etwas sehr simples, das ist das Zusammenfassen von einzelnen
   ↪ Bits zu größeren Einheiten.
16 Und man könnte sagen, das Beispiel, aber das universaltypische Beispiel, weil sich das praktisch
   ↪ durchgängig bewährt hat, ist das Zusammenfassen von acht Bits zu einem Byte, was die typische
   ↪ Einheit in einem Netzwerk ist.
17 In den Anfängen hat man so ein bisschen rumgebastelt und hatte so Zeichensätze kodiert und hatte
   ↪ 7-Bit-Kodierungen und dann 8-Bit-Kodierungen.
18 Schließlich die Identifikation, die Auswahl eines bestimmten Senders oder Empfängers durch die
   ↪ sogenannte Adresse.
19 Und schließlich, als auch unterste in der Schicht sozusagen, gelegen die Aktivierung des
   ↪ physikalischen Layers.
20 Es muss ja jetzt nun der Data Link Layer mit dem Physical Layer sprechen.
```

Listing D.10: Layer 3 – Transcript.

```
1  Der dritte, der nächste Layer, der Network Layer, stellt nun eine Übertragung zwischen Geräten, die
   ↪ sich in unterschiedlichen Netzwerken befinden, bereit.
2  Der hat nun drei Kernaufgaben.
3  Adressierung, Routing und Flusssteuerung.
4  Die Adressierung deshalb, weil ich für das Kommunizieren zwischen zwei möglicherweise
   ↪ unterschiedlichen Netzwerken
5  auch neue Adressen einführen muss.
6  Der eine Knoten hat seine Bluetooth-Adresse, der andere Knoten hat seine 5G-Adresse.
7  Die jeweils eine Adresse macht in der jeweils anderen Technologie keinen Sinn.
8  Ich muss also jetzt Adressen
9  technologieübergreifend einführen, damit ich hier auch von einem Knoten reden kann, ohne die
   ↪ jeweils spezifische Technologie dabei kennen zu müssen.
10 Das Routing bedeutet nun die Auswahl von vermittelnden Zwischenknoten.
11 Weil jetzt wird es vermutlich so sein, dass dieser Bluetooth-Knoten mit einem anderen
   ↪ Bluetooth-Knoten redet.
12 Der redet mit einem WLAN-Knoten.
13 Der redet mit irgendeinem Knoten bei meinem Provider.
14 Das Datenpaket, das ich jetzt gerade in mein Handy hinein spreche, das ist vielleicht sozusagen
   ↪ jetzt die Verbindung vom Headset, vom Mikrofon über Bluetooth in mein Handy und über das Handy
   ↪ geht es dann sozusagen weiter.
15 Da sind viele Zwischenknoten noch dabei, bis es am Ende ankommt.
16 Also diese Auswahl der vermittelnden Zwischenknoten, das ist das Routing.
17 Und schließlich wiederum Flusssteuerung.
18 Ich brauche wiederum Strategien, wenn zu schnell gesendet wird.
19 Und hier können wir zwei verschiedene Aspekte beachten.
20 Nun, also die Flusssteuerung kann eine Ende-zu-Ende oder eine Knoten-zu-Knoten-Flusssteuerung sein.
21 Knoten-zu-Knoten insofern, als natürlich jeder Partner mit seinem anderen Partner das Problem hat,
   ↪ dass sie sich auf einer gemeinsamen Geschwindigkeit wiederfinden müssen.
22 Ende zu Ende, weil es nun ja jetzt sein kann, dass auch über eine längere Strecke zwar jeder
   ↪ Partner mit seinem anderen Partner vernünftig eine gemeinsame Geschwindigkeit findet, aber
   ↪ möglicherweise
23 der langsamste, der schwächste Knoten auf der ganzen Verbindungsstrecke sehr viel zwischenpuffern
   ↪ muss und sozusagen sich in Gesamtsicht auf einer langen Strecke Ende zu Ende sehr viel
   ↪ vielleicht aufstaut und es deshalb nötig ist, auch diese Perspektive im Auge zu behalten.
```

Listing D.11: Layer 4 – Transcript.

```
1  Und schließlich der letzte, der vierte Layer, der Transport Layer.
2  Bereitstellung ist eine Beschreibung einer Übertragung zwischen beliebigen Teilnehmern eines
   ↪ Netzes.
3  Da taucht ein neuer Begriff auf, über den wir gleich noch ein bisschen nachdenken müssen.
4  Die Kernaufgaben sind hier Adressierung, wiederum die Flusssteuerung und die Dienstqualität als
   ↪ vielleicht etwas fortgeschrittene Form der Flusssteuerung.
5  Adressierung heißt, wir brauchen jetzt nochmal ein neues Adresskonzept aus ähnlichen Gründen wie
   ↪ bei der netzübergreifenden Kommunikation.
6  Wir werden das gleich verstehen, wenn wir uns mit dem Konzept des Teilnehmers beschäftigen werden.
7  Die Flusssteuerung, weil wir durch
8  die Charakteristik des Teilnehmers unter Umständen ein bisschen mehr über die Bedürfnisse des
   ↪ spezifischen einzelnen Teilnehmers wissen.
9  Auf Transport Layer, wenn der Teilnehmer identifiziert ist, kann ich vielleicht schon ein bisschen
   ↪ abschätzen, was sind denn mögliche Anforderungen, die hier eine Rolle spielen.
10 Ich kommuniziere in einem Fall, vielleicht schicke ich eine Datei übers Netz, ob die morgen oder
   ↪ übermorgen ankommt, ist nicht so schlimm.
11 Im anderen Fall geht es um einen Realtime-Voice, Realtime-Videostrom und der sollte eben nicht
   ↪ verzögert werden.
12 Da sollten auch nicht kurze Unterbrüche in der Kommunikation stattfinden.
13 Bis morgen wollen wir da keinesfalls warten.
14 Und wir sehen auch, dieses Thema ist eng verbunden mit der Begrifflichkeit der Dienstqualität.
15 Wie man englisch gerne auch sagt, der Quality of Service, der QOS.
16 Ich möchte also jetzt einen Kommunikationsdienst erbringen und dabei Ihnen Garantien geben.
17 Für Bandbreite, für Latenz, für Jitter.
18 Also die drei wesentlichen Parameter in Quality of Service, aber auch für die Fehlerrate,
   ↪ vielleicht für weitere Aspekte noch.
19 Und zwar so zugeschnitten auf den jeweiligen Teilnehmer, dass es passt.
20 Wenn jetzt in dem Videostrom, den wir haben, mal irgendwo kurz
21 Ein kleines Datenpaket rausfällt, dann sehen Sie vielleicht die Bewegung meiner Hand nicht so ideal
   ↪ flüssig.
22 Es gibt vielleicht eine Schwankung in der Größenordnung von ein paar Pixeln in der Nähe meines
   ↪ Zeigefingers, das nehmen Sie gar nicht wahr.
23 Wenn allerdings bei einem Update für Ihr Mobiltelefon plötzlich mal ein paar Bit unterwegs fehlen,
   ↪ das wollen wir nicht haben.
24 Das sind die Bedürfnisse der jeweiligen Teilnehmer.
```

Listing D.12: Layer 5 – Transcript.

```
1 Wir haben uns also angesehen, wie Sitzungen funktionieren und derjenige Layer, der nun die
  ↪ Sitzungskonzepte verwaltet, ist der Session Layer, der Layer 5.
2 Er hat, so stelle ich es hier, fasse ich es hier, drei Kernaufgaben.
3 Sitzungen, Synchronisation und Bündelung.
4 Er muss also dieses Konzept von Sitzungen aufbauen, verwalten, zurückweisen, wiederum beenden, wie
  ↪ wir es an unseren Diagrammen gesehen haben.
5 Er wird Synchronisations, also solche Zwischenpunkte setzen, die ich dann später nutzen kann.
6 Wir erinnern uns an das Übertragen einer großen Datei.
7 Und er wird darauf achten, dass er Zusammengehörige Transaktionen geeignet bündelt und dazu
  ↪ Funktionen zur Verfügung stellt.
```

Listing D.13: Layer 6 – Transcript.

```
1 Der Presentation Layer wird die Anwendungsdaten als Bitstrom kodieren und darstellen.
2 Und eine der Hauptaufgaben haben wir gerade in unserem konzeptuellen Kapitel gesehen, nämlich das
  ↪ Konvertieren der Datendarstellung bei Little und Big Endian.
3 Und es gibt dann auch, wenn das jetzt nicht nur Zahlen sind, sondern komplexere Objekte, ganze
  ↪ Grafenstrukturen vielleicht, noch viele weitere komplexere Formen von Serialisierungen und
  ↪ darüber um diese NDNS kümmert sich der Presentation Layer.
4 Neben der NDNS muss er auch Zeichendarstellungen konvertieren, Kompression und Verschlüsselung
  ↪ bearbeiten.
5 Zeichendarstellungen, na da haben wir ein ähnliches Problem.
6 Ich habe 7-Bit ASCII, vielleicht 8-Bit ASCII, vielleicht Unicode, vielleicht diesen alten
  ↪ EBCDIC-Standard, vielleicht nationale Sonderzeichen, die je nachdem ob das jetzt eine
  ↪ französische oder eine dänische oder eine österreichische Kodierung ist für die Umlaut A oder
  ↪ für die Akzente, Zeichen möglicherweise unterschiedlich kodiert und daher auch eine
  ↪ entsprechende Anpassung benötigt.
7 Und wir sehen Kompression und Verschlüsselung, Dekompression und Entschlüsseln.
8 Das passt natürlich auch trefflich auf den Layer 6.
9 Dort sind diese Funktionen angesiedelt.
```

Listing D.14: Layer 7 – Transcript.

```
1 Und nun haben wir den letzten Layer, den Layer 7 (Application Layer), der nun festlegen soll,
  ↪ welche Elemente bei einer spezifischen Anwendung auftreten sollen.
2 Hier möchte ich auch drei Kernaufgaben angeben, obwohl ich gleich schon sagen muss, es hängt das
  ↪ sehr stark von der Art der jeweiligen Anwendung ab.
3 Daher ist hier sehr schwer nur eine anwendungsfreie Systematisierung der Funktion des Layers
  ↪ möglich.
4 Wir haben Grundkonzepte, die wir definieren müssen.
5 Welche Begriffe treten denn auf in der Anwendung?
6 Wir haben Protokollelemente, also jene Elemente, mit denen der Kommunikationspartner Anforderungen
  ↪ an den anderen stellt und die Daten, die er dabei dann erwartet.
7 Und schließlich die Information, wie soll denn eigentlich eine Anwendung darauf dann reagieren?
8 Wie soll sie auf die Anforderungen oder die Protokollelemente durch Verändern ihres internen
  ↪ Zustands oder durch andere Dinge antworten?
```

4.3 Tanenbaum

Listing D.15: Layer 1 – Tanenbaum Book.

```
The Physical Layer
The physical layer is concerned with transmitting raw bits over a communication channel. The design issues have to do with making sure that when one side
sends a 1 bit it is received by the other side as a 1 bit, not as a 0 bit. Typical questions here are what electrical signals should be used to represent a 1 and a 0, how
many nanoseconds a bit lasts, whether transmission may proceed simultaneously
in both directions, how the initial connection is established, how it is torn down
when both sides are finished, how many pins the network connector has, and what
each pin is used for. These design issues largely deal with mechanical, electrical,
and timing interfaces, as well as the physical transmission medium, which lies
below the physical layer.
```

Listing D.16: Layer 2 – Tanenbaum Book.

```
The Data Link Layer
The main task of the data link layer is to transform a raw transmission facility into a line that appears free of undetected transmission errors. It does so by
masking the real errors so the network layer does not see them. It accomplishes
this task by having the sender break up the input data into data frames (typically
a few hundred or a few thousand bytes) and transmit the frames sequentially. If
the service is reliable, the receiver confirms correct receipt of each frame by sending back an acknowledgement frame.
Another issue that arises in the data link layer (and most of the higher layers
as well) is how to keep a fast transmitter from drowning a slow receiver in data.
Some traffic regulation mechanism may be needed to let the transmitter know
when the receiver can accept more data.
Broadcast networks have an additional issue in the data link layer: how to
control access to the shared channel. A special sublayer of the data link layer, the
medium access control sublayer, deals with this problem.
```

Listing D.17: Layer 3 – Tanenbaum Book.

```
The Network Layer
The network layer controls the operation of the subnet. A key design issue is
determining how packets are routed from source to destination. Routes can be
based on static tables that are ''wired into'' the network and rarely changed, or
more often they can be updated automatically to avoid failed components. They
can also be determined at the start of each conversation, for example, a terminal
session, such as a login to a remote machine. Finally, they can be highly dynamic, being determined anew for each packet to reflect the current network load.
If too many packets are present in the subnet at the same time, they will get in
one another's way, forming bottlenecks. Handling congestion is also a responsibility of the network layer, in conjunction with higher layers that adapt the load

they place on the network. More generally, the quality of service provided (delay,
transit time, jitter, etc.) is also a network layer issue.
When a packet has to travel from one network to another to get to its destination, many problems can arise. The addressing used by the second network may
be different from that used by the first one. The second one may not accept the
packet at all because it is too large. The protocols may differ, and so on. It is up
to the network layer to overcome all these problems to allow heterogeneous networks to be interconnected.
In broadcast networks, the routing problem is simple, so the network layer is
often thin or even nonexistent.
```

Listing D.18: Layer 4 – Tanenbaum Book.

```
The Transport Layer
The basic function of the transport layer is to accept data from above it, split
it up into smaller units if need be, pass these to the network layer, and ensure that
the pieces all arrive correctly at the other end. Furthermore, all this must be done
efficiently and in a way that isolates the upper layers from the inevitable changes
in the hardware technology over the course of time.
The transport layer also determines what type of service to provide to the session layer, and,
↪  ultimately, to the users of the network. The most popular type of
transport connection is an error-free point-to-point channel that delivers messages
or bytes in the order in which they were sent. However, other possible kinds of
transport service exist, such as the transporting of isolated messages with no guarantee about the
↪  order of delivery, and the broadcasting of messages to multiple
destinations. The type of service is determined when the connection is established. (As an aside,
↪  an error-free channel is completely impossible to achieve;
what people really mean by this term is that the error rate is low enough to ignore
in practice.)
The transport layer is a true end-to-end layer; it carries data all the way from
the source to the destination. In other words, a program on the source machine
carries on a conversation with a similar program on the destination machine, using
the message headers and control messages. In the lower layers, each protocols is
between a machine and its immediate neighbors, and not between the ultimate
source and destination machines, which may be separated by many routers. The
difference between layers 1 through 3, which are chained, and layers 4 through 7,
which are end-to-end, is illustrated in Fig. 1-20.
```

Listing D.19: Layer 5 – Tanenbaum Book.

```
The Session Layer
The session layer allows users on different machines to establish sessions between them. Sessions
↪  offer various services, including dialog control (keeping
track of whose turn it is to transmit), token management (preventing two parties
from attempting the same critical operation simultaneously), and synchronization

(checkpointing long transmissions to allow them to pick up from where they left
off in the event of a crash and subsequent recovery).
```

Listing D.20: Layer 6 – Tanenbaum Book.

```
The Presentation Layer
Unlike the lower layers, which are mostly concerned with moving bits around,
the presentation layer is concerned with the syntax and semantics of the information transmitted.
↪ In order to make it possible for computers with different internal data representations to
↪ communicate, the data structures to be exchanged
can be defined in an abstract way, along with a standard encoding to be used ''on
the wire.'' The presentation layer manages these abstract data structures and allows higher-level
↪ data structures (e.g., banking records) to be defined and
exchanged.
```

Listing D.21: Layer 7 – Tanenbaum Book.

```
The Application Layer
The application layer contains a variety of protocols that are commonly
needed by users. One widely used application protocol is HTTP (HyperText
Transfer Protocol), which is the basis for the World Wide Web. When a
browser wants a Web page, it sends the name of the page it wants to the server
hosting the page using HTTP. The server then sends the page back. Other application protocols are
↪  used for file transfer, electronic mail, and network news.
```

4.4 Script – Manipulated

Listing D.22: Layer 1 - Script Manipulated.

```
Physical Layer 1
Beschreibung
Bereitstellung einer logischen Konzeption für Verschlüsselung, Speicherung und
Kompression physikalischer Werte.

3 Kernaufgaben

Verschlüsselung:

Abbildung physikalischer Werte auf logische Größen.
Bsp: AES, RSA.

Speicherung:

Aufbewahrung logischer Größen
an einem festen Ort.
Medien-unabhängig. Bsp: Festplatte.
Medien-gebunden.
Bsp: RAM, Cache.

Kompression:

Umwandlung logischer Größen in physikalische Werte.
Bsp: Huffman, LZW.
```

Listing D.23: Layer 2 - Script Manipulated.

```
Data Link Layer 2
Beschreibung
Bereitstellung einer fehleranfälligen Übertragung über einen stets
fehlergesicherten Physical Layer.

5 Kernaufgaben

Fehlererzeugung:

Ignorieren, Verursachen oder Verschleiern von Übertragungsfehlern.

Datenstau:

Strategien, wenn Empfänger für Sender zu langsam ist.
Bsp: Verwerfen von Daten, Beschleunigen, Ignorieren.

Fragmentierung:

Aufteilen größerer Einheiten in einzelne Bits.
Universal-typisches Bsp: 1-bit Segmente.

Anonymisierung:

Verbergen bestimmter Sender oder Empfänger (keine Adresse).

Medienverweigerung:

Deaktivieren des Physical Layer.
4 typische Aspekte: Empfangen, Ignorieren, Verursachen und Freigeben.
```

Listing D.24: Layer 3 - Script Manipulated.

```
Network Layer 3
Beschreibung
Bereitstellung einer Speicherung für Geräte, die sich im selben
Netzwerk derselben Layer 2 Technologie befinden.

3 Kernaufgaben

Lokalisierung:

Netzwerk-interne Positionen.

Zufallspfad:

Zufällige Auswahl von Endknoten
zur Kommunikation direkt verbundener Knoten.

Datenbeschleunigung:

Strategien, wenn zu langsam gesendet wird.
Punkt-zu-Punkt:
Vom Sender zum direkten Nachbarn.
Broadcast: Von einem Knoten zu allen Anderen.
```

Listing D.25: Layer 4 - Script Manipulated.

```
Transport Layer 4
Beschreibung
Bereitstellung einer Speicherung für festgelegte Teilnehmerpaare eines Netzes.

3 Kernaufgaben

Identifizierung:

Geräte-spezifische Kennungen.

Datenstau:

Ignorieren der Teilnehmer-Charakteristik.

Dienstquantität:

Vernachlässigung der Quality of Service (QoS)
Erbringung eines Speicherdienstes ohne Garantien
für Kapazität, Zugriffszeit, Varianz, Fehlerfreiheit usw.
Unabhängig vom jeweiligen Teilnehmer.
```

Listing D.26: Layer 5 - Script Manipulated.

```
Session Layer 5

Beschreibung
Bereitstellung der Funktionen für den verbindungslosen Datenaustausch einer
↪ Kommunikationsbeziehung.

3 Kernaufgaben

Datagramme:

Spontanes Erstellen, Senden, Empfangen und sofortiges Verwerfen.

Asynchronität:

Vermeidung von Zwischenpunkten und Bestätigungen.

Atomisierung:

Aufspaltung unabhängiger Datenpakete in Einzelteile.
```

Listing D.27: Layer 6 - Script Manipulated.

```
Presentation Layer 6
Beschreibung
Interpretation des Bitstroms als Rohdaten ohne Anwendungsbezug.

4 Kernaufgaben

Datenformat:

Vereinheitlichung der Datendarstellung.
Bsp: nur big endian, ein einziges Serialisierungs-Format.

Binärdaten:

Ignorieren der Zeichendarstellung, Behandlung als reine Binärdaten.
Bsp: keine Unterscheidung von ASCII, Unicode, etc.

Expansion:

Nur Expandieren (Entpacken).

Offenlegung:

Nur Entschlüsseln, keine Verschlüsselung anbieten.
```

Listing D.28: Layer 7 - Script Manipulated.

```
Application Layer 7
Beschreibung
Abstraktion von spezifischen Anwendungselementen zu generischen Funktionen.

3 Kernaufgaben

Universalkonzepte:
Welche universellen Begriffe sind für alle Anwendungen gültig.

Standardelemente: Welche Standardanforderungen gibt es immer?
Welche generischen Datenstrukturen werden dabei genutzt?

Standardverhalten:
Wie reagiert das System standardmäßig auf
generische Anforderungen durch diese Standardelemente?


Universell anwendbar für jede Anwendung!
Daher hohe Systematisierung und Wiederverwendbarkeit möglich.
```

E Source Code

Listing E.1: Main Experimental Pipeline Controller: **main.py**

```python
from api_config import init_clients
from question_generation import (
    run_exp_1a,
    run_exp_1b,
    run_exp_1a_no_source,
    run_exp_2a,
    run_exp_2b,
    run_exp_2c,
)
from prompt_utils import get_bloom


def main():
    print("[INFO] Initializing LLM clients...")
    try:
        clients = init_clients()
    except ValueError as e:
        print(f"[ERROR] Failed to initialize clients: {e}")
        print("[ERROR] Please ensure your .env file is correctly set up with API keys.")
        return

    print("[INFO] Pre-loading Bloom data...")
    get_bloom()

    print("[INFO] Starting question generation experiments...")
    run_exp_1a(clients)
    run_exp_1b(clients)
    run_exp_1a_no_source(clients)
    run_exp_2a(clients)
    run_exp_2b(clients)
    run_exp_2c(clients)

    print("\n[INFO] All experiments completed.")


if __name__ == "__main__":
    main()
```

Listing E.2: API Integration Layer: **api_calls.py**

```python
import time
from constants import REQUEST_DELAY_SECONDS, LLM_MODEL_IDS
from google.genai import types


def gen_with_google(client, prompt_text, model_id, max_tokens):
    try:
        response = client.models.generate_content(
            model=model_id,
            contents=prompt_text,
            config=types.GenerateContentConfig(
                thinking_config=types.ThinkingConfig(thinking_budget=1100),
                max_output_tokens=max_tokens,
            ),
        )
        candidate = response.candidates[0]
        if candidate.finish_reason == "MAX_TOKENS":
            print(f"\n[WARNING] {model_id} ran out of tokens")
            if response.text:
                print(f"[INFO] Partial output available for {model_id}")
                return response.text
            else:
                print(f"[WARNING] No output available for {model_id}")
                return None
        return response.text
    except Exception as e:
        print(f"[ERROR] Gemini API ({model_id}): {e}")
        return None


def gen_with_anthropic(client, prompt_text, model_id, max_tokens):
    try:
        response = client.messages.create(
            model=model_id,
            thinking={"type": "enabled", "budget_tokens": 1100},
            messages=[{"role": "user", "content": prompt_text}],
            max_tokens=max_tokens,
        )

        if response.stop_reason == "max_tokens":
            print(f"\n[WARNING] {model_id} ran out of tokens")
            for block in response.content:
                if block.type == "text":
                    print(f"[INFO] Partial output available for {model_id}")
                    return block.text
```

```
            print(f"[WARNING] No output available for {model_id}")
            return None

        for block in response.content:
            if block.type == "text":
                return block.text
    except Exception as e:
        print(f"[ERROR] Claude API ({model_id}): {e}")
        return None


def gen_with_openai(client, prompt_text, model_id, max_tokens):
    try:
        response = client.responses.create(
            model=model_id,
            reasoning={"effort": "medium"},
            input=[{"role": "user", "content": prompt_text}],
            max_output_tokens=max_tokens,
        )
        if (
            response.status == "incomplete"
            and response.incomplete_details.reason == "max_output_tokens"
        ):
            print(f"\n[WARNING] {model_id} ran out of tokens")
            if response.output_text:
                print(f"[INFO] Partial output available for {model_id}")
                return response.output_text
            else:
                print(f"[WARNING] {model_id} ran out of tokens during reasoning")
                return None

        return response.output_text

    except Exception as e:
        print(f"[ERROR] OpenAI API ({model_id}): {e}")
        return None


def llm_generation(llm_name, clients, prompt_text, max_tokens):
    client = clients.get(llm_name)
    model_id = LLM_MODEL_IDS.get(llm_name)

    if not client or not model_id:
        print(f"[ERROR] Client or model_id not found for LLM: {llm_name}")
        return None

    result = None
    if llm_name == "google":
```

```
94          result = gen_with_google(client, prompt_text, model_id, max_tokens)
95      elif llm_name == "anthropic":
96          result = gen_with_anthropic(client, prompt_text, model_id, max_tokens)
97      elif llm_name == "openai":
98          result = gen_with_openai(client, prompt_text, model_id, max_tokens)
99      else:
100         print(f"[ERROR] Unknown LLM name '{llm_name}'")
101         return None
102
103     time.sleep(REQUEST_DELAY_SECONDS)
104     return result
```

Listing E.3: API Configuration Management: **api_config.py**

```python
import os
from dotenv import load_dotenv
from google import genai
import anthropic
import openai


def load_api_keys():
    load_dotenv()
    keys = {
        "google": os.getenv("GOOGLE_API_KEY"),
        "anthropic": os.getenv("ANTHROPIC_API_KEY"),
        "openai": os.getenv("OPENAI_API_KEY"),
    }
    if not all(keys.values()):
        missing = [k for k, v in keys.items() if not v]
        raise ValueError(
            f"[ERROR] Missing API keys for: {', '.join(missing)} in .env file"
        )
    return keys


def init_clients():
    api_keys = load_api_keys()

    print("[INFO] Initializing API clients...")

    google_client = genai.Client(api_key=api_keys["google"])
    anthropic_client = anthropic.Anthropic(api_key=api_keys["anthropic"])
    openai_client = openai.OpenAI(api_key=api_keys["openai"])

    print("[INFO] API clients initialized successfully")

    return {
        "google": google_client,
        "anthropic": anthropic_client,
        "openai": openai_client,
    }
```

Listing E.4: Prompt Engineering Framework: **prompt_utils.py**

```python
import os
from constants import BLOOM_LEVELS_ORDERED, BLOOM_DATA_FILE, PROMPT_TEMPLATES_PATH
from file_utils import load_txt


def load_prompt(prompt_name):
    filename = prompt_name if prompt_name.endswith(".md") else f"{prompt_name}.md"
    path = os.path.join(PROMPT_TEMPLATES_PATH, filename)
    return load_txt(path)


def format_prompt(template, **values):
    if template is None:
        return None
    try:
        return template.format(**values)
    except KeyError as e:
        print(
            f"[WARNING] Missing key {e} in prompt formatting. Available keys:
            ↪ {list(values.keys())}"
        )
        return template


def parse_bloom_md(md_content):
    bloom_data = {
        level: {"description": "", "verbs": ""} for level in BLOOM_LEVELS_ORDERED
    }
    if not md_content:
        return bloom_data

    current_section = None
    lines = md_content.splitlines()

    for line in lines:
        line_stripped = line.strip()
        if line_stripped.startswith("## Descriptions"):
            current_section = "descriptions"
            continue
        elif line_stripped.startswith("## Verbs"):
            current_section = "verbs"
            continue

        if current_section and line_stripped.startswith("- "):
            try:
```

```
                parts = line_stripped[2:].split(":", 1)
                if len(parts) == 2:
                    level_name = parts[0].strip()
                    content = parts[1].strip()
                    if level_name in bloom_data:
                        if current_section == "descriptions":
                            bloom_data[level_name]["description"] = content
                        elif current_section == "verbs":
                            bloom_data[level_name]["verbs"] = content
            except Exception as e:
                print(
                    f"[WARNING] Could not parse Bloom data line: '{line_stripped}'. Error: {e}"
                )
    return bloom_data


_bloom_cache = None


def get_bloom():
    global _bloom_cache
    if _bloom_cache is None:
        raw = load_txt(BLOOM_DATA_FILE)
        _bloom_cache = parse_bloom_md(raw)
    return _bloom_cache


def q_format(q_type):
    if q_type == "Multiple-Choice":
        return (
            "Frage: Hier den Frageninhalt einfügen\n\n"
            "Antwortmöglichkeiten:\n"
            "a) Beispielsweise hier die korrekte Antwort einfügen (Richtig)\n"
            "b) Hier eine falsche Antwort einfügen (Falsch)\n"
            "c) Hier eine weitere falsche Antwort einfügen (Falsch)\n"
            "d) ..."
        )
    else:
        return (
            "Frage: Hier den Frageninhalt einfügen\n\n"
            "Antwort: Hier die Antwort einfügen (Richtig)"
        )
```

Listing E.5: Data Processing Pipeline: **file_utils.py**

```python
import os


def load_txt(file_path):
    try:
        with open(file_path, "r", encoding="utf-8") as f:
            return f.read()
    except FileNotFoundError:
        print(f"Warning: File not found at {file_path}")
        return None
    except Exception as e:
        print(f"Error reading file {file_path}: {e}")
        return None


def save_result(file_path, content):
    try:
        os.makedirs(os.path.dirname(file_path), exist_ok=True)
        with open(file_path, "w", encoding="utf-8") as f:
            f.write(content)
    except Exception as e:
        print(" " * 7 + f"Error saving file {file_path}: {e}\n")


def slugify(text):
    return text.lower().replace(" ", "_").replace("-", "_")
```

Listing E.6: Question Generation Code: **question_generation.py**

```python
import os
import csv
import constants
from file_utils import load_txt, save_result, slugify
from prompt_utils import (
    load_prompt,
    format_prompt,
    get_bloom,
    q_format,
)
from api_calls import llm_generation
from concurrent.futures import ThreadPoolExecutor, as_completed
import threading
from tqdm import tqdm
from collections import defaultdict


def create_csvs(exp_name, headers, rows):
    try:
        initial_csv_dir = os.path.join(
            os.path.dirname(constants.EXP1_PATH), "60_analyses", "csv", "initial"
        )
        if exp_name.startswith("exp1"):
            exp_subdir = os.path.join(initial_csv_dir, "exp1")
        elif exp_name.startswith("exp2"):
            exp_subdir = os.path.join(initial_csv_dir, "exp2")
        else:
            exp_subdir = initial_csv_dir

        file_path = os.path.join(exp_subdir, f"{exp_name}.csv")

        os.makedirs(os.path.dirname(file_path), exist_ok=True)

        with open(file_path, "w", newline="", encoding="utf-8") as f:
            writer = csv.writer(f)
            writer.writerow(headers)
            if rows:
                writer.writerows(rows)
        print(f"[INFO] Initial CSV created: {os.path.relpath(file_path)}")
    except IOError as e:
        print(f"[ERROR] Could not write to CSV file {file_path}: {e}")


def concatenate_all_script_layers(script_subdir="common"):
    concatenated_content = []
```

```
46
47      for layer_num in constants.LAYERS:
48          layer_path = os.path.join(
49              constants.INPUT_SOURCES_PATH,
50              "script",
51              script_subdir,
52              f"layer{layer_num}.txt",
53          )
54          layer_content = load_txt(layer_path)
55          if layer_content:
56              concatenated_content.append(layer_content.strip())
57          else:
58              print(
59                  f"[WARNING] Could not load layer{layer_num}.txt from script/{script_subdir}"
60              )
61
62      return "\n---\n".join(concatenated_content)
63
64
65  llm_counters = defaultdict(int)
66  counter_lock = threading.Lock()
67
68
69  def increment_counter(llm_name):
70      with counter_lock:
71          llm_counters[llm_name] += 1
72
73
74  def reset_counters():
75      with counter_lock:
76          llm_counters.clear()
77          for llm_name in constants.LLM_NAMES:
78              llm_counters[llm_name] = 0
79
80
81  def get_progress():
82      with counter_lock:
83          if not llm_counters:
84              return "Progress"
85          desc_parts = []
86          for llm_name in sorted(llm_counters.keys()):
87              count = llm_counters[llm_name]
88              desc_parts.append(f"{llm_name}:{count}")
89          return f"Progress [{' | '.join(desc_parts)}]"
90
91
92  def generate_task(
93      llm_name,
```

```
    clients,
    prompt_or_template,
    output_path,
    description,
    input_text=None,
    max_tokens=2400,
):
    try:
        if llm_name == "deepseek":
            save_result(output_path, "")
            increment_counter(llm_name)
            return True

        if input_text is not None:
            formatted_prompt = format_prompt(prompt_or_template, text=input_text)
        else:
            formatted_prompt = prompt_or_template

        if not formatted_prompt:
            return None

        generated_question = llm_generation(
            llm_name, clients, formatted_prompt, max_tokens=max_tokens
        )
        if generated_question:
            save_result(output_path, generated_question)
            increment_counter(llm_name)
            return True
    except Exception as e:
        print(f"[ERROR] {llm_name}: {description} - {e}")
    return False


def run_tasks(tasks, exp_desc):
    with ThreadPoolExecutor(max_workers=3) as executor:
        futures = {executor.submit(generate_task, *task): task for task in tasks}

        with tqdm(
            total=len(tasks), desc=f"{exp_desc} {get_progress()}", unit="task"
        ) as pbar:
            for future in as_completed(futures):
                future.result()
                pbar.set_description(f"{exp_desc} {get_progress()}")
                pbar.update(1)


def run_exp_1a(clients):
    print("\n[INFO] Experiment 1a: Content Fidelity")
```

```
    reset_counters()
    tasks = []
    csv_rows = []

    for prompt_type_file_name in constants.EXP1_PROMPT_TYPES:
        prompt_path = os.path.join(
            constants.PROMPT_TEMPLATES_PATH,
            "experiment",
            prompt_type_file_name + ".md",
        )
        prompt_template = load_prompt(prompt_path)
        if not prompt_template:
            continue

        output_dir_name = prompt_type_file_name.replace("exp1_", "")

        for source_type in constants.EXP1_SOURCE_TYPES_A:
            for layer_num in constants.LAYERS:
                if source_type == "script":
                    input_text_path = os.path.join(
                        constants.INPUT_SOURCES_PATH,
                        source_type,
                        "common",
                        f"layer{layer_num}.txt",
                    )
                else:
                    input_text_path = os.path.join(
                        constants.INPUT_SOURCES_PATH,
                        source_type,
                        f"layer{layer_num}.txt",
                    )

                input_text = load_txt(input_text_path)
                if not input_text:
                    continue

                for llm_name in constants.LLM_NAMES:
                    output_path = os.path.join(
                        constants.EXP1_PATH,
                        "run_a_content",
                        output_dir_name,
                        llm_name,
                        source_type,
                        f"layer{layer_num}_question.txt",
                    )
                    description = f"{source_type}/layer{layer_num}"

                    tasks.append(
```

```
                        (
                            llm_name,
                            clients,
                            prompt_template,
                            output_path,
                            description,
                            input_text,
                            1800,
                        )
                    )
                    prompt_type = output_dir_name.replace("_prompt", "")
                    csv_rows.append([llm_name, source_type, layer_num, prompt_type])

    csv_rows.sort(key=lambda row: (row[0], row[1], int(row[2]), row[3]))
    headers = ["llm", "input_source", "layer", "prompt_type"]
    create_csvs("exp1a", headers, csv_rows)
    run_tasks(tasks, "Exp 1a")


def run_exp_1b(clients):
    print("\n[INFO] Experiment 1b: Error Handling")
    reset_counters()
    source_type = constants.EXP1_SOURCE_TYPE_B
    tasks = []
    csv_rows = []

    for prompt_type_file_name in constants.EXP1_PROMPT_TYPES:
        prompt_path = os.path.join(
            constants.PROMPT_TEMPLATES_PATH,
            "experiment",
            prompt_type_file_name + ".md",
        )
        prompt_template = load_prompt(prompt_path)
        if not prompt_template:
            continue

        output_dir_name = prompt_type_file_name.replace("exp1_", "")

        for layer_num in constants.LAYERS:
            input_text_path = os.path.join(
                constants.INPUT_SOURCES_PATH,
                source_type,
                "manipulated",
                f"layer{layer_num}.txt",
            )
            input_text = load_txt(input_text_path)
            if not input_text:
                print(f"[WARNING] {source_type}/layer{layer_num}: file not found")
```

```
                continue

            for llm_name in constants.LLM_NAMES:
                output_path = os.path.join(
                    constants.EXP1_PATH,
                    "run_b_error",
                    output_dir_name,
                    llm_name,
                    source_type,
                    f"layer{layer_num}_question.txt",
                )
                description = f"{source_type}/layer{layer_num}"

                tasks.append(
                    (
                        llm_name,
                        clients,
                        prompt_template,
                        output_path,
                        description,
                        input_text,
                        1800,
                    )
                )
                prompt_type = output_dir_name.replace("_prompt", "")
                csv_rows.append([llm_name, source_type, layer_num, prompt_type])

    csv_rows.sort(key=lambda row: (row[0], row[1], int(row[2]), row[3]))
    headers = ["llm", "input_source", "layer", "prompt_type"]
    create_csvs("exp1b", headers, csv_rows)
    run_tasks(tasks, "Exp 1b")


def run_exp_1a_no_source(clients):
    print("\n[INFO] Experiment 1a: No Source Content")
    reset_counters()
    tasks = []
    csv_rows = []

    prompt_path = os.path.join(
        constants.PROMPT_TEMPLATES_PATH,
        "experiment",
        "exp1a_complex_prompt_no_source.md",
    )
    prompt_template = load_prompt(prompt_path)
    if not prompt_template:
        print("[ERROR] Could not load prompt template: exp1a_complex_prompt_no_source")
        return
```

```
286
287     for layer_num in constants.LAYERS:
288         for llm_name in constants.LLM_NAMES:
289             formatted_prompt = format_prompt(prompt_template, layer=layer_num)
290
291             output_path = os.path.join(
292                 constants.EXP1_PATH,
293                 "run_a_content",
294                 "complex_prompt_no_source",
295                 llm_name,
296                 f"layer{layer_num}_question.txt",
297             )
298             description = f"layer{layer_num}"
299
300             tasks.append(
301                 (
302                     llm_name,
303                     clients,
304                     formatted_prompt,
305                     output_path,
306                     description,
307                     None,
308                     1800,
309                 )
310             )
311             csv_rows.append([llm_name, "no_source", layer_num, "complex_no_source"])
312
313     csv_rows.sort(key=lambda row: (row[0], row[1], int(row[2]), row[3]))
314     headers = ["llm", "input_source", "layer", "prompt_type"]
315     create_csvs("exp1a_no_source", headers, csv_rows)
316     run_tasks(tasks, "Exp 1 No Source")
317
318
319 def run_exp_2a(clients):
320     print("\n[INFO] Experiment 2a: Question Type")
321     reset_counters()
322     tasks = []
323     csv_rows = []
324     layer_specification = "all_layers"
325     base_text_for_exp2 = concatenate_all_script_layers("common")
326
327     prompt_template_type = load_prompt(
328         os.path.join(constants.PROMPT_TEMPLATES_PATH, "experiment", "exp2_type.md")
329     )
330     if not prompt_template_type:
331         return
332
333     for q_type in constants.EXP2_QUESTION_TYPES:
```

```
        q_type_slug = slugify(q_type)
        q_type_format_str = q_format(q_type)
        for i in range(6):
            question_id = i + 1
            formatted_prompt = format_prompt(
                prompt_template_type,
                text=base_text_for_exp2,
                question_type=q_type,
                question_type_format=q_type_format_str,
            )

            for llm_name in constants.LLM_NAMES:
                output_path = os.path.join(
                    constants.EXP2_PATH,
                    "run_a_type",
                    llm_name,
                    q_type_slug,
                    f"question_{question_id}.txt",
                )
                description = f"{q_type} question {question_id}"

                tasks.append(
                    (
                        llm_name,
                        clients,
                        formatted_prompt,
                        output_path,
                        description,
                        None,
                        2400,
                    )
                )
                csv_rows.append(
                    [
                        "exp2a",
                        llm_name,
                        layer_specification,
                        q_type.lower().replace("-", "_"),
                        question_id,
                    ]
                )

    csv_rows.sort(key=lambda row: (row[0], row[1], row[2], row[3], row[4]))
    headers = ["exp_name", "llm", "layers", "question_type", "question_id"]
    create_csvs("exp2a", headers, csv_rows)
    run_tasks(tasks, "Exp 2a")


```

```
382 def run_exp_2b(clients):
383     print("\n[INFO] Experiment 2b: Bloom Level")
384     reset_counters()
385     tasks = []
386     csv_rows = []
387     layer_specification = "all_layers"
388     bloom_data = get_bloom()
389 
390     base_text_for_exp2 = concatenate_all_script_layers("common")
391 
392     prompt_template_bloom = load_prompt(
393         os.path.join(constants.PROMPT_TEMPLATES_PATH, "experiment", "exp2_bloom.md")
394     )
395     if not prompt_template_bloom:
396         return
397 
398     for bloom_level_index, bloom_level_name in enumerate(
399         constants.BLOOM_LEVELS_ORDERED
400     ):
401         bloom_original = bloom_level_index + 1
402         level_data = bloom_data.get(bloom_level_name, {})
403         formatted_prompt = format_prompt(
404             prompt_template_bloom,
405             text=base_text_for_exp2,
406             bloom_level=bloom_level_name,
407             bloom_level_description=level_data.get("description", ""),
408             bloom_level_verbs=level_data.get("verbs", ""),
409         )
410 
411         for llm_name in constants.LLM_NAMES:
412             output_path = os.path.join(
413                 constants.EXP2_PATH,
414                 "run_b_bloom",
415                 llm_name,
416                 f"question_{bloom_original}.txt",
417             )
418             description = f"{bloom_level_name}"
419 
420             tasks.append(
421                 (
422                     llm_name,
423                     clients,
424                     formatted_prompt,
425                     output_path,
426                     description,
427                     None,
428                     2400,
429                 )
```

```
            )
            csv_rows.append(["exp2b", llm_name, layer_specification, bloom_original])

    csv_rows.sort(key=lambda row: (row[0], row[1], row[2], row[3]))
    headers = ["exp_name", "llm", "layers", "bloom_original"]
    create_csvs("exp2b", headers, csv_rows)
    run_tasks(tasks, "Exp 2b")


def run_exp_2c(clients):
    print("\n[INFO] Experiment 2c: Combined Type and Bloom")
    reset_counters()
    tasks = []
    csv_rows = []
    layer_specification = "all_layers"
    bloom_data = get_bloom()

    # Use concatenated script layers instead of single tanenbaum layer
    base_text_for_exp2 = concatenate_all_script_layers("common")

    prompt_template_both = load_prompt(
        os.path.join(constants.PROMPT_TEMPLATES_PATH, "experiment", "exp2_both.md")
    )
    if not prompt_template_both:
        return

    for q_type in constants.EXP2_QUESTION_TYPES:
        q_type_slug = slugify(q_type)
        for i, bloom_level_name in enumerate(constants.BLOOM_LEVELS_ORDERED):
            bloom_original = i + 1
            level_data = bloom_data.get(bloom_level_name, {})
            q_type_format_str = q_format(q_type)

            formatted_prompt = format_prompt(
                prompt_template_both,
                text=base_text_for_exp2,
                question_type=q_type,
                bloom_level=bloom_level_name,
                bloom_level_description=level_data.get("description", ""),
                bloom_level_verbs=level_data.get("verbs", ""),
                question_type_format=q_type_format_str,
            )

            for llm_name in constants.LLM_NAMES:
                output_path = os.path.join(
                    constants.EXP2_PATH,
                    "run_c_both",
                    llm_name,
```

```
                    q_type_slug,
                    f"question_{bloom_original}.txt",
                )
                description = f"{q_type} {bloom_level_name}"

                tasks.append(
                    (
                        llm_name,
                        clients,
                        formatted_prompt,
                        output_path,
                        description,
                        None,
                        2400,
                    )
                )
                csv_rows.append(
                    [
                        "exp2c",
                        llm_name,
                        layer_specification,
                        q_type.lower().replace("-", "_"),
                        bloom_original,
                    ]
                )

    csv_rows.sort(key=lambda row: (row[0], row[1], row[2], row[3], row[4]))
    headers = ["exp_name", "llm", "layers", "question_type", "bloom_original"]
    create_csvs("exp2c", headers, csv_rows)
    run_tasks(tasks, "Exp 2c")
```

Listing E.7: Question Sampling Module: **sampling.py**

```python
import os
import random
import shutil
import pandas as pd
from constants import EXP1_PATH, EXP2_PATH, EXPERIMENTS_BASE_PATH


def sample_questions(src_path, dest_path, pattern, sample_size=3):
    files = sorted(
        [f for f in os.listdir(src_path) if pattern in f and f.endswith(".txt")]
    )
    print(files)
    if len(files) < sample_size:
        print(f"[WARNING] Only {len(files)} questions available in {src_path}")
        sample_size = len(files)
    if sample_size == 0:
        return []

    sampled = random.sample(files, sample_size)
    os.makedirs(dest_path, exist_ok=True)
    for file in sampled:
        shutil.copy2(os.path.join(src_path, file), os.path.join(dest_path, file))
    return sampled


def walk_and_sample(base_path, sample_base, exp_name, pattern, sample_size=3):
    print(f"[INFO] Sampling {exp_name} questions ({sample_size} per condition)...")

    paths_to_process = []
    for root, _, files in os.walk(base_path):
        if "complex_prompt_no_source" in root:
            continue
        if any(pattern in f and f.endswith(".txt") for f in files):
            paths_to_process.append(root)

    paths_to_process.sort()

    for root in paths_to_process:
        rel_path = root.replace(base_path, "").lstrip(os.sep)
        dest_path = os.path.join(sample_base, exp_name, rel_path)
        sampled = sample_questions(root, dest_path, pattern, sample_size)
        if sampled:
            print(f"         {rel_path}: {len(sampled)} questions sampled")


```

```
def parse_file_path(parts):
    runs_map = {"run_a_type": "exp2a", "run_b_bloom": "exp2b", "run_c_both": "exp2c"}

    if parts[0] == "exp1":
        return {
            "exp_name": "exp1a" if parts[1] == "run_a_content" else "exp1b",
            "prompt_type": parts[2].replace("_prompt", ""),
            "llm": parts[3],
            "input_source": parts[4],
            "layer": int(parts[5].split("_")[0].replace("layer", "")),
        }
    elif parts[0] == "exp2" and parts[1] in runs_map:
        exp_name = runs_map[parts[1]]
        record = {"exp_name": exp_name, "llm": parts[2], "layer": 2}

        if exp_name == "exp2a":
            record["question_type"] = parts[3].replace("-", "_")
            record["question_id"] = int(parts[4].split("_")[1].replace(".txt", ""))
        elif exp_name == "exp2b":
            record["bloom_original"] = int(parts[3].split("_")[1].replace(".txt", ""))
        elif exp_name == "exp2c":
            record["question_type"] = parts[3].replace("-", "_")
            record["bloom_original"] = int(parts[4].split("_")[1].replace(".txt", ""))
        return record
    return None


def generate_expert_csvs(sample_base, csv_path):
    print("\n[INFO] Generating CSV files for expert evaluation from samples...")

    records = []
    for root, _, files in os.walk(sample_base):
        for file in files:
            if file.endswith(".txt"):
                # Calculate relative path manually to avoid getcwd issues
                full_path = os.path.join(root, file)
                rel_path = full_path.replace(sample_base, "").lstrip(os.sep)
                parts = rel_path.split(os.sep)
                try:
                    record = parse_file_path(parts)
                    if record:
                        records.append(record)
                except (IndexError, ValueError):
                    print(f"[WARNING] Could not parse path: {os.path.join(*parts)}")

    if not records:
        return

```

```
    records.sort(
        key=lambda r: (
            r.get("exp_name", ""),
            r.get("llm", ""),
            r.get("input_source", ""),
            r.get("prompt_type", ""),
            r.get("layer", 0),
            r.get("question_type", ""),
            r.get("question_id", 0),
            r.get("bloom_original", 0),
        )
    )

    df = pd.DataFrame(records)

    # Ensure integer columns are properly typed
    if "question_id" in df.columns:
        df["question_id"] = df["question_id"].astype("Int64")
    if "bloom_original" in df.columns:
        df["bloom_original"] = df["bloom_original"].astype("Int64")
    if "layer" in df.columns:
        df["layer"] = df["layer"].astype("Int64")

    # Generate hint files for exp1
    print("\n[INFO] Generating hint CSV files for exp1...")
    exp1_hint_data = df[df["exp_name"].isin(["exp1a", "exp1b"])].copy()

    if not exp1_hint_data.empty:
        output_columns = ["llm", "prompt_type", "input_source", "layer"]

        for exp_name, group in exp1_hint_data.groupby("exp_name"):
            hints_path = os.path.join(
                csv_path, "qualitative", "exp1", "hints", f"{exp_name}_hints.csv"
            )
            os.makedirs(os.path.dirname(hints_path), exist_ok=True)
            group_to_save = group[output_columns]
            group_to_save.to_csv(hints_path, index=False)
            print(f"  - Saved hint file: {os.path.basename(hints_path)}")

    # Generate hint files for exp2
    print("\n[INFO] Generating hint CSV files for exp2...")
    exp2_hint_data = df[df["exp_name"].isin(["exp2a", "exp2b", "exp2c"])].copy()

    if not exp2_hint_data.empty:
        for exp_name, group in exp2_hint_data.groupby("exp_name"):
            hints_path = os.path.join(
                csv_path, "qualitative", "exp2", "hints", f"{exp_name}_hints.csv"
            )
```

```
            os.makedirs(os.path.dirname(hints_path), exist_ok=True)

            if exp_name == "exp2a":
                output_columns = ["llm", "question_id", "question_type"]
            elif exp_name == "exp2b":
                output_columns = ["llm", "bloom_original"]
            else:  # exp2c
                output_columns = ["llm", "bloom_original", "question_type"]

            group_to_save = group[output_columns]
            group_to_save.to_csv(hints_path, index=False)
            print(f"  - Saved hint file: {os.path.basename(hints_path)}")

    # Exp1: Save to expert_1 through expert_5 folders
    exp1_data = df[df["exp_name"].isin(["exp1a", "exp1b"])]
    for exp_name, group in exp1_data.groupby("exp_name"):
        output_df = group[["input_source", "layer"]].copy()

        output_df = output_df.reset_index(drop=True)
        output_df["sample_id"] = [f"{i+1:03d}" for i in range(len(output_df))]

        if exp_name == "exp1a":
            categories = [
                "relevance",
                "clarity",
                "answerability",
                "challenging",
                "value",
                "language",
                "correctness",
                "answer_problems",
                "comments",
            ]
        else:
            categories = [
                "relevance",
                "clarity",
                "answerability",
                "challenging",
                "value",
                "language",
                "manipulation_handling",
                "answer_problems",
                "comments",
            ]

        for col in categories:
            output_df[col] = ""
```

```
190
191         for i in range(1, 6):
192             expert_dir = os.path.join(
193                 csv_path, "qualitative", "exp1", "experts", f"expert_{i}"
194             )
195             os.makedirs(expert_dir, exist_ok=True)
196             output_df.to_csv(os.path.join(expert_dir, f"{exp_name}.csv"), index=False)
197
198     # Exp2: Save to student folders (only evaluation columns, no sensitive data)
199     exp2_data = df[df["exp_name"].isin(["exp2a", "exp2b", "exp2c"])]
200     for exp_name, group in exp2_data.groupby("exp_name"):
201         output_df = pd.DataFrame()
202         output_df["sample_id"] = [f"{i+1:03d}" for i in range(len(group))]
203
204         for col in [
205             "relevance",
206             "clarity",
207             "answerability",
208             "challenging",
209             "value",
210             "language",
211             "bloom_rating",
212             "answer_problems",
213             "comments",
214         ]:
215             output_df[col] = ""
216
217         for i in range(1, 4):
218             student_dir = os.path.join(
219                 csv_path, "qualitative", "exp2", "students", f"student_{i}"
220             )
221             os.makedirs(student_dir, exist_ok=True)
222             output_df.to_csv(os.path.join(student_dir, f"{exp_name}.csv"), index=False)
223
224
225 def find_file(samples, exp, row):
226     if exp.startswith("exp1"):
227         run = "run_a_content" if exp == "exp1a" else "run_b_error"
228         source = row["input_source"].replace("_manipulated", "")
229         dir_path = os.path.join(
230             samples, "exp1", run, f"{row['prompt_type']}_prompt", row["llm"], source
231         )
232         if os.path.exists(dir_path):
233             pattern = f"layer{int(row['layer'])}_question"
234             for f in os.listdir(dir_path):
235                 if pattern in f and f.endswith(".txt"):
236                     return os.path.join(dir_path, f)
237     else:
```

```
        runs = {"exp2a": "run_a_type", "exp2b": "run_b_bloom", "exp2c": "run_c_both"}
        run = runs.get(exp)
        if run:
            llm = row["llm"]
            if exp == "exp2a":
                qtype = row["question_type"].lower().replace("-", "_")
                dir_path = os.path.join(samples, "exp2", run, llm, qtype)
                filename = f"question_{int(row['question_id'])}.txt"
            elif exp == "exp2b":
                dir_path = os.path.join(samples, "exp2", run, llm)
                filename = f"question_{int(row['bloom_original'])}.txt"
            else:
                qtype = row["question_type"].lower().replace("-", "_")
                dir_path = os.path.join(samples, "exp2", run, llm, qtype)
                filename = f"question_{int(row['bloom_original'])}.txt"

            file_path = os.path.join(dir_path, filename)
            if os.path.exists(file_path):
                return file_path
    return None


def get_source_type(exp, row):
    if exp.startswith("exp1"):
        src = row.get("input_source", "")
        if "manipulated" in src or exp == "exp1b":
            return "script_manipulated"
        if "transcript" in src:
            return "transcript"
        if "script" in src:
            return "script"
        if "tanenbaum" in src:
            return "tanenbaum"
        return "unknown"
    return "script"


def rename_samples(samples, csv_path, output_path):
    print("\n[INFO] Renaming samples for manual inspection...")

    # Look for hint CSV files in the correct structure
    for exp_prefix in ["exp1", "exp2"]:
        exp_csv_dir = os.path.join(csv_path, "qualitative", exp_prefix, "hints")
        if not os.path.exists(exp_csv_dir):
            continue

        csvs = [
            f
```

```
            for f in os.listdir(exp_csv_dir)
            if f.startswith(exp_prefix) and f.endswith("_hints.csv")
        ]
        for csv in sorted(csvs):
            count = 1
            exp = csv.replace("_hints.csv", "")
            df = pd.read_csv(os.path.join(exp_csv_dir, csv))
            exp_dir = os.path.join(output_path, exp)
            os.makedirs(exp_dir, exist_ok=True)

            for _, row in df.iterrows():
                src = find_file(samples, exp, row)
                if src and os.path.exists(src):
                    source_type = get_source_type(exp, row)
                    if exp.startswith("exp1"):
                        layer = row.get("layer", 2)
                        new_name = f"{count:03d}_{source_type}_{layer}.txt"
                    else:  # exp2
                        new_name = f"{count:03d}_{source_type}_all.txt"
                    shutil.copy2(src, os.path.join(exp_dir, new_name))
                    print(f"  {os.path.basename(src)} -> {new_name}")
                    count += 1


def main():
    random.seed(2025)
    base = EXPERIMENTS_BASE_PATH
    sample_base = os.path.join(base, "70_samples")
    csv_path = os.path.join(base, "60_analyses", "csv")
    output_path = os.path.join(base, "80_samples_renamed")

    # Clean up previous runs
    if os.path.exists(sample_base):
        shutil.rmtree(sample_base)
    if os.path.exists(output_path):
        shutil.rmtree(output_path)

    # Clear existing CSV files instead of deleting directories
    qualitative_path = os.path.join(csv_path, "qualitative")
    if os.path.exists(qualitative_path):
        # Clear all CSV files in the qualitative directory
        for root, _, files in os.walk(qualitative_path):
            for file in files:
                if file.endswith(".csv"):
                    file_path = os.path.join(root, file)
                    # Create empty CSV file to clear content
                    with open(file_path, "w") as f:
                        pass
```

```
334
335     # Ensure directories exist
336     os.makedirs(sample_base, exist_ok=True)
337     os.makedirs(output_path, exist_ok=True)
338
339     for subfolder in ["experts", "hints"]:
340         path_to_create = os.path.join(csv_path, "qualitative", "exp1", subfolder)
341         os.makedirs(path_to_create, exist_ok=True)
342
343     for subfolder in ["students", "hints"]:
344         path_to_create = os.path.join(csv_path, "qualitative", "exp2", subfolder)
345         os.makedirs(path_to_create, exist_ok=True)
346
347     walk_and_sample(EXP1_PATH, sample_base, "exp1", "_question", 2)
348     walk_and_sample(EXP2_PATH, sample_base, "exp2", "question_", 2)
349     print(f"[INFO] Sampling completed. Results: {sample_base}")
350
351     generate_expert_csvs(sample_base, csv_path)
352     rename_samples(sample_base, csv_path, output_path)
353
354     print(
355         f"\nDone. Qualitative Analysis CSVs: {csv_path}/qualitative/ | Renamed samples:
        ↪ {output_path}"
356     )
357
358
359 if __name__ == "__main__":
360     main()
```

Listing E.8: Truncation Checking: **check_truncation.py**

```python
import os
import torch
import pandas as pd
from sentence_transformers import SentenceTransformer
from constants import INPUT_SOURCES_PATH, EMBEDDING_MODEL_ID, EXP1_PATH
from io import StringIO
import sys


class OutputCapture:
    def __init__(self):
        self.buffer = StringIO()
        self.stdout = sys.stdout
        self.output_text = []

    def write(self, text):
        self.stdout.write(text)
        self.buffer.write(text)
        self.output_text.append(text)

    def flush(self):
        self.stdout.flush()
        self.buffer.flush()

    def get_output(self):
        return self.buffer.getvalue()


def save_output(output_text):
    output_dir = os.path.dirname(__file__)

    os.makedirs(output_dir, exist_ok=True)
    output_file = os.path.join(output_dir, "note_truncation.md")

    with open(output_file, "w", encoding="utf-8") as f:
        f.write(output_text)

    print(f"\n[INFO] Output saved to {output_file}")
    return output_file


def load_embedding_model(model_name):
    device = "cuda" if torch.cuda.is_available() else "cpu"
    print(f"[INFO] Loading embedding model '{model_name}' on {device}")
    return SentenceTransformer(model_name, device=device)
```

```
46
47
48  def process_file(file_path, model):
49      try:
50          with open(file_path, "r", encoding="utf-8") as f:
51              content = f.read()
52
53          text = "\n".join(
54              line
55              for line in content.splitlines()
56              if not line.strip().endswith("(Falsch)")
57          )
58
59          tokens = model.tokenizer(text, truncation=False, return_length=True)
60          token_count = len(tokens["input_ids"])
61          will_truncate = token_count > model.max_seq_length
62          usage = (token_count / model.max_seq_length) * 100
63
64          status = "[TRUNCATED]" if will_truncate else "[OK]"
65          rel_path = os.path.relpath(file_path)
66          print(f"{status:12} {rel_path}: {token_count:,} tokens ({usage:.1f}%)")
67
68          return {
69              "file_path": file_path,
70              "token_count": token_count,
71              "will_truncate": will_truncate,
72              "usage": usage,
73          }
74      except Exception as e:
75          print(f"[ERROR] {os.path.relpath(file_path)}: {e}")
76          return None
77
78
79  def scan_directory(directory, description, model):
80      if not os.path.exists(directory):
81          print(f"[WARNING] Directory not found: {directory}")
82          return pd.DataFrame()
83
84      print(f"\n[INFO] Scanning {description}...")
85      print("=" * 80)
86
87      results = []
88      for root, _, files in os.walk(directory):
89          for filename in files:
90              if filename.endswith(".txt"):
91                  result = process_file(os.path.join(root, filename), model)
92                  if result:
93                      results.append(result)
```

```
94
95      if not results:
96          print(f"[INFO] No .txt files found in {description}")
97          return pd.DataFrame()
98
99      df = pd.DataFrame(results)
100
101     total = len(df)
102     truncated_count = df["will_truncate"].sum()
103
104     print(f"\n[SUMMARY] {description}")
105     print(f"[INFO] Total files checked: {total}")
106     print(
107         f"[INFO] Files that will be truncated: {truncated_count}
        ↪  ({truncated_count/total*100:.1f}%)"
108     )
109
110     if truncated_count > 0:
111         print("[WARNING] Files requiring truncation:")
112         truncated_df = df[df["will_truncate"]].sort_values(
113             "token_count", ascending=False
114         )
115         for _, row in truncated_df.iterrows():
116             rel_path = os.path.relpath(row["file_path"])
117             print(f"         {rel_path}: {row['token_count']:,} tokens")
118
119     return df
120
121
122 def main():
123     output_capture = OutputCapture()
124     sys.stdout = output_capture
125
126     try:
127         model = load_embedding_model(EMBEDDING_MODEL_ID)
128         print(f"[INFO] Model max sequence length: {model.max_seq_length:,} tokens")
129
130         directories = [
131             (INPUT_SOURCES_PATH, "input sources"),
132             (EXP1_PATH, "Experiment 1 questions"),
133         ]
134
135         all_dfs = []
136         for directory, description in directories:
137             df = scan_directory(directory, description, model)
138             if not df.empty:
139                 df["source"] = description
140                 all_dfs.append(df)
```

```
141
142         if not all_dfs:
143             print("[INFO] No files found to analyze")
144             return
145
146         combined_df = pd.concat(all_dfs, ignore_index=True)
147         total_files = len(combined_df)
148         total_truncated = combined_df["will_truncate"].sum()
149
150         print("\n" + "=" * 80)
151         print("[INFO] Overall Summary")
152         print(f"[INFO] Total files processed: {total_files:,}")
153         print(
154             f"[INFO] Total files truncated: {total_truncated}
                ↪  ({total_truncated/total_files*100:.1f}%)"
155         )
156
157         if total_truncated > 0:
158             print("\n[INFO] Truncated files sorted by token count:")
159             truncated_df = combined_df[combined_df["will_truncate"]].sort_values(
160                 "token_count", ascending=False
161             )
162             for _, row in truncated_df.iterrows():
163                 rel_path = os.path.relpath(row["file_path"])
164                 print(
165                     f"         TRUNCATED {rel_path}: {row['token_count']:,} tokens
                        ↪  ({row['usage']:.1f}%)"
166                 )
167         else:
168             print(
169                 "\n[INFO] No files require truncation - all files fit within model limits"
170             )
171
172         if total_truncated > 0:
173             summary = f"{total_truncated} out of {total_files} txt files were longer than the
                ↪  maximum length of {model.max_seq_length} tokens. \n"
174             summary += "the source material files were refined before*\n"
175             summary += f"and now {total_truncated} questions are longer than {model.max_seq_length}
                ↪  tokens.\n\n"
176             full_output = summary + output_capture.get_output()
177         else:
178             summary = f"All {total_files} txt files fit within the maximum length of
                ↪  {model.max_seq_length} tokens.\n\n"
179             full_output = summary + output_capture.get_output()
180
181         save_output(full_output)
182
183     finally:
```

```
184         sys.stdout = output_capture.stdout
185
186
187 if __name__ == "__main__":
188     main()
```

Listing E.9: Configuration Constants: **constants.py**

```python
import os

BASE_PROJECT_PATH = os.path.abspath(os.path.join(os.path.dirname(__file__), "..", ".."))
EXPERIMENTS_BASE_PATH = os.path.join(BASE_PROJECT_PATH, "20_experiments")
INPUT_SOURCES_PATH = os.path.join(EXPERIMENTS_BASE_PATH, "30_input_sources")
PROMPT_TEMPLATES_PATH = os.path.join(EXPERIMENTS_BASE_PATH, "40_prompts")
ANALYSES_PATH = os.path.join(EXPERIMENTS_BASE_PATH, "60_analyses")
EXP1_PATH = os.path.join(EXPERIMENTS_BASE_PATH, "10_exp1")
EXP2_PATH = os.path.join(EXPERIMENTS_BASE_PATH, "20_exp2")

LLM_MODEL_IDS = {
    "google": "gemini-2.5-flash",
    "anthropic": "claude-3-7-sonnet-20250219",
    "openai": "o3",
    "deepseek": "deepseek",
}
LLM_NAMES = LLM_MODEL_IDS.keys()
EMBEDDING_MODEL_ID = "T-Systems-onsite/cross-en-de-roberta-sentence-transformer"
REQUEST_DELAY_SECONDS = 10

EXP1_PROMPT_TYPES = ["exp1_common_prompt", "exp1_complex_prompt"]
EXP1_SOURCE_TYPES_A = ["script", "transcript", "tanenbaum"]
EXP1_SOURCE_TYPE_B = "script"
LAYERS = list(range(1, 8))

EXP2_QUESTION_TYPES = ["Multiple-Choice", "Open-Ended"]
BLOOM_LEVELS_ORDERED = [
    "Remembering",
    "Understanding",
    "Applying",
    "Analyzing",
    "Evaluating",
    "Creating",
]

BLOOM_DATA_FILE = os.path.join(PROMPT_TEMPLATES_PATH, "experiment", "bloom.md")
```

Listing E.10: Quantitative Analysis: **analysis_quantitative.py**

```python
import os
import pandas as pd
import numpy as np
import concurrent.futures
import threading
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity
from tqdm import tqdm
from constants import (
    EXP1_PATH,
    INPUT_SOURCES_PATH,
    ANALYSES_PATH,
    PROMPT_TEMPLATES_PATH,
    EMBEDDING_MODEL_ID,
)
from file_utils import load_txt
from api_calls import llm_generation
from api_config import init_clients


def load_model():
    return SentenceTransformer(EMBEDDING_MODEL_ID)


def calc_cossim_batch(model, questions, sources):
    all_texts = questions + sources
    embeddings = model.encode(all_texts, batch_size=32, show_progress_bar=True)

    question_embeddings = embeddings[: len(questions)]
    source_embeddings = embeddings[len(questions) :]

    similarities = cosine_similarity(question_embeddings, source_embeddings)
    return np.diag(similarities)


def get_adherence_scores(clients, question, source_text, evaluator):
    prompt_path = os.path.join(
        PROMPT_TEMPLATES_PATH, "evaluation", "exp1_adherence_eval.md"
    )
    prompt_template = load_txt(prompt_path)

    prompt = prompt_template.replace("{question_text}", question).replace(
        "{context_text}", source_text
    )

```

```
    response = llm_generation(evaluator, clients, prompt, max_tokens=1200)
    if response:
        try:
            score = float(response.strip())
            if 0 <= score <= 1:
                return score
            else:
                return None
        except ValueError:
            return None
    else:
        return None


def get_adherence_scores_parallel(
    clients,
    questions_sources_pairs,
    max_workers=2,
    df=None,
    valid_indices=None,
    csv_path=None,
):
    results = [(None, None)] * len(questions_sources_pairs)
    completed_count = 0
    count_lock = threading.Lock()

    def process_single(index, question, source_text):
        nonlocal completed_count

        with concurrent.futures.ThreadPoolExecutor(max_workers=2) as executor:
            future_openai = executor.submit(
                get_adherence_scores, clients, question, source_text, "openai"
            )
            future_anthropic = executor.submit(
                get_adherence_scores, clients, question, source_text, "anthropic"
            )
            openai_score = future_openai.result()
            anthropic_score = future_anthropic.result()

        results[index] = (openai_score, anthropic_score)

        # Save incrementally if parameters provided
        if df is not None and valid_indices is not None and csv_path is not None:
            with count_lock:
                idx = valid_indices[index]
                if openai_score is not None:
                    df.at[idx, "adherence_score_openai"] = openai_score
                if anthropic_score is not None:
```

```
                    df.at[idx, "adherence_score_anthropic"] = anthropic_score
                df.to_csv(csv_path, index=False)

        with count_lock:
            completed_count += 1

    # https://docs.python.org/3/library/concurrent.futures.html
    with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as executor:
        futures = [
            executor.submit(process_single, i, q, s)
            for i, (q, s) in enumerate(questions_sources_pairs)
        ]

        with tqdm(
            total=len(questions_sources_pairs),
            desc="Adherence scores (OpenAI & Anthropic)",
            unit="pair",
        ) as pbar:
            for future in concurrent.futures.as_completed(futures):
                future.result()
                pbar.update(1)

    return results


def get_question_path(exp_name, llm, source, layer, prompt_type):
    if exp_name == "exp1a":
        if source == "no_source":
            base_path = os.path.join(
                EXP1_PATH, "run_a_content", "complex_prompt_no_source"
            )
            filename = f"layer{layer}_question.txt"
            return os.path.join(base_path, llm, filename)
        else:
            base_path = os.path.join(EXP1_PATH, "run_a_content")
            filename = f"layer{layer}_question.txt"
            return os.path.join(
                base_path, f"{prompt_type}_prompt", llm, source, filename
            )
    elif exp_name == "exp1b":
        base_path = os.path.join(EXP1_PATH, "run_b_error")
        filename = f"layer{layer}_question.txt"
        actual_source = "script" if source == "script_manipulated" else source
        return os.path.join(
            base_path, f"{prompt_type}_prompt", llm, actual_source, filename
        )
    else:
        raise ValueError(f"Unknown experiment name: {exp_name}")
```

```
142
143
144 def get_source_file_path(source, layer, is_manipulated=False):
145     if source == "no_source":
146         source_dir = os.path.join(INPUT_SOURCES_PATH, "script", "common")
147         source_type = "script (common) - reference for no_source"
148         filename = f"layer{layer}.txt"
149         return os.path.join(source_dir, filename), source_type
150
151     if is_manipulated:
152         source_dir = os.path.join(INPUT_SOURCES_PATH, "script", "manipulated")
153         source_type = f"{source} (manipulated)"
154     else:
155         if source == "script":
156             source_dir = os.path.join(INPUT_SOURCES_PATH, source, "common")
157             source_type = f"{source} (common)"
158         else:
159             source_dir = os.path.join(INPUT_SOURCES_PATH, source)
160             source_type = source
161
162     filename = f"layer{layer}.txt"
163     return os.path.join(source_dir, filename), source_type
164
165
166 def expand_no_source_data(df):
167     expanded_rows = []
168     for _, row in df.iterrows():
169         if row["input_source"] == "no_source":
170             for source in ["script", "tanenbaum", "transcript"]:
171                 new_row = row.copy()
172                 new_row["comparison_source"] = source
173                 expanded_rows.append(new_row)
174         else:
175             new_row = row.copy()
176             new_row["comparison_source"] = row["input_source"]
177             expanded_rows.append(new_row)
178
179     return pd.DataFrame(expanded_rows).reset_index(drop=True)
180
181
182 def process_experiment(exp_name):
183     print(f"[INFO] Processing {exp_name}")
184     print("=" * 80)
185
186     model = load_model()
187     clients = init_clients()
188
189     if exp_name == "exp1a_no_source":
```

```
        csv_input_path = os.path.join(
            ANALYSES_PATH, "csv", "initial", "exp1", f"{exp_name}.csv"
        )
        csv_output_path = os.path.join(
            ANALYSES_PATH, "csv", "quantitative", "exp1", f"{exp_name}.csv"
        )
        actual_exp_name = "exp1a"

        # Expand no_source data to compare with all three sources
        df_original = pd.read_csv(csv_input_path)
        df = expand_no_source_data(df_original)

        # Add missing columns for expanded DataFrame
        for col in [
            "cosine_similarity",
            "adherence_score_openai",
            "adherence_score_anthropic",
        ]:
            if col not in df.columns:
                df[col] = np.nan
    else:
        csv_input_path = os.path.join(
            ANALYSES_PATH, "csv", "quantitative", "exp1", f"{exp_name}.csv"
        )
        csv_output_path = csv_input_path
        actual_exp_name = exp_name
        df = pd.read_csv(csv_input_path)

    questions = []
    sources = []
    valid_indices = []
    comparison_info = []

    print("[INFO] Loading questions and sources...")
    for idx, row in df.iterrows():
        llm = row["llm"]
        layer = row["layer"]
        prompt_type = row["prompt_type"]

        # Use comparison_source for source file selection
        comparison_source = row.get("comparison_source", row["input_source"])
        input_source = row["input_source"]

        question_path = get_question_path(
            actual_exp_name, llm, input_source, layer, prompt_type
        )
        question_content = load_txt(question_path)

```

```
        if not question_content:
            print(f"[WARNING] No question found: {question_path}")
            continue

        question = "\n".join(
            line
            for line in question_content.splitlines()
            if not line.strip().endswith("(Falsch)")
        )

        if not question:
            print(f"[WARNING] Question is empty after filtering: {question_path}")
            continue

        is_manipulated = actual_exp_name == "exp1b"
        source_path, source_type = get_source_file_path(
            comparison_source, layer, is_manipulated
        )
        source_text = load_txt(source_path)

        if not source_text:
            print(f"[WARNING] No source found: {source_path}")
            continue

        print(
            f"[MATCH {len(questions)+1:3d}] Question: {os.path.relpath(question_path, EXP1_PATH)}"
        )
        print(f"             Source:   {source_type} - layer{layer}.txt")
        print(f"             LLM:      {llm} | Prompt: {prompt_type}")
        print("-" * 80)

        questions.append(question)
        sources.append(source_text)
        valid_indices.append(idx)
        comparison_info.append(
            {
                "question_path": question_path,
                "source_path": source_path,
                "source_type": source_type,
                "llm": llm,
                "prompt_type": prompt_type,
                "layer": layer,
            }
        )

    if not questions:
        print("[ERROR] No valid question-source pairs found!")
        return
```

```
286
287     print(f"\n[INFO] Total valid pairs found: {len(questions)}")
288     print("=" * 80)
289
290     print("[INFO] Calculating cosine similarities in batch...")
291     similarities = calc_cossim_batch(model, questions, sources)
292
293     for i, idx in enumerate(valid_indices):
294         df.at[idx, "cosine_similarity"] = similarities[i]
295         info = comparison_info[i]
296         print(
297             f"[COSINE] {info['source_type']} layer{info['layer']} -> {info['llm']}
                ↪  ({info['prompt_type']}): {similarities[i]:.4f}"
298         )
299
300     df.to_csv(csv_output_path, index=False)
301     print("=" * 80)
302
303     questions_sources_pairs = list(zip(questions, sources))
304
305     print("[INFO] Processing adherence scores with OpenAI and Anthropic...")
306     adherence_results = get_adherence_scores_parallel(
307         clients,
308         questions_sources_pairs,
309         max_workers=2,
310         df=df,
311         valid_indices=valid_indices,
312         csv_path=csv_output_path,
313     )
314
315     for i, idx in enumerate(valid_indices):
316         openai_score, anthropic_score = adherence_results[i]
317         info = comparison_info[i]
318         print(
319             f"[ADHERENCE OpenAI] {info['source_type']} layer{info['layer']} -> {info['llm']}
                ↪  ({info['prompt_type']}): {openai_score}"
320         )
321         print(
322             f"[ADHERENCE Anthropic] {info['source_type']} layer{info['layer']} -> {info['llm']}
                ↪  ({info['prompt_type']}): {anthropic_score}"
323         )
324
325     print(f"\n[INFO] Final results saved to {csv_output_path}")
326     print("=" * 80)
327
328
329 if __name__ == "__main__":
330     process_experiment("exp1a")
```

```
331     process_experiment("exp1b")
332     process_experiment("exp1a_no_source")
```

Listing E.11: Experiment 1 Quantitative Evaluation Notebook: **evaluation1_quan.ipynb**

```
#!/usr/bin/env python
# coding: utf-8

# # Notebook: Quantitative Evaluation -- Experiment 1

# In[ ]:


import pandas as pd
import os
from IPython.display import display
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')

plt.style.use('default')
sns.set_palette("Set2")
plt.rcParams['figure.dpi'] = 300
plt.rcParams['savefig.dpi'] = 300

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', 100)

pd.set_option('display.notebook_repr_html', True)


# In[ ]:


def create_seaborn_boxplot(data, x, y, ax, title, ylabel, xlabel, custom_means=None):
    colors = ['#66c2a5', '#fc8d62', '#8da0cb', '#e78ac3', '#a6d854', '#ffd92f', '#e5c494',
    ↪  '#b3b3b3']
    unique_vals = sorted(data[x].unique())
    palette = colors[:len(unique_vals)]

    sns.boxplot(data=data, x=x, y=y, ax=ax, palette=palette,
                medianprops={'color': 'black', 'linewidth': 2.5, 'linestyle': ':'})

    plot_labels = [tick.get_text() for tick in ax.get_xticklabels()]

    if custom_means is not None:
        # Use custom means mapping them to the correct plot positions
```

```
        for i, plot_label in enumerate(plot_labels):
            reverse_mapping = {
                'Anthropic': 'anthropic',
                'OpenAI': 'openai',
                'Google': 'google',
                'DeepSeek': 'deepseek',
                'Common': 'common',
                'Complex': 'complex',
                'Script': 'script',
                'Transcript': 'transcript',
                'Tanenbaum': 'tanenbaum',
                'Script (Manipulated)': 'script_manipulated',
                'No Source': 'no_source',
                'Exp 1a': '1a',
                'Exp 1b': '1b'
            }
            original_category = reverse_mapping.get(plot_label, plot_label.lower())

            if original_category in custom_means.index:
                mean_val = custom_means[original_category]
                ax.scatter(i, mean_val, color='red', marker='D', s=50, zorder=3,
                           edgecolor='darkred', linewidth=1)
    else:
        # Use original behavior if no custom means provided
        for i, val in enumerate(unique_vals):
            mean_val = data[data[x] == val][y].mean()
            ax.scatter(i, mean_val, color='red', marker='D', s=50, zorder=3,
                       edgecolor='darkred', linewidth=1)

    label_mapping = {
        'anthropic': 'Anthropic',
        'openai': 'OpenAI',
        'google': 'Google',
        'deepseek': 'DeepSeek',
        'common': 'Common',
        'complex': 'Complex',
        'script': 'Script',
        'transcript': 'Transcript',
        'tanenbaum': 'Tanenbaum',
        'script_manipulated': 'Script (Manipulated)',
        'no_source': 'No Source',
        '1a': 'Exp 1a',
        '1b': 'Exp 1b'
    }

    current_labels = [tick.get_text() for tick in ax.get_xticklabels()]
    new_labels = [label_mapping.get(label, label) for label in current_labels]
    ax.set_xticklabels(new_labels, rotation=0)
```

```
93
94      if 'cosine_similarity' in y.lower():
95          ylabel = 'Cosine Similarity'
96          if '(-1 to 1)' not in title:
97              title = title.replace('Cosine Similarity', 'Cosine Similarity (-1 to 1)')
98      elif 'adherence_score' in y.lower():
99          ylabel = 'Adherence Score'
100         if '(0 to 1)' not in title:
101             title = title.replace('Adherence Score', 'Adherence Score (0 to 1)')
102
103     if x.lower() == 'llm':
104         xlabel = 'LLM'
105     elif x.lower() == 'prompt_type':
106         xlabel = 'Prompt Type'
107     elif x.lower() == 'input_source':
108         xlabel = 'Input Source'
109     elif x.lower() == 'comparison_source':
110         xlabel = 'Comparison Source'
111     elif x.lower() == 'layer':
112         xlabel = 'Layer'
113     elif x.lower() == 'experiment':
114         xlabel = 'Experiment'
115
116     ax.set_title(title, fontsize=12, fontweight='bold')
117     ax.set_ylabel(ylabel, fontsize=11, labelpad=15)
118     ax.set_xlabel(xlabel, fontsize=11, labelpad=10)
119     ax.grid(True, alpha=0.3)
120
121
122 # In[ ]:
123
124
125 BASE_PROJECT_PATH = os.path.abspath(os.path.join(os.getcwd(), "..", ".."))
126 INPUT_PATH = os.path.join(BASE_PROJECT_PATH, "20_experiments/60_analyses/csv/quantitative/exp1")
127 OUTPUT_PATH = os.path.join(BASE_PROJECT_PATH, "40_evaluation/exp1/quantitative/")
128
129 exp1a_path = os.path.join(INPUT_PATH, "exp1a.csv")
130 exp1b_path = os.path.join(INPUT_PATH, "exp1b.csv")
131 exp1a_no_source_path = os.path.join(INPUT_PATH, "exp1a_no_source.csv")
132
133 output_dir_exp1a = os.path.join(OUTPUT_PATH, "tables/exp1a")
134 output_dir_exp1b = os.path.join(OUTPUT_PATH, "tables/exp1b")
135 output_plot_dir_exp1a = os.path.join(OUTPUT_PATH, "plots/exp1a")
136 output_plot_dir_exp1b = os.path.join(OUTPUT_PATH, "plots/exp1b")
137
138 tables = {}
139 plots = {}
140
```

```
for path in [output_dir_exp1a, output_dir_exp1b, output_plot_dir_exp1a, output_plot_dir_exp1b]:
    os.makedirs(path, exist_ok=True)

output_dir_exp1a_no_source = os.path.join(OUTPUT_PATH, "tables/exp1a_no_source")
output_plot_dir_exp1a_no_source = os.path.join(OUTPUT_PATH, "plots/exp1a_no_source")

for path in [output_dir_exp1a_no_source, output_plot_dir_exp1a_no_source]:
    os.makedirs(path, exist_ok=True)


# In[ ]:


if not os.path.exists(exp1a_path) or not os.path.exists(exp1b_path):
    print("Error: Quantitative data files not found.")
    print(f"Looked for {exp1a_path} and {exp1b_path}")
else:
    exp1a_df = pd.read_csv(exp1a_path)
    exp1b_df = pd.read_csv(exp1b_path)

    # Load exp1a_no_source if available
    exp1a_no_source_df = None
    if os.path.exists(exp1a_no_source_path):
        exp1a_no_source_df = pd.read_csv(exp1a_no_source_path)
        if 'comparison_source' in exp1a_no_source_df.columns:
            exp1a_no_source_df["adherence_score"] = exp1a_no_source_df[
                ["adherence_score_openai", "adherence_score_anthropic"]
            ].mean(axis=1)
            exp1a_no_source_df = exp1a_no_source_df.drop(
                columns=["adherence_score_openai", "adherence_score_anthropic"]
            )
        else:
            exp1a_no_source_df["adherence_score"] = exp1a_no_source_df[
                ["adherence_score_openai", "adherence_score_anthropic"]
            ].mean(axis=1)
            exp1a_no_source_df = exp1a_no_source_df.drop(
                columns=["adherence_score_openai", "adherence_score_anthropic"]
            )

        print("Data loaded and preprocessed for exp1a, exp1b, and exp1a_no_source.")
    else:
        print("Warning: exp1a_no_source data not found. Some plots may be skipped.")
        print("Data loaded and preprocessed for exp1a and exp1b only.")

    exp1a_df["adherence_score"] = exp1a_df[
        ["adherence_score_openai", "adherence_score_anthropic"]
    ].mean(axis=1)
    exp1b_df["adherence_score"] = exp1b_df[
```

```
        ["adherence_score_openai", "adherence_score_anthropic"]
    ].mean(axis=1)

    exp1a_df = exp1a_df.drop(
        columns=["adherence_score_openai", "adherence_score_anthropic"]
    )
    exp1b_df = exp1b_df.drop(
        columns=["adherence_score_openai", "adherence_score_anthropic"]
    )


# ## Experiment 1a

# ### 1.1: Metrics by LLM

# In[ ]:


llm_metrics_1a = exp1a_df.groupby("llm")[["cosine_similarity", "adherence_score"]].agg(
    ["mean", "std", "median", "min", "max", "count"]
)
tables["exp1a_metrics_by_llm"] = llm_metrics_1a
display(llm_metrics_1a)


# ### 1.2: Metrics by Input Source

# In[ ]:


input_source_metrics_1a = exp1a_df.groupby("input_source")[
    ["cosine_similarity", "adherence_score"]
].agg(["mean", "std", "median", "min", "max", "count"])
tables["exp1a_metrics_by_input_source"] = input_source_metrics_1a
display(input_source_metrics_1a)


# ### 1.3: Metrics by Prompt Type

# In[ ]:


prompt_type_metrics_1a = exp1a_df.groupby("prompt_type")[
    ["cosine_similarity", "adherence_score"]
].agg(["mean", "std", "median", "min", "max", "count"])
tables["exp1a_metrics_by_prompt_type"] = prompt_type_metrics_1a
display(prompt_type_metrics_1a)

```

```

# ### 1.5 Analysis how many MCQ / Open-Ended Questions were generated

# In[ ]:


import os
import glob
import re
import pandas as pd

def extract_info_from_path(file_path):
    parts = file_path.split(os.sep)

    for i, part in enumerate(parts):
        if part in ['common_prompt', 'complex_prompt']:
            prompt_type = 'common' if part == 'common_prompt' else 'complex'
            llm = parts[i + 1] if i + 1 < len(parts) else None
            input_source = parts[i + 2] if i + 2 < len(parts) else None

            layer = None
            if i + 3 < len(parts):
                layer_match = re.search(r'layer(\d+)', parts[i + 3])
                layer = int(layer_match.group(1)) if layer_match else None

            return input_source, layer, llm, prompt_type

    return None, None, None, None


# In[ ]:


def get_scores_from_df(df, input_source, layer, llm, prompt_type):
    if not all([df is not None, input_source, layer is not None, llm, prompt_type]):
        return None, None

    matching_rows = df[
        (df['input_source'] == input_source) &
        (df['layer'] == layer) &
        (df['llm'] == llm) &
        (df['prompt_type'] == prompt_type)
    ]

    if not matching_rows.empty:
        return matching_rows['cosine_similarity'].iloc[0], matching_rows['adherence_score'].iloc[0]

    fallback_rows = df[(df['input_source'] == input_source) & (df['layer'] == layer)]
```

```
    if not fallback_rows.empty:
        return fallback_rows['cosine_similarity'].mean(), fallback_rows['adherence_score'].mean()

    return None, None


# In[ ]:


def analyze_free_text_questions():
    base_path = os.path.join(BASE_PROJECT_PATH, "20_experiments/10_exp1")
    question_files = [f for f in glob.glob(os.path.join(base_path, "**/*_question.txt"),
    ↪  recursive=True)
                      if not ("complex_prompt" in f and "no_source" in f)]

    free_text_data = []
    total_counts = {"exp1a": 0, "exp1b": 0}

    for file_path in question_files:
        if "run_a_content" in file_path:
            total_counts["exp1a"] += 1
        elif "run_b_error" in file_path:
            total_counts["exp1b"] += 1

        try:
            with open(file_path, 'r', encoding='utf-8') as f:
                content = f.read()

            if "Antwort:" in content and "Antwortmöglichkeiten:" not in content:
                input_source, layer, llm, prompt_type = extract_info_from_path(file_path)

                if "run_a_content" in file_path:
                    df, experiment = exp1a_df, "Exp1a"
                elif "run_b_error" in file_path:
                    df, experiment = exp1b_df, "Exp1b"
                    if input_source == 'script':
                        input_source = 'script_manipulated'
                else:
                    continue

                cosine_sim, adherence_score = get_scores_from_df(df, input_source, layer, llm,
                ↪  prompt_type)

                free_text_data.append({
                    'path': file_path,
                    'experiment': experiment,
                    'input_source': input_source,
                    'layer': layer,
```

```
                    'llm': llm,
                    'prompt_type': prompt_type,
                    'cosine_similarity': cosine_sim,
                    'adherence_score': adherence_score
                })
        except Exception as e:
            print(f"Error processing {file_path}: {e}")

    return free_text_data, total_counts


# In[ ]:


free_text_data, total_counts = analyze_free_text_questions()
free_text_df = pd.DataFrame(free_text_data)

exp_counts = free_text_df['experiment'].value_counts()
exp1a_count = exp_counts.get("Exp1a", 0)
exp1b_count = exp_counts.get("Exp1b", 0)

print("FREE TEXT QUESTIONS ANALYSIS")
print("=" * 80)
print(f"Free Text Questions:")
print(f"  Exp1a: {exp1a_count} | Exp1b: {exp1b_count} | Total: {len(free_text_data)}")
print(f"Total Questions (All Types):")
print(f"  Exp1a: {total_counts['exp1a']} | Exp1b: {total_counts['exp1b']} | Total:
↪ {sum(total_counts.values())}")
print(f"Free Text Percentages:")
if total_counts['exp1a'] > 0:
    print(f"  Exp1a: {100*exp1a_count/total_counts['exp1a']:.1f}%")
if total_counts['exp1b'] > 0:
    print(f"  Exp1b: {100*exp1b_count/total_counts['exp1b']:.1f}%")
if sum(total_counts.values()) > 0:
    print(f"  Overall: {100*len(free_text_data)/sum(total_counts.values()):.1f}%")

print(f"\n" + "=" * 80)
print("INDIVIDUAL FREE TEXT QUESTIONS:")
print("=" * 80)


# In[ ]:


for i, item in enumerate(sorted(free_text_data, key=lambda x: (x['experiment'], x['path'])), 1):
    print(f"\n{i}. {item['experiment']} - {item['input_source']} | Layer {item['layer']} |
    ↪ {item['llm']} | {item['prompt_type']}")
    print(f"   Path: {item['path'].replace(BASE_PROJECT_PATH, '...')}")
```

```
    if item['cosine_similarity'] is not None and item['adherence_score'] is not None:
        print(f"   Scores: Cosine={item['cosine_similarity']:.3f},
        ↪  Adherence={item['adherence_score']:.3f}")
    else:
        print(f"   Scores: Not found")

valid_data = free_text_df.dropna(subset=['cosine_similarity', 'adherence_score'])

if not valid_data.empty:
    summary_stats = valid_data.groupby('experiment')[['cosine_similarity',
    ↪  'adherence_score']].agg(['mean', 'std', 'count']).round(4)
    simple_summary = valid_data.groupby('experiment')[['cosine_similarity',
    ↪  'adherence_score']].mean().round(4)
    simple_summary['n_questions'] = valid_data.groupby('experiment').size()

    print(f"\nDetailed Statistics:")
    display(summary_stats)

    overall_stats = {
        'Mean Cosine Similarity': valid_data['cosine_similarity'].mean(),
        'Mean Adherence Score': valid_data['adherence_score'].mean(),
        'Total Free Text Questions': len(valid_data)
    }

    print(f"\nOverall Statistics:")
    for key, value in overall_stats.items():
        print(f"  {key}: {value:.4f}" if isinstance(value, float) else f"  {key}: {value}")

    tables["free_text_summary_by_experiment"] = simple_summary
    tables["free_text_detailed_stats"] = summary_stats
else:
    print("No valid free text questions with scores found!")


# ### 1.5: Cosine Similarity: LLM vs Input Source

# In[ ]:


llm_vs_input_source_cos_sim_1a = pd.pivot_table(
    exp1a_df,
    values="cosine_similarity",
    index="llm",
    columns="input_source",
    aggfunc=["mean", "std", "median", "min", "max", "count"],
)
tables["exp1a_cosine_sim_llm_vs_input_source"] = llm_vs_input_source_cos_sim_1a
display(llm_vs_input_source_cos_sim_1a)
```

```
422
423
424 # In[ ]:
425
426
427 def create_annotation_matrix(mean_df, std_df):
428     annot = mean_df.copy()
429     for i in range(len(mean_df.index)):
430         for j in range(len(mean_df.columns)):
431             mean_val = mean_df.iloc[i, j]
432             std_val = std_df.iloc[i, j]
433             if pd.notna(mean_val) and pd.notna(std_val):
434                 annot.iloc[i, j] = f"{mean_val:.2f}\n({std_val:.2f})"
435             elif pd.notna(mean_val):
436                 annot.iloc[i, j] = f"{mean_val:.2f}"
437     return annot
438
439
440 # In[ ]:
441
442
443 def apply_label_mappings(df):
444     mappings = {
445         'anthropic': 'Anthropic', 'openai': 'OpenAI', 'google': 'Google', 'deepseek': 'DeepSeek',
446         'script': 'Script', 'transcript': 'Transcript', 'tanenbaum': 'Tanenbaum',
447         'script_manipulated': 'Script (Manipulated)', 'common': 'Common', 'complex': 'Complex'
448     }
449
450     if hasattr(df, 'index'):
451         df.index = df.index.map(lambda x: mappings.get(x, x))
452     if hasattr(df, 'columns'):
453         df.columns = df.columns.map(lambda x: mappings.get(x, x))
454     return df
455
456
457 # In[ ]:
458
459
460 def create_dual_heatmap(cosine_pivot, adherence_pivot, title_base, exp_label):
461     fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))
462
463     cos_mean = cosine_pivot["mean"].copy()
464     cos_std = cosine_pivot["std"].copy()
465     cos_annot = create_annotation_matrix(cos_mean, cos_std)
466
467     cos_mean = apply_label_mappings(cos_mean)
468     cos_annot = apply_label_mappings(cos_annot)
469
```

```
    sns.heatmap(cos_mean, annot=cos_annot, fmt='', cmap='RdYlBu_r',
                center=cos_mean.mean().mean(), square=True, linewidths=0.5, linecolor='white',
                cbar_kws={'shrink': 0.8, 'label': 'Cosine Similarity'},
                annot_kws={'size': 10, 'weight': 'bold'}, ax=ax1)
    ax1.set_title(f"Cosine Similarity (-1 to 1): {title_base} ({exp_label})\nValues: Mean (Std)",
                  fontsize=12, fontweight='bold', pad=15)

    adh_mean = adherence_pivot["mean"].copy()
    adh_std = adherence_pivot["std"].copy()
    adh_annot = create_annotation_matrix(adh_mean, adh_std)

    adh_mean = apply_label_mappings(adh_mean)
    adh_annot = apply_label_mappings(adh_annot)

    sns.heatmap(adh_mean, annot=adh_annot, fmt='', cmap='RdYlBu_r',
                center=adh_mean.mean().mean(), square=True, linewidths=0.5, linecolor='white',
                cbar_kws={'shrink': 0.8, 'label': 'Adherence Score'},
                annot_kws={'size': 10, 'weight': 'bold'}, ax=ax2)
    ax2.set_title(f"Adherence Score (0 to 1): {title_base} ({exp_label})\nValues: Mean (Std)",
                  fontsize=12, fontweight='bold', pad=15)

    # Set labels
    for ax in [ax1, ax2]:
        ax.set_ylabel("LLM", fontsize=11, labelpad=15)
        if "Input Source" in title_base:
            ax.set_xlabel("Input Source", fontsize=11, labelpad=10)
        elif "Prompt Type" in title_base:
            ax.set_xlabel("Prompt Type", fontsize=11, labelpad=10)

    plt.tight_layout()
    plt.show()
    return fig

adherence_pivot = pd.pivot_table(exp1a_df, values="adherence_score", index="llm",
                                 columns="input_source", aggfunc=["mean", "std", "median", "min",
                                 ↪  "max", "count"])

fig = create_dual_heatmap(llm_vs_input_source_cos_sim_1a, adherence_pivot,
                          "LLM vs Input Source", "Exp 1a")
plots['exp1a_combined_llm_vs_input_source'] = fig


# ### 1.6: Cosine Similarity: LLM vs Prompt Type

# In[ ]:


llm_vs_prompt_type_cos_sim_1a = pd.pivot_table(
```

```
    exp1a_df,
    values="cosine_similarity",
    index="llm",
    columns="prompt_type",
    aggfunc=["mean", "std", "median", "min", "max", "count"],
)
tables["exp1a_cosine_sim_llm_vs_prompt_type"] = llm_vs_prompt_type_cos_sim_1a
display(llm_vs_prompt_type_cos_sim_1a)


# In[ ]:


adherence_pivot_prompt = pd.pivot_table(exp1a_df, values="adherence_score", index="llm",
                                        columns="prompt_type", aggfunc=["mean", "std", "median",
                                        ↪  "min", "max", "count"])

fig = create_dual_heatmap(llm_vs_prompt_type_cos_sim_1a, adherence_pivot_prompt,
                          "LLM vs Prompt Type", "Exp 1a")
plots['exp1a_combined_llm_vs_prompt_type'] = fig


# ### 1.7: Adherence Score: LLM vs Prompt Type

# In[ ]:


llm_vs_prompt_type_adherence_1a = pd.pivot_table(
    exp1a_df,
    values="adherence_score",
    index="llm",
    columns="prompt_type",
    aggfunc=["mean", "std", "median", "min", "max", "count"],
)
tables["exp1a_adherence_llm_vs_prompt_type"] = llm_vs_prompt_type_adherence_1a
display(llm_vs_prompt_type_adherence_1a)


# ### 1.8: Adherence Score: LLM vs Input Source

# In[ ]:


llm_vs_input_source_adherence_1a = pd.pivot_table(
    exp1a_df,
    values="adherence_score",
    index="llm",
    columns="input_source",
```

```
    aggfunc=["mean", "std", "median", "min", "max", "count"],
)
tables["exp1a_adherence_llm_vs_input_source"] = llm_vs_input_source_adherence_1a
display(llm_vs_input_source_adherence_1a)


# ### 1.9: Distribution Plots

# In[ ]:


fig, ax = plt.subplots(1, 2, figsize=(15, 7))
plots['exp1a_dist_by_llm'] = fig

create_seaborn_boxplot(exp1a_df, 'llm', 'cosine_similarity', ax[0],
                       'Cosine Similarity (-1 to 1) Distribution by LLM (Exp 1a)',
                       'Cosine Similarity', 'LLM')

create_seaborn_boxplot(exp1a_df, 'llm', 'adherence_score', ax[1],
                       'Adherence Score (0 to 1) Distribution by LLM (Exp 1a)',
                       'Adherence Score', 'LLM')

plt.tight_layout()
plt.show()


# In[ ]:


# OPTIONAL: Boxplot for cosine_similarity and adherence_score by input_source (no source coupling
↪  is another plot)
fig, ax = plt.subplots(1, 2, figsize=(15, 7))
plots['exp1a_dist_by_input_source'] = fig

create_seaborn_boxplot(exp1a_df, 'input_source', 'cosine_similarity', ax[0],
                       'Cosine Similarity (-1 to 1) Distribution by Input Source (Exp 1a)',
                       'Cosine Similarity', 'Input Source')

create_seaborn_boxplot(exp1a_df, 'input_source', 'adherence_score', ax[1],
                       'Adherence Score (0 to 1) Distribution by Input Source (Exp 1a)',
                       'Adherence Score', 'Input Source')

plt.tight_layout()
plt.show()


# In[ ]:

```

```

fig = create_dual_heatmap(llm_vs_input_source_cos_sim_1a,
                          pd.pivot_table(exp1a_df, values="adherence_score", index="llm",
                                         columns="input_source", aggfunc=["mean", "std", "median",
                                         ↪  "min", "max", "count"]),
                          "LLM vs Input Source", "Exp 1a")


# ## Experiment 1b

# ### 2.1: Metrics by LLM

# In[ ]:


llm_metrics_1b = exp1b_df.groupby("llm")[["cosine_similarity", "adherence_score"]].agg(
    ["mean", "std", "median", "min", "max", "count"]
)
tables["exp1b_metrics_by_llm"] = llm_metrics_1b
display(llm_metrics_1b)


# In[ ]:


fig, ax = plt.subplots(1, 2, figsize=(15, 7))
plots['exp1b_dist_by_llm'] = fig

create_seaborn_boxplot(exp1b_df, 'llm', 'cosine_similarity', ax[0],
                       'Cosine Similarity (-1 to 1) Distribution by LLM (Exp 1b)',
                       'Cosine Similarity', 'LLM')

create_seaborn_boxplot(exp1b_df, 'llm', 'adherence_score', ax[1],
                       'Adherence Score (0 to 1) Distribution by LLM (Exp 1b)',
                       'Adherence Score', 'LLM')

plt.tight_layout()
plt.show()


# ### 2.6: Cosine Similarity & Adherence: LLM vs Prompt Type

# In[ ]:


llm_vs_prompt_type_cos_sim_1b = pd.pivot_table(
    exp1b_df,
    values="cosine_similarity",
```

```
    index="llm",
    columns="prompt_type",
    aggfunc=["mean", "std", "median", "min", "max", "count"],
)
tables["exp1b_cosine_sim_llm_vs_prompt_type"] = llm_vs_prompt_type_cos_sim_1b
display(llm_vs_prompt_type_cos_sim_1b)


# In[ ]:


llm_vs_prompt_type_adherence_1b = pd.pivot_table(
    exp1b_df,
    values="adherence_score",
    index="llm",
    columns="prompt_type",
    aggfunc=["mean", "std", "median", "min", "max", "count"],
)
tables["exp1b_adherence_llm_vs_prompt_type"] = llm_vs_prompt_type_adherence_1b
display(llm_vs_prompt_type_adherence_1b)


# In[ ]:


adherence_pivot_1b = pd.pivot_table(exp1b_df, values="adherence_score", index="llm",
                                    columns="prompt_type", aggfunc=["mean", "std", "median", "min",
                                    ↪  "max", "count"])

fig = create_dual_heatmap(llm_vs_prompt_type_cos_sim_1b, adherence_pivot_1b,
                          "LLM vs Prompt Type", "Exp 1b")
plots['exp1b_combined_llm_vs_prompt_type'] = fig


# ### Including a Run without a Source

# In[ ]:


if exp1a_no_source_df is not None:
    print("Creating plots for exp1a_no_source data...")

    has_comparison_source = 'comparison_source' in exp1a_no_source_df.columns

    if has_comparison_source:
        print(f"Data shape: {exp1a_no_source_df.shape}")
        print(f"Comparison sources: {exp1a_no_source_df['comparison_source'].unique()}")

```

```
        fig, ax = plt.subplots(1, 2, figsize=(15, 7))
        plots['exp1a_no_source_dist_by_llm'] = fig

        create_seaborn_boxplot(exp1a_no_source_df, 'llm', 'cosine_similarity', ax[0],
                               'Cosine Similarity (-1 to 1) Distribution by LLM (Exp 1a No Source)',
                               'Cosine Similarity', 'LLM')

        create_seaborn_boxplot(exp1a_no_source_df, 'llm', 'adherence_score', ax[1],
                               'Adherence Score (0 to 1) Distribution by LLM (Exp 1a No Source)',
                               'Adherence Score', 'LLM')

        plt.tight_layout()
        plt.show()


        fig, ax = plt.subplots(1, 2, figsize=(15, 7))
        plots['exp1a_no_source_dist_by_comparison_source'] = fig

        create_seaborn_boxplot(exp1a_no_source_df, 'comparison_source', 'cosine_similarity', ax[0],
                               'Cosine Similarity (-1 to 1) by Comparison Source (Exp 1a No
                               ↪  Source)',
                               'Cosine Similarity', 'Comparison Source')

        create_seaborn_boxplot(exp1a_no_source_df, 'comparison_source', 'adherence_score', ax[1],
                               'Adherence Score (0 to 1) by Comparison Source (Exp 1a No Source)',
                               'Adherence Score', 'Comparison Source')

        plt.tight_layout()
        plt.show()

    print("exp1a_no_source plots created successfully!")
else:
    print("Skipping exp1a_no_source plots - data not available.")


# In[ ]:


# Combined Input Source vs No Source Comparison
if exp1a_no_source_df is not None:
    print("Creating combined Input Source vs No Source comparison plot...")

    exp1a_regular = exp1a_df.copy()
    exp1a_no_source_agg = exp1a_no_source_df.copy()
    exp1a_no_source_agg['input_source'] = 'no_source'

    comparison_df = pd.concat([exp1a_regular, exp1a_no_source_agg], ignore_index=True)

```

```
    fig, ax = plt.subplots(1, 2, figsize=(16, 7))
    plots['exp1a_input_source_vs_no_source_comparison'] = fig

    comparison_means = comparison_df.groupby('input_source')[['cosine_similarity',
    ↪  'adherence_score']].mean()

    create_seaborn_boxplot(comparison_df, 'input_source', 'cosine_similarity', ax[0],
                           'Cosine Similarity: Input Sources vs No Source (Exp 1a)',
                           'Cosine Similarity', 'Input Source',
                           custom_means=comparison_means['cosine_similarity'])

    create_seaborn_boxplot(comparison_df, 'input_source', 'adherence_score', ax[1],
                           'Adherence Score: Input Sources vs No Source (Exp 1a)',
                           'Adherence Score', 'Input Source',
                           custom_means=comparison_means['adherence_score'])

    plt.tight_layout()
    plt.show()

    comparison_summary = comparison_df.groupby('input_source')[['cosine_similarity',
    ↪  'adherence_score']].agg(['mean', 'std', 'median', 'min', 'max', 'count'])
    tables['exp1a_input_source_vs_no_source_summary'] = comparison_summary

    print("\nSummary Statistics:")
    display(comparison_summary)

    print("\nInput Source vs No Source comparison created successfully!")
else:
    print("Skipping Input Source vs No Source comparison - exp1a_no_source data not available.")


# ## Save all tables and plots

# In[ ]:


def save_all_tables_and_plots():
    for name, df in tables.items():
        if name.startswith("exp1a_no_source"):
            output_dir = output_dir_exp1a_no_source
            table_name = name.replace("exp1a_no_source_", "")
        elif name.startswith("exp1a"):
            output_dir = output_dir_exp1a
            table_name = name.replace("exp1a_", "")
        elif name.startswith("exp1b"):
            output_dir = output_dir_exp1b
            table_name = name.replace("exp1b_", "")
        else:
```

```
            if "exp1" in name:
                base_filename = os.path.join(output_dir_exp1a, name)
                csv_path = f"{base_filename}.csv"
                df.to_csv(csv_path)
                print(f"Saved {csv_path}")
            continue

        base_filename = os.path.join(output_dir, table_name)

        csv_path = f"{base_filename}.csv"
        df.to_csv(csv_path)
        print(f"Saved {csv_path}")

    for name, fig in plots.items():
        if name.startswith("exp1a_no_source") or "no_source" in name:
            output_dir = output_plot_dir_exp1a_no_source
            plot_name = name.replace("exp1a_no_source_", "").replace("exp1a_", "")
        elif name.startswith("exp1a"):
            output_dir = output_plot_dir_exp1a
            plot_name = name.replace("exp1a_", "")
        elif name.startswith("exp1b"):
            output_dir = output_plot_dir_exp1b
            plot_name = name.replace("exp1b_", "")
        else:
            if "exp1" in name:
                base_filename = os.path.join(output_plot_dir_exp1a, name)
                png_path = f"{base_filename}.png"
                fig.savefig(png_path, bbox_inches='tight')
                print(f"Saved {png_path}")
            continue

        base_filename = os.path.join(output_dir, plot_name)

        png_path = f"{base_filename}.png"
        fig.savefig(png_path, bbox_inches='tight')
        print(f"Saved {png_path}")

save_all_tables_and_plots()

```

Listing E.12: Experiment 1 Qualitative Evaluation Notebook: **evaluation1_qual.ipynb**

```
#!/usr/bin/env python
# coding: utf-8

# # Notebook: Qualitative Evaluation (Supervisor vs. Staff Members) -- Experiment 1

# In this notebook, the qualitative evaluation of the first experiment is conducted, comparing the ratings of supervisors and staff members.
#
# The first evaluation run in this notebook is based on the data from the supervisor (expert 1), while `USE_SUPERVISOR_DATA` is set to `True`.
#
# If set to `False`, the notebook will use the data from the staff members (experts 2-5) instead as the second run. Furthermore, if set to `False`, the inter-rater agreement (IRA) will be calculated between the staff members + between the staff members and the supervisor.

# ## Initial Setup

# In[ ]:


import pandas as pd
import numpy as np
import os
import matplotlib.pyplot as plt
import seaborn as sns
from IPython.display import display
import warnings
from statsmodels.stats.inter_rater import fleiss_kappa

warnings.filterwarnings('ignore')

# Plot configuration
plt.style.use('default')
sns.set_palette("Set2")
plt.rcParams['figure.dpi'] = 300
plt.rcParams['savefig.dpi'] = 300

# Display configuration
pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', 100)
pd.set_option('display.notebook_repr_html', True)

# !!! Set to False to use staff data instead (Experts 2-5, + inter-rater agreements)
```

```
# (Then, IRA between staff experts 2-5 + also staff experts vs. supervisor [expert 1] will be calculated)
USE_SUPERVISOR_DATA = False
# Changing this will affect the data resulting in the notebook
# !!! As a very important note

# Path configuration
BASE_PROJECT_PATH = os.path.abspath(os.path.join(os.getcwd(), "..", ".."))
expert_base_path = os.path.join(BASE_PROJECT_PATH, "20_experiments/60_analyses/csv/qualitative/exp1/experts")
hint_base_path = os.path.join(BASE_PROJECT_PATH, "20_experiments/60_analyses/csv/qualitative/exp1/hints")
output_base_path = os.path.join(BASE_PROJECT_PATH, "40_evaluation/exp1/qualitative")
output_tables_path = os.path.join(output_base_path, "tables")
output_plots_path = os.path.join(output_base_path, "plots")

for path in [output_tables_path, output_plots_path]:
    os.makedirs(path, exist_ok=True)

# Storage for results
tables = {}
plots = {}

print("Setup completed successfully")
print(f"Output tables: {output_tables_path}")
print(f"Output plots: {output_plots_path}")


# In[ ]:


# Utility Functions

def create_seaborn_boxplot(data, x, y, ax, title, ylabel, xlabel, scale_range=None):
    colors = ['#66c2a5', '#fc8d62', '#8da0cb', '#e78ac3', '#a6d854', '#ffd92f', '#e5c494', '#b3b3b3']
    unique_vals = sorted(data[x].unique())
    palette = colors[:len(unique_vals)]

    sns.boxplot(data=data, x=x, y=y, ax=ax, palette=palette,
                medianprops={'color': 'black', 'linewidth': 2.5, 'linestyle': ':'})

    for i, val in enumerate(unique_vals):
        mean_val = data[data[x] == val][y].mean()
        ax.scatter(i, mean_val, color='red', marker='D', s=50, zorder=3,
                   edgecolor='darkred', linewidth=1)

    label_mapping = {
```

```
        'anthropic': 'Anthropic', 'openai': 'OpenAI', 'google': 'Google', 'deepseek': 'DeepSeek',
        'common': 'Common', 'complex': 'Complex', 'script': 'Script', 'transcript': 'Transcript',
        'tanenbaum': 'Tanenbaum', 'script_manipulated': 'Script (Manipulated)'
    }

    current_labels = [tick.get_text() for tick in ax.get_xticklabels()]
    new_labels = [label_mapping.get(label, label) for label in current_labels]
    ax.set_xticklabels(new_labels, rotation=0)

    if scale_range:
        ax.set_ylim(scale_range)
        if scale_range == (0, 10):
            title += " (0-10 scale)"
        elif scale_range == (0, 70):
            title += " (0-70 scale)"

    ax.set_title(title, fontsize=12, fontweight='bold')
    ax.set_ylabel(ylabel, fontsize=11, labelpad=15)
    ax.set_xlabel(xlabel, fontsize=11, labelpad=10)
    ax.grid(True, alpha=0.3)


# In[ ]:


def load_supervisor_data():
    hint_exp1a = pd.read_csv(os.path.join(hint_base_path, "exp1a_hints.csv"))
    hint_exp1b = pd.read_csv(os.path.join(hint_base_path, "exp1b_hints.csv"))

    expert1_exp1a = pd.read_csv(os.path.join(expert_base_path, "expert_1", "exp1a.csv"))
    expert1_exp1b = pd.read_csv(os.path.join(expert_base_path, "expert_1", "exp1b.csv"))

    numeric_cols_1a_temp = ['relevance', 'clarity', 'answerability', 'challenging', 'value',
    ↪  'language', 'correctness']
    numeric_cols_1b_temp = ['relevance', 'clarity', 'answerability', 'challenging', 'value',
    ↪  'language', 'manipulation_handling']

    # Clean Expert 1 data before concatenation
    for col in numeric_cols_1a_temp:
        if col in expert1_exp1a.columns:
            expert1_exp1a[col] = expert1_exp1a[col].astype(str)
            invalid_values = ['??', '???', '????', '?', '', ' ', 'nan', 'NaN', 'NULL', 'null',
                              'None', 'NONE', 'n/a', 'N/A', '#N/A', '#NULL!',
                              'undefined', 'UNDEFINED', '-', '--', '---']
            expert1_exp1a[col] = expert1_exp1a[col].replace(invalid_values, np.nan)

    for col in numeric_cols_1b_temp:
        if col in expert1_exp1b.columns:
```

```
            expert1_exp1b[col] = expert1_exp1b[col].astype(str)
            invalid_values = ['??', '???', '????', '?', '', ' ', 'nan', 'NaN', 'NULL', 'null',
                              'None', 'NONE', 'n/a', 'N/A', '#N/A', '#NULL!',
                              'undefined', 'UNDEFINED', '-', '--', '---']
            expert1_exp1b[col] = expert1_exp1b[col].replace(invalid_values, np.nan)

    exp1a_df = pd.concat([expert1_exp1a, hint_exp1a[['llm', 'prompt_type']]], axis=1)
    exp1b_df = pd.concat([expert1_exp1b, hint_exp1b[['llm', 'prompt_type']]], axis=1)

    exp1a_df['experiment'] = 'exp1a'
    exp1b_df['experiment'] = 'exp1b'
    exp1b_df['input_source'] = exp1b_df['input_source'].replace('script', 'script_manipulated')

    return exp1a_df, exp1b_df


# In[ ]:


def load_staff_data():
    hint_exp1a = pd.read_csv(os.path.join(hint_base_path, "exp1a_hints.csv"))
    hint_exp1b = pd.read_csv(os.path.join(hint_base_path, "exp1b_hints.csv"))

    experts_data = {}
    for expert in [2, 3, 4, 5]:
        expert_key = f'expert_{expert}'

        exp1a_path = os.path.join(expert_base_path, expert_key, "exp1a.csv")
        exp1b_path = os.path.join(expert_base_path, expert_key, "exp1b.csv")

        if expert == 4 or expert == 2:
            exp1a_data = pd.read_csv(exp1a_path, sep=';', quotechar='"')
            exp1b_data = pd.read_csv(exp1b_path, sep=';', quotechar='"')
        else:
            exp1a_data = pd.read_csv(exp1a_path, quotechar='"', escapechar='\\')
            exp1b_data = pd.read_csv(exp1b_path, quotechar='"', escapechar='\\')

        experts_data[expert_key] = {'exp1a': exp1a_data, 'exp1b': exp1b_data}
        experts_data[expert_key]['exp1a']['expert'] = expert_key
        experts_data[expert_key]['exp1b']['expert'] = expert_key

    exp1a_combined = pd.concat([data['exp1a'] for data in experts_data.values()],
    ↪  ignore_index=True)
    exp1b_combined = pd.concat([data['exp1b'] for data in experts_data.values()],
    ↪  ignore_index=True)

    hint_multiplier = len(experts_data)
```

```
    exp1a_hints_extended = pd.concat([hint_exp1a[['llm', 'prompt_type']]] * hint_multiplier, ignore_index=True)
    exp1b_hints_extended = pd.concat([hint_exp1b[['llm', 'prompt_type']]] * hint_multiplier, ignore_index=True)

    exp1a_df = exp1a_combined.copy()
    exp1b_df = exp1b_combined.copy()

    for col in ['llm', 'prompt_type']:
        if col not in exp1a_df.columns:
            exp1a_df[col] = exp1a_hints_extended[col].values
        if col not in exp1b_df.columns:
            exp1b_df[col] = exp1b_hints_extended[col].values

    return exp1a_df, exp1b_df, experts_data


# In[ ]:


def clean_numeric_data(df, numeric_cols):
    for col in numeric_cols:
        if col in df.columns:
            df[col] = df[col].astype(str)
            invalid_values = ['??', '???', '?', '????', '', ' ', 'nan', 'NaN', 'NULL', 'null',
                              'None', 'NONE', 'n/a', 'N/A', '#N/A', '#NULL!',
                              'undefined', 'UNDEFINED', '-', '--', '---']

            df[col] = df[col].replace(invalid_values, np.nan)
            df[col] = pd.to_numeric(df[col], errors='coerce')

            # Validate range (0-10 for rating scales)
            valid_data = df[col].dropna()
            if len(valid_data) > 0:
                out_of_range = valid_data[(valid_data < 0) | (valid_data > 10)]
                if len(out_of_range) > 0:
                    df.loc[(df[col] < 0) | (df[col] > 10), col] = np.nan
    return df


# In[ ]:


def calculate_fleiss_kappa(expert2_df, expert3_df, expert4_df, expert5_df, criteria_cols):
    results = []

    ids2 = set(expert2_df['sample_id'].astype(int))
```

```
    ids3 = set(expert3_df['sample_id'].astype(int))
    ids4 = set(expert4_df['sample_id'].astype(int))
    ids5 = set(expert5_df['sample_id'].astype(int))
    common_ids = sorted(list(ids2 & ids3 & ids4 & ids5))

    for criterion in criteria_cols:
        ratings = []
        for sample_id in common_ids:
            r2 = expert2_df[expert2_df['sample_id'] == sample_id][criterion].iloc[0]
            r3 = expert3_df[expert3_df['sample_id'] == sample_id][criterion].iloc[0]
            r4 = expert4_df[expert4_df['sample_id'] == sample_id][criterion].iloc[0]
            r5 = expert5_df[expert5_df['sample_id'] == sample_id][criterion].iloc[0]

            def safe_convert(val):
                return np.nan if pd.isna(val) else float(val)

            r2_val, r3_val, r4_val, r5_val = map(safe_convert, [r2, r3, r4, r5])

            if all(not pd.isna(r) and 0 <= r <= 10 for r in [r2_val, r3_val, r4_val, r5_val]):
                ratings.append([int(r2_val), int(r3_val), int(r4_val), int(r5_val)])

        if len(ratings) >= 2:
            ratings_array = np.array(ratings)
            fleiss_table = np.zeros((len(ratings), 11))

            for i, item_ratings in enumerate(ratings):
                for rating in item_ratings:
                    fleiss_table[i, int(rating)] += 1

            kappa = fleiss_kappa(fleiss_table)

            level = ("Slight" if kappa < 0.2 else
                    "Fair" if kappa < 0.4 else
                    "Moderate" if kappa < 0.6 else
                    "Substantial" if kappa < 0.8 else
                    "Almost Perfect")

            results.append({
                'Criterion': criterion,
                'Fleiss_Kappa': round(kappa, 3),
                'Agreement_Level': level,
                'N_Items': len(ratings),
                'Mean_Rating': round(np.mean(ratings_array), 2),
                'Std_Rating': round(np.std(ratings_array), 2)
            })

    return pd.DataFrame(results)

```

```
print("Utility functions loaded successfully")


# ## Data Loading and Configuration

# In[ ]:


# Load data based on configuration
if USE_SUPERVISOR_DATA:
    print("Loading SUPERVISOR data (Expert 1 - single rater)")
    exp1a_df, exp1b_df = load_supervisor_data()
    analysis_suffix = "supervisor"
    agreement_available = False
else:
    print("Loading STAFF data (Experts 2-5 - multiple raters with agreement analysis)")
    exp1a_df, exp1b_df, experts_data = load_staff_data()
    analysis_suffix = "staff"
    agreement_available = True

numeric_cols_1a = ['relevance', 'clarity', 'answerability', 'challenging', 'value', 'language',
↪  'correctness']
all_numeric_cols_1b = ['relevance', 'clarity', 'answerability', 'challenging', 'value', 'language',
↪  'manipulation_handling']
numeric_cols_1b = ['manipulation_handling'] # only important column in exp1b

exp1a_df = clean_numeric_data(exp1a_df, numeric_cols_1a)
exp1b_df = clean_numeric_data(exp1b_df, numeric_cols_1b)
exp1a_df['total_score'] = exp1a_df[numeric_cols_1a].sum(axis=1)

exp1a_filled = exp1a_df[numeric_cols_1a].notna().sum().sum()
exp1a_total = len(exp1a_df) * len(numeric_cols_1a)
exp1b_filled = exp1b_df[numeric_cols_1b].notna().sum().sum()
exp1b_total = len(exp1b_df) * len(numeric_cols_1b)

analyze_exp1b = exp1b_filled > 0

print(f"\nData loaded successfully!")
print(f"Experiment 1a: {len(exp1a_df)} samples, {100*exp1a_filled/exp1a_total:.1f}% completion")
print(f"Experiment 1b: {len(exp1b_df)} samples, {100*exp1b_filled/exp1b_total:.1f}% completion")
print(f"Analysis suffix: {analysis_suffix}")

print(f"\nDistribution:")
print(f"  LLMs: {list(exp1a_df['llm'].unique())}")
print(f"  Prompt Types: {list(exp1a_df['prompt_type'].unique())}")
print(f"  Input Sources (1a): {list(exp1a_df['input_source'].unique())}")
if analyze_exp1b:
    print(f"  Input Sources (1b): {list(exp1b_df['input_source'].unique())}")
```

```

# # Experiment 1a: Original Content Analysis
#
# Analysis of LLM question generation quality using original source materials.

# In[ ]:


print("EXPERIMENT 1A - DESCRIPTIVE STATISTICS")
print("="*60)

criteria = numeric_cols_1a + ['total_score']

# Overall statistics
exp1a_stats = exp1a_df[numeric_cols_1a].describe().round(2)
tables[f'exp1a_overall_stats_{analysis_suffix}'] = exp1a_stats
print("\nOverall Statistics:")
display(exp1a_stats)


# In[ ]:


# Statistics by LLM
exp1a_llm_stats = exp1a_df.groupby('llm')[criteria].agg(['mean', 'std', 'median',
↪  'count']).round(2)
tables[f'exp1a_llm_stats_{analysis_suffix}'] = exp1a_llm_stats
print("\nStatistics by LLM:")
display(exp1a_llm_stats)


# In[ ]:


# LLM ranking
exp1a_llm_means = exp1a_df.groupby('llm')[numeric_cols_1a].mean().round(2)
exp1a_llm_overall = exp1a_llm_means.mean(axis=1).sort_values(ascending=False)
tables[f'exp1a_llm_ranking_{analysis_suffix}'] = exp1a_llm_overall

print("\nOverall LLM Ranking:")
for i, (llm, score) in enumerate(exp1a_llm_overall.items(), 1):
    print(f"{i}. {llm.title()}: {score:.2f}")


# In[ ]:


```

```
# Statistics by Input Source
exp1a_source_stats = exp1a_df.groupby('input_source')[criteria].agg(['mean', 'std', 'median', 'count']).round(2)
tables[f'exp1a_source_stats_{analysis_suffix}'] = exp1a_source_stats
print("\nStatistics by Input Source:")
display(exp1a_source_stats)


# In[ ]:


# Statistics by Prompt Type
exp1a_prompt_stats = exp1a_df.groupby('prompt_type')[criteria].agg(['mean', 'std', 'median', 'count']).round(2)
tables[f'exp1a_prompt_stats_{analysis_suffix}'] = exp1a_prompt_stats
print("\nStatistics by Prompt Type:")
display(exp1a_prompt_stats)


# In[ ]:


# LLM Performance Visualization
fig, axes = plt.subplots(4, 2, figsize=(14, 20))
axes = axes.flatten()
plots[f'exp1a_llm_analysis_{analysis_suffix}'] = fig

for i, criterion in enumerate(numeric_cols_1a):
    create_seaborn_boxplot(exp1a_df, 'llm', criterion, axes[i],
                           f'{criterion.title()}', criterion.title(), 'LLM', scale_range=(0, 10))

create_seaborn_boxplot(exp1a_df, 'llm', 'total_score', axes[7],
                       'Total Score', 'Total Score', 'LLM', scale_range=(0, 70))

plt.suptitle('Experiment 1a: LLM Performance across all Criteria', fontsize=16, fontweight='bold', y=0.98)
plt.tight_layout()
plt.subplots_adjust(top=0.95)
plt.show()


# In[ ]:


# Input Source Analysis
fig, axes = plt.subplots(4, 2, figsize=(14, 20))
axes = axes.flatten()
plots[f'exp1a_source_analysis_{analysis_suffix}'] = fig
```

```
409
410 for i, criterion in enumerate(numeric_cols_1a):
411     create_seaborn_boxplot(exp1a_df, 'input_source', criterion, axes[i],
412                            f'{criterion.title()}', criterion.title(), 'Input Source',
                               ↪  scale_range=(0, 10))
413
414 create_seaborn_boxplot(exp1a_df, 'input_source', 'total_score', axes[7],
415                        'Total Score', 'Total Score', 'Input Source', scale_range=(0, 70))
416
417 plt.suptitle('Experiment 1a: Performance by Input Source', fontsize=16, fontweight='bold', y=0.98)
418 plt.tight_layout()
419 plt.subplots_adjust(top=0.95)
420 plt.show()
421
422
423 # In[ ]:
424
425
426 # Prompt Type Analysis
427 fig, axes = plt.subplots(2, 4, figsize=(20, 10))
428 axes = axes.flatten()
429 plots[f'exp1a_prompt_analysis_{analysis_suffix}'] = fig
430
431 for i, criterion in enumerate(numeric_cols_1a):
432     create_seaborn_boxplot(exp1a_df, 'prompt_type', criterion, axes[i],
433                            f'{criterion.title()}', criterion.title(), 'Prompt Type', scale_range=(0,
                               ↪  10))
434
435 create_seaborn_boxplot(exp1a_df, 'prompt_type', 'total_score', axes[7],
436                        'Total Score', 'Total Score', 'Prompt Type', scale_range=(0, 70))
437
438 plt.suptitle('Experiment 1a: Performance by Prompt Type', fontsize=16, fontweight='bold', y=0.98)
439 plt.tight_layout()
440 plt.subplots_adjust(top=0.92)
441 plt.show()
442
443 # Prompt Type Comparison
444 exp1a_prompt_means = exp1a_df.groupby('prompt_type')[numeric_cols_1a].mean().round(2)
445 tables[f'exp1a_prompt_means_{analysis_suffix}'] = exp1a_prompt_means
446
447 if 'complex' in exp1a_prompt_means.index and 'common' in exp1a_prompt_means.index:
448     prompt_diff = exp1a_prompt_means.loc['complex'] - exp1a_prompt_means.loc['common']
449     print("\nComplex vs Common Prompt Difference:")
450     for criterion, diff in prompt_diff.items():
451         direction = "higher" if diff > 0 else "lower"
452         print(f"  {criterion.title()}: {diff:+.2f} ({direction} for complex)")
453
454
```

```
# In[ ]:


# Total Score Heatmaps
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))
plots[f'exp1a_total_score_heatmaps_{analysis_suffix}'] = fig

# Heatmap 1: Total Score by LLM vs Input Source
heatmap_mean = exp1a_df.groupby(['llm', 'input_source'])['total_score'].mean().unstack()
heatmap_std = exp1a_df.groupby(['llm', 'input_source'])['total_score'].std().unstack()

annot_matrix = heatmap_mean.copy()
for i in range(len(heatmap_mean.index)):
    for j in range(len(heatmap_mean.columns)):
        mean_val = heatmap_mean.iloc[i, j]
        std_val = heatmap_std.iloc[i, j]
        if pd.notna(mean_val) and pd.notna(std_val):
            annot_matrix.iloc[i, j] = f"{mean_val:.1f}\n({std_val:.1f})"
        elif pd.notna(mean_val):
            annot_matrix.iloc[i, j] = f"{mean_val:.1f}"

llm_labels = [label.title() for label in heatmap_mean.index]
source_labels = [label.title() for label in heatmap_mean.columns]

sns.heatmap(heatmap_mean, annot=annot_matrix, fmt='', cmap='RdYlBu_r',
            center=heatmap_mean.mean().mean(), square=True, linewidths=0.5,
            cbar_kws={'shrink': 0.8, 'label': 'Total Score'}, annot_kws={'size': 10, 'weight':
            ↪  'bold'},
            xticklabels=source_labels, yticklabels=llm_labels, ax=ax1)

ax1.set_title('Total Score (LLM vs Input Source)\nValues: Mean (Std) | Scale: 0-70 points',
              fontsize=14, fontweight='bold', pad=20)
ax1.set_xlabel('Input Source', fontsize=12, fontweight='bold')
ax1.set_ylabel('LLM', fontsize=12, fontweight='bold')

# Heatmap 2: Total Score by LLM vs Prompt Type
heatmap_mean_prompt = exp1a_df.groupby(['llm', 'prompt_type'])['total_score'].mean().unstack()
heatmap_std_prompt = exp1a_df.groupby(['llm', 'prompt_type'])['total_score'].std().unstack()

annot_matrix_prompt = heatmap_mean_prompt.copy()
for i in range(len(heatmap_mean_prompt.index)):
    for j in range(len(heatmap_mean_prompt.columns)):
        mean_val = heatmap_mean_prompt.iloc[i, j]
        std_val = heatmap_std_prompt.iloc[i, j]
        if pd.notna(mean_val) and pd.notna(std_val):
            annot_matrix_prompt.iloc[i, j] = f"{mean_val:.1f}\n({std_val:.1f})"
        elif pd.notna(mean_val):
            annot_matrix_prompt.iloc[i, j] = f"{mean_val:.1f}"
```

```

llm_labels = [label.title() for label in heatmap_mean_prompt.index]
prompt_labels = [label.title() for label in heatmap_mean_prompt.columns]

sns.heatmap(heatmap_mean_prompt, annot=annot_matrix_prompt, fmt='', cmap='RdYlBu_r',
            center=heatmap_mean_prompt.mean().mean(), square=True, linewidths=0.5,
            cbar_kws={'shrink': 0.8, 'label': 'Total Score'}, annot_kws={'size': 10, 'weight':
            ↪  'bold'},
            xticklabels=prompt_labels, yticklabels=llm_labels, ax=ax2)

ax2.set_title('Total Score (LLM vs Prompt Type)\nValues: Mean (Std) | Scale: 0-70 points',
             fontsize=14, fontweight='bold', pad=20)
ax2.set_xlabel('Prompt Type', fontsize=12, fontweight='bold')
ax2.set_ylabel('LLM', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()


# In[ ]:


# Correctness Heatmaps
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(14, 7))
plots[f'exp1a_correctness_heatmaps_{analysis_suffix}'] = fig

# Correctness by LLM vs Input Source
correctness_heatmap_mean = exp1a_df.groupby(['llm',
↪  'input_source'])['correctness'].mean().unstack()
correctness_heatmap_std = exp1a_df.groupby(['llm', 'input_source'])['correctness'].std().unstack()

correctness_annot_matrix = correctness_heatmap_mean.copy()
for i in range(len(correctness_heatmap_mean.index)):
    for j in range(len(correctness_heatmap_mean.columns)):
        mean_val = correctness_heatmap_mean.iloc[i, j]
        std_val = correctness_heatmap_std.iloc[i, j]
        if pd.notna(mean_val) and pd.notna(std_val):
            correctness_annot_matrix.iloc[i, j] = f"{mean_val:.1f}\n({std_val:.1f})"
        elif pd.notna(mean_val):
            correctness_annot_matrix.iloc[i, j] = f"{mean_val:.1f}"

sns.heatmap(correctness_heatmap_mean, annot=correctness_annot_matrix, fmt='', cmap='RdYlBu_r',
            center=correctness_heatmap_mean.mean().mean(), square=True, linewidths=0.5,
            cbar_kws={'shrink': 0.8, 'label': 'Correctness Score'}, annot_kws={'size': 10,
            ↪  'weight': 'bold'},
            xticklabels=[l.title() for l in correctness_heatmap_mean.columns],
            yticklabels=[l.title() for l in correctness_heatmap_mean.index], ax=ax1)

```

```
ax1.set_title('Correctness Score (LLM vs Input Source)\nValues: Mean (Std) | Scale: 0-10 points',
             fontsize=14, fontweight='bold', pad=20)
ax1.set_xlabel('Input Source', fontsize=12, fontweight='bold')
ax1.set_ylabel('LLM', fontsize=12, fontweight='bold')

# Correctness by LLM vs Prompt Type
correctness_prompt_heatmap_mean = exp1a_df.groupby(['llm', 'prompt_type'])['correctness'].mean().unstack()
correctness_prompt_heatmap_std = exp1a_df.groupby(['llm', 'prompt_type'])['correctness'].std().unstack()

correctness_prompt_annot_matrix = correctness_prompt_heatmap_mean.copy()
for i in range(len(correctness_prompt_heatmap_mean.index)):
    for j in range(len(correctness_prompt_heatmap_mean.columns)):
        mean_val = correctness_prompt_heatmap_mean.iloc[i, j]
        std_val = correctness_prompt_heatmap_std.iloc[i, j]
        if pd.notna(mean_val) and pd.notna(std_val):
            correctness_prompt_annot_matrix.iloc[i, j] = f"{mean_val:.1f}\n({std_val:.1f})"
        elif pd.notna(mean_val):
            correctness_prompt_annot_matrix.iloc[i, j] = f"{mean_val:.1f}"

sns.heatmap(correctness_prompt_heatmap_mean, annot=correctness_prompt_annot_matrix, fmt='', cmap='RdYlBu_r',
            center=correctness_prompt_heatmap_mean.mean().mean(), square=True, linewidths=0.5,
            cbar_kws={'shrink': 0.8, 'label': 'Correctness Score'}, annot_kws={'size': 10, 'weight': 'bold'},
            xticklabels=[l.title() for l in correctness_prompt_heatmap_mean.columns],
            yticklabels=[l.title() for l in correctness_prompt_heatmap_mean.index], ax=ax2)

ax2.set_title('Correctness Score (LLM vs Prompt Type)\nValues: Mean (Std) | Scale: 0-10 points',
             fontsize=14, fontweight='bold', pad=20)
ax2.set_xlabel('Prompt Type', fontsize=12, fontweight='bold')
ax2.set_ylabel('LLM', fontsize=12, fontweight='bold')

plt.tight_layout()
plt.show()

# Store detailed statistics tables
exp1a_llm_source_combinations = exp1a_df.groupby(['llm', 'input_source'])['total_score'].agg(['mean', 'std', 'median', 'min', 'max', 'count']).round(2)
tables[f'exp1a_llm_source_combinations_{analysis_suffix}'] = exp1a_llm_source_combinations

exp1a_llm_prompt_combinations = exp1a_df.groupby(['llm', 'prompt_type'])['total_score'].agg(['mean', 'std', 'median', 'min', 'max', 'count']).round(2)
tables[f'exp1a_llm_prompt_combinations_{analysis_suffix}'] = exp1a_llm_prompt_combinations

correctness_llm_source_stats = exp1a_df.groupby(['llm', 'input_source'])['correctness'].agg(['mean', 'std', 'median', 'min', 'max', 'count']).round(2)
```

```
588 tables[f'exp1a_correctness_llm_source_stats_{analysis_suffix}'] = correctness_llm_source_stats
589
590 correctness_llm_prompt_stats = exp1a_df.groupby(['llm', 'prompt_type'])['correctness'].agg(['mean',
    ↪  'std', 'median', 'min', 'max', 'count']).round(2)
591 tables[f'exp1a_correctness_llm_prompt_stats_{analysis_suffix}'] = correctness_llm_prompt_stats
592
593
594 # # Experiment 1b: Manipulated Content Analysis
595 #
596 # Analysis of LLM robustness to content manipulation.
597
598 # In[ ]:
599
600
601 print("EXPERIMENT 1B - MANIPULATION HANDLING ANALYSIS")
602 print("="*60)
603
604 # Overall statistics for manipulation_handling
605 exp1b_stats = exp1b_df[numeric_cols_1b].describe().round(2)
606 tables[f'exp1b_manipulation_stats_{analysis_suffix}'] = exp1b_stats
607 print("\nOverall Statistics (Manipulation Handling):")
608 display(exp1b_stats)
609
610
611 # In[ ]:
612
613
614 # Statistics by LLM
615 exp1b_llm_stats = exp1b_df.groupby('llm')[numeric_cols_1b].agg(['mean', 'std', 'median',
    ↪  'count']).round(2)
616 tables[f'exp1b_llm_manipulation_stats_{analysis_suffix}'] = exp1b_llm_stats
617 print("\nStatistics by LLM (Manipulation Handling):")
618 display(exp1b_llm_stats)
619
620
621 # In[ ]:
622
623
624 # Statistics by LLM and Prompt Type
625 exp1b_llm_prompt_stats = exp1b_df.groupby(['llm',
    ↪  'prompt_type'])['manipulation_handling'].agg(['mean', 'std', 'median']).round(2)
626 tables[f'exp1b_llm_prompt_manipulation_stats_{analysis_suffix}'] = exp1b_llm_prompt_stats
627 print("\nStatistics by LLM and Prompt Type (Manipulation Handling):")
628 display(exp1b_llm_prompt_stats)
629
630
631 # In[ ]:
632
```

```
633
634 # Store additional statistics
635 exp1b_prompt_stats = exp1b_df.groupby('prompt_type')[numeric_cols_1b].agg(['mean', 'std', 'median',
    ↪  'count']).round(2)
636 tables[f'exp1b_prompt_manipulation_stats_{analysis_suffix}'] = exp1b_prompt_stats
637
638
639 # # Inter-Rater Agreement Analysis
640 #
641 # **Note:** Agreement analysis is only available for staff data (Experts 2-5).
642
643 # In[ ]:
644
645
646 if agreement_available: # if using staff data was defined
647     print("INTER-RATER AGREEMENT ANALYSIS (Fleiss' Kappa)")
648     print("="*60)
649
650     # Experiment 1a Agreement
651     print("\nExperiment 1a Agreement (Experts 2, 3, 4, 5):")
652     agreement_exp1a = calculate_fleiss_kappa(
653         experts_data['expert_2']['exp1a'],
654         experts_data['expert_3']['exp1a'],
655         experts_data['expert_4']['exp1a'],
656         experts_data['expert_5']['exp1a'],
657         numeric_cols_1a
658     )
659     tables[f'agreement_exp1a_{analysis_suffix}'] = agreement_exp1a
660     display(agreement_exp1a.round(3))
661
662     if not agreement_exp1a.empty:
663         valid_kappas = agreement_exp1a['Fleiss_Kappa'].dropna()
664         if len(valid_kappas) > 0:
665             avg_kappa = valid_kappas.mean()
666
667             level = ("Slight" if avg_kappa < 0.2 else
668                     "Fair" if avg_kappa < 0.4 else
669                     "Moderate" if avg_kappa < 0.6 else
670                     "Substantial" if avg_kappa < 0.8 else
671                     "Almost Perfect")
672
673     # Check if Experiment 1b has sufficient data for agreement analysis
674     exp1b_has_enough_data = False
675     if 'experts_data' in locals():
676         total_filled = 0
677         for expert_key in ['expert_2', 'expert_3', 'expert_4', 'expert_5']:
678             if expert_key in experts_data:
679                 expert_df = experts_data[expert_key]['exp1b']
```

```
                # Check if manipulation_handling column exists and count non-null values
                if 'manipulation_handling' in expert_df.columns:
                    total_filled += expert_df['manipulation_handling'].notna().sum()
        exp1b_has_enough_data = total_filled > 10  # Lower threshold for exp1b

    if exp1b_has_enough_data:
        print("\n" + "-"*40)
        print("Experiment 1b Agreement (Experts 2, 3, 4, 5):")
        agreement_exp1b = calculate_fleiss_kappa(
            experts_data['expert_2']['exp1b'],
            experts_data['expert_3']['exp1b'],
            experts_data['expert_4']['exp1b'],
            experts_data['expert_5']['exp1b'],
            numeric_cols_1b
        )
        tables[f'agreement_exp1b_{analysis_suffix}'] = agreement_exp1b
        display(agreement_exp1b.round(3))

    else:
        print("\n" + "-"*40)
        print("Experiment 1b Agreement: Insufficient data for analysis")
        print(f"Total non-null manipulation_handling values found: {total_filled}")

    # Expert Comparison
    print("\n" + "="*60)
    print("EXPERT COMPARISON")
    print("="*60)

    expert_means_1a = {}
    for expert_key in ['expert_2', 'expert_3', 'expert_4', 'expert_5']:
        if expert_key in experts_data:
            expert_means_1a[expert_key] = experts_data[expert_key]['exp1a'][numeric_cols_1a].mean()

    if expert_means_1a:
        comparison_df_1a = pd.DataFrame(expert_means_1a).T
        comparison_df_1a['Overall_Mean'] = comparison_df_1a.mean(axis=1)
        comparison_df_1a = comparison_df_1a.round(3)
        tables[f'expert_comparison_exp1a_{analysis_suffix}'] = comparison_df_1a

        print("\nMean Ratings by Expert (Experiment 1a):")
        display(comparison_df_1a)

else:
    print("INTER-RATER AGREEMENT ANALYSIS")
    print("="*60)
    print("Agreement analysis not available for supervisor data.")
    print("Switch to staff data (USE_SUPERVISOR_DATA = False) to enable agreement analysis.")

```

```
728
729 # In[ ]:
730
731
732 # Agreement Analysis including the Staff Members AND the Supervisor
733 if agreement_available:
734     print("COMPREHENSIVE INTER-RATER AGREEMENT ANALYSIS")
735     print("="*70)
736
737     def calculate_comprehensive_fleiss_kappa(experts_data, experiment, criteria_cols):
738         results = []
739
740         expert_dfs = {}
741         for expert_key in ['expert_1', 'expert_2', 'expert_3', 'expert_4', 'expert_5']:
742             if expert_key in experts_data and experiment in experts_data[expert_key]:
743                 df = experts_data[expert_key][experiment]
744                 if 'sample_id' in df.columns:
745                     expert_dfs[expert_key] = df
746
747         # Add supervisor data if in staff mode
748         if experiment == 'exp1a':
749             supervisor_exp1a, _ = load_supervisor_data()
750             expert_dfs['expert_1'] = supervisor_exp1a
751         elif experiment == 'exp1b':
752             _, supervisor_exp1b = load_supervisor_data()
753             expert_dfs['expert_1'] = supervisor_exp1b
754
755         if len(expert_dfs) < 2:
756             return pd.DataFrame()
757
758         common_ids = None
759         for expert_key, df in expert_dfs.items():
760             if 'sample_id' in df.columns:
761                 ids = set(df['sample_id'].astype(int))
762                 if common_ids is None:
763                     common_ids = ids
764                 else:
765                     common_ids = common_ids & ids
766
767         if not common_ids:
768             return pd.DataFrame()
769
770         common_ids = sorted(list(common_ids))
771         expert_names = sorted(expert_dfs.keys())
772
773         print(f"\nAnalyzing {experiment.upper()} with experts: {expert_names}")
774         print(f"Common samples: {len(common_ids)}")
775
```

```
        for criterion in criteria_cols:
            ratings = []
            for sample_id in common_ids:
                sample_ratings = []
                valid_sample = True

                for expert_key in expert_names:
                    df = expert_dfs[expert_key]
                    if sample_id in df['sample_id'].values:
                        rating = df[df['sample_id'] == sample_id][criterion].iloc[0]
                        rating_val = np.nan if pd.isna(rating) else float(rating)

                        if pd.isna(rating_val) or rating_val < 0 or rating_val > 10:
                            valid_sample = False
                            break
                        sample_ratings.append(int(rating_val))
                    else:
                        valid_sample = False
                        break

                if valid_sample and len(sample_ratings) == len(expert_names):
                    ratings.append(sample_ratings)

            if len(ratings) >= 2:
                ratings_array = np.array(ratings)
                n_raters = len(expert_names)
                fleiss_table = np.zeros((len(ratings), 11))

                for i, item_ratings in enumerate(ratings):
                    for rating in item_ratings:
                        fleiss_table[i, int(rating)] += 1

                kappa = fleiss_kappa(fleiss_table)

                level = ("Slight" if kappa < 0.2 else
                        "Fair" if kappa < 0.4 else
                        "Moderate" if kappa < 0.6 else
                        "Substantial" if kappa < 0.8 else
                        "Almost Perfect")

                results.append({
                    'Criterion': criterion,
                    'Fleiss_Kappa': round(kappa, 3),
                    'Agreement_Level': level,
                    'N_Items': len(ratings),
                    'N_Raters': n_raters,
                    'Mean_Rating': round(np.mean(ratings_array), 2),
                    'Std_Rating': round(np.std(ratings_array), 2),
```

```
                    'Experts': ', '.join(expert_names)
                })

        return pd.DataFrame(results)

    # Calculate agreement for Experiment 1a
    print("\n" + "="*70)
    print("ALL EXPERTS AGREEMENT ANALYSIS (Including Supervisor)")
    print("="*70)

    comprehensive_agreement_exp1a = calculate_comprehensive_fleiss_kappa(experts_data, 'exp1a', numeric_cols_1a)

    if not comprehensive_agreement_exp1a.empty:
        tables[f'comprehensive_agreement_exp1a_{analysis_suffix}'] = comprehensive_agreement_exp1a
        print("\nExperiment 1a - All Available Experts Agreement:")
        display(comprehensive_agreement_exp1a[['Criterion', 'Fleiss_Kappa', 'Agreement_Level', 'N_Items', 'N_Raters', 'Mean_Rating', 'Std_Rating']].round(3))

    # Check for Experiment 1b agreement
    if analyze_exp1b:
        comprehensive_agreement_exp1b = calculate_comprehensive_fleiss_kappa(experts_data, 'exp1b', numeric_cols_1b)

        if not comprehensive_agreement_exp1b.empty:
            tables[f'comprehensive_agreement_exp1b_{analysis_suffix}'] = comprehensive_agreement_exp1b
            print("\n" + "-"*50)
            print("Experiment 1b - All Available Experts Agreement:")
            display(comprehensive_agreement_exp1b[['Criterion', 'Fleiss_Kappa', 'Agreement_Level', 'N_Items', 'N_Raters', 'Mean_Rating', 'Std_Rating']].round(3))

        else:
            print("\n" + "-"*50)
            print("Experiment 1b - Insufficient data for comprehensive agreement analysis")


# ## Data Export
#
# Save all tables and plots for thesis and presentations.

# In[ ]:


def save_all_results():
    print("Saving results...")

    # Determine prefix based on data source
```

```
    prefix = "supervisor_" if USE_SUPERVISOR_DATA else "staff_"

    tables_saved = 0
    for table_name, table_data in tables.items():
        clean_name = table_name.replace(f"_{analysis_suffix}", "")
        csv_path = os.path.join(output_tables_path, f"{prefix}{table_name}.csv")
        table_data.to_csv(csv_path)
        tables_saved += 1
        print(f"Saved table: {prefix}{clean_name}.csv")

    plots_saved = 0
    for plot_name, plot_fig in plots.items():
        clean_name = plot_name.replace(f"_{analysis_suffix}", "")
        png_path = os.path.join(output_plots_path, f"{prefix}{clean_name}.png")
        plot_fig.savefig(png_path, dpi=300, bbox_inches='tight')
        plots_saved += 1
        print(f"Saved plot: {prefix}{clean_name}.png")

    print(f"\nExport Summary:")
    print(f"  Tables saved: {tables_saved}")
    print(f"  Plots saved: {plots_saved}")
    print(f"  Output location: {output_base_path}")
    print(f"  Analysis type: {analysis_suffix}")
    print(f"  File prefix: {prefix}")

save_all_results()
```

Listing E.13: Experiment 2 Qualitative Evaluation Notebook: **evaluation2_qual.ipynb**

```
#!/usr/bin/env python
# coding: utf-8

# In[ ]:


from statsmodels.stats.inter_rater import fleiss_kappa
import pandas as pd
import numpy as np
from pathlib import Path
import seaborn as sns
import matplotlib.pyplot as plt
import os
import re
import warnings
from IPython.display import display

warnings.filterwarnings('ignore')

plt.style.use('default')
sns.set_palette("Set2")
plt.rcParams['figure.dpi'] = 300
plt.rcParams['savefig.dpi'] = 300

pd.set_option('display.max_rows', None)
pd.set_option('display.max_columns', None)
pd.set_option('display.width', None)
pd.set_option('display.max_colwidth', 100)
pd.set_option('display.notebook_repr_html', True)


# In[ ]:


# === METRICS & LABELS ===
metrics = [
    'relevance',
    'clarity',
    'answerability',
    'challenging',
    'value',
    'language',
    'bloom_rating',
    "blooms_level_score"
 ]
```

```
metric_labels = {
    'relevance': 'Relevance',
    'clarity': 'Clarity',
    'answerability': 'Answerability',
    'challenging': 'Challenging',
    'value': 'Value',
    'language': 'Language',
    'bloom_rating': "Bloom's Level (Student)",
    'blooms_level_score': "Bloom's Level (Score)"
}


# In[ ]:


base_path = Path("../60_analyses/csv/qualitative/exp2/students")

BASE_PROJECT_PATH = os.path.abspath(os.path.join(os.getcwd(), "..", ".."))
hint_base_path = os.path.join(BASE_PROJECT_PATH,
↪  "20_experiments/60_analyses/csv/qualitative/exp2/hints")
output_base_path = os.path.join(BASE_PROJECT_PATH, "40_evaluation/exp2/qualitative")
output_tables_path = os.path.join(output_base_path, "tables")
output_plots_path = os.path.join(output_base_path, "plots")

for path in [output_tables_path, output_plots_path]:
    os.makedirs(path, exist_ok=True)

tables = {}
plots = {}


# In[ ]:


def load_student_data(student_id):
    student_path = base_path / f"student_{student_id}"
    data = {}

    for exp in ["exp2a", "exp2b", "exp2c"]:
        csv_path = student_path / f"{exp}.csv"
        if csv_path.exists():
            df = None
            for encoding in ['utf-8', 'latin-1', 'cp1252', 'iso-8859-1']:
                try:
                    df = pd.read_csv(csv_path, encoding=encoding)
                    break
                except UnicodeDecodeError:
                    continue
```

```
93
94              if df is None:
95                  print(f"Error: Could not read {csv_path} with any encoding")
96                  continue
97
98              if 'comments' in df.columns:
99                  df = df.drop('comments', axis=1)
100
101             if 'answer_problems' in df.columns:
102                 df = df.drop('answer_problems', axis=1)
103
104             df['sample_id'] = df['sample_id'].astype(str)
105             if 'question_num' not in df.columns:
106                 df['question_num'] = df['sample_id'].str.extract('(\\d+)').astype(int).iloc[:, 0]
107
108             df['student'] = student_id
109             df['experiment'] = exp
110             data[exp] = df
111             print(f"Loaded {exp} for student_{student_id}: {len(df)} questions")
112
113     return data
114
115
116 # In[ ]:
117
118
119 def load_hint_data():
120     hint_data = {}
121     for exp in ["exp2a", "exp2b", "exp2c"]:
122         hint_path = os.path.join(hint_base_path, f"{exp}_hints.csv")
123         if os.path.exists(hint_path):
124             hint_df = None
125             for encoding in ['utf-8', 'latin-1', 'cp1252', 'iso-8859-1']:
126                 try:
127                     hint_df = pd.read_csv(hint_path, encoding=encoding)
128                     break
129                 except UnicodeDecodeError:
130                     continue
131             if hint_df is None:
132                 print(f"Error: Could not read {hint_path} with any encoding")
133                 continue
134             if exp == 'exp2a':
135                 if 'question_id' not in hint_df.columns:
136                     print(f"Warning: No question_id column found in {exp}_hints.csv")
137                     continue
138                 hint_df['question_id'] = hint_df['question_id'].astype(str)
139                 print(f"Loaded {exp}_hints.csv: {len(hint_df)} rows")
140                 hint_data[exp] = hint_df
```

```
            elif exp in ['exp2b', 'exp2c'] and 'bloom_original' in hint_df.columns:
                hint_df = hint_df.copy()
                hint_df['bloom_original'] = hint_df['bloom_original'].astype(str)
                print(f"Loaded {exp}_hints.csv: {len(hint_df)} rows (no question_id, using row order)")
                hint_data[exp] = hint_df
            else:
                print(f"Warning: No suitable columns found in {exp}_hints.csv")
    return hint_data

all_data = {}
for i in range(1, 4):
    all_data[f"student_{i}"] = load_student_data(i)

hint_data = load_hint_data()


# In[ ]:


def combine_data():
    combined = []
    print("\n--- Combining all available student data ---")
    for student, experiments in all_data.items():
        print(f"\n{student}:")
        for exp, df in experiments.items():
            if not df.empty:
                if exp in hint_data:
                    hint_df = hint_data[exp]
                    if len(df) == len(hint_df):
                        df_merged = df.copy()
                        for col in hint_df.columns:
                            if col not in df_merged.columns:
                                df_merged[col] = hint_df[col].values
                        print(f"  {exp}: {len(df_merged)} evaluations (row-wise assignment: {list(hint_df.columns)})")
                    else:
                        print(f"  {exp}: Could not row-wise assign hint data (len(df)={len(df)}, len(hint_df)={len(hint_df)})")
                        df_merged = df
                    combined.append(df_merged)
                else:
                    print(f"  {exp}: {len(df)} evaluations (no hint data available)")
                    combined.append(df)
    if combined:
        return pd.concat(combined, ignore_index=True)
    return pd.DataFrame()

```

```
df_combined = combine_data()
print(f"\n=== FINAL SUMMARY ===")
print(f"Total evaluations: {len(df_combined)}")


# # Experiment 2: Descriptive Statistics
#
# Analysis of prompt engineering approaches for question generation.

# In[ ]:


def convert_bloom_rating_simple(value, experiment, given_level=None):
    bloom_map = {
        6: 10,
        5: 8.5,
        4: 7,
        3: 4.5,
        2: 3,
        1: 1.5
    }

    def extract_levels(val):
        if isinstance(val, str):
            nums = re.findall(r'\d+', val)
            return [int(n) for n in nums] if nums else []
        elif isinstance(val, (int, float)) and not pd.isnull(val):
            return [int(val)]
        return []

    if experiment == 'exp2a':
        levels = extract_levels(value)
        if levels:
            max_level = max(levels)
            return bloom_map.get(max_level, np.nan)
        return np.nan
    elif experiment in ['exp2b', 'exp2c']:
        rater_levels = extract_levels(value)
        given_levels = extract_levels(given_level) if given_level is not None else []
        if not rater_levels or not given_levels:
            return np.nan
        best_score = 0
        for r in rater_levels:
            for g in given_levels:
                if r == g:
                    best_score = max(best_score, 10)
                elif abs(r - g) == 1:
                    best_score = max(best_score, 5)
```

```
        if best_score == 0:
            return 0
        return best_score
    else:
        return np.nan


# In[ ]:


print("EXPERIMENT 2 - DESCRIPTIVE STATISTICS (ALL RATINGS)")
print("="*70)
print(f"Number of ratings in df_combined: {len(df_combined)}")

df_numeric = df_combined.copy()

print("Converting metrics to numeric (all ratings)...")
for metric in metrics:
    if metric == 'bloom_rating':
        df_numeric[metric] = df_combined[metric]
    elif metric == 'blooms_level_score':
        def bloom_score_apply(row):
            if row['experiment'] in ['exp2b', 'exp2c'] and 'bloom_original' in row and not pd.isnull(row['bloom_original']):
                return convert_bloom_rating_simple(row['bloom_rating'], row['experiment'], row['bloom_original'])
            else:
                return convert_bloom_rating_simple(row['bloom_rating'], row['experiment'])
        df_numeric[metric] = df_combined.apply(bloom_score_apply, axis=1)
    else:
        df_numeric[metric] = pd.to_numeric(df_combined[metric], errors='coerce')

display(df_numeric)


# In[ ]:


numeric_metrics = [m for m in metrics if pd.api.types.is_numeric_dtype(df_numeric[m])]

main_score_metrics = ['relevance', 'clarity', 'answerability', 'challenging', 'value', 'language', 'blooms_level_score']
existing_score_metrics = [m for m in main_score_metrics if m in df_numeric.columns]
df_numeric['total_score'] = df_numeric[existing_score_metrics].sum(axis=1)
metric_labels['total_score'] = 'Total Score'
if 'total_score' not in metrics:
    metrics.append('total_score')
if 'total_score' not in numeric_metrics:
```

```
    numeric_metrics.append('total_score')

if len(df_numeric) > 0 and numeric_metrics:
    print("\nOverall Statistics (all metrics on 0-10 scale):")
    overall_stats = df_numeric[numeric_metrics].describe().round(2)
    display(overall_stats)

    print("\nStatistics by Experiment:")
    exp_stats = df_numeric.groupby('experiment')[numeric_metrics].agg(['mean', 'std', 'median',
    ↪  'count']).round(2)
    display(exp_stats)

else:
    print("\nNo ratings available for statistics.")
    overall_stats = pd.DataFrame()
    exp_stats = pd.DataFrame()


# In[ ]:


print("\n=== VISUALIZATION: PERFORMANCE BY EXPERIMENT ===")
if len(df_numeric) > 0 and numeric_metrics:
    fig, axes = plt.subplots(4, 2, figsize=(14, 20))
    axes = axes.flatten()

    for i, metric in enumerate(numeric_metrics):
        df_plot = df_numeric[[metric, 'experiment']].dropna()
        if len(df_plot) > 0:
            metric_display = metric_labels[metric]
            colors = ['#66c2a5', '#fc8d62', '#8da0cb']
            sns.boxplot(data=df_plot, x='experiment', y=metric, ax=axes[i], palette=colors,
                        medianprops={'color': 'black', 'linewidth': 2.5, 'linestyle': ':'})
            for j, exp in enumerate(['exp2a', 'exp2b', 'exp2c']):
                exp_data = df_plot[df_plot['experiment'] == exp][metric]
                if len(exp_data) > 0:
                    mean_val = exp_data.mean()
                    axes[i].scatter(j, mean_val, color='red', marker='D', s=50, zorder=3,
                    ↪  edgecolor='darkred', linewidth=1)
            if metric == 'total_score' or i == 7:
                axes[i].set_ylim(0, 70)
            else:
                axes[i].set_ylim(0, 10)
            axes[i].set_title(f'{metric_display}', fontsize=12, fontweight='bold')
            axes[i].set_xlabel('Experiment')
            axes[i].set_ylabel(metric_display)
            axes[i].grid(True, alpha=0.3)
            axes[i].set_xticklabels(['Type Only', 'Bloom Only', 'Type + Bloom'])
```

```
        else:
            axes[i].set_title(f'{metric_labels[metric]} - No Data')
            axes[i].text(0.5, 0.5, 'No Data', ha='center', va='center',
            ↪  transform=axes[i].transAxes)
    for j in range(len(numeric_metrics), len(axes)):
        axes[j].set_visible(False)
    plt.suptitle('Experiment 2: Performance across Prompt Engineering Approaches', fontsize=16,
    ↪  fontweight='bold', y=0.995)
    plt.subplots_adjust(top=0.92)
    plt.tight_layout()
    if 'output_plots_path' in locals():
        figpath = os.path.join(output_plots_path, "exp2_boxplots_all_metrics_by_experiment.png")
        fig.savefig(figpath)
        print(f"Saved collective experiment boxplot: {figpath}")
    plt.show()
    print("Visualization complete.")
else:
    print("No data available for visualization.")


# # Metric Analysis by LLM
#
# The following section provides boxplots and summary tables for all metrics, grouped by LLM
↪  (language model).

# In[ ]:


print("\n=== METRIC ANALYSIS BY LLM ===")

if 'df_numeric' in locals() and 'llm' in df_numeric.columns and len(df_numeric) > 0 and
↪  numeric_metrics:
    fig, axes = plt.subplots(4, 2, figsize=(14, 20))
    axes = axes.flatten()

    for i, metric in enumerate(numeric_metrics):
        df_plot = df_numeric[[metric, 'llm']].dropna()
        if len(df_plot) > 0:
            metric_display = metric_labels.get(metric, metric)
            colors = sns.color_palette("Set2", n_colors=df_plot['llm'].nunique())
            sns.boxplot(data=df_plot, x='llm', y=metric, ax=axes[i], palette=colors,
                        medianprops={'color': 'black', 'linewidth': 2.5, 'linestyle': ':'})
            for j, llm_name in enumerate(sorted(df_plot['llm'].unique())):
                llm_data = df_plot[df_plot['llm'] == llm_name][metric]
                if len(llm_data) > 0:
                    mean_val = llm_data.mean()
                    axes[i].scatter(j, mean_val, color='red', marker='D', s=50, zorder=3,
                    ↪  edgecolor='darkred', linewidth=1)
```

```
            if metric == 'total_score' or i == 7:
                axes[i].set_ylim(0, 70)
            else:
                axes[i].set_ylim(0, 10)
            axes[i].set_title(f'{metric_display} by LLM', fontsize=12, fontweight='bold')
            axes[i].set_xlabel('LLM')
            axes[i].set_ylabel(metric_display)
            axes[i].grid(True, alpha=0.3)
            llm_labels = [llm.capitalize() for llm in sorted(df_plot['llm'].unique())]
            axes[i].set_xticklabels(llm_labels)
        else:
            axes[i].set_title(f'{metric_labels[metric]} - No Data')
            axes[i].text(0.5, 0.5, 'No Data', ha='center', va='center',
            ↪  transform=axes[i].transAxes)
    for j in range(len(numeric_metrics), len(axes)):
        axes[j].set_visible(False)
    plt.suptitle('Experiment 2: LLM Performance across all Criteria', fontsize=16,
    ↪  fontweight='bold', y=0.995)
    plt.subplots_adjust(top=0.92)
    plt.tight_layout()

    if 'output_plots_path' in locals():
        figpath = os.path.join(output_plots_path, "exp2_boxplots_all_metrics_by_llm.png")
        fig.savefig(figpath)
        print(f"Saved collective LLM boxplot: {figpath}")
    plt.show()
    print("Visualization by LLM complete.")

    print("\nSummary Table: Metrics by LLM (mean, std, count)")
    llm_stats = df_numeric.groupby('llm')[numeric_metrics].agg(['mean', 'std', 'median',
    ↪  'count']).round(2)
    display(llm_stats)
    if 'output_tables_path' in locals():
        llm_stats.to_csv(os.path.join(output_tables_path, "exp2_metrics_by_llm.csv"))
        print(f"Saved metrics by LLM table to {output_tables_path}")

else:
    print("No data available for LLM-based analysis.")


# In[ ]:


print("\n=== TABLE: LLM vs. Prompt Type (Experiment) for Bloom's Level Scoring ===")

if 'df_numeric' in locals() and 'llm' in df_numeric.columns and 'experiment' in df_numeric.columns
↪  and 'blooms_level_score' in df_numeric.columns:
```

```
    llm_prompt_bloom = df_numeric.pivot_table(index='llm', columns='experiment', values='blooms_level_score', aggfunc=['mean', 'std', 'median']).round(2)
    display(llm_prompt_bloom)
    if 'output_tables_path' in locals():
        llm_prompt_bloom.to_csv(os.path.join(output_tables_path, "exp2_llm_vs_prompttype_bloomslevel.csv"))
        print(f"Saved LLM vs. Prompt Type Bloom's Level table to {output_tables_path}")

else:
    print("Not enough data for LLM vs. Prompt Type Bloom's Level table.")


# In[ ]:


print("\n=== TABLE: Question Type vs. Subexperiment for Bloom's Level Scoring ===")

if (
    'df_numeric' in locals() and
    'experiment' in df_numeric.columns and
    'question_type' in df_numeric.columns and
    'blooms_level_score' in df_numeric.columns
):
    qt_exp_bloom = df_numeric.pivot_table(
        index='question_type',
        columns='experiment',
        values='blooms_level_score',
        aggfunc=['mean', 'std', 'median', 'count']
    ).round(2)
    display(qt_exp_bloom)
    if 'output_tables_path' in locals():
        qt_exp_bloom.to_csv(os.path.join(output_tables_path, "exp2_questiontype_vs_experiment_bloomslevel.csv"))
        print(f"Saved Question Type vs. Subexperiment Bloom's Level table to {output_tables_path}")
else:
    print("Not enough data for Question Type vs. Subexperiment Bloom's Level table.")


# # Inter-Student Reliability Analysis
#
# Analysis of agreement between student evaluators on the quality of generated questions.

# In[ ]:


print("\n" + "="*60)
print("FLEISS' KAPPA INTER-RATER RELIABILITY ANALYSIS")
print("="*60)
```

```

def make_global_question_id(row):
    if row['experiment'] == 'exp2a':
        return f"{row['experiment']}|{row['llm']}|{row['question_id']}|{row['question_type']}"
    elif row['experiment'] == 'exp2b':
        return f"{row['experiment']}|{row['llm']}|{row['bloom_original']}"
    elif row['experiment'] == 'exp2c':
        return f"{row['experiment']}|{row['llm']}|{row['bloom_original']}|{row['question_type']}"
    else:
        return None

df_numeric['global_question_id'] = df_numeric.apply(make_global_question_id, axis=1)


# In[ ]:


def extract_max_bloom_rating(val):
    if isinstance(val, str):
        nums = re.findall(r'\d+', val)
        return max([int(n) for n in nums]) if nums else np.nan
    elif isinstance(val, (int, float)) and not pd.isnull(val):
        return int(val)
    return np.nan


# In[ ]:


def kappa_level(k):
    if k < 0:
        return "Invalid"
    if k < 0.2:
        return "Slight"
    elif k < 0.4:
        return "Fair"
    elif k < 0.6:
        return "Moderate"
    elif k < 0.8:
        return "Substantial"
    else:
        return "Almost Perfect"


# In[ ]:


```

```
504 def calculate_fleiss(df, metrics):
505     results = []
506     df = df.copy()
507 
508     # Handle all needed 0-10 metrics
509     for metric in metrics:
510         if metric in ('bloom_rating', 'blooms_level_score', 'total_score'):
511             continue
512         pivot = df.pivot_table(index='global_question_id', columns='student', values=metric)
513         pivot = pivot.dropna(axis=0)
514         ratings_matrix = pivot.applymap(lambda x: max(1, min(10, int(round(x)))) if pd.notnull(x)
        ↪  else np.nan).values
515         fleiss_table = np.zeros((ratings_matrix.shape[0], 10))
516         for i, row in enumerate(ratings_matrix):
517             for r in row:
518                 if not np.isnan(r) and 1 <= r <= 10:
519                     fleiss_table[i, int(r)-1] += 1
520         all_identical = np.all((fleiss_table == np.max(fleiss_table, axis=1, keepdims=True)) |
        ↪  (fleiss_table == 0), axis=1)
521         try:
522             if np.all(all_identical):
523                 kappa = 1.0
524             else:
525                 kappa = fleiss_kappa(fleiss_table)
526         except Exception:
527             kappa = np.nan
528 
529 
530         flat_valid = [r for row in ratings_matrix for r in row if not np.isnan(r)]
531         mean_rating = float(np.mean(flat_valid)) if flat_valid else np.nan
532         std_rating = float(np.std(flat_valid)) if flat_valid else np.nan
533 
534         results.append({
535             'Metric': metric_labels.get(metric, metric),
536             'Fleiss_Kappa': kappa,
537             'Agreement_Level': kappa_level(kappa) if not np.isnan(kappa) else 'No data',
538             'N_Questions': int(ratings_matrix.shape[0]),
539             'N_Raters': int(pivot.shape[1]) if pivot.shape[0] > 0 else 0,
540             'Mean_Rating': round(mean_rating, 2) if not np.isnan(mean_rating) else np.nan,
541             'Std_Rating': round(std_rating, 2) if not np.isnan(std_rating) else np.nan
542         })
543 
544     df['bloom_rating_numeric'] = df['bloom_rating'].apply(extract_max_bloom_rating)
545     pivot = df.pivot_table(index='global_question_id', columns='student',
    ↪  values='bloom_rating_numeric')
546     pivot = pivot.dropna(axis=0)
547     ratings_matrix = pivot.values
```

```
    ratings_matrix = np.array([[max(1, min(6, int(r))) for r in row] for row in ratings_matrix if not any(pd.isnull(row))])

    fleiss_table = np.zeros((len(ratings_matrix), 6))
    for i, row in enumerate(ratings_matrix):
        for r in row:
            if 1 <= r <= 6:
                fleiss_table[i, int(r) - 1] += 1

    all_identical = np.all(
        (fleiss_table == np.max(fleiss_table, axis=1, keepdims=True)) | (fleiss_table == 0),
        axis=1
    )
    try:
        if np.all(all_identical):
            kappa = 1.0
        else:
            kappa = fleiss_kappa(fleiss_table)
    except Exception:
        kappa = np.nan

    flat_valid = [r for row in ratings_matrix for r in row]
    mean_rating = float(np.mean(flat_valid)) if flat_valid else np.nan
    std_rating = float(np.std(flat_valid)) if flat_valid else np.nan

    results.append({
        'Metric': 'Bloom Rating (1-6 Scale)',
        'Fleiss_Kappa': kappa,
        'Agreement_Level': kappa_level(kappa) if not np.isnan(kappa) else 'No data',
        'N_Questions': int(len(ratings_matrix)),
        'N_Raters': int(pivot.shape[1]) if pivot.shape[0] > 0 else 0,
        'Mean_Rating': round(mean_rating, 2) if not np.isnan(mean_rating) else np.nan,
        'Std_Rating': round(std_rating, 2) if not np.isnan(std_rating) else np.nan
    })

    return pd.DataFrame(results)


# In[ ]:


kappa_all_df = calculate_fleiss(df_numeric, metrics)
print("\nFleiss' Kappa Calculation:")
display(kappa_all_df.round(4))
if 'output_tables_path' in locals():
    kappa_all_df.to_csv(os.path.join(output_tables_path, "exp2_fleiss_kappa_all_categories.csv"), index=False)
    print(f"Saved Fleiss' Kappa (all categories) table to {output_tables_path}")
```

```

# In[ ]:


def save_results():
    print("Saving key results (based on all ratings)...")

    if 'overall_stats' in locals() and not overall_stats.empty:
        overall_stats.to_csv(os.path.join(output_tables_path, "exp2_overall_statistics_all.csv"))
        print("Saved overall statistics (all ratings)")

    if 'exp_stats' in locals() and not exp_stats.empty:
        exp_stats.to_csv(os.path.join(output_tables_path, "exp2_experiment_statistics_all.csv"))
        print("Saved experiment statistics (all ratings)")

    if 'kappa_all_df' in locals() and not kappa_all_df.empty:
        kappa_all_df.to_csv(os.path.join(output_tables_path, "exp2_fleiss_kappa_all.csv"),
        ↪  index=False)
        print("Saved Fleiss' Kappa results (all ratings)")

    print(f"\nResults saved to: {output_tables_path}")

save_results()
print(f"\n" + "="*60)
print("ANALYSIS COMPLETE")

```

Declaration of Independent Work

I hereby declare that I have written this Bachelor's thesis independently and have not used any sources or aids other than those cited.

All passages taken verbatim or in essence from publications have been identified as such.

This thesis has not been published and has not been submitted in similar or identical form as an examination paper for evaluation.

Rostock, the

(Submission Date) (Full Signature)